

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

=====

Дата генерации: 14.09.2026, 21:52:49
Файлов исходного кода: 110
Суммарный объём кода: 1.94 MB
Глав/билетов в пособии: 36

=====

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

=====

1. 1. Дерево отрезков с операциями на отрезках [id: ...]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref]
6. 6. Графы. Представление графов, DFS, BFS [id: ...]
7. 7. Графы. Планарные. Покраска [id: graph-plana]
8. 7. Графы. Планарные. Покраска [id: planarity-e]
9. 8. Графы. Компоненты связности [id: graph-comp]
10. 9. Графы. Топологическая сортировка [id: top-s]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articul]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cyc]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-a]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: ...]
22. 18. Графы. Остовное дерево. Краскал [id: mst-k]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Б
24. 19. Графы. Остовное дерево. Прима [id: mst-pri]
25. 20. Графы. Остовное дерево. Борувка [id: mst-b]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/collect-code.mjs

95. scripts/export/common.mjs

96. scripts/export/html-to-blocks.mjs

97. scripts/export/render-pages.mjs

98. scripts/verify/browser-checks.tsx

99. scripts/verify/check-links.mjs

100. scripts/verify/check-reference-code.mjs

101. scripts/verify/dom-env.mjs

102. scripts/verify/live-matrix.test.mjs

103. scripts/verify/probe-compiler.tsx

104. scripts/verify/README.md

105. scripts/verify/run-browser-checks.mjs

106. scripts/verify/run-probe.mjs

CI / автоматизация

107. .github/workflows/deploy-pages.yml

108. .github/workflows/rebuild-pdf.yml

109. .github/workflows/verify.yml

Прочее

110. .ai_skills/universal_guide_authoring/SKILL.md

```
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
deploy:
```

```
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
+       name: github-pages
```

```
      default: "150"

concurrency:
  group: rebuild-pdf-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot: ')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends imagemagick
          # На Ubuntu политика ImageMagick иногда запрещает
          # разрешаем то, что нужно пайплайну (text -> pdf)
          for policy in /etc/ImageMagick-6/policy.xml /usr/share/ImageMagick-6/policy.xml; do
            if [ -f "$policy" ]; then
              sudo sed -i 's/rights="none" pattern="\{P
```

```
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action
    git add -- public/export/full_code_all_pages.
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full
    git push origin "HEAD:${GITHUB_REF_NAME}"
....

### Patch относительно `main`

````diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
 - name: Checkout
```

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, >
    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
    <title> Дискретка для тупых – Интерактивный гид по
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, >
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
     <title> Дискретка для тупых – Интерактивный гид по
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useState<
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefined;
 const next = index < chapters.length - 1 ? chapters[index + 1] : undefined;

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLInputElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters.length]);

 return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
 </div>
 </>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное пособие
</footer>
</div>
);
}
....

Patch относительно `main`

```

```
`src/components/Navbar.tsx`
```

```
Полное содержимое после изменений
```

```
```tsx
```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler') => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab }) => {
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };

  return (
```

```
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center gap-3 w-full">
          <div className="bg-gradient-to-tr from-indigo-500 to-purple-500 h-6 w-6 rounded-sm">
            <Sparkles className="w-6 h-6" />
          </div>
          <div className="min-w-0 flex-1">
            <h1 className="text-xl font-extrabold text-white">
              Welcome to the Guide
            </h1>
          </div>
        </div>
      </div>
    </header>
  );
};
```

```

        uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }` }
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full_
    download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
    over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех страниц"
    >
    <FileDown className="w-4 h-4" /> Скачать PDF
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full_
    download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
    :bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом"
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center
    er:bg-amber-600/20 border border-amber-500/

```



```

const [code, setCode] = useState(STARTER_CODE);
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Нажмите «Запустить»");
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(false);
const [runtimeMessage, setRuntimeMessage] = useState("");

const runCode = useCallback(async () => {
  if (running) return;

  setRunning(true);
  setOutput("");
  setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

  const stdout: string[] = [];
  const stderr: string[] = [];
  const inputLines = stdin.split(/\r?\n/);

  try {
    const runtime = await getPythonRuntime();
    setRuntimeReady(true);
    setRuntimeMessage("Python готов — код выполняется");

    runtime.setStdout({ batched: (message) => stdout.push(message) });
    runtime.setStderr({ batched: (message) => stderr.push(message) });
    runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
    await runtime.runPythonAsync(code);

    const text = [...stdout, ...stderr].join("");
    setOutput(text || "Готово: программа ничего не выполнила");
  } catch (error) {
    const captured = [...stdout, ...stderr].join("");
    setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
    setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
  } finally {
    setRunning(false);
  }
}, [code, stdin, running, runtimeReady]);

```

```

</div>
<div className="flex items-center gap-2">
  <button
    type="button"
    onClick={reset}
    className="inline-flex items-center gap-2
ate-800 transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" />
  </button>
  <button
    type="button"
    onClick={() => void runCode()}
    disabled={running}
    className="inline-flex items-center gap-2
60 text-white text-xs font-bold transition-
  >
    {running ? <Loader2 className="w-3.5 h-3.5" /> : <span>
    {running ? "Запускаю..." : "Запустить"}}
  </button>
</div>
</div>

<textarea
  value={code}
  onChange={(event) => setCode(event.target.value)}
  onKeyDown={(event) => {
    if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
      void runCode();
    }
  }}
  spellCheck={false}
  aria-label="Код Python"
  className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-blue-500"
  placeholder="Напишите Python-код..."
/>

```

```

        <Info className="w-4 h-4 shrink-0 text-indigo-500">
        <span>Ctrl или Cmd + Enter запускает код. В
    </div>
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
        <Info className="w-4 h-4 shrink-0 text-amber-500">
        <span>Первый запуск скачивает примерно неск
    </div>
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
        <Info className="w-4 h-4 shrink-0 text-emerald-500">
        <span>Код выполняется в браузере и не отпра
    </div>
    </div>
</div>
</main>
);
}
....
```

Patch относительно `main`

```

```diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и установит библиотеки
+name = "алгоритмы"
+
+for number in range(1, 4):
+ console.log(\`Выполнение цикла \${number}\`);
+ // Здесь можно добавить код, который будет выполняться в Python
+ // Например:
+ // console.log("Hello from Python!");
+ // console.log(2 + 2);
+ // console.log("Сканирование завершено");
+`;
```



```

+ const runCode = useCallback(async () => {
+ if (running) return;
+
+ setRunning(true);
+ setOutput("");
+ setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+ const stdout: string[] = [];
+ const stderr: string[] = [];
+ const inputLines = stdin.split(/\r?\n/);
+
+ try {
+ const runtime = await getPythonRuntime();
+ setRuntimeReady(true);
+ setRuntimeMessage("Python готов – код выполняется");
+
+ runtime.setStdout({ batched: (message) => stdout
+ runtime.setStderr({ batched: (message) => stderr
+ runtime.setStdin({ stdin: () => (inputLines.length
+ await runtime.runPythonAsync(code);
+
+ const text = [...stdout, ...stderr].join("");
+ setOutput(text || "Готово: программа ничего не выв
+ } catch (error) {
+ const captured = [...stdout, ...stderr].join("");
+ setOutput(`${captured}${captured && !captured.endsWith
+ setRuntimeMessage("Не удалось выполнить код – пр
+ } finally {
+ setRunning(false);
+ }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резул
+ };
+
+

```

```

+ >
+ <RotateCcw className="w-3.5 h-3.5" />
+ </button>
+ <button
+ type="button"
+ onClick={() => void runCode()}
+ disabled={running}
+ className="inline-flex items-center p-
-60 text-white text-xs font-bold transition-colors"
+ >
+ {running ? <Loader2 className="w-3.5
+ {running ? "Запускаю..." : "Запустить"}
+ </button>
+ </div>
+ </div>
+
+ <textarea
+ value={code}
+ onChange={(event) => setCode(event.target.value)}
+ onKeyDown={(event) => {
+ if ((event.ctrlKey || event.metaKey) &
+ event.preventDefault();
+ void runCode();
+ }
+ }}
+ spellCheck={false}
+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код..."
+ />
+
+ <div className="border-t border-slate-800
+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea
+ id="python-stdin"

```

## Универсальное пособие • полный исходный код

```

+ <div className="flex gap-2 rounded-xl border
+ <Info className="w-4 h-4 shrink-0 text-eme
+ Код выполняется в браузере и не отпра
+ </div>
+ </div>
+ </div>
+ </main>
+);
+}
````
```

`vite.config.ts`

Полное содержимое после изменений

```
````typescript  
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локаль
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
```

Универсальное пособие • полный исходный код

-----  
-----  
  
# AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное Пособие).

**\*\*CRITICAL DIRECTIVE\*\*:**

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md`.`

This file contains the mandatory rich HTML templates, strictly required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML tags.

-----  
ФАЙЛ 3/110: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2048 байт

-----  
  
# Gemini App Canvas Instructions

Ты – AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу, ты должен **\*\*строго\*\*** придерживаться готовых шаблонов разметки, описанных в файле ``.ai_skills/universal_guide_authoring/SKILL.md``.

## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML-шаблоны, описанные в файле ``.ai_skills/universal_guide_authoring/SKILL.md``.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть описана в файле ``.ai_skills/universal_guide_authoring/SKILL.md`` в формате `{ "id": "...", "content": "...`" } для вставки в `.src/data/content.json`.`
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-blue-200``).
  - Обязательная сетка аналогий (2 колонки: неформальный язык / Формальный язык).
  - "Душные" детали (код, формулы, сложные доказательства).

---

ФАЙЛ 6/110: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

---

```
{
 "name": "algv0-exam-guide",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "audit:content": "node scripts/audit-coverage.mjs",
 "audit:terms": "node scripts/audit-terms.mjs",
 "export:txt": "node scripts/export/collect-code.mjs",
 "export:png": "node scripts/export/render-pages.mjs",
 "export:pdf": "node scripts/export/build-pdf.mjs",
 "export": "npm run export:txt && npm run export:png",
 "export:guide": "node scripts/export/build-guide-pd",
 "export:shots": "node scripts/export/capture-viz-sh",
 "verify:unit": "node scripts/verify/live-matrix.tes",
 "verify:code": "node scripts/verify/check-reference",
 "verify:links": "node scripts/verify/check-links.mj",
 "verify:dom": "node scripts/verify/run-browser-chec",
 "verify:probe": "node scripts/verify/run-probe.mjs",
 "verify": "npm run verify:unit && npm run verify:li",
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 "tailwind-merge": "3.4.0"
```

Универсальное пособие • полный исходный код

```

npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export` | всё сразу | txt + png + pdf |

Скрипты лежат в `scripts/export/`. Требуется ImageMagic

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core
```
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` — DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` — оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается ****на каждой новой коммита**** и пересобирает TXT → PNG → PDF, кладёт PNG-страницы и документ PDF в папку `public/export/full\_code\_all\_pages` и коммитит обновлённые `public/export/full\_code\_all\_pages` (с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code\_all\_pages`

Аудит контента

```
```bash  
node scripts/audit-coverage.mjs # какие темы без
```

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы.
- \* Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторные

- \* Они помещены внутрь оберток с классом `<div class="img">`
- \* Все отрисовки находятся внутри векторного тега `<svg>`
- \* Подпись к картинке, если она есть, идет сразу после SVG

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```

* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование
```

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

| Глава                                      | id                    | Комментарий             |
|--------------------------------------------|-----------------------|-------------------------|
| 15. Планарность: K5, K3,3 и формула Эйлера | `planarity-euler`     | «на пальцах».           |
| 16. Флойд (Флойд–Уоршелл)                  | `floyd`               | Есть матричный алгоритм |
| 17. Алгоритм Джонсона                      | `johnson-algo`        | Перевзвешивание         |
| Обзор: Кратчайшие пути и Графы             | `intro`               | Вводная страница        |
| Эйлер: Путь ≠ Цикл                         | `euler-path-vs-cycle` | Ключевая концепция      |

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сложно»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока не реализована

| Глава                                | id                   | Чего не хватает              |
|--------------------------------------|----------------------|------------------------------|
| 3. Декартово дерево (Treap)          | `treap`              | Нет ни визуализации, ни кода |
| 8. Графы. Компоненты связности       | `graph-components`   | Нет кода                     |
| 24. Классы сложности, сведение задач | `complexity-classes` | Нет кода                     |

## Сводная таблица по всем главам

| # | Глава                                           | Аналогий | 2D-виз. | SVG | Картинка |
|---|-------------------------------------------------|----------|---------|-----|----------|
| 2 | 2. Двумерная разреженная матрица (Sparse Table) | Нет      | Нет     | Нет | Нет      |



ФАЙЛ 9/110: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

---

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],

 /* Bundler mode */
 "moduleResolution": "bundler",
 "allowImportingTsExtensions": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

---

```
});
```

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

ФАЙЛ 11/110: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 206 | РАЗМЕР: 10.2 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { LiteGuide } from "../components/LiteGuide";
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite=1`
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {
 if (typeof window === "undefined") return { lite: false };
 const q = new URLSearchParams(window.location.search);
 const lite = q.has("lite") || q.get("view") === "lite";
 return { lite, topic: q.get("topic") ?? undefined };
}
```

```
export default function App() {
 const liteMode = useMemo(readLiteMode, []);
 const [activeChapterId, setActiveChapterId] = useState<string>();
```

```

 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.innerWidth - compilerWidth;
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", apply);
 }, [compilerOpen, compilerWidth]);

const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefined;
const next = index < chapters.length - 1 ? chapters[index + 1] : undefined;

const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {
 requestAnimationFrame(() => {
 document.getElementById("chapter-viz")?.scrollIntoView();
 });
 });
}, []);

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target.tagName)) return;
 if (e.key === "ArrowLeft") {
 if (prev) goTo(prev.id);
 } else if (e.key === "ArrowRight") {
 if (next) goTo(next.id);
 }
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, prev, next]);

```

```

 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center gap-
border-slate-700 bg-slate-900/90 px-1 py-3
 aria-label={sidebarCollapsed ? "Развернуть" : "Свернуть"}
 title={sidebarCollapsed ? "Развернуть содержимое" : "Свернуть"}
 >
 {sidebarCollapsed ? <PanelLeftOpen className="text-slate-400">

 </button>

<main id="main-content" className="flex-1 min-w-0" >
 { /* Шапка главы с быстрыми переходами */ }
 <div className="flex items-center justify-between" >
 <div className="min-w-0">
 <div className="text-[10px] uppercase font-medium">
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium">
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" >
 <div
 className="h-full bg-gradient-to-r from-slate-800 to-slate-700"
 style={{ width: `${progress}%` }}
 />
 </div>

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 />

```

```

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

/** Числовые константы TreeWalker: глобального `NodeFile
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)|\

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
 const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l))
 // границы «не буква/цифра» – \b не работает с кириллицей
 return new RegExp(`(?<![\\p{L}\\p{N} _-])(${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, `term-${l.toLowerCase().replace(/ /g, '-')}`]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * , чтобы на них
 * повесить всплывающую карточку (как превью ссылок в Википедии)
 */
export function annotateTerms(root: HTMLElement): void {
 if (!cachedRegex) cachedRegex = buildRegex();
 const re = cachedRegex;

```

```

 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
 const run = () => {
 const el = ref.current;
 if (!el) return;
 try {
 annotateTerms(el);
 } catch {
 /* если браузер не умеет lookbehind – просто оставим как есть */
 }
 };
}

```

```
 scrollbar-width: none;
 }
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
 from {
 transform: translateX(-100%);
 }
 to {
 transform: translateX(0);
 }
}

@keyframes hintIn {
 from {
 opacity: 0;
 transform: translateY(4px);
 }
 to {
 opacity: 1;
 }
}
```

```
padding: 0 1px;
cursor: pointer;
transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
 background: rgba(99, 102, 241, 0.25);
 color: #fff;
 outline: none;
}

/* Контент главы приходит готовым HTML — чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}

.chapter-body :where(pre, code) {
 white-space: pre-wrap;
 word-break: break-word;
}

.chapter-body pre {
 overflow-x: auto;
 max-width: 100%;
 font-size: clamp(11px, 2.9vw, 13px);
}

/* Пользователь: «поля вокруг теории слишком большие».
 глав ужимаем на ступень, не трогая семантику разметки
.chapter-body [class~="p-6"] { padding: 1.25rem; }
.chapter-body [class~="sm:p-6"] { padding: 1.25rem; }
```



```
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализациях */
@keyframes demoProgress {
 from {
 width: 0%;
 }
 to {
 width: 100%;
 }
}

/* -----
Печать и «Сохранить в PDF» из браузера.
```

Печатаем ТОЛЬКО саму тему: текст страницы + её демонстрация. Меню и приложения (шапка, содержание, панель Python-компилятора, карточки терминов, кнопки управления) убирается, темный фон упрощается до чёрного по белому – как «лёгкая» версия. Интерактивная демонстрация печатается как есть (со с

```
article.chapter-body .prose pre,
article.chapter-body .prose code {
 background-color: #f3f4f6 !important;
 color: #111827 !important;
 font-size: 9pt;
}
```

```
article.chapter-body .prose table {
 border-collapse: collapse;
}
```

```
article.chapter-body .prose th,
article.chapter-body .prose td {
 border: 1px solid #9ca3af !important;
}
```

```
article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
 page-break-after: avoid;
}
```

```
article.chapter-body .prose details,
article.chapter-body .prose table,
article.chapter-body .prose pre {
 page-break-inside: avoid;
}
```

/\* демонстрация — на отдельной странице, со своими цв

```
#chapter-viz {
 page-break-before: always;
 print-color-adjust: exact;
 -webkit-print-color-adjust: exact;
}
```

```
#chapter-viz svg {
 max-height: 68vh;
}
```

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

---

## РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

---

---

ФАЙЛ 17/110: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 1197 | РАЗМ:

---

```
import { Chapter } from "../types";
```

```
export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
```

```

 <div class="grid grid-cols-1 sm:grid-cols-3"
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-rose-300 t
 <p class="text-[11px] text-slate-50
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-30
 <p class="text-[11px] text-slate-50
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-rose-300 t
 <p class="text-[11px] text-slate-50
 </div>
 </div>
 </div>

 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4
 <div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-80
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Сетка из вафельных
 </div>
 <p class="text-slate-300 text-sm mb
о накопилось от угла (0,0) до меня». Хочешь
у и верни дважды отрезанный угол. Вычитание
адывать, но и убирать обратно. Поэтому хран
 <div class="rounded-lg bg-slate-950
 <div><span class="text-slate-50
) </div>
 <div><span class="text-slate-50
/ div>
 </div>

```

```

<table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 border-b"
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate-200"
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 </tr>
 <tr class="border-b border-slate-200"
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 </tr>
 <tr class="border-b border-slate-200"
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3">только
 <td class="py-2 pr-3">min,
 </tr>
 <tr class="border-b border-slate-200"
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3">нельзя
 <td class="py-2 pr-3">нельзя
 </tr>
 <tr class="align-top"
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3">«сумма
 <td class="py-2 pr-3">«миним
 </tr>
 </tbody>
</table>
</div>

```

```

1 и
 Нужны обновления. Разрежьте пополам (билет 1), а не она.

</div>

<details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код: построение 1D (тот же, что вып
 </summary>
 <div class="p-5 text-sm text-slate-300" style="border: 1px solid #000; padding: 10px;">
 <pre class="bg-slate-950 p-4 rounded" style="font-family: monospace; font-size: 0.9em; margin: 0;">
 = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
st = []
for row in a:
 st.append([row] + [None] * (LOG - 1)) # уровень 0

for j in range(1, LOG): # уровень 1
 for i in range(n - (1 <<< j) + 1):
 st[i][j] = min(st[i][j - 1], st[i + (1 <<< j) - 1])
 <p class="text-slate-300 text-sm">3.
 min
 text-emerald-300">a, n, LOG, st, i, j
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код: построение и запрос в 2D (C++-
 </summary>
 <div class="p-5 text-sm text-slate-300" style="border: 1px solid #000; padding: 10px;">
 <pre class="bg-slate-950 p-4 rounded" style="font-family: monospace; font-size: 0.9em; margin: 0;">
 / построение: st[r][c][kx][ky] из четырёх к
for (int kx = 0; (1 <<< kx) <= N; ++kx)
 for (int ky = 0; (1 <<< ky) <= M; ++ky)

```

```

<ul class="list-disc pl-5 space-y-2 text-sm">
 <a class="ticket-link" data-goto="p
умм – вторая половина этого сравнения:
 <a class="ticket-link" data-goto="s
у запросами меняются элементы: $O(\log N)$ на
 <a class="ticket-link" data-goto="f
м, но там уровни означают «разрешённые пром

</div>

</div>
</section>`,
 }, {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приор
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 двоичная куча. Пара – точка на декартовой
 </div>

 <p class="text-slate-300 text-sm mb-4">Постр
 платит $\approx n \cdot \log n$ сравнений, counting по ма
 точка вставляется спуском от корня: $x \leq \text{уз}$
 единственно. Весь процесс – в симуляторе под
 «соединить сам» – кликаешь две точки,

```

```

 <p class="text-slate-400 text-sm mb-1">
быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «Уб
аются merge за $O(h)$.</p>
</div>

<p class="text-slate-300 text-sm mt-4"><b c
"bg-slate-950 px-2 py-0.5 rounded text-xs f
оение – <code class="bg-slate-950 px-2 py-0
code> (сортировка + n спусков). Худший случ
text-xs font-mono text-rose-300 border bord
ed text-xs font-mono text-rose-300 border b

<p class="text-slate-500 text-xs mt-3 leading
и останется набор букв, как treap без у оста
но кричат по «!»: корень самый громкий, род
; равные x – идут влево по уставу.</p>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: split (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}</pre>

```



```

 },
 {
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Самонастраивающееся дерево: поднимаем",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ротов поднимает x в корень. Единственный ин
 </div>
 </div>

 <div class="grid grid-cols-1 md:grid-cols-3">
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-white font-bold">мож
 <p class="text-slate-400 text-xs mt-1">
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-emerald-300 font-bold">
 <p class="text-slate-400 text-xs mt-1">
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-white font-bold">0(n
 <p class="text-slate-400 text-xs mt-1">
 </div>
 </div>
 </div>
`
 }
]
}

```

омать дерево поиска поворотами невозможно в момент «отстёжки», а врезка «Механика одного

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Смотрим
<div class="overflow-x-auto -mx-2 px-2">
 <table class="w-full text-xs sm:text-sm">
 <thead>
 <tr class="text-left text-slate-300">
 <th class="py-2 pr-3 font-bold">
 <th class="py-2 pr-3 font-bold">
 <th class="py-2 pr-3 font-bold">
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-300">
 <td class="py-3 pr-3 font-bold">
 <td class="py-3 pr-3">деда и
 <td class="py-3 pr-3">сам х
 <td class="py-3">х — корень
 </tr>
 <tr class="border-b border-slate-300">
 <td class="py-3 pr-3 font-bold">
 <td class="py-3 pr-3">х и п
 <td class="py-3 pr-3">
 <td class="py-3">ветка укоротил
 </tr>
 <tr>
 <td class="py-3 pr-3 font-bold">
 <td class="py-3 pr-3">х и п
 <td class="py-3 pr-3">
 <td class="py-3">п и г стали
 </tr>
 </tbody>
 </table>
</div>
<p class="text-slate-300 text-sm mt-3">И по
```

```
>O(n)</code>; n вставок подряд – амортизированно
-300 border border-slate-800">O(n log n)</code>
```

```
<p class="text-slate-300 text-sm mt-4">Все операции (включая splay последнего на пути); insert – BST, но левая мума левой половины не будет правого сына – а под «горячие» ключи, дешёвые split/merge. Это не меняется даже при чтении (плохо для многих)
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
```

Код: rotate + splay + find (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
```

```
узел: { key, left, right, parent }
```

```
def rotate_up(x): # x встаёт на место
 p = x.parent
 g = p.parent
 if p.left is x: # правый поворот
 p.left = x.right
 if x.right: x.right.parent = p
 x.right = p
 else: # левый (зеркально)
 p.right = x.left
 if x.left: x.left.parent = p
 x.left = p
 p.parent = x
 x.parent = g
 if g:
 if g.left is p: g.left = x
 else: g.right = x
```

```
def splay(x): # поднять x в корень
 while x.parent:
 p, g = x.parent, x.parent.parent
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
 <h3 class="text-xl font-bold text-emerald-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-emerald-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 <p class="text-sm font-mono text-slate-400">
```

```
 </div>
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-blue-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 <p class="text-sm font-mono text-slate-400">
```

```
 [cl]</p>
```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">3дес
```

```
угла
```

```
улевая строка и нулевой столбец – рамка из
```

```
lass="font-mono text-emerald-300">+1
```

```
<div class="bg-slate-800/70 p-5 rounded-lg">
```

```
 <p class="text-sm font-bold text-white">
```

```
 <table class="w-full text-xs text-left">
```

```
 <thead>
```

```
 <tr class="text-slate-500 border">
```

```
 <th class="py-1.5 pr-2 font">
```

```
 <th class="py-1.5 pr-2 font">
```

```
 <th class="py-1.5 pr-2 font">
```

```
 </tr>
```

```
 </thead>
```

```
 <tbody>
```

```
 <tr class="border-b border-slate">
```

```
 <td class="py-2 pr-3 font-b">
```

```
 <td class="py-2 pr-3 font-m">
```

```
 <td class="py-2 pr-3 font-m">
```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 о накопилось от угла (0,0) до меня». Хочешь
 у и верни дважды отрезанный угол. Вычитание
 адывать, но и убирать обратно. Поэтому храни
 <div class="rounded-lg bg-slate-950 p-6">
 <div>
)</div>
 <div>
 /div>
 </div>
 <p class="text-[12px] text-slate-400">
 Работает ТОЛЬКО для суммы: менять элементы
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: Торт-пирамидка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ия, зато красивый, как пирамидка: каждый из
 сех блоков $2^{k_x} \times 2^{k_y}$), и ответ на запрос
 ой операции нет: крем обратно в торт не запы
 $n(x, x) = x$), и цена $O(1)$ – сто коржей вмес
 <div class="rounded-lg bg-slate-950 p-6">
 <div>
 n $O(1)$ </div>
 <div>
 = 100 копий матрицы</div>
 </div>
 <p class="text-[12px] text-slate-400">
 шает). Менять элементы тоже нельзя. Сумму ч
 </div>
 </div>
 <p class="text-slate-400 text-sm mt-4">Поря
 нные
 углов запроса – переменные <span class="font
 </div>

```

```

 <td class="py-2 pr-3 font-mono">
 <td class="py-2 pr-3 font-mono">
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3 font-bolder">
 <td class="py-2 pr-3 font-mono">
 <td class="py-2 pr-3 font-mono">
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3 font-bolder">
 <td class="py-2 pr-3">сумма
 <td class="py-2 pr-3">min,
 </tr>
 <tr class="align-top">
 <td class="py-2 pr-3 font-bolder">
 <td class="py-2 pr-3">нет</td>
 <td class="py-2 pr-3">нет</td>
 </tr>
</tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
ого решения, а решения двух разных задач.</p>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-amber-400">
 <div class="bg-rose-950/30 p-5 rounded-lg border">
 <ul class="list-disc pl-5 space-y-2 text-slate-200">
 Смешали индексации. В формуле
 $S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]$
 т ответ со сдвигом на клетку.
 Переполнение. Суммы до 1018
 –
 Минимум через префиксные.
 Обновления между запросами.

). С обновлениями – дерево отрезков (билет
 Знак угла. Левый верхний

```

```

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]
print("итого:", ans)</code></pre>

```

<p class="text-slate-300 text-sm">П  
 вигают 2D-вкладку демонстрации: при построении  
 при запросе подсвечиваются четыре угла формируемого

</div>  
 </details>

<details class="bg-slate-900/40 rounded-lg border-1  
 <summary class="p-4 font-bold text-slate-300  
 -b border-slate-700/50">

Почему формула верна (доказательство)  
 </summary>

<div class="p-5 text-sm text-slate-300">  
 <p class="text-slate-300 text-sm">Получим сумму значений  
 от (0,0) до (i-1, j-1). Тогда прямоугольник с вершинами  
 class="font-mono text-emerald-300">A[i-1][j-1] - A[0][0]  
 -1, j-1)</span>, полоса слева <span class="font-mono text-emerald-300">A[0][j-1] - A[0][0]  
 ая, получаем <span class="font-mono text-emerald-300">S[i][j] - S[0][j] - S[i][0] + S[0][0]  
 <span class="font-mono text-emerald-300">S[i][j] - S[i][0] - S[0][j] + S[0][0]  
 <p class="text-slate-300 text-sm">Значит, формула верна.  
 ычитенный угол.</p>

</div>  
 </details>

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-500">  
 <h3 class="text-xl font-bold text-indigo-400">Асимптотика  
 <ul class="list-disc pl-5 space-y-2 text-sm">  
 <li><a class="ticket-link" data-goto="section">Асимптотика  
 – та же скорость запроса, но в десятки раз меньше  
 <li><a class="ticket-link" data-goto="section">Асимптотика  
 нт матрицы меняется между запросами:  $O(\log N)$   
 <li><a class="ticket-link" data-goto="section">Асимптотика  
 клеткой по соседям; отличается тем, что та же матрица  
 </ul>

```

</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
 еская сортировка – это расписание, которое
 >
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">

```



```

 </div>
 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-rose-400 mb-2">
 <div class="bg-slate-900 p-4 rounded-lg mb-2">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-2">
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрывая
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Если сходится нечетное число линий, значит, из
 каждой вершины должно быть четное число (зашел-вышел)
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",
 description: "",
 category: "Графы. Пути",
 content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-10">

```

```

</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb"
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r"
 <h2 class="text-2xl sm:text-3xl font-bold text-v"
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

 <p class="text-xl font-mono text-white">
ается с соседом.</p>
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb
изкому племени для объединения. К вечеру пл
ства шлют гонцов к ближайшему чужому царств
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
i], который одновременно является её суффикс
 </div>

```

```

 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
щегося с i.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Копипаста
 </div>
 <p class="text-slate-300 text-sm mb">
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной коп
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate">
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded

int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}

```

```

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #800080; border-radius: 10px; box-shadow: 5px 5px 0px #800080;">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-6">
 Это отличный переводчик на английский (Сведения о переводе: В, то В – это NP-Полная (сосредоточена на переводе))
 </p>
 </div>
 </div>
</section>`,
 },
];

```

ФАЙЛ 18/110: src/data/content\_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 484 | РАЗМЕР: 10.5 КБ

```

import type { Chapter } from "../types";

export const chapters: Chapter[] = [
 {
 id: "euler-path-vs-cycle",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Эйлеров путь и эйлеров цикл: признаки и свойства",
 category: "Графы",
 content: `
<section id="euler-path-vs-cycle" class="euler-path-vs-cycle">
<div class="flex items-center mb-6">
 13. Графы. Эйлеров цикл
 <h2 class="text-3xl font-bold text-white">Эйлеров цикл</h2>
</div>

```

</div>

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md bg-slate-950">
```

Ловушка: цикл – это вернулся домой.

</div>

```
<p class="text-slate-300 text-sm mb-4">
```

**Гамильтон – это болезнь.**

ng> ровно раз и вернуться домой.

Ты обходишь все бары (вершины) ровно

Главное – зайти и выйти, не заходя

Никто не знает, как решать быстро.

```

плоту).
```

</p>

</div>

</div>

```
<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md bg-slate-950">
 <summary class="p-4 font-bold text-slate-400">
 Душный режим: Формальное доказательство
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p class="text-blue-400">Теорема Эйлера
```

Душный режим: Формальное доказательство

</summary>

```
<div class="p-5 text-sm text-slate-300 space-y-4">
```

<p class="text-blue-400">Теорема Эйлера

<p>

Связный граф содержит эйлеров цикл

<br>Эйлеров путь (не цикл)  $\Leftrightarrow$  ровно 2

</p>

```
<p class="text-rose-400">Почему Гамильтон
```

<p>

Задача коммивояжёра (TSP) – это взв

Доказано: полиномиальный алгоритм д

В общем, не парься. Просто запомни

</p>

</div>

</details>

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-emerald-500 shadow-md">
 Мост: Единственная веревка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спустился в пещеру.
 Из пещеры и нет других выходов — только одна веревка.
 Если верёвка (ребро v-u) оборвётся, ты погибнешь.
 Если бы из пещеры и был чёрный ход, ты бы его нашел.
 </p>
</div>

<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 Аналогия: Кровеносный сосуд
</div>
<p class="text-slate-300 text-sm mb-4">
 Граф — это кровеносная система. Ребра (сосуды) — это не мост, живём.
 Если же перерезать единственную артерию, ты погибнешь.
 Алгоритм DFS ищет такие «критические» ребра.
</p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
 <summary class="p-4 font-bold text-slate-400">
 Полный код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Классическая реализация — <code>class Graph {</code>
 оходов.</p>
 </div>
 <div class="bg-slate-950 p-4 rounded-lg">
```



```

tin.assign(n, -1);
low.assign(n, -1);

for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
}
}
}

</div>
<p class="text-xs text-slate-400">Визуализация происходит в момент проверки low[u] > tin[v]. Это позволяет самому — шаги кода двинут демонстрацию.</p>
</div>
</details>
</div>
</section>`,
},
{
 id: "planarity-euler-formula",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Планарные графы: формула Эйлера, K5 и K4, 4 краски.",
 category: "Графы. Теория",
 content: `<section id="planarity-euler-formula" class="text-slate-400">
<div class="flex items-center mb-6">
 7. Графы. Планарные. Покраска
 <h2 class="text-3xl font-bold text-white">Планарные графы
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">Формула Эйлера
 <p>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Формула Эйлера:
 <p class="text-3xl font-mono text-white">
 <div class="text-sm text-slate-400 space-x-2">
 <p><span class="text-blue-400 font-

```

```
 Почему? Потому что у двудольного графа $2E \geq 3F$.
 Отсюда более строгое ограничение: $2E \geq 3F \Rightarrow 20 \geq 21$.
 <code class="text-white">9 > 8</code>
 </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-400">
 Доказательство непланарности K_5 (Скрыть)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p class="text-blue-400">Доказательство непланарности K_5
 <p>
 Пусть K_5 планарен. $V = 5$, $E = 10$.

 $F = E - V + 2 = 10 - 5 + 2 = 7$.

Каждая грань ограничена минимумом 3 рёбрами.

Сумма длин граней $= 2E = 20$.

Отсюда: $2E \geq 3F \Rightarrow 20 \geq 21$ — противоречие.

Значит, K_5 не может быть планарным.
 </p>
 </div>
</details>

<div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600/50">
 <h3 class="text-xl font-bold text-fuchsia-400">Задача 1.3
 </h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-fuchsia-300 italic">Докажите, что граф K_5 непланарен.
 <p class="text-xl font-mono text-white">Решение:
 </div>
 <p> — минимум цветов. Жадная раскраска даёт хроматический индекс $\chi(K_5) = 4$.
</div>

<p class="text-slate-300 text-sm mb-4">Планарные графы (которые не
отличались цветом) — это раскраска вершин графа K_5 в 4 цвета.
итая теорема о четырёх красках (Аппеля-Хансона).
доказательство, где решающий перебор сделан с помощью компьютера.
```

```

 <tr class="border-b border-slate-200">
<td class="py-2 pr-3 font-mono"> $\chi = 2 \Rightarrow$ нет
) – единственная «лёгкая» раскраска</td></tr>
 <tr class="border-b border-slate-200">
td><td class="py-2 pr-3 font-mono"> $\chi \leq 4$ (и
мна, 4 – только перебором</td></tr>
 <tr class="border-b border-slate-200">
td><td class="py-2 pr-3 font-mono"> $\Delta \leq \chi' \leq$
 <tr class="align-top"><td class="font-mono">
o"> $\chi \leq 2$ – P; $\chi \leq 3$ – NP-полно</td><td class="font-mono">
 </tbody>
 </table>
</div>

```

```

<p class="text-slate-300 text-sm mb-4">Пример 1.1.1.
и суммы степеней $2E = \sum \deg(v)$ следует, что
, то есть $E \geq 3V$ – противоречие). Убираем е
й цвет точно свободен. Чтобы спуститься до
а две вершины с цветами 1 и 3 и проверяем,
обожаем цвет.</p>

```

```

<div class="bg-slate-900/40 rounded-lg border-1 border-slate-700">
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
фа красятся в 4 цвета (это dual к вершинам)
 <p class="text-slate-400 text-sm mb-1">
2 (дерево двудольно).</p>
 <p class="text-slate-400 text-sm mb-1">
е решается 5 цветами, но 4 цветами – уже не
).</p>
 <p class="text-slate-400 text-sm">• Пример 1.1.2.
проходит, но непланарен. Достаточный критер
разделением K_5 или $K_{3,3}$. Эквивалентная форм
</div>

```

```

<p class="text-slate-300 text-sm mt-4">Пример 1.1.3.
950 px-2 py-0.5 rounded text-xs font-mono text-slate-400
e class="bg-slate-950 px-2 py-0.5 rounded text-slate-400"

```

```

 dq.append(to)
 elif col[to] == col[v]: # одноцветные
 bipartite = False</pre>
 <p class="text-slate-400 text-xs">П
ике даёт заметно меньше цветов, чем случайн
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 category: "Графы. Продвинутые",
 content: `<section id="graph-articulation" class="ml
<div class="flex items-center mb-6">
 <span class="bg-rose-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Точки
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-rose-300 italic
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>
</div>
`
 }
]

```

```

<div class="p-5 text-sm text-slate-300 space-x-4">
 <p class="text-blue-400 font-bold">Особый случай (если нет родителей, то выше неё прыгать некуда). Поэтому для неё у него больше 1 независимого ребенка.
 </p>
 <div class="bg-slate-950 p-4 rounded-lg">
 <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
 if (visited[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}</pre>
 </div>
 </div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border">
 <h4 class="text-lg font-bold text-indigo-400">
 <p class="text-slate-300 text-sm mb-4">
 Это глубокий дуальный мост между ориентацией
 </p>

```

```

* поздние переписанные страницы (Эйлер, Мосты, Планарн
*
* Страниц в содержании меньше, чем тем: одна страница
* несколько билетов сразу (18–20 «Основное дерево: Кра
* три алгоритма на одном графе с тремя вкладками в виз
*/
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24,
 ...ticket1Segtree,
 ...tickets18to20Mst,
].filter((c) => c && c.id);

/**
* Устаревшие/заменённые главы:
* - graph-planar-colors, graph-bridges, graph-euler,
* pathfinding-accel – короткие версии, заменены раз
* (graph-articulation НЕ удаляется: его короткая вер
* развёрнутой из content_temp – так работает dedup
* - mst-kruskal, mst-prima, mst-boruvka – объединены
* - intro, alg-map – заглушки без визуализации (полезн
* карте» переехало в билет 17 «Ускорения»: сжатие к
*/
const toRemove = [
 "graph-planar-colors",
 "graph-bridges",
 "graph-euler",
 "pathfinding-accel",
 "mst-kruskal",
 "mst-prima",
 "mst-boruvka",
 "alg-map",
];

const dedup = new Map<string, Chapter>();

```

```

 "graph-components": [8],
 "top-sort": [9],
 "scc-kosaraju": [10],
 "bridges-code": [11],
 "graph-articulation": [12],
 "euler-path-vs-cycle": [13],
 dijkstra: [14],
 "bellman-ford": [15],
 floyd: [16],
 "johnson-algo": [17],
 mst: [18, 19, 20],
 "string-kmp": [21],
 "string-z-func": [22],
 "aho-corasick": [23],
 "complexity-classes": [24],
};

/**
 * Разделы содержания: связанные темы стоят рядом.
 *
 * Номера билетов при этом не меняются – порядок страниц
 * Раньше сайдбар группировал страницы по полю `category`
 * встречи, из-за чего графы расползались по семи группам
 * отдельно, 9 и 13 отдельно, 10–11 отдельно, а 12 «Точки»
 * уезжали в «Прочие темы» – подальше от 11 «Мосты», с которыми
 * Теперь раздел один на семью тем, порядок разделов фиксирован
 */
export const SECTIONS: { id: string; title: string; from: number; to: number }[] = [
 { id: "intro", title: "Введение", from: 0, to: 0 },
 { id: "arrays", title: "Массивы и деревья", from: 1, to: 1 },
 { id: "graphs", title: "Графы: обходы и структура", from: 8, to: 13 },
 { id: "paths", title: "Графы: пути и остов", from: 14, to: 14 },
 { id: "strings", title: "Строки", from: 21, to: 23 },
 { id: "theory", title: "Сложность и сведения", from: 24, to: 24 }
];

/** Раздел страницы – по её первому билету (у `mst` их несколько)
export function sectionOf(chapterId: string) {

```

```

 * JSON-снимок доступных имён (globals + locals). В о
 * можно напрямую отдать визуализации: i/j остаются ч
 */
 values?: Record<string, DebugValue>;
}

export interface DebugResult {
 steps: DebugStep[];
 /** Локали в момент возврата из модуля – «результат»
 finalLocals?: Record<string, string>;
 /** Финальный машинно-читаемый снимок для визуализации
 finalValues?: Record<string, DebugValue>;
 /** Текст последней необработанной ошибки (или пустая
 error: string;
 /** true, если уперлись в лимит шагов. */
 truncated: boolean;
}

/**
 * Лимит шагов трассы – защита от бесконечных циклов и
 *
 * Поднят с 500 до 1500: эталонный код Флойда–Уоршелла
 * делает 869 шагов, «Классы сложности» – 585, Ахо–Кора
 * лимите трасса обрывалась на середине алгоритма, и ви
 * итоговую матрицу. Замер: 869 шагов Флойда выполняются
 */
export const DEBUG_STEP_CAP = 1500;

/**
 * Строит самодостаточный python-скрипт: выполняет userSrc
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
export function buildDebugRunner(userSrc: string): string {
 const srcLiteral = JSON.stringify(userSrc); // JSON-с
 return `import sys as __sys, json as __json

 src = ${srcLiteral}

```



```
def __visible(items):
 return {
 k: v for k, v in items
 if not k.startswith("__") and type(v).__name__
 }

def __capture(frame):
 # Фрейм модуля: f_locals – это тот же словарь, что и
 # на каждом событии строки НЕЛЬЗЯ: в CPython 3.12 в
 # списковые включения шлют событие на своей строке
 # обращение к f_locals между итерациями обнуляет пер
 # код вида E = [(a, b) for a in V for b in V if a[0
 # TypeError: 'NoneType' object is not subscriptable
 # RuntimeWarning: assigning None to unbound local.
 if frame.f_code.co_name == "<module>":
 scope_values = __visible(frame.f_globals.items())
 return {
 {k: __safe(v) for k, v in scope_values.items()}
 {k: __snapshot(v) for k, v in scope_values.items()}
 }
 local_values = __visible(frame.f_locals.items())
 # Внутри функции алгоритма важные структуры (memo,
 # глобальные. Локальные значения имеют приоритет на
 scope_values = __visible(frame.f_globals.items())
 scope_values.update(local_values)
 loc = {k: __safe(v) for k, v in local_values.items()}
 values = {k: __snapshot(v) for k, v in scope_values.items()}
 return loc, values

def __tracer(frame, event, arg):
 global __was_truncated
 # f_lineno == 0 – это RESUME модуля, а не строка ис
 # событие приходит даже для файла, где одни коммента
 # код давал «шаг 1/1», и панель прятала редактор за
 if event == "line" and frame.f_code.co_filename ==
 and not (frame.f_code.co_name.startswith("<
 loc, values = __capture(frame)
```

```

 OUT
 \;
}

```

```
/**
```

```
 * Какие переменные изменились на этом шаге относительно
```

```
 * они подсвечиваются (как amber-подсветка изменений в
```

```
 * Новые переменные тоже считаются изменёнными.
```

```
 */
```

```

export function changedVars(
 prev: Record<string, string> | null | undefined,
 cur: Record<string, string>
): Record<string, boolean> {
 const out: Record<string, boolean> = {};
 for (const k of Object.keys(cur)) {
 out[k] = !prev || !(k in prev) || prev[k] !== cur[k];
 }
 return out;
}

```

```
/** Базовое имя из карточки переменной: "tin[v]" → "tin"
```

```
export const baseVarName = (name: string): string => name
```

```
/**
```

```
 * Порядок показа в панели переменных: сначала переменные
```

```
 * страницы (те, что подсвечивает визуализация), остальные
```

```
 */
```

```

export function orderVars(
 cur: Record<string, string>,
 syncNames: string[]
): [string, string][] {
 const inSync = new Set(syncNames);
 const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
 const rest = Object.entries(cur)
 .filter(([k]) => !inSync.has(k))
 .sort(([a], [b]) => a.localeCompare(b));
 return [...syncEntries, ...rest];
}

```

```

 "Волна от источника: сначала все соседи, потом со
formal:
 "Очередь (FIFO). Кладём старт, достаём вершину —
 у рёбер.",
complexity: "O(V + E)",
demo: "bfs",
simulator: "queue-bfs",
},
{
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину"],
 title: "DFS — обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся — откатываешься на
formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
complexity: "O(V + E)",
demo: "dfs",
simulator: "stack-dfs",
},
{
 id: "binsect",
 labels: ["binsect", "бинсект", "бинарный поиск", "д
 title: "Бинарный поиск (и его самозванец binsect)",
 short:
 "Бинарный поиск НАХОДИТ готовый элемент в уже отс
 о не сортирует. «binsect» из конспекта — шутлив
 quicksort.",
formal:
 "Поиск одного элемента в отсортированном массиве —
 O(n log n). Бинарный поиск сортировку не выполн
complexity: "поиск: O(log n); сортировкой не являетс
},
{
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "к
 title: "Куча (priority queue)",
 short:

```

```

 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "т
 title: "Бор (trie, префиксное дерево)",
 short:
 "Дерево, где путь от корня по буквам и есть слово
 formal: "Из корня по символам; в вершине хранится ф.
 complexity: "поиск слова — $O(|\text{слова}|)$ ",
 demo: "trie",
 simulator: "aho-corasick",
},
{
 id: "bfs-on-trie",
 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то об
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффиксн
 о есть строго по уровням.",
 formal:
 "Кладём в очередь детей корня (их ссылка = корень
 кладём с в очередь.",
 complexity: "O(суммарной длины словаря × алфавит)",
 demo: "trie-bfs",
 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки",
 title: "Суффиксная ссылка",

```

```

 complexity: "препроцессинг $O(n)$, запрос $O(1)$ ",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "sparse-table",
 labels: [
 "разреженная таблица", "разреженная таблица", "sp",
 "разреженная", "двоичные подьёмы", "двоичный подь",
],
 title: "Разреженная таблица (метод двоичных подьёмов)",
 short:
 "Заранее считаем ответ для всех отрезков длины 1,
 м.",
 formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
 complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ ",
 demo: "sparse-table",
 simulator: "sparse-table",
},
{
 id: "segment-tree",
 labels: ["дерево отрезков", "дереве отрезков", "дер"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь ма
 товых кусков.",
 formal: "Строится за $O(n)$, запрос и обновление — $O(\log n)$ ",
 demo: "segment-tree",
 simulator: "segment-trees",
},
{
 id: "bridge",
 labels: ["мост", "моста", "мосты", "мостов", "мостов"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается",
 formal: "Ребро (u,v) — мост $\iff \text{low}[v] > \text{tin}[u]$, где
 ребро в родителя — нет).",

```

```

 ассив.",
 demo: "amortized",
 },
 {
 id: "np",
 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полнота",
 "класс P", "полином", "полиномиальное",
],
 title: "Класс NP и NP-полнота",
 short:
 "NP — «решение проверить легко, найти трудно» (суть NP-полноты в NP.",
 formal: "Задача NP-полна, если она в NP и к ней полнота сводится",
 demo: "np",
 },
 {
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов", "потенциальность"],
 title: "Потенциалы (перевзвешивание Джонсона)",
 short:
 "Трюк «поднять все дороги на одинаковую высоту»: для любых $u, v \in V$ и $e \in E$ выполняется $w'(u, v) = w(u, v) + h(u) - h(v)$, где h — потенциал, а w' — вес ребра e в новом графе. При этом кратчайший путь сохраняется.",
 formal: " $w'(u, v) = w(u, v) + h(u) - h(v)$, где h — потенциал, а w' — вес ребра e в новом графе. При этом кратчайший путь сохраняется.",
 demo: "relax",
 simulator: "johnson-algo",
 },
 {
 id: "scc",
 labels: ["компонента сильной связности", "компоненты сильной связности"],
 title: "Компоненты сильной связности",
 short:
 "Группы вершин, где из каждой можно дойти до каждой другой вершины в той же группе.",
 formal: "Косарайю: DFS с сохранением порядка выхода из вершин.",
 complexity: " $O(V + E)$ ",
 demo: "scc",
 simulator: "scc-kosaraju",
 },

```

```

 short:
 "Решаем задачу через ответы на её меньшие копии и
formal:
 "Нужны: оптимальная подструктура + перекрывающиеся
 ция).",
demo: "dp",
simulator: "dynamic-programming",
},
{
 id: "shortest-path",
 labels: ["кратчайший путь", "кратчайшие пути", "кра
title: "Кратчайший путь",
short:
 "Самый дешёвый маршрут из A в B. «Дешёвый» – по с
formal:
 "Дейкстра – неотрицательные веса, $O(E \log V)$. Фор
 ы, $O(V^3)$.",
demo: "relax",
simulator: "dijkstra",
},
{
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикл
title: "Отрицательный цикл",
short:
 "Круг, пройдя по которому вы становитесь «богаче»
formal:
 "Форд–Беллман: если на V -й итерации ещё удаётся с
demo: "relax",
simulator: "bellman-ford",
},
{
 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешёвое ребро", "самое дешёвое ребро", "са
],
title: "Жадный алгоритм",

```

```

 "Три случая: zig (родитель – корень), zig-zig (уз
 рацию.",
 demo: "amortized",
 simulator: "splay-tree",
},
{
 id: "adjacency",
 labels: ["матрица смежности", "список смежности", "
 title: "Способы хранить граф",
 short:
 "Матрица смежности – таблица «есть ли ребро», быс
 й, экономно для разреженных графов.",
 formal: "Матрица: проверка $O(1)$, память $O(V^2)$. Спис
},
{
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший
 льные.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали
 complexity: " $O(E \log V)$ с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "bellman-ford",
 labels: [
 "Форд–Беллман", "Форда–Беллмана", "Форду–Беллману
 "Форд-Беллман", "Форда-Беллмана", "Беллман-Форд",
],
 title: "Алгоритм Форда–Беллмана",
 short:
 "Тупой, но честный: $V-1$ раз проходим по ВСЕМ рёбра
 ",
 formal:

```



```

 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
 title: "Алгоритм Борувки",
 short:
 "Все компоненты одновременно шлют «гонца» к ближайшей вершине",
 formal:
 "Фаза: для каждой компоненты ищем минимальное исходящее ребро",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst",
},
{
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
 title: "Алгоритм Краскала",
 short:
 "Сортируем все рёбра по весу и берём подряд те, которые не образуют цикла",
 formal:
 "Сортировка O(E log E) + E операций union/find. Сложность O(E log E)",
 complexity: "O(E log E)",
 demo: "mst",
 simulator: "mst",
},
{
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "компонент",
 "компоненте связности", "связности", "связный", "связность",
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мостов",
 formal:
 "Считаются одним проходом: пока есть непосещённая вершина",
 complexity: "O(V + E)",
 demo: "components",
 simulator: "graph-dfs-bfs",
},

```

```

 demo: "heap",
 },
 {
 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска",
 title: "Бинарное дерево поиска",
 short:
 "Слева всё меньше корня, справа — всё больше. Поиск",
 formal:
 "Без балансировки дерево может вырождаться в список",
 complexity: "O(высоты): от $O(\log n)$ до $O(n)$ ",
 demo: "segment-tree",
 },
 {
 id: "inclusion-exclusion",
 labels: [
 "включений-исключений", "включения-исключения", "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём больший и меньший, а затем вычитаем. Если один из отрезков вырезали дважды.",
 formal:
 " $\text{Sum}(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]$ ",
 complexity: "запрос $O(1)$ после $O(nm)$ предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "reduction",
 labels: ["сведение", "сведения", "сведениями", "сведённый"],
 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно свести к B, то B можно решить, решив A.",
 formal:
 "A $\leq_p$ B: есть полиномиальная функция, переводящая экземпляры A в экземпляры B, такая, что A разрешима тогда и только тогда, когда B разрешима.",
 }
}

```

```

 simulator: "splay-tree",
},
{
 id: "zig",
 labels: ["Zig", "Zag", "зиг"],
 title: "Zig – одиночный поворот",
 short:
 "Самый простой случай: родитель x уже сидит в корне,
 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию",
 demo: "zig",
 simulator: "splay-tree",
},
{
 id: "zig-zig",
 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig – x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: g → p → x идут в одну сторону,
 м (p, x). На длинном бамбуке это реально расплюснуть",
 formal:
 "Если крутить наивно (два раза поднимать x), бамбук превращается в
 звезду",
 demo: "zig-zig",
 simulator: "splay-tree",
},
{
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p – левый сын g, а x – правый сын
 p и g становятся двумя детьми x.",
 formal: "Эквивалентно двойному повороту AVL-дерева",
 demo: "zig-zag",
 simulator: "splay-tree",
},
];

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-indigo-400 mb
 <p class="text-slate-300 text-sm mb-4">Дерево н
 единицы, то родственников выдаёт простая ари
 чейках <span class="font-mono text-emerald-300
 n /.
 па.</p>
 <pre class="bg-slate-950 border border-slate-700
tree = [None, 40, 20, 60, 10, 30, 50, 70] # tree[0] н
корень n=1
дети 40: 2*1=2 → 20 и 2*1+1=3 → 60
дети 20: 2*2=4 → 10 и 2*2+1=5 → 30
родитель 30: 5 // 2 = 2 → 20</pre>
 <p class="text-slate-300 text-sm mb-4">Цена: пу
 > деревья – кучу (ниже) и дерево отрезков из
 emerald-300">2n+1 и <span class="font-m
 ">(n-1)//2.</p>
 <details class="bg-slate-900 border border-slate
 <summary class="cursor-pointer px-4 py-2 text
 <div class="px-4 pb-4 text-slate-300 text-sm
 На уровне <span class="font-mono text-emera
 /span> вершин, а перед ним суммарно <span c
 class="font-mono text-emerald-300">k
 lass="font-mono text-emerald-300">n
 0">2n и <span class="font-mono text-l
 де, включая heapq.
 </div>
 </details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-emerald-400 mb
 <p class="text-slate-300 text-sm mb-4">Куча
 ель ≤ своих детей. Отсюда магия: минимум всег
 минимума стоят <span class="font-mono text-e
 елей/детей. Полностью сортировать кучу не нуж
 <pre class="bg-slate-950 border border-slate-700
import heapq

```

```

o text-emerald-300">O(n log n) в худшем
анных данных. Внутри он ищет уже упорядоченные
вставками на короткихранах. Сортировка ус
<pre class="bg-slate-950 border border-slate-700
sorted([3, 1, 2]) # [1, 2, 3] – новый с
sorted([(1, "6"), (1, "a")], key=lambda t: t[0]) # ра
<p class="text-slate-300 text-sm">Что лежит вну
отвечают: слияние двух ранов длины k стоит O(
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-l
<h3 class="text-xl font-bold text-purple-400 mb
<p class="text-slate-300 text-sm mb-4">AVL-д
>каждой вершины выполнено условие баланса
class="font-mono text-emerald-300">|h_L – h_R
зывают фактором баланса; из условия следует в
авка и удаление за O(log n) даже в худшем слу
<pre class="bg-slate-950 border border-slate-700
Малый поворот (цепочка вправо-вправо стрелочками):

```

```

z x
└> x => ↙ ↘
└> y z y

```

Зигзаг (вправо-влево): сначала малый поворот нижнего из  
потом малый поворот верхнего – две перерисовки стрелок:

```

z z x
└> x => └> x => ↙ ↘
↙ y └> y z y</pre>

```

```

<p class="text-slate-300 text-sm mb-4">Поворот
родителя y среднего звена и перевесить одно
отов вдоль пути вставки, и условие <span clas
<details class="bg-slate-900 border border-slat
<summary class="cursor-pointer px-4 py-2 text
<div class="px-4 pb-4 text-slate-300 text-sm
x = right(z)

right(z) = left(x) # внутреннее поддерев

```

```

 border-emerald-500 shadow-md">
 Аналогия 2: Турнирная сетка
 </div>
 <p class="text-slate-300 text-sm">Куча
 ь, не нужно переигрывать все матчи. Снял по
 тавшихся.</p>
 </div>
 </div>
 </div>

 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-indigo-400 mb
 <p class="text-slate-300 text-sm leading-relaxe
 вут в билетах 14–16 (Дейкстра, Форд-Беллман,
 роки – в 21–23. Начинать удобно с билета 6: т
 </div>

 </div>
</section>`
 },
 {
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",
 content: `<section id="dijkstra-content" class="mb-
 <div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Дейкс
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <p class="text-xl font-mono text-white">

```

```

 </div>
</section>`
 },
 {
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class=
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Форд-Б
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400">Аналогия 1: Параноик

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Параноик
 <p class="text-xl font-mono text-white">Параноик
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 <div class="border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-4">Параноик
 берет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l

```

### Аналогия 1: Автомагистрали

```
</div>
<p class="text-slate-300 text-sm mb-6">
 евле долететь из Москвы в Париж, если мы сд
 <div id="slot-chapter-image-floyd" class="w-100 h-100">
</div>

<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb-6">
 три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
</div>

<div id="slot-floyd-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12">
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-blue-400 mb-4">

```



```

<div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>
 ссылку <code>term_link</code> (ближайшая мен
 <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
 map<char, int> go; // Предвычисленные пер
 int link = 0; // Суффиксная ссылка: куда
 int term_link = 0; // Терминальная ссылка: мат
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход
 nodes[child].link = nodes[nodes[v].link].go

 // Если суффиксная ссылка сама является сло
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].l
 } else {
 nodes[child].term_link = nodes[nodes[ch
 }

 q.push(child);

```

```
export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "sparse-table": [
 {
 question: "Сколько памяти занимает двумерная разреженная матрица?",
 options: [
 "N × M – столько же, сколько сама матрица",
 "N × M × log N × log M: на каждом уровне почти M копий",
 "N × M × 4 – по одной ячейке на каждый угол каждого блока",
],
 correctIndex: 1,
 explanation: "Уровней log N по строкам и log M по столбцам. Для матрицы 1000 × 1000 уровней по строкам и по столбцам о примерно 10 × 10 = 100 копий – ровно такова цена хранения разреженной матрицы.",
 },
 {
 question: "Можно ли считать сумму на прямоугольнике?",
 options: [
 "Да: минимум и сумма считаются одним приёмом",
 "Нет: блоки перекрываются, а вычесть перекрытие",
 "Да, если заранее хранить суммы в нулевом уровне",
],
 correctIndex: 1,
 explanation: "Два (в 2D – четыре) блока накрываются. Если вычесть перекрытия, обратная операция, но применить её к перекрытиям. Ответ: нет, билет 5.",
 },
 {
 question: "Какие операции подходят разреженной таблице?",
 options: [
 "Любые ассоциативные",
 "Идемпотентные: min, max, gcd, побитовые И и ИЛИ",
 "Только сумма и произведение"
]
 }
]
```

```

 вариант без +1 даёт ответ со сдвигом на клетку —
 },
 {
 question: "Почему левый верхний угол в формуле пр
 options: [
 "Он входит и в левую, и в верхнюю полосу — его
 "Он всегда равен нулю, поэтому прибавление безо
 "Так компенсируется рамка из нулей"
],
 correctIndex: 0,
 explanation: "Отрезая полосу слева и полосу сверх
 аланс: это и есть формула включений-исключений.
 },
 {
 question: "Можно ли тем же приёмом считать миниму
 options: [
 "Да, если заменить вычитание на min",
 "Нет: у минимума нет обратной операции, нужна р
 "Да, но только для неотрицательных чисел"
],
 correctIndex: 1,
 explanation: "Весь приём держится на вычитании. З
 ной операции нет. Для min, max и gcd берут разр
 },
 {
 question: "Изменили один элемент матрицы. Что ста
 options: [
 "Поправится одна клетка за $O(1)$ ",
 "Нужен пересчёт $O(N \cdot M)$: от этой клетки зависит
 "Ничего не станет: таблица остаётся верной"
],
 correctIndex: 1,
 explanation: "Каждая клетка $S[i][j]$ включает в се
 я частые, это задача дерева отрезков (билет 1).
 }
},
"euler-path-vs-cycle": [
 {

```

```

 "Нет, low[u] должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: low[u] > tin[v]. В н
},
{
 question: "Для чего нужен массив low[v]?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого",
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: "low[v] хранит минимальное время входа
 поиска мостов и точек сочленения."
},
{
 question: "Какова временная сложность алгоритма п
 options: [
 "O(V^2) — ищем мосты в квадрате",
 "O(V + E) — каждый элемент один раз",
 "O(2^V) — полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
 }
},
"planarity-euler-formula": [
 {
 question: "Планарный граф имеет V=6, E=12. Возможен ли такой граф?",
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство E ≤ 3V - 6",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: "E ≤ 3V - 6 ⇒ 12 ≤ 12 — на грани пр
 }
]

```

```

* выполненного Python-кода. Визуализация читает захард
* (i, j, k, v, dist, P, st...) и подсвечивает соответств
* любой пользовательский код работает, если в нём испо
*/
```

```
export const VizChapterContext = createContext("");
```

```
const CHANNEL = "algo:viz-state";
```

```
export interface VizStateEventDetail {
 chapterId: string;
 /** Строка и функция реально выполненного пользовател
 line?: number;
 func?: string;
 /** Машинно-читаемые globals + locals: числа остаются
 variables?: Record<string, DebugValue>;
 /** Имена, изменённые выделенной строкой. */
 changed?: string[];
}
```

```
/** Последний снимок хранится, чтобы поздно смонтирован
const latestByChapter = new Map<string, VizStateEventDe
```

```
/** Транслировать реальные переменные визуализациям тек
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
 const detail: VizStateEventDetail = { chapterId, ...s
 latestByChapter.set(chapterId, detail);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}
```

```
/** Вернуть демонстрацию в ручной режим при закрытии/ре
export function clearVizState(chapterId: string) {
 if (typeof window === "undefined") return;
 latestByChapter.delete(chapterId);
```

```

const chapterId = useContext(VizChapterContext);

useEffect(() => {
 if (!chapterId) return;

 const apply = (detail?: VizStateEventDetail) => {
 if (!detail || detail.chapterId !== chapterId) re
 // Без явного селектора компонент либо рисует use
 // либо остаётся ручным. Номер Python-строки нико
 const requested =
 selectByVariables && detail.variables ? selectB
 // null означает: компонент рисует runtime-перемен
 // не увидел знакомых захардкоженных имён.
 if (requested === null || !Number.isFinite(requested))
 const target = Math.max(0, Math.min(Math.max(0, requested), maxStep));
 if (target !== currentStep) goToStep(target);
 };

 apply(latestByChapter.get(chapterId));
 const handler = (e: Event) => apply((e as CustomEvent).detail);
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, handler);
}, [chapterId, currentStep, maxStep, goToStep, selectByVariables]);

/** Утилиты без небезопасных cast-ов для визуализаторов */
export const vizNumber = (value: DebugValue | undefined): number | null {
 typeof value === "number" && Number.isFinite(value) ? value : null;
}

export const vizString = (value: DebugValue | undefined): string | null {
 typeof value === "string" ? value : null;
}

export const vizArray = (value: DebugValue | undefined): Array<any> | null {
 Array.isArray(value) ? value : null;
}

export const vizRecord = (value: DebugValue | undefined): Record<string, any> | null {
 value !== null && typeof value === "object" && !Array.isArray(value)
 ? (value as Record<string, DebugValue>) : null;
}

```

```

 return Number.isFinite(n) ? n : null;
 }
 return null;
};

/** Ключ-пара вида "{0, 3}" или "0, 3" → [0, 3]. */
const parsePairKey = (key: string): [number, number] | null {
 const m = key.match(/^\(?\s*(-?\d+)\s*,\s*(-?\d+)\s*\)/);
 if (!m) return null;
 return [Number(m[1]), Number(m[2])];
};

export function liveMatrix(value: DebugValue | undefined) {
 if (value === undefined || value === null) return null;

 const cells = new Map<string, number>();
 let rows = 0;
 let cols = 0;
 const put = (i: number, j: number, v: DebugValue | undefined) => {
 if (!Number.isInteger(i) || !Number.isInteger(j) ||
 i < 0 || j < 0 || i >= MAX_MATRIX_SIDE || j >= MAX_MATRIX_SIDE) return;
 const n = toCellNumber(v);
 if (n === null) return;
 cells.set(`${i}:${j}`, n);
 rows = Math.max(rows, i + 1);
 cols = Math.max(cols, j + 1);
 };

 const asArray = vizArray(value);
 if (asArray) {
 asArray.slice(0, MAX_MATRIX_SIDE).forEach((row, i) => {
 const inner = vizArray(row);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((cell, j) => {
 put(i, j, cell);
 });
 }
 const record = vizRecord(row);
 if (record) {
 record.forEach((cell, j) => {
 put(i, j, cell);
 });
 }
 });
 }
}

```

```

 hasNumbers: cells.size > 0,
 };
}

/** Первое имя из списка, которое реально есть в снимке
export function pickLiveVar(
 vars: Record<string, DebugValue> | undefined,
 names: string[]
): { name: string; value: DebugValue } | null {
 if (!vars) return null;
 for (const name of names) {
 if (Object.prototype.hasOwnProperty.call(vars, name)
 }
 }
 return null;
}

/** Индекс, ограниченный реальной размерностью данных
export const clampIndex = (value: number | null, size: number): number {
 if (value === null || !Number.isFinite(value) || size <= 0)
 return 0;
 return Math.max(0, Math.min(size - 1, Math.trunc(value)));
}

/* — Выбор демонстрации внутри страницы (вкладки ID / chapterId)

const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные
 demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта
export function emitVizDemo(chapterId: string, demoId: string) {
 if (typeof window === "undefined") return;
 latestDemoByChapter.set(chapterId, demoId);
 window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {

```



```

 role: string;
 /** Характерный диапазон/значение на демонстрации. */
 range?: string;
}

export interface PageSync {
 /** Заголовок демонстрации на странице (если интерактивна)
 vizTitle?: string;
 /** Чему соответствует один шаг визуализации (для тулза)
 stepNote?: string;
 /** Переменные синхронизации. Пусто – на странице нет
 variables: SyncVariable[];
 /** Полный референсный Python-код, который кнопка-главн
 code?: string;
}

/** Справочные значения, указанные внутри самих симулято
export const PAGE_SYNC: Record<string, PageSync> = {
 "segment-trees": {
 vizTitle: "Дерево отрезков",
 stepNote: "Шаг = спуск по вершине v: либо читаем тр
 code: `n = 8 # длина массива
tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
 variables: [
 { name: "n", role: "длина исходного массива; лист
 { name: "i", role: "индекс элемента массива; его
 { name: "v", role: "текущая вершина дерева отрезк
 { name: "l, r", role: "границы отрезка, за которы
 { name: "m", role: "середина отрезка m = (l + r)
],
 },
 treap: {
 vizTitle: "Декартово дерево: build → split → merge

```

```

 t.r = a
 return (t, b)
 a, b = split(t.l, m)
 t.l = b
 return (a, t)

def merge(a, b):
 """Слияние: все x в a меньше всех x в b."""
 global step
 if a is None: return b
 if b is None: return a
 step += 1
 if a.y > b.y:
 a.r = merge(a.r, b)
 return a
 b.l = merge(a, b.l)
 return b

def erase(t, k):
 """Удалить точку с x = k: два split и один merge."""
 a, b = split(t, k - 1) # a: x <= k-1, b: x >= k
 mid_t, c = split(b, k) # mid_t: ровно x = k
 return merge(a, c)

def inorder(t, acc):
 if t:
 inorder(t.l, acc); acc.append(t.x); inorder(t.r)
 return acc

def h(t):
 return 0 if t is None else 1 + max(h(t.l), h(t.r))

root = build(points)
height = h(root)
print("обход по x:", inorder(root, []), "| высота:", height)

step = 0
L, R = split(root, mid)

```

```

g = p.p if p else None
if p and p.l is x:
 p.l = x.r
 if x.r: x.r.p = p
 x.r = p
elif p:
 p.r = x.l
 if x.l: x.l.p = p
 x.l = p
if p: p.p = x
x.p = g
if g:
 if g.l is p: g.l = x
 else: g.r = x

```

```

def inorder(n, acc): # обход слева направо —
 if n:
 inorder(n.l, acc); acc.append(n.k); inorder(n.r)
 return acc

```

```

def splay(x):
 global case, step, p, g, root, xk, pk, gk
 while x.p:
 p = x.p; g = p.p
 step += 1
 if g is None:
 case = "zig" # родители
 rot(x)
 elif (g.l is p) == (p.l is x):
 case = "zig-zig" # х и р —
 rot(p); rot(x)
 else:
 case = "zig-zag" # змейка
 rot(x); rot(x)
 xk, pk, gk = x.k, p.k, (g.k if g else 0)
 print("шаг", step, "->", case, "| x =", xk, "p =", pk, "g =", gk)
 root = x
 print(" корень теперь", x.k, "| обход:", inorder(root, []))

```

```

print(row) # готовая таблица: None т

запрос min на отрезке за O(1): отрезок покрывают ДВА
L, R = 1, 6
K = (R - L + 1).bit_length() - 1
print("min на [" + str(L) + ", " + str(R) + "] =",
 min(st[L][K], st[R - (1 << K) + 1][K]),
 "| блоки длины", 1 << K)`,
variables: [
 { name: "a", role: "исходный массив, из которого "
 { name: "n", role: "длина массива a", range: "n = "
 { name: "LOG", role: "число уровней таблицы: log2"
 { name: "st", role: "разреженная таблица: st[i][j]"
 { name: "i", role: "начало блока, за который отве"
 { name: "j", role: "уровень таблицы: ячейка храни"
 { name: "row", role: "очередной элемент a, из кото"
 { name: "L / R", role: "границы отрезка запроса –"
 { name: "K", role: "степень двойки: длина блока 2"
],
},
"prefix-sums-2d": {
 vizTitle: "Префиксные суммы (1D и 2D)",
 stepNote: "Шаг = заполнение очередного P[i] = P[i-1"
 code: `a = [3, 1, 4, 1, 5, 9, 2, 6] # исходный ма
n = len(a) # длина: префиксов будет
i = 0 # текущая граница префикс

P = [0] # P[0] = 0 – пустой преф
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)`,
variables: [
 { name: "a", role: "исходный массив 1D-демонстрац"
 { name: "n", role: "длина массива a; префиксов бу"
 { name: "P", role: "префиксные суммы: P[i] = a[0]"
 { name: "i", role: "граница префикса: на шаге счи
],
},

```

```

 { name: "i", role: "индекс элемента, который сейчас обрабатываем",
 { name: "(i-1)//2", role: "индекс родителя вершины",
 { name: "2·i+1, 2·i+2", role: "индексы левого и правого ребёнка",
],
},
"stack-dfs": {
 vizTitle: "Стек и обход в глубину",
 stepNote: "Шаг = push новой вершины на стек или pop",
 code: `st = [] # стек = рекурсия
st.append("A") # push – зашли в вершину
st.append("B")
v = st[-1] # вершина на вершине стека:
top = st.pop() # pop – сняли её со стека
print("v =", v, "| pop →", top, "| стек:", st)\`,
 variables: [
 { name: "top", role: "вершина стека – то, откуда пришли",
 { name: "v", role: "текущая вершина графа, которую обрабатываем",
],
},
"queue-bfs": {
 vizTitle: "Очередь и обход в ширину",
 stepNote: "Шаг = dequeue вершины из головы очереди",
 code: `from collections import deque

q = deque(["A"]) # очередь BFS
u = "B" # сосед, найденный из текущей вершины
q.append(u) # enqueue соседа – в конец очереди
front = q[0] # голова очереди: её обрабатываем
v = q.popleft() # шаг: достали голову и обрабатываем
print("front =", front, "| v =", v, "| очередь:", list(q))
 variables: [
 { name: "front", role: "голова очереди – вершина, которую обрабатываем",
 { name: "v", role: "вершина, извлечённая из очереди",
 { name: "u", role: "сосед вершины v, которого кладем в очередь",
],
},
"everyday-basics": {
 vizTitle: "Бытовой тренажёр: стек, очередь, куча",

```

```

 if v in visited:
 continue
 visited.add(v); order.append(v)
 for u in adj[v]: # соседи v → в конец очереди
 q.append(u)
print("BFS:", order)`
variables: [
 { name: "v", role: "текущая вершина обхода", group: "main" },
 { name: "to / u", role: "очередной сосед вершины", group: "main" },
 { name: "visited", role: "множество уже посещённых", group: "main" },
 { name: "order", role: "порядок, в котором обход", group: "main" },
],
},
"graph-components": {
 vizTitle: "Компоненты связности: DFS-покраска",
 stepNote: "Шаг = вершина v достаётся из стека/очереди",
 code: `
n = 10 # вершин в графе
adj = [[1, 3], [0, 2, 3], [1, 3], [0, 1, 2], [5, 6], [4, 6]]
comp = [-1] * n # -1 = вершина ещё не покрашена
used = [False] * n # то же самое булевым массивом
comps = 0 # счётчик найденных компонент
c = 0 # номер текущей компоненты
v = -1 # вершина, в которой обход начался
stack = [] # фронт обхода (стек для DFS)
i = 0 # номер шага — двигает по очереди

--- тело алгоритма ---
for start in range(n):
 if comp[start] != -1:
 continue
 comps += 1 # непокрашенная вершина => новая компонента
 c = comps
 stack = [start]
 comp[start] = c
 used[start] = True
 v = start
 while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)

```

## Универсальное пособие • полный исходный код

```

answer = order[::-1] # разворот – топологический
for i, v in enumerate(answer):
 print(f"позиция {i}: вершина {v}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "вершина, куда ведёт ребро из
 { name: "i", role: "перебор стартовых вершин во в
 { name: "order", role: "список выхода из рекурсии
],
},
"scc-kosaraju": {
 vizTitle: "Компоненты сильной связности (Косарайю)"
 stepNote: "Шаг = строка алгоритма: в фазе 1 DFS кла
 рованному графу и красит свою SCC.",
 code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [4], 4:
n = 5

фаза 1: DFS по обычному графу, запоминаем ПОРЯДОК ВЫХ
used = {}
for v in adj:
 used[v] = False
order = []

def dfs1(v):
 used[v] = True
 for to in adj[v]:
 if not used[to]:
 dfs1(to)
 order.append(v) # выходим из вершины – она

for v in adj:
 if not used[v]:
 dfs1(v)
 print("DFS из", v, "закончен | order:", order)

транспонирование: каждое ребро разворачиваем
adjT = {}
for v in adj:
```

```

 },
 "graph-articulation": {
 vizTitle: "Точки сочленения",
 stepNote: "Шаг = обновление low[v] по ребру; вершина",
 code: `tin, low, timer = {}, {}, 0

i = 0 # номер шага DFS
v, to = 0, 1 # текущее ребро DFS: v → to
tin[v] = tin.get(v, timer); low[v] = tin[v]

шаг: возврат из to в v
i += 1
low[v] = min(low[v], low.get(to, tin[v]))
critical = low.get(to, 0) >= tin[v] # → v — точка соч.
print(f"i={i} v={v} to={to} critical={critical}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед: либо ребёнок в DFS-дер
 { name: "tin[v]", role: "время входа в вершину (п
 { name: "low[v]", role: "минимальный tin, достижи
 { name: "i", role: "перебор стартовых вершин для
],
 },
 "bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v,
 code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3 # как на демо

v, to = "B", "G" # шаг: возврат из G
is_bridge = low[to] > tin[v] # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед; ребро (v, to) проверя
 { name: "tin[v]", role: "время входа в вершину —
 { name: "low[v]", role: "минимум tin, куда можно
 { name: "timer", role: "глобальный счётчик времен

```



```

print("нечётных вершин:", len(odd), odd, "->", "цикл" if len(odd) > 1 else "нет", "цикл")
print("маршрут:", path, "| рёбер пройдено:", sum(used), "шагов")
variables: [
 { name: "path", role: "маршрут – последовательность вершин"},
 { name: "last / v", role: "вершина, где обход находится"},
 { name: "used", role: "какие рёбра уже пройдены (или нет)"},
 { name: "deg", role: "степени вершин: чётные всюду, нечётные в цикле"},
 { name: "odd", role: "список нечётных вершин – по порядку"},
 { name: "stack", role: "стек отката алгоритма Иерархического поиска"},
 { name: "step / i", role: "номер шага обхода", range: 1, step: 1},
],
},
"planarity-euler-formula": {
 vizTitle: "Покраска вершин: $\chi \leq \Delta + 1$ ",
 stepNote: "Шаг = одна вершина v: смотрим цвета её соседей",
 code: `
n = 8 # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 5]]
delta = max(len(a) for a in adj) # максимальная степень вершины
color = [0] * n # 0 = ещё не покрашена
chi = 0 # сколько цветов понадобилось
v = -1 # вершина, которую красим сейчас
i = 0 # номер шага

V = n # вершин для формулы
E = sum(len(a) for a in adj) // 2 # рёбер: в списке смежности
F = 2 - V + E # граней: из V - E + 2

--- тело алгоритма ---
print("V =", V, "| E =", E, "| F =", F, "| V - E + F =", 2)
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 `

```

```

 if du + w < dist.get(v, float("inf")):
 dist[v] = du + w
 prev[v] = u
 heapq.heappush(pq, (dist[v], v))
print("итог:", dist)\,
 variables: [
 { name: "u", role: "вершина с минимальным dist среди",
 { name: "v", role: "сосед вершины u, до которого",
 { name: "w", role: "вес ребра (u, v)", range: "по",
 { name: "dist[v]", role: "лучшее известное расстояние"},
],
},
"bellman-ford": {
 vizTitle: "Форд–Беллман",
 stepNote: "Шаг = одна релаксация ребра (u, v, w) в графе",
 code: `INF = float("inf")

демо-граф страницы: 7 вершин, 9 односторонних рёбер;
E = [("S", "A", 4), ("S", "B", 5), ("B", "C", -2), ("A",
 ("C", "D", 3), ("C", "E", 4), ("D", "F", 2), ("E",
 ("D", "A", -2))] # замените -2 на -6 – появится цикл

nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes) # 7 вершин => n - 1 = 6 работ
m = len(E) # 9 рёбер

dist = {v: INF for v in nodes}
dist["S"] = 0 # старт – База

--- тело алгоритма ---
last_changed = ""
for i in range(1, n): # итерация i = 1 ... n-1
 last_changed = ""
 for u, v, w in E: # шаг: релаксация ребра (u, v, w)
 if dist[u] != INF and dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 last_changed = v
 if last_changed == "":

```

```

print("итог: все пары посчитаны")`,
 variables: [
 { name: "n", role: "число вершин – размер квадрата" },
 { name: "k", role: "промежуточная вершина – ВНЕШНЯЯ" },
 { name: "i", role: "начало пути – строка матрицы" },
 { name: "j", role: "конец пути – столбец матрицы" },
 { name: "dist[i][j]", role: "кратчайший путь i → j" },
],
},
"johnson-algo": {
 vizTitle: "Алгоритм Джонсона (перевзвешивание)",
 stepNote: "Шаг = Беллман-Форд из фиктивной вершины",
 code: `# потенциалы из Беллмана-Форда от фиктивной вершины
h = {"A": 0, "B": -3, "C": -1, "D": 1, "E": 2, "F": 4}

u, v, w = "A", "B", -3 # ребро с отрицательным весом
w_new = w + h[u] - h[v] # w' = w + h[u] - h[v] ≥ 0
print(f"w'({u}→{v}) = {w_new}")
после Дейкстры: ответ = d[v] - h[start] + h[v]`,
 variables: [
 { name: "h[v]", role: "потенциал вершины – кратчайшее расстояние от Беллмана-Форда" },
 { name: "u, v", role: "ребро (u, v) перевзвешивается" },
 { name: "d[v]", role: "расстояние Дейкстры в новой метрике" },
],
},
},
mst: {
 vizTitle: "Остовное дерево: Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `n = 9 # вершин: A..I – то же, что в предыдущем
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "C"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "D", "F"),
 (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGHI"} # DSU: представителем каждой вершины
rank = {v: 0 for v in "ABCDEFGHI"} # высота дерева

```

```

n = len(s)
i, j = 1, 0
pi = [None] * n # пустые клетки: значения п
pi[0] = 0

for i in range(1, n): # шаг: считаем pi[i]
 j = pi[i - 1] # длина совпадения
 while j > 0 and s[i] != s[j]:
 j = pi[j - 1] # откат назад
 if s[i] == s[j]:
 j += 1
 pi[i] = j
print(pi)`
 variables: [
 { name: "n", role: "длина строки s, для которой с
 { name: "i", role: "индекс текущего символа – счи
 { name: "j", role: "длина текущего наилучшего сов
 { name: "π[i]", role: "префикс-функция: длина наи
 ица" },
],
},
"string-z-func": {
 vizTitle: "Z-функция",
 stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l
 code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * n # пустые клетки: значения п
z[0] = 0

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:

```

```

for w in words:
 add_word(w)

BFS по уровням: fail-ссылка ребенка = куда приведёт f
fail[0] = 0
queue = sorted(v for (p, ch), v in nxt.items() if p == 0)
for v in queue:
 fail[v] = 0
qi = 0
while qi < len(queue):
 v = queue[qi]
 qi += 1
 for (p, ch), u in sorted(nxt.items()):
 if p != v:
 continue
 f = fail[v]
 while f != 0 and (f, ch) not in nxt:
 f = fail[f]
 fail[u] = nxt.get((f, ch), 0)
 if fail[u] == u:
 fail[u] = 0
 queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

def go(v, ch):
 """Переход автомата: сначала по бору, иначе спускаем
 while v != 0 and (v, ch) not in nxt:
 v = fail[v]
 return nxt.get((v, ch), 0)

for i, c in enumerate(text):
 state = go(state, c)
 v = state
 while v != 0:
 # собираем все образцы
 if v in term:
 matches.append((i, term[v]))

```



```

 if color[start] != 0:
 continue
 color[start] = 1
 queue = [start]
 while queue and ok:
 v = queue.pop(0)
 for u in adj[v]:
 if color[u] == 0:
 color[u] = 3 - color[v] # противоположный цвет
 queue.append(u)
 i += 1
 elif color[u] == color[v]:
 ok = False # нечётный цикл
 print("конфликт на ребре", (v, u))
print("двудольный:", ok, "| разметка:", color)`
variables: [
 { name: "color", role: "разметка на две доли: 1 и 2"},
 { name: "v", role: "вершина, из которой BFS смотрит"},
 { name: "u", role: "сосед, которого красим (или с которого смотрим)"},
 { name: "queue", role: "очередь BFS", range: "≤ n"},
 { name: "ok", role: "вердикт: граф двудольный (не планарный)"},
 { name: "i / step", role: "номер кадра", range: "0..n-1"}
],
},
"planarity-euler-formula#edges": {
 vizTitle: "Покраска рёбер (Визинг: $\Delta \leq \chi' \leq \Delta + 1$)",
 stepNote: "Шаг = ребро e получает наименьший цвет, не имеющий соседей",
 code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4), (4, 5), (5, 6), (6, 7), (7, 8)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4, 6], [3, 4, 5, 7], [4, 5, 6, 8], [5, 6, 7, 8]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро, которое имеет цвет
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

--- тело алгоритма ---

```

```

 c = comps
 stack = [start]
 comp[start] = c
 used[start] = True
 v = start
 while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)
 i += 1
 for to in adj[v]:
 if comp[to] == -1:
 comp[to] = c
 used[to] = True
 stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)\,
 variables: [
 { name: "comp", role: "номер компоненты каждой вер",
 { name: "v", role: "текущая вершина обхода", rang
 { name: "stack", role: "стек обхода (для BFS заме
 { name: "comps / c", role: "счётчик компонент и н
 { name: "i", role: "номер шага — перематывает кад
],
},
"graph-components#dsu": {
 vizTitle: "Компоненты связности: DSU по рёбрам",
 stepNote: "Шаг = одно ребро (u, v): union склеивает
 code: `n = 10 # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)
parent = list(range(n)) # DSU: parent[v] = представи
rank = [0] * n # высота дерева — по ней реш
comps = n # сначала каждая вершина — о
e = -1 # номер обрабатываемого ребр
u, v = -1, -1 # концы текущего ребра
step = 0 # номер шага — двигает покад

--- тело алгоритма ---
def find(x):
 while parent[x] != x:

```



```
--- тело алгоритма ---
P: решается за полином – сортировка
a = [5, 2, 8, 1, 9, 3]
for i in range(len(a)):
 for j in range(len(a) - 1):
 ops += 1
 if a[j] > a[j + 1]:
 a[j], a[j + 1] = a[j + 1], a[j]
print(cls, "отсортировано за", ops, "операций:", a)\,
 variables: [
 { name: "cls", role: "имя класса – P, NP, PSPACE"},
 { name: "n", role: "размер входа задачи, про которую"},
 { name: "ops", role: "сколько операций потребовалось"},
],
},
"complexity-classes#reduce": {
 vizTitle: "Сведение $A \leq_p B$ (3-SAT $\rightarrow$ КЛИКА)",
 stepNote: "Шаг = один кадр схемы сведения: вход $x \rightarrow$ ",
 code: `cls = "NP" # класс задачи A,
step = 0 # кадр сведения $A \leq_p B$: 0...6
n = 9 # размер входа x (литералов в дизъюнктах)
k = 3 # размер клики, которую ищем

--- тело алгоритма ---
Сведение 3-SAT $\leq_p$ КЛИКА: вершина = (номер дизъюнкта, номер литерала)
ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
 print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима $\Leftrightarrow$ в графе есть клика размера k")
 variables: [
 { name: "step", role: "номер кадра сведения (0...6)"},
 { name: "cls", role: "класс задачи A, которую сводим к клике"},
 { name: "n", role: "размер входа x; после сведения к клике"},
 { name: "k", role: "размер клики, которую ищет алгоритм"}]
```

```

 for i in range(n):
 for j in range(i, n):
 ops += 1
 print("n =", n, "ops =", ops)

Экспоненциальный (полный перебор подмножеств) – раскол
for n in range(1, 15):
ops = 2 ** n
print("n =", n, "ops =", ops)`,
 variables: [
 { name: "n", role: "размер входа – по нему строится"},
 { name: "ops", role: "число операций алгоритма на"},
 { name: "i", role: "внутренний счётчик цикла – то"},
],
},
"mst#kruskal": {
 vizTitle: "Остовное дерево: Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `n = 9 # вершин: A...I – т
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10
 (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию
parent = {v: v for v in "ABCDEFGH I"} # DSU: представител
rank = {v: 0 for v in "ABCDEFGH I"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):

```

```

w, x, frm = heapq.heappop(heap) # самое лёгкое
if x in taken:
 continue # устаревшая за
taken.append(x)
u, v = frm, x
if frm != x:
 mst.append((w, frm, x))
 total += w
for to, wt in adj[x]:
 if to not in taken:
 heapq.heappush(heap, (wt, to, x))
print("вес остова:", total, "| вершин захвачено:", len(taken))
print(mst)`
variables: [
 { name: "start", role: "вершина, из которой растёт"},
 { name: "heap", role: "куча рёбер-кандидатов (вес, вершина)"},
 { name: "taken", role: "вершины, уже вошедшие в остова"},
 { name: "u, v, w", role: "ребро, которое только что добавили"},
 { name: "mst / total", role: "рёбра остова и его вес"},
],
},
"mst#boruvka": {
 vizTitle: "Остовное дерево: Борувка (фазы-«пятилетки»)",
 stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРШИНОВА",
 code: `n = 9 # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "C"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "D", "F"),
 (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"} # DSU: корень компоненты
cheapest = {} # корень компоненты -> самое лёгкое ребро
mst = [] # рёбра остова
total = 0 # вес остова (= 51)
phase = 0 # номер фазы («пятилетки») — от 0 до 4
u, v, w = None, None, None # ребро-кандидат

--- тело алгоритма ---
def find(x):
 while comp[x] != x:

```

```

code: `A = [[45, 12, 88, 34, 11, 76, 23, 90], [67,
 88, 29, 60], [25, 41, 16, 92, 9, 17, 56, 31], [5
 1, 42, 85, 14]]
r, c, k = 0, 0, 0
st2 = {} # пустой каркас виден до

--- тело алгоритма ---
База не отдельный уровень вычислений: блок 1x1 — это
Записываем её одним шагом, а не проигрываем 64 одноти
Генератор остаётся отдельным фреймом и не конфликтует
st2.update(dict(({r, c, 0, 0}, A[r][c]) for r in range(0
for k in range(1, 4):
 print(f"уровень k={k}: блоки {1 << k}x{1 << k}")
 size = 9 - (1 << k)
 half = 1 << (k - 1)
 for r in range(size):
 for c in range(size):
 st2[(r, c, k, k)] = min(st2[(r, c, k - 1, k
 + half, c + half, k - 1, k - 1)])`,
variables: [
 { name: "r, c", role: "левый верхний угол блока,
 { name: "k", role: "уровень: блоки 2^k x 2^k (шаг
 { name: "st2[(r,c,k,k)]", role: "минимум квадратн
],
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери
 code: `# st2[(r, c, kx, ky)] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваем

```

```

* Формат: короткий комментарий-шапка + исполняемый реф
* Визуализация читает из его трассы знакомые имена (i,
* но точно так же примет пользовательский код с теми же
*/
export function buildSyncTemplate(chapterId: string, chapterTitle: string, sync: Sync) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} .`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 return `${head}\n${sync.code}\n`;
}

/**
* Инициализация скелета: только объявление демо-данных
* (без for/while/def/if/print). Именно это показывается
* умолчанию – дальше пользователь пишет свой вариант,
* заменяет редактор полным референсным кодом.
*/
function extractInit(code: string): string {
 const keep: string[] = [];
 for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
 keep.push(line);
 }
 }
 // убрать пустые строки с конца
 while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
 keep.pop();
 }
 return keep.join("\n");
}

/**
* Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
* переменные демо (без всего кода алгоритма).
*/
export function buildInitTemplate(chapterId: string, chapterTitle: string, sync: Sync) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} .`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 return `${head}\n${sync.code}\n`;
}
```

```

 type: "html",
 description:
 "Структура для запросов на отрезке и обновлений з
 category: "Продвинутые структуры",
 content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-blue-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic m
 <p class="text-xl font-mono text-white">
ree[2v+1])), где $m = (l + r) / 2$. Запрос и о
 </div>

 <p class="text-slate-300 text-sm mb-4">Масс
 [0, m

 — это одиночные элементы массива. В узле л
 чество единиц, И/ИЛИ — любая операция, котор
 >).</p>

 <p class="text-slate-300 text-sm mb-4">Отсю
 а каждом уровне встречается один из трёх случ
 не пересекается — возвращаем нейтральный э
 склеиваем. Частичных пересечений на уровне
 одного элемента — спуск к листу и пересчёт

 <div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300

```

на  $[L, R]$ » – всё это делается одним спуском

`ss="font-mono text-emerald-300">tree[v] +=`

`</b>`: перед любым спуском в детей вызываем

`<table class="w-full text-xs text-left`

`<thead>`

`<tr class="text-slate-500 borde`

`<th class="py-1.5 pr-2 font`

`<th class="py-1.5 pr-2 font`

`<th class="py-1.5 font-bold`

`</tr>`

`</thead>`

`<tbody class="text-slate-300 font-m`

`<tr class="border-b border-slate`

`x · len</td><td class="py-1.5">lazy[v] +=`

`<tr class="border-b border-slate`

`c · len</td><td class="py-1.5">lazy[v] = c`

`<tr class="border-b border-slate`

`len - tree[v]</td><td class="py-1.5">lazy[`

`<tr><td class="py-1.5 pr-2">min`

`-1.5">lazy[v] = min(lazy[v], c)</td></tr>`

`</tbody>`

`</table>`

`<p class="text-slate-500 text-xs mt-3">`

кивают в строгом порядке: сначала присваива

`</div>`

`<div class="bg-slate-900/40 rounded-lg bord`

`<p class="text-slate-300 text-sm mb-2">`

`<p class="text-slate-400 text-sm mb-1">`

за  $O(1)$ , но строятся один раз и умирают при

ми.`</p>`

`<p class="text-slate-400 text-sm mb-1">`

идемпотентные запросы (`min`, `gcd`, `И`) по ста

`<p class="text-slate-400 text-sm mb-1">`

$n$  ячеек (не  $2n!$ ) – иначе на  $n = 6$  поймать п

$n \dots 2n-1]$  и  $n$  – степень двойки (или использ

`<p class="text-slate-400 text-sm">` · `<b>`

ть лень во всё дерево – это  $O(n)$  и весь смы

```

 int m = (l + r) / 2;
 return sum(2 * v, l, m, L, R) // части
 + sum(2 * v + 1, m + 1, r, L, R);
}

```

```

void update(int v, int l, int r, int i, int x) { // a[i]
 if (l == r) { tree[v] += x; return; }
 int m = (l + r) / 2;
 if (i <= m) update(2 * v, l, m, i, x);
 else update(2 * v + 1, m + 1, r, i, x);
 tree[v] = tree[2 * v] + tree[2 * v + 1]; // перес
}

```

</div>  
</details>

<details class="bg-slate-900/40 rounded-lg l  
<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">

Код: массовое обновление, lazy pr

</summary>

<div class="p-5 text-sm text-slate-300

<pre class="bg-slate-950 p-4 rounded  
y[4 \* MAXN]; // «обещание»: прибавит

```

void push(int v, int l, int r) {
 if (lazy[v] == 0) return;
 int m = (l + r) / 2;
 tree[2 * v] += lazy[v] * (m - l + 1); // п
 lazy[2 * v] += lazy[v];
 tree[2 * v + 1] += lazy[v] * (r - m); // и
 lazy[2 * v + 1] += lazy[v];
 lazy[v] = 0; // о
}

```

```

void add(int v, int l, int r, int L, int R, int x) { //
 if (r < L || R < l) return;
 if (L <= l && r <= R) {
 tree[v] += x * (r - l + 1);
 }
}

```



```

 </div>
 </details>

 <details class="bg-slate-900/40 rounded-lg p-4 font-bold text-slate-300 border-slate-700/50">
 <summary>
 Вопросы, которые любят задавать (и не любят)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>• Почему $4n$, а не $2n$? – при n , не $2n$.

 пример: $n = 6$).</p>
 <p>• Сколько вершин посещает запрос

 ва.</p>
 <p>• Почему построение $O(n)$, а не $O(n^2)$?

 <p>• Какие операции подходят? – любые, но

 количество различных на отрезке» обычное дерево.

 <p>• Зачем push в запросе, если ничего не

 й; push делает данные детей достоверными.</p>
 </div>
 </details>
 </div>
 </div>
</section>
\
 },
];

```

ФАЙЛ 27/110: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 1194 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from '../types';

export const tickets14to20: Chapter[] = [
 {
 id: "dijkstra",

```

```

 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Курьер с навигатором
 </div>
 <p class="text-slate-300 text-sm mb
ка, потом следующая по близости и так далее
удет, потому что все дороги платные (веса ≥
волна расстояний расходится от старта в по
 </div>

 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Ограничения: минус ломает жадно
 </div>
 <p class="text-slate-300 text-sm mb
class="font-mono text-emerald-300">s→a = 2</
ono text-emerald-300">b→a = -2. Верн
начала вытащит <span class="font-mono text-
o text-emerald-300">3 - 2 = 1 придёт
Джонсона (билет 17).</p>
 </div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400

```

```

 <tr class="border-b border-slate-400">
 <td class="py-2 font-mono">Парк С (2)</td><td class="py-2 font-mono">
 <tr class="border-b border-slate-400">
 <td class="py-2 font-mono">Дом 1 В (3)</td><td class="py-2 font-mono">
 <tr class="border-b border-slate-400">
 <td class="py-2 font-mono">Метро G (5)</td><td class="py-2 font-mono">
 <tr class="border-b border-slate-400">
 <td class="py-2 font-mono">Офис D (6)</td><td class="py-2 font-mono">
 <tr class="border-b border-slate-400">
 <td class="py-2 font-mono">Дом 2 Е (7)</td><td class="py-2 font-mono">
 <tr class="align-top"><td class="py-2 font-mono">исходящих рёбер нет</td><td class="py-2 font-mono">
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-3">
 F 8. Из 11 проверок рёбер 10 улучшились.
 : A → C → G → E → F = 2 + 3 + 2 + 1 = 8
 </div>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">Найден лучший путь!</p>
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-1.5 pr-2 font-normal">Путь</th>
 <th class="py-1.5 pr-2 font-normal">Длина</th>
 <th class="py-1.5 font-bold">Описание</th>
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-400">
 <td class="py-2 pr-2">лучший известный путь s → v</td><td class="py-2 pr-2">10</td><td class="py-2 pr-2">предок v на текущем лучшем пути</td><td class="py-2 pr-2">вершины с окончательным dist</td></tr>
 <tr class="align-top"><td class="py-2 pr-2">лучший известный путь s → v</td><td class="py-2 pr-2">10</td><td class="py-2 pr-2">предок v на текущем лучшем пути</td><td class="py-2 pr-2">вершины с окончательным dist</td></tr>
 </tbody>
 </table>

```

```

<details class="bg-slate-900/40 rounded-lg border-b border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">Откуда берётся сложность?
 <div class="p-5 text-sm text-slate-300">
 <ul class="list-disc pl-5 space-y-2">
 Куча. Каждая вершина имеет pop), каждое ребро push. Итого E пар, но логарифм от этого не меняет
 Массив. Поиск минимума V: итого E ≈ V^2) это быстрее кучи и
 Куча Фибоначчи. E операций, константа: в олимпиадах почти не используется
 Специальные веса. Вес E; малые целые веса → корзины Диала. Всё это

 </div>
</details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-b border-slate-700/50">
 <h3 class="text-xl font-bold text-emerald-400">Сложности
 <div class="overflow-x-auto my-4">
 <table class="w-full text-xs text-left border-collapse">
 <thead>
 <tr class="text-slate-500 border-b border-slate-600/50">
 <th class="py-2 pr-3 font-bold">Алгоритм
 <th class="py-2 pr-3 font-bold">Сложность
 <th class="py-2 font-bold">Примечания
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-600/50">
 <td>priority_queue (C++)</td><td class="py-2 pr-3">O(n log n)
 й выбор</td></tr>
 <tr class="border-b border-slate-600/50">

```

```

 heapq.heappush(pq, (dist[v], v))
 return dist, prev

def restore(prev, start, target):
 if prev[target] is None and target != start:
 return None # вершина недо
 path = [target]
 while path[-1] != start:
 path.append(prev[path[-1]])
 return path[::-1]

демо-граф страницы (доставка пиццы, рёбра односторонни
g = {
 "A": [("B", 4), ("C", 2)],
 "B": [("G", 4), ("D", 5)],
 "C": [("B", 1), ("G", 3), ("E", 8)],
 "D": [("F", 3)],
 "E": [("F", 1)],
 "F": [],
 "G": [("D", 1), ("E", 2)],
}
dist, prev = dijkstra(g, "A")
print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
 <p class="text-slate-400 text-xs">П
 же вершины, потому что читает ваши <span c
nt-mono">done.</p>
 </div>
</details>

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">> Код: C++ (priority
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <queue>;
using namespace std;

```

```

<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">» Код: $O(V^2)$ без куд
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 ти $n \times n$, $w[i][j] = \text{INF}$ если ребра нет
d = [INF] * n
used = [False] * n
d[s] = 0

for _ in range(n):
 u = -1
 for v in range(n): # ищем минимум
 if not used[v] and (u == -1 or d[v] < d[u]):
 u = v
 if u == -1 or d[u] == INF:
 break # остались тол
 used[u] = True # фиксируем d[u]
 for v in range(n): # релаксируем
 if d[u] + w[u][v] < d[v]:
 d[v] = d[u] + w[u][v]</code></pre>
 <p class="text-slate-400 text-xs">Т
 . Память – матрица
 </div>
 </details>
 </div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-amber-400

 <div class="bg-rose-950/30 p-5 rounded-lg b
 <p class="text-sm font-bold text-rose-3
 <ul class="list-disc pl-5 space-y-2 tex
 Отрицательные веса. Алгор
 ксация вершины необратима. Нужен Форд–Беллм
 Отрицательные циклы. При
 ечное число. Сначала проверьте граф Фордом–
 Ленивое удаление. Без пр
```

```

 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
есть отрицательный цикл.</p>
</div>

```

```

<p class="text-slate-300 text-sm mb-4">Форд-
ных весов. Плата – скорость: вместо 0(V · E)
font-mono text-emerald-300">(u, v, w)
ком, релаксируя каждое ребро.</p>

```

```

<p class="text-slate-300 text-sm mb-4">Логи
ез повторов вершин), значит в нём не больше
nt-mono text-emerald-300">i-й итераци
00">≤ i рёбер. Поэтому <span class="
nt-mono text-emerald-300">V-я – диагн
ь мы попали в отрицательный цикл.</p>

```

```

<div class="grid grid-cols-1 sm:grid-cols-3"
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">

```

Аналогия 1: Слухи в деревне

```
</tr>
</thead>
<tbody class="text-slate-300">
 <tr class="border-b border-slate-200 text-emerald-300"><td class="py-2 pr-2 font-mono">да</td><td class="py-2 pr-2 font-mono">нет</td><td class="py-2 pr-2 font-mono">без изменений</td></tr>
 <tr class="border-b border-slate-200 text-emerald-300"><td class="py-2 pr-2 font-mono">нет</td><td class="py-2 pr-2 font-mono">гарантия V –</td><td class="py-2 pr-2 font-mono">отрицательная</td></tr>
</tbody>
</table>
<p class="text-slate-400 text-xs mt-3">«в том же порядке: S→A, порядок рёбер местами — и понадобится до шести шагов»</p>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white mb-3">Шаг 1</p>
 <p class="text-slate-300 text-sm mb-3">Вспомогательная таблица A имеет суммарный вес S→A, который равен «весу» на 2.</p>
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-bottom">
 <th class="py-1.5 pr-2 font-normal">Элементы</th>
 <th class="py-1.5 pr-2 font-normal">Вес</th>
 <th class="py-1.5 pr-2 font-normal">Путь</th>
 <th class="py-1.5 pr-2 font-normal">Суммарный вес</th>
 <th class="py-1.5 pr-2 font-normal">Граничные элементы</th>
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200 font-mono"><td class="py-2 pr-2 font-mono">0</td><td class="py-2 pr-2 font-mono">—</td><td class="py-2 pr-2 font-mono">—</td><td class="py-2 pr-2 font-mono">—</td><td class="py-2 pr-2 font-mono">—</td></tr>
 <tr class="border-b border-slate-200 font-mono"><td class="py-2 pr-2 font-mono">1</td><td class="py-2 pr-2 font-mono">—</td><td class="py-2 pr-2 font-mono">—</td><td class="py-2 pr-2 font-mono">—</td><td class="py-2 pr-2 font-mono">—</td></tr>
```



$\leq \text{dist}[u] + w(u, v) \leq |P|$

**Следствие 1.** Если отрицать уть прост и содержит  $\leq \text{dist}$  точны.

**Следствие 2.** Если  $V$  рёбер, который вес  $\geq 0$ , выкидывание цикла не ухудшило бы п

```

 y = prev[y]

 cycle = [y]
 v = prev[y]
 while v != y:
 cycle.append(v)
 v = prev[v]
 cycle.append(y)
 cycle.reverse()
 return dist, prev, cycle

демо-граф страницы (9 рёбер, режим с отрицательным ци
nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
 {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D",
print([dist[i] for i in range(len(nodes))]) # [0, 4,
print(cyc) # None —

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D",
print([nodes[i] for i in cyc2]) # ['D',
print([dist2[i] for i in range(len(nodes))]) # всё
 <p class="text-slate-400 text-xs">П
 имая вершина с INF<
 ля float('inf')</sp
 </div>
 </details>

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">> Код: C++ (структур
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <iostream>;

```

```

}

int main() {
 int s = 0;
 int x = bellman_ford(s);
 if (x == -1) cout &&& "отрицательных циклов не
 else {
 cout &&& "цикл: ";
 for (int v : negative_cycle(x)) cout &&& v
 cout &&& "\n";
 }
}
}

```

</div>  
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">» Оптимизации: ранни  
 <div class="p-5 text-sm text-slate-300  
 <pre class="bg-slate-950 p-4 rounded overflow-x-auto te  
 rt deque

```

def spfa(n, adj, s):
 """adj[u] = [(v, w), ...]. Релаксируем только «живые»
 INF = float("inf")
 dist = [INF] * n
 in_queue = [False] * n
 counter = [0] * n # сколько раз вершина
 dist[s] = 0
 q = deque([s])
 in_queue[s] = True
 while q:
 u = q.popleft()
 in_queue[u] = False
 for v, w in adj[u]:
 if dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 if not in_queue[v]:

```

```

 Цикл есть, но не от старта.
 . Нужна проверка всего графа – добав
 а, билет 17) и запустите от неё.
 «Ответ $-\infty$ ». Если отрицат
 s="font-mono">dist[v] = $-\infty$: печатать
 ходом в обратном графе.
 SPFA как «быстрый Форд–Беллм
 в условии V, E ≤ 1

 </div>

 <p class="text-slate-300 text-sm mt-4">С
 отенциалы <span class="font-mono text-emera
 я Дейкстрой. Флойд–Уоршелл (16) решает все
 порядок для DAG – билет 9.</p>
 </div>
 </div>
</section>`
 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">

```

```

 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 </div>

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Таблица пересадок в
 </div>
 <p class="text-slate-300 text-sm mb
це только соседние станции. Затем говорите:
 class="font-mono">через 1 там, где
плакат окончательный. Порядок «разрешений»
an>.</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Почему k обязан быть
 </div>
 <p class="text-slate-300 text-sm mb
s="font-mono">k один раз и бо
садку не соберётся. На демо-графе этой стра
ono text-emerald-300">d[2][1] = ∞, х
пользует «поздние» посредники). Проверка пер
 </div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400
 <p class="text-slate-300 text-sm mb-4">Демо

```



```

 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
 return d

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1]) # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good[i][j] != bad[i][j]))
4 клетки отличаются; при порядке (i,k,j) их 6

```

В остальных «почти работают» (большинство клеток с

Код: Флойд–Уоршелл

```

def floyd_warshall(n, edges, directed=True):
 d = [[INF] * n for _ in range(n)]
 nxt = [[-1] * n for _ in range(n)]
 for i in range(n):
 d[i][i] = 0 # диагональ обнулена

 for u, v, w in edges:
 if w < d[u][v]: # кратчайший путь
 d[u][v] = w
 nxt[u][v] = v
 if not directed and w < d[v][u]:
 d[v][u] = w
 nxt[v][u] = u

```

```

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-white" style="font-family: monospace;">
    t;
#include <algorithm>
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d(n, vector<long long>(n, INF));

void floyd() {
    for (int i = 0; i < n; ++i) d[i][i] = 0;
    for (int k = 0; k < n; ++k) // k
        for (int i = 0; i < n; ++i) {
            if (d[i][k] == INF) continue; // защищаем от переполнения
            for (int j = 0; j < n; ++j) {
                if (d[k][j] == INF) continue;
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
}

bool has_negative_cycle() {
    for (int i = 0; i < n; ++i) if (d[i][i] < 0) return true;
    return false;
}

long long diameter() { // самая длинная дистанция
    long long res = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (d[i][j] < INF) res = max(res, d[i][j]);
    return res;
}

int center() { // вершина с минимальным значением радиуса
    int best = -1; long long bestEcc = INF;
    for (int i = 0; i < n; ++i) {
        long long ecc = 0;
        for (int j = 0; j < n; ++j)
            ecc = max(ecc, d[i][j]);
        if (ecc < bestEcc) best = i, bestEcc = ecc;
    }
    return best;
}

```


```



```

 </table>
 <p class="text-slate-400 text-xs mt-2">
 > при r[i][k]
 V3.</p>
 </div>
 </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-amber-400">

 <div class="bg-rose-950/30 p-5 rounded-lg border">
 <p class="text-sm font-bold text-rose-300">
 <ul class="list-disc pl-5 space-y-2 text-rose-300">
 Порядок циклов. (i, j, k) на демо-графе 4 клетки
] = 999 вместо 4).
 Диагональ. d[i][j]
 «починит» её только при наличии цикла, а при отсутствии
 Переполнение INF. long long уходит в минус и «улучшает» всё подряд. При
 no">long long.
 Параллельные рёбра. Во вводе d[u][v] стартует с худшего веса.
 Отрицательные циклы. Формулы с циклом, ответ d[i][j] = -∞, если существует
 d[i][k] < ∞
 Недостижимость ≠ 0. В ответе 0 для пар (i, j) таких пар из 49). Выводить 999 или 0 как «решение»
 Память. nxt[i][j] значений (≈100 МБ на nxt[i][j] истока берите Дейкстру.
 Сам путь. Матрица nxt[i][j] (как в задаче)

 </div>
 </div>

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Такси с навигатором
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ы одинаково. А* – тот же таксист, но с навигатором.
 /span> прибавляется оценка остатка пути по карте.
 т туда, где сумма меньше. Навигатор обязан найти оптимальный путь.
 /span>): занижать можно, завышать нельзя – это сфера
 0">h = 0 это снова Дейкстра.</p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 2: Две бригады туннеля
 </div>
 <p class="text-slate-300 text-sm mb-4">
 пускают две: от А и от В. Каждая зона роста имеет свой
 я не в момент встречи, а когда сумма затрат минимальна.
 – иначе можно пропустить более дешёвый вариант.
 ирует.</p>
 </div>
 </div>
 <div class="overflow-x-auto my-6">
 <table class="w-full text-xs text-left" style="border-collapse: collapse; font-family: monospace;">
 <thead>
 <tr class="text-slate-500 border-bottom">
 <th class="py-2 pr-3 font-bold">Путь
 <th class="py-2 pr-3 font-bold">Сумма
 <th class="py-2 pr-3 font-bold">Остаток
 <th class="py-2 font-bold">Дейкстра
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3">разрешает Дейкстра

```

```

 </div>
 <p class="text-slate-300 text-sm mb-3">
 Дейкстра (билет 14) быстрая, но
 ленный. Алгоритм Джонсона делает финт: пере-
 ь теми же.
 </p>
 <ol class="text-sm text-slate-300">
 Добавляем фиктивную вершину
 Запускаем от s один цикл
 отрицательный цикл: если цикл есть, Джонсон
 Перевзвешиваем: d(u, v) =
 тво следует из того, что $h(v) \leq h(u) + w(u, v)$
 Магия: для любого пути p
) – $h(\text{финиш})$, то есть все пути между фиктивной
 еняется.
 Запускаем быструю Дейкстру
 lass="font-mono text-emerald-300">d(u, v) =

 <p class="text-slate-400 text-xs mt-3">
 еженном графе ($E \approx V$) это $O(V^2 \log V)$ против
 ойд – для маленьких или очень плотных.</p>
 </div>
 </div>

```

```

<h3 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Если
азад из t (по перевёрнутым рёбрам). Волны р
: в «широком» графе (дорожная сеть) это эконо
огда волны «встретились» в одной вершине; на
огда g(v) (уже
(v) = g(v) + h(v), где $h(v)$ – оценка
а требования: допустимость ($h(v) \leq$ и
монотонность/согласованность ($h(u) \leq w(u, v)$

```

```

<h3 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Дейк
по g(v) (уже
(v) = g(v) + h(v), где $h(v)$ – оценка
а требования: допустимость ($h(v) \leq$ и
монотонность/согласованность ($h(u) \leq w(u, v)$

```



```

dist, v = heapq.heappop(qf)
if dist > df.get(v, INF): continue
for to, w in g[v]:
 nd = dist + w
 if nd < df.get(to, INF):
 df[to] = nd; heapq.heappush(qf, (nd, to))
 if to in db:
 mu = min(mu, nd + db[to]) # встреча:
dist, v = heapq.heappop(qb)
if dist > db.get(v, INF): continue
for to, w in rg[v]:
 nd = dist + w
 if nd < db.get(to, INF):
 db[to] = nd; heapq.heappush(qb, (nd, to))
 if to in df:
 mu = min(mu, nd + df[to])
return mu

--- A*: приоритет f = g + h ---
def astar(g, s, t, h): # h(v) – допустимая оценка
 INF = float("inf")
 gdist = {s: 0}
 heap = [(h(s), 0, s)] # (f, g, v)
 closed = set()
 while heap:
 f, gv, v = heapq.heappop(heap)
 if v == t:
 return gv # ранний выход:
 if v in closed:
 continue
 closed.add(v)
 for to, w in g[v]:
 nd = gv + w
 if nd < gdist.get(to, INF):
 gdist[to] = nd
 heapq.heappush(heap, (nd + h(to), nd, to))
 return INF

```

```
</section>
```

```
 },
```

```
];
```

```
ФАЙЛ 28/110: src/data/tickets/ticket18_20_mst.ts
```

```
КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 224 | РАЗМ
```

```
import { Chapter } from '../types';
```

```
/**
```

```
 * Билеты 18 · 19 · 20 — одна страница на три алгоритма
```

```
 * минимального остовного дерева (MST). Так же, как в с
```

```
 * KruskalSimulator, три алгоритма живут на одном графе
```

```
 * переключаются вкладками «Краскал / Прима / Борувка».
```

```
 *
```

```
 * Синхронизация с компилятором: PAGE_SYNC["mst"] + под
```

```
 * "mst#kruskal", "mst#prim", "mst#boruvka" (вкладка со
```

```
 * через emitVizDemo, код в панели следует за ней).
```

```
 */
```

```
export const tickets18to20Mst: Chapter[] = [
```

```
 {
```

```
 id: "mst",
```

```
 title: "18–20. Графы. Остовное дерево: Краскал, При
```

```
 type: "html",
```

```
 description:
```

```
 "Три способа построить минимальное остовное дерево
```

```
 .",
```

```
 category: "Графы. Остовные",
```

```
 content: `
```

```
<section id="mst" class="mb-12 scroll-mt-10">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
 <span class="bg-blue-600 text-white px-4 py-1 r
```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-r
```

```
 </div>
```

```

 <td class="py-2 pr-3 font-b
 <td class="py-2 pr-3">Пастёр
 <td class="py-2 pr-3 font-m
 <td class="py-2 pr-3 font-m
 /td>
 <td class="py-2">плотный гр
 </tr>
 <tr class="align-top">
 <td class="py-2 pr-3 font-b
 <td class="py-2 pr-3">Все к
 <td class="py-2 pr-3 font-m
 <td class="py-2 pr-3 font-m
 <td class="py-2">параллельн
 </tr>
</tbody>
</table>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-3
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
der border-indigo-500 shadow-md">

```

Аналогия к билету 18 • Краск

```

 </div>
 <p class="text-slate-300 text-sm">С
» . По всей карте растут независимые кусочки
дерево. Порядок рёбер важен, порядок верши
</div>

```

```

 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">

```

★ Аналогия к билету 19 • Прим:

```

 </div>
 <p class="text-slate-300 text-sm">П
а захватывает ближайший непосещённый город.
ва» против «снаружи» .</p>
</div>

```

```
<summary class="p-4 font-bold text-slate-300 border-slate-700/50">
```

Код 18: Краскал + DSU (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300"
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
 lt;pair<int, pair<int,int>>>>
vector<int> p, rnk;
```

```
int find(int v) { return p[v] == v ? v : p[v] = find(p[v]); }
```

```
bool unite(int a, int b) {
 a = find(a); b = find(b);
 if (a == b) return false; // уже в одной компоненте
 if (rnk[a] < rnk[b]) swap(a, b);
 p[b] = a;
 if (rnk[a] == rnk[b]) rnk[a]++;
 return true;
}
```

```
sort(edges.begin(), edges.end()); // O(E log E)
long long cost = 0; int taken = 0;
for (auto [w, uv] : edges) {
 if (unite(uv.first, uv.second)) { // не замыкает цикл
 cost += w;
 if (++taken == n - 1) break; // V - 1 ребро
 }
}
```

```
</pre>
```

```
</div>
```

```
</details>
```

```
<details class="bg-slate-900/40 rounded-lg"
```

```
<summary class="p-4 font-bold text-slate-300 border-slate-700/50">
```

Код 19: Прим с кучей (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300"
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
```

```
 heapq
```



```
comps = n
total = 0
while comps > 1:
 best = {} # компонента
 for w, u, v in edges:
 cu, cv = find(u), find(v)
 if cu == cv:
 continue # уже внутри
 if cu not in best or w < best[cu][0]:
 best[cu] = (w, u, v)
 if cv not in best or w < best[cv][0]:
 best[cv] = (w, u, v)
 if not best:
 break # граф несвя
 for w, u, v in best.values():
 if find(u) == find(v):
 continue # дубль: два
 parent[find(u)] = find(v)
 comps -= 1
 total += w
print(total)</pre>
```

<p class="text-slate-400 text-xs">3.  
ну), значит фаз не больше  $\log_2 V$ .</p>

</div>  
</details>

<details class="bg-slate-900/40 rounded-lg l  
<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">

Вопросы, которые любят задавать (C  
</summary>

<div class="p-5 text-sm text-slate-300  
<p>• Почему жадность здесь работает  
меном ребра в предполагаемом MST.</p>

<p>• Чем Прим отличается от Дейкстры  
ного веса против дерева кратчайших путей).</p>

<p>• Какой алгоритм лучше на плотном

```
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
сом, оканчивающимся в позиции i.</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb">
 Мы идём по строке слева направо. К
строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слов
x</code>. Тупой алгоритм начнет искать заново
cab</code> суффикс <code>ab</code> – это ро
буквы шаблона». Это поиск за $O(N)$ без возвр
 </p>
 </div>
```

```
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb">
 Для LLM, которая стримит токены
каждом шаге. Если мы частично совпали с зап
акого места этого слова мы можем продолжить
 </p>
 </div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate
```

```

<h3 class="text-lg font-bold text-white mt-4">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">
 p>
</div>
<p class="text-slate-300 text-sm mb-4">Префикс
 для шаблона (pattern) один раз, затем идём
 пало – $j += 1$. Не совпало – откатываемся на
 у заново сравнивать его не нужно. Когда j до
 ждения), делаем $j = \pi[j - 1]$.</p>
<p class="text-slate-300 text-sm mb-4">П
 ждый откат j уменьшает. Значит суммарно отк
 классическая амортизационная оценка, её люб
<div class="bg-slate-900/40 rounded-lg border"
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
 льшего собственного префикса, равного суффи
 <p class="text-slate-400 text-sm mb-1">
 ый алгоритм.</p>
 <p class="text-slate-400 text-sm mb-1">
 т онлайн (символы могут приходить потоком),
 <p class="text-slate-400 text-sm">• P
 5] = 4, период 6 – 4 = 2 («ab»)</p>
</div>
<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код: поиск подстроки КМР (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded"
fix_function(s):
n = len(s)
pi = [0] * n
for i in range(1, n):
 j = pi[i - 1]
 while j > 0 and s[i] != s[j]:

```

```

type: "html",
category: "Строки",
content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ca 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Тут мы берём всю строку и «приклеива
 я начну читать строку отсюда, насколько длинная
 Это самый быстрый способ найти индекс
 >, считаешь Z-функцию, и там, где <code>Z[i]</code>
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Аналогия 2: LLM Самокопираст
 </div>

```

```

 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}

```

```

 </div>

```

```

 </details>

```

```

 </div>

```

```

 </div>

```

```

</section>`

```

```

 },

```

```

 {

```

```

 id: "aho-corasick",

```

```

 title: "23. Алгоритм Ахо-Корасик",

```

```

 type: "html",

```

```

 category: "Строки",

```

```

 content: `

```

```

<section id="aho-corasick" class="mb-12 scroll-mt-10">

```

```

 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```



```

```

 <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

 </div>

```

```

<div class="space-y-8">

```

```

 <div class="bg-slate-700/50 p-6 rounded-xl border-l">

```

```

 <h3 class="text-xl font-bold text-blue-400 mb-4">

```

```

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text">

```

```

 <p class="text-sm text-blue-300 italic mb-2">Осн

```

```

 <p class="text-lg font-mono text-white">Бор (Тр

```

```

 </div>

```

```

 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>

```

```

 </div>

```

```

<!-- Аналогии -->

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">

```

```

 <div class="bg-slate-800 p-6 rounded-lg border bo

```

```

 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">

```

```

 Аналогия 1: Паутина (Бор)

```

```

 </details>
 </div>
</section>`
},
{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description:
 "P, NP, co-NP, NP-трудные и NP-полные, PSPACE и E. Если ли задача NP-полна.",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 24
 <h2 class="text-2xl sm:text-3xl font-bold text-white">Классы сложности</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Если A сводится к B, то решение задачи A можно найти, решив задачу B.</p>
 <p class="text-xl font-mono text-white">Пример: Если A сводится к B, то решение задачи A можно найти, решив задачу B.</p>
 </div>
 <p class="text-slate-300 text-sm mb-4">Класс сложности (например, машина Тьюринга) с выбранным бюджетом ресурсов (например, время или память) для решения задачи. Мы хотим доказать, что задача A сводится к задаче B, а не про «в среднем».</p>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P против NP)
 </div>

```



```

 Код: верификатор сертификата и пр
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
 ьтонов цикл NP-полон: сертификат = порядок
def verify_hamilton(n, edges, cert):
 if sorted(cert) != list(range(n)): # каждая
 return False
 allowed = {(u, v) for u, v in edges} | {(v, u) for
 for i in range(n):
 a, b = cert[i], cert[(i + 1) % n] # замык
 if (a, b) not in allowed:
 return False
 return True

3-SAT ≤p КЛИКА: формула с k дизъюнктов → граф из 3k в
def reduce_3sat_to_clique(clauses):
 # вершина = (номер дизъюнкта, литерал); ребро, если
 verts = [(ci, lit) for ci, cl in enumerate(clauses)
 edges = []
 for i in range(len(verts)):
 for j in range(i + 1, len(verts)):
 ci, li = verts[i]; cj, lj = verts[j]
 if ci == cj: # из од
 continue
 if li.lstrip("-") == lj.lstrip("-") and (li
 continue # x и n
 edges.append((i, j))
 return verts, edges, len(clauses) # клика

Проверка задачи из P для контраста: двудольность = 2-р
def is_bipartite(n, graph):
 from collections import deque
 col = [-1] * n
 for s in range(n):
 if col[s] != -1:
 continue
 col[s] = 0

```



```
];
```

ФАЙЛ 30/110: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 594 | РАЗМ

```
import { Chapter } from "../../types";
```

```
export const tickets6to13: Chapter[] = [
```

```
{
```

```
 id: "graph-dfs-bfs",
```

```
 title: "6. Графы. Представление графов, DFS, BFS",
```

```
 type: "html",
```

```
 description: "Представление графов и базовые обходы
```

```
 category: "Графы. База",
```

```
 content: `
```

```
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```

```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">
```

```
 </div>
```

```
 <div class="space-y-8">
```

```
 { /* База представлений */ }
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
 <h3 class="text-xl font-bold text-slate-300">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-slate-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 (V + E).</p>
```

```
 </div>
```

```
 <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
 <div class="bg-slate-800 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4 border"
```

```
 border-emerald-500 shadow-md">
```

```
 Аналогия: Матрица = Таблица
```

```
 </div>
```

```

 <!-- Аналогия BFS -->
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 BFS (Поиск в ширину) = Волна
 </div>
 <p class="text-slate-300 text-sm mb-0">
 Что делает: Концентрические волны от источника.
 Работает через Очередь (FIFO). Продвигается в 4 стороны.
 </p>
 <ul class="text-xs text-emerald-300 list-disc list-inside">
 Где используется:
 Кратчайший путь в Невзвешенном графе
 Алгоритм Ахо-Корасик (сборка слов)
 Алгоритм Форда-Фалкерсона / Поиск в ширину
 Двунаправленный поиск для ускорения

 </div>
</div>

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #008080; border-radius: 10px 10px 0px 0px; box-shadow: 5px 5px 0px #008080;">
 Код обходов C++ (Основа)
 </summary>
 <div class="grid grid-cols-1 md:grid-cols-2" style="border: 1px solid #008080; border-radius: 10px 10px 10px 10px; box-shadow: 5px 5px 0px #008080;">
 <pre class="bg-slate-950 p-3 rounded" style="border: 1px solid #008080; border-radius: 10px 10px 10px 10px; box-shadow: 5px 5px 0px #008080;">
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
 vis[v] = true;
 for (int u : adj[v]) {
 if (!vis[u]) dfs(u);
 }
}</pre>

```

```

 <p class="text-xl font-mono text-white">
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 r
 rder border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего 4
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description:
 "Компоненты связности: обходами DFS/BFS за $O(V + E)$ ",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500/50">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 рой вершин есть путь. Внешний цикл «для каж

```

```

 <th class="py-2 pr-3 font-b
 <th class="py-2 pr-3 font-b
 <th class="py-2 font-bold">
 </tr>
</thead>
<tbody class="text-slate-300">
 <tr class="border-b border-slate
/td><td class="py-2 pr-3">забыли про стрелки
у графу</td><td class="py-2 font-mono">0(V
 <tr class="border-b border-slate
 (SCC)</td><td class="py-2 pr-3">из любой в
><td class="py-2 font-mono">0(V + E)</td></
 <tr class="align-top"><td class
r-3">до кого дойдёт старт</td><td class="py
V + E)</td></tr>
 </tbody>
</table>
</div>
<p class="text-slate-400 text-xs mt-3">Клас
ультат SCC. Обход из s даёт только вершины,

<h3 class="text-lg font-bold text-white mt-1
<p class="text-slate-300 text-sm mb-3">Пров
разметка связанных областей на изображении (п
чёт «островов» в матрице 0/1, поиск частей
влении рёбер (DSU), а в обратной стороне –
тном порядке). Мосты (билет 11) и точки соч
.</p>

<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
«слабой» связности орграфа), вторые – толь
 <p class="text-slate-400 text-sm mb-1">
е-реверс времени или в динамическую связнос
 <p class="text-slate-400 text-sm mb-1">
явный стек или увеличение лимита рекурсии.<
 <p class="text-slate-400 text-sm">•

```

```

 comp[start] = color
 dq = deque([start])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if not used[to]:
 used[to] = True
 comp[to] = color
 dq.append(to)

3) DSU: компоненты при динамическом добавлении рёбер
parent = list(range(n))
rank = [0] * n
comps = n # сколько компонент
def find(v):
 while parent[v] != v:
 parent[v] = parent[parent[v]] # сжатие пути
 v = parent[v]
 return v
def union(a, b):
 global comps
 ra, rb = find(a), find(b)
 if ra == rb:
 return False # уже в одной компоненте
 if rank[ra] < rank[rb]:
 ra, rb = rb, ra
 parent[rb] = ra
 if rank[ra] == rank[rb]:
 rank[ra] += 1
 comps -= 1
 return True

for u, v in edges:
 union(u, v)
print("компонент после всех рёбер:", comps)

```

<p class="text-slate-400 text-xs">Т  
 n вернул True.</p>
 </div>

## Ограничения: Циклы

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 получить опыт, нужна работа". Алгоритм выявляет циклы.
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Душный мод: Код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-indigo-400 font-bold">
 <div class="bg-slate-950 p-4 rounded">
 <pre class="text-xs font-mono">
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
 visited[v] = true;
 for (int to : adj[v]) {
 if (!visited[to]) {
 dfs(to);
 }
 }
 ans.push_back(v); // Добавляем в момент ВЫХОДА (post-order)
}

void topological_sort(int n) {
 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
 reverse(ans.begin(), ans.end()); // Разворачиваем с
}
 </pre>
 </div>
 </div>
 </div>

```

</ul>

```
<div class="grid grid-cols-1 xl:grid-cols-2" data-bbox="267 143 1000 235">
 <div class="bg-slate-800 p-6 rounded-lg" data-bbox="339 169 1000 235">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md" data-bbox="405 195 1000 235">
```

Аналогия: Интриги в офисе

</div>

<p class="text-slate-300 text-sm mb" data-bbox="401 285 1000 307">

Сильная связность – это "клуб" с  
ни в одной SCC. Добавить курьера Гошу, который  
йти наружу (к директору), но обратно в клуб

</p>

</div>

```
 <div class="bg-slate-900 p-6 rounded-lg" data-bbox="339 451 1000 520">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md" data-bbox="405 477 1000 520">
```

Связь с Билетом 12

</div>

<p class="text-slate-300 text-sm mb" data-bbox="401 569 1000 592">

<b>SCC (Билет 10)</b> склеивает

</br><br>

<b>Точки сочленения (Билет 12)</b>  
ость</strong> монолита. Вырвешь точку сочле

</p>

</div>

</div>

```
<details class="bg-slate-900/40 rounded-lg" data-bbox="267 783 1000 853">
 <summary class="p-4 font-bold text-slate-300" data-bbox="339 809 1000 853">
 -b border-slate-700/50">
```

Душный мод: Код Алгоритм Косарайю

</summary>

<div class="p-5 text-sm text-slate-300" data-bbox="334 902 1000 925">

<p class="text-emerald-400 font-bol" data-bbox="401 926 1000 948">

<ol class="list-decimal list-inside" data-bbox="401 950 1000 972">

<li>Запускаем DFS на исходном графе





```

 <p class="text-blue-400 font-bold">Особ
 <p>
 Для стартовой вершины обхода правил
 то выше неё прыгать некуда). Поэтому для не
 у него больше 1 независимого ребенка
 </p>
 <div class="bg-slate-950 p-4 rounded-lg
 <pre class="text-xs font-mono text-
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
 if (visited[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}</pre>
 </div>
</div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl bor
 <h4 class="text-lg font-bold text-indigo-40
 <p class="text-slate-300 text-sm mb-4">
 Это глубокий дуальный мост между ориент
 </p>
 <ul class="list-disc list-inside text-sm te

```

```
 </div>
 <p class="text-slate-300 text-sm mb-2">
 е сходится нечетное число линий, значит, из
 а везде должно быть четное число (зашел-вышел)
 </div>
 </div>
</div>
</section>`,
 },
];
```

---

## РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

---

---

ФАЙЛ 31/110: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from "../data/vizState";
import { Play, Pause, RotateCcw, Info } from "lucide-react";

interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface Edge {
 u: number;
 v: number;
}
```

```

}

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 useVizStepSync(currentStepIndex, setCurrentStepIndex,
 const v = vizNumber(vars.v);
 if (v === null) return null;
 return steps.map((step, index) => ({ step, index }
));
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);

 const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
 };

 recordStep("Начало DFS (Тарьян) для поиска точек со

 const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

```

```

 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он
 v,
);
 }
};

for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
}

recordStep("Алгоритм завершен. Найдены все точки со
setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
};

useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length -
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

```

```
</div>
```

```
<div className="grid grid-cols-1 lg:grid-cols-3 g
```

```
 <div className="lg:col-span-2 relative h-96 lg:h-
```

```
 <svg className="w-full h-full" viewBox="0 0 8
```

```
 {/* Edges */}
```

```
 {edges.map((edge, i) => {
```

```
 const u = nodes.find((n) => n.id === edge
```

```
 const v = nodes.find((n) => n.id === edge
```

```
 const isBridge =
```

```
 (edge.u === 3 && edge.v === 4) ||
```

```
 (edge.u === 4 && edge.v === 3);
```

```
 return (
```

```
 <line
```

```
 key={"edge-" + i}
```

```
 x1={u.x}
```

```
 y1={u.y}
```

```
 x2={v.x}
```

```
 y2={v.y}
```

```
 stroke={isBridge ? "#f43f5e" : "#475568"}
```

```
 strokeWidth={isBridge ? "4" : "2"}
```

```
 strokeDasharray={isBridge ? "8,4" : ""}
```

```
 />
```

```
);
```

```
])}>
```

```
{/* Nodes */}
```

```
{nodes.map((node) => {
```

```
 const isCurrent = currentStep.currentNode
```

```
 const isVisited = currentStep.visited.has
```

```
 const isAp = currentStep.ap.has(node.id);
```

```
 const fill = isCurrent
```

```
 ? "#3b82f6"
```

```
 : isAp
```

```
 ? "#e11d48"
```

```

 <text
 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
)}
 </g>
);
}}
</svg>
</div>

```

```

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

```

```

<div className="bg-slate-900 border border-sl

```

```

 xt-slate-500"}` }
 >
 {step.desc}
 </div>
)}}
 </div>
</div>
</div>
<div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 rounded">
 <div
 className="bg-rose-500 h-full transition"
 style={{
 width: `${currentStepIndex / Math.max(
 1, steps.length - 1
)} * 100%`
 }}
 />
 </div>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 32/110: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from 'viz';
import { LiveDistTable } from '../LiveDistTable';

```

```

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
}

```

```

 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
};

const adjList = getAdjList(edges);

const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start)' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0;' },
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1' },
 { line: 4, text: ' for (u, v, weight) in E:' },
 { line: 5, text: ' if dist[u] + weight < dist[v]' },
 { line: 6, text: ' dist[v] = dist[u] + weight;' },
 { line: 7, text: ' for (u, v, weight) in E: // Проверка на отрицательный цикл' },
 { line: 8, text: ' if dist[u] + weight < dist[v]' },
 { line: 9, text: ' return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"' },
 { line: 10, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 99 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({

```



```

 }
 }

 if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.weight < dist[edge.v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `    ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}`,
 activeCodeLine: 9,
 });
 break;
 }
 }

 if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: `    Проверка завершена. Ни одно ребро не содержит цикла`,
 activeCodeLine: 10,
 });
 }
 }

 setSteps(simulationSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
}, [isPlaying, currentStepIndex, steps.length]);

```

```

<div className="bg-slate-900 p-1 rounded-lg b
 <button onClick={() => setHasCycle(false)}
 bg-blue-600 text-white' : 'text-slate-400'}
 <button onClick={() => setHasCycle(true)} c
 r gap-1 ${hasCycle ? 'bg-rose-600 text-whit
</div>
<button onClick={() => { setIsPlaying(false);
 TEXT : "Вернуться к первому шагу"} className
 bled:cursor-not-allowed text-white px-3 py-1
<button onClick={() => setIsPlaying(!isPlaying
 нную демонстрацию"} className={`flex items-
 sabled:opacity-40 disabled:cursor-not-allow
 }</button>
<button onClick={() => { setIsPlaying(false);
 ; }} disabled={compilerLinked} title={compil
 gap-1 bg-blue-600 hover:bg-blue-500 disable
 ansition-colors">⚡</button>
</div>
</div>

<LiveDistTable
 dist={liveDist}
 demoIds={nodes.map((node) => node.id)}
 title="Ваши расстояния из кода (Форд–Беллман)"
/>

<div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-blue-950/20
 y-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-blue
 ld shadow-sm">
 Итерация {currentStep.iteration} (⚡ {curr
 </div>

 {currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-r
 animate-pulse shadow-lg shadow-rose-600/50

```

```

 });
 }}}

 {nodes.map((node) => {
 const dist = currentStep.distances[node.id];
 const isCheckingNode = currentStep.checkingNode ===
 node.id);

 return (
 <g key={node.id} className="transition-...
 <circle cx={node.x} cy={node.y} r={isC
 ckingNode ? '#7dd3fc' : '#64748b'} strokeWid
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y - 28} fill=
 ight="bold" textAnchor="middle" className="
 <text x={node.x} y={node.y + 35} fill=
 split(' ')[0]}</text>
 </g>
);
 }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400 m
 <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap g
 -colors">
 <span className="font-bold text-white
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingE

```

```

 <h4 className="text-lg font-bold text-slate-600">
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5 px-4 ${
 item.line === 1 ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'
 }`} >
 {item.line}
 {item.code}
 </div>
 })}
 </div>
 </div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 33/110: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useCallback } from 'react';
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';

```

```

const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];

```

```

const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },

```

```
});
```

```
while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}`
 });
```

```
const curr = queue.shift()!;
steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
});
```

```
const neighbors = ADJ[curr];
if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не`
 });
```

```
 visited.add(n);
 queue.push(n);
```

```
 steps.push({
 activeLine: 9,
 queue: [...queue],
```

```

 }, []));

 // @ts-expect-error зарезервировано под кнопку «назад»
 const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
 }, []);

 useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

 return (
 <div className="bg-slate-900 border-2 border-slate-700 p-4">
 <div className="flex flex-col md:flex-row items-center justify-between">
 <div>
 <h3 className="text-2xl font-bold text-white">
 Анонс Албума
 </h3>
 <p className="text-slate-400 text-sm mt-1">Видео-презентация
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded">
 <button
 onClick={() => { setIsPlaying(false); setStepIdx(0); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-blue-300"
 title={isPlaying ? 'Пауза' : 'Воспроизведение'}
 >
 <RotateCw className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>
);
 }
}

```

```

const isInQueue = step.queue.includes(n.id)
const isVisited = step.visited.includes(n.id)

let fill = '#1e293b'; // slate-800
let stroke = '#475569'; // slate-600
let scale = 1;
void scale;

if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
} else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
} else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
}

return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors duration
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"

```

```

 });
 })}
 </div>
 </div>

 <div className="bg-slate-800/80 border border-
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? {
 <span className="text-slate-500 text-x
 } : {
 step.queue.map(({qId, i}) => {
 <span key={`${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
 {qId}

 })
 }}
 </div>
 <div className="mt-2 text-sm text-amber-30
ms-center justify-center">
 "{step.logs}"
 </div>
 </div>
</div>
</div>
</div>
);
}

```



```
 <p className={`text-center text-xs mt-2 font-bold`} >
 </div>
);
}
```

ФАЙЛ 35/110: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```
interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}
```

```
/**
```

```
 * Переходы «предыдущая / следующая тема» — как на Codecademy
```

```
 * Компактный вариант живёт в шапке главы, крупный — по центру
```

```
 */
```

```
export const ChapterNav: React.FC<Props> = ({ prev, next, index, total, onGo, compact }) => {
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1.5 rounded-md
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
 <ChevronLeft />
 {prev ? prev.title : "Это первая тема"}
 </button>
 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1.5 rounded-md
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
 {next ? next.title : "Это последняя тема"}
 <ChevronRight />
 </button>
 </div>
);
 } else {
 return (
 <div className="flex items-center gap-2 text-sm">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-3 py-2 rounded-md
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
 <ChevronLeft />
 {prev ? prev.title : "Это первая тема"}
 </button>
 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-3 py-2 rounded-md
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
 {next ? next.title : "Это последняя тема"}
 <ChevronRight />
 </button>
 </div>
);
 }
};
```

```
 {next && (
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg-
 transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end
 text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3
 </div>
 <div className="text-sm font-bold text-slate-
 </button>
)}
 </nav>
);
};
```

---

ФАЙЛ 36/110: src/components/ChapterQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { ArrowRight, CheckCircle2, RotateCcw, XCircle, }
import { quizzes } from "../data/quizzes";
```

```
interface Props {
 chapterId: string;
}
```

```
/**
```

- \* Блиц по прочитанной теме: 3–5 коротких вопросов с не
- \*
- \* Вопросы живут в `src/data/quizzes.ts` (по id главы).
- \* компонент-викторина существовали, но не были подклю
- \* контент был, а добраться до него было нельзя. Теперь

```

 }
 };

 const restart = () => {
 setIdx(0);
 setPicked(null);
 setScore(0);
 setFinished(false);
 };

 return (
 <section aria-label="Блиц по теме" className="mt-8">
 <div className="bg-slate-700/50 p-5 rounded-xl border">
 <div className="flex items-center justify-between">
 <h3 className="flex items-center gap-2 text-brown-300">
 <Zap className="w-4 h-4 text-indigo-400" />
 Блиц: {list.length} вопроса по этой теме
 </h3>

 {finished ? `итог ${score}/${list.length}` : ''}

 </div>

 {finished ? (
 <div className="text-center py-3">
 <p className="text-sm text-slate-300 mb-1">
 {score === list.length
 ? "Все верно: тему можно закрывать."
 : score * 2 >= list.length
 ? "Половина и больше верно – стоит пересмотреть."
 : "Стоит вернуться к тексту страницы:"}
 </p>
 <p className="text-2xl font-bold text-white">
 {score} / {list.length}
 </p>
 <button
 type="button"
 onClick={restart}
 >

```

```

 <div role="status" className="mt-3 rounded
 <p className={`text-xs font-bold mb-1 $
 {correct ? "Верно" : "Неверно"}
 </p>
 <p className="text-[13px] leading-relax
 </div>
)}

 <div className="mt-3 flex justify-end">
 <button
 type="button"
 onClick={next}
 disabled={!answered}
 className="inline-flex items-center gap
nt-bold text-indigo-200 transition-colors h
-500/15"
 >
 {idx + 1 < list.length ? "Следующий воп
 <ArrowRight className="w-3.5 h-3.5" />
 </button>
 </div>
 </>
)}
</div>
</section>
);
};

export default ChapterQuiz;

```

ФАЙЛ 37/110: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useCallback, useEffect, useRef, useState
import { Check, Download, FileText, Loader2, Maximize2,

```

```

const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
}, []);

const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(
), [cancelHide]);

useEffect(() => {
 setAnchor(null);
 setSaving("idle");
}, [chapter.id]);

/**
 * Печать/«сохранить в PDF» этой темы: раскрываем все
 * чтобы на бумагу попал весь текст (доказательства,
 * а после печати возвращаем их в исходное состояние.
 */
useEffect(() => {
 const each = () => Array.from(contentRef.current?.q
 const openAll = () => {
 each().forEach((d) => {
 d.dataset.wasOpen = d.open ? "1" : "0";
 d.open = true;
 });
 };
 const restore = () => {
 each().forEach((d) => {
 if (d.dataset.wasOpen === "0") d.open = false;
 delete d.dataset.wasOpen;
 });
 };
 window.addEventListener("beforeprint", openAll);
 window.addEventListener("afterprint", restore);

```

```

const handleClick = (e: React.MouseEvent) => {
 // ссылка на другой билет: переходим внутри приложения
 const link = (e.target as HTMLElement).closest<HTMLAnchorElement>('a')
 if (link) {
 e.preventDefault();
 onGo(link.dataset.goto ?? "");
 return;
 }
 const el = (e.target as HTMLElement).closest<HTMLAnchorElement>('a')
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLAnchorElement>('a')
 if (el) open(el);
};

const viz = getViz(chapter.id);

return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-sm font-medium border border-emerald-500 rounded-md px-3 py-2"
 >
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (

```

```
<p className="mt-6 flex items-start gap-2 text-
ed-lg px-3 py-2 print:hidden">
 <MousePointerClick className="w-4 h-4 shrink-0" />

 Подчёркнутые термины – интерактивные: наведи
 объяснение на пальцах, мини-анимацию и кноп

</p>
```

```
{viz && {
 <section id="chapter-viz" className="mt-8 scr
 <div className="flex items-center justify-b
 <h3 className="flex items-center gap-2 te
 <Sparkles className="w-4 h-4 text-indigo
 Демонстрация: {viz.title}
 </h3>
 <div className="flex items-center gap-2">
 <button
 onClick={() => onOpenSimulator(chapte
 className="flex items-center gap-1.5
 slate-300 hover:text-white hover:border-ind
 >
 <Maximize2 className="w-3.5 h-3.5" />
 </button>
 {onOpenCompiler && {
 <button
 onClick={onOpenCompiler}
 title="Открыть Python: код выполняе
рации"
 className="flex items-center gap-1.5
 t-emerald-400 hover:text-white hover:border
 >
 <Terminal className="w-3.5 h-3.5" />
 </button>
 }}
 </div>
 </div>
```

```

/**
 * Билет 24 – «Классы сложности, сведение задач».
 *
 * Четыре вкладки:
 * 1. «Иерархия» – $P \subseteq NP \subseteq PSPACE \subseteq EXP$ и NP-полные
 * 2. «Сведение» – $A \leq_p B$ по шагам: преобразовал вход
 * 3. «Верификатор» – проверка сертификата для 3-SAT по
 * 4. «Рост» – живой график по переменным n и ops
 * сколько операций насчитал твой алгоритм
 * похоже (полином или экспонента).
 *
 * Синхронизация с компилятором (PAGE_SYNC["complexity-
 * cls – имя класса (подсветит нужный овал), step/i –
 * k – номер проверяемого дизъюнкта, cert/assign – сер
 * n и ops – точки на графике роста, ok/ans – вердикт
 */

```

```

type Tab = "hierarchy" | "reduce" | "verify" | "growth"

```

```

const TABS: { id: Tab; label: string; icon: React.ReactElement } = [
 { id: "hierarchy", label: "1 · Иерархия классов", icon: },
 { id: "reduce", label: "2 · Сведение A ≤p B", icon: },
 { id: "verify", label: "3 · Верификатор (NP)", icon: },
 { id: "growth", label: "4 · Рост по n и ops", icon: },
];

```

```

/* ----- Иерархия -----

```

```

const CLASSES = [
 { id: "EXP", label: "EXP", sub: "2^(n^k)", color: "#0000FF",
 (кванторные формулы)" } },
 { id: "PSPACE", label: "PSPACE", sub: "полиномиальная
 с полной информацией" } },
 { id: "NP", label: "NP", sub: "сертификат проверяется
 AT", "гамильтонов цикл", "раскраска в 3 цвета", "кли
 { id: "P", label: "P", sub: "решается за O(n^k)", color: "#00FF00",
 йд", "MST", "2-SAT, двудольность" } },
];

```



```

 [2, -3, -4],
 [-2, 1, 3],
];
 /** Сертификат, который проверяем: $x = (1, 0, 1, 0)$ — ok
 const CERT = [1, 0, 1, 0];

 const VERIFY_STEPS = CLAUSES.map((cl, i) => ({
 title: `Дизъюнкт $\{i + 1\}$: $\{ \{cl.map((l) => (l > 0 ?$
 note: `Подставляем сертификат $x = (\{CERT.join(", ")$
 .map((l) => $(l > 0 ? \neg x\{l\}=\{CERT[l - 1]\}$ ` : $\neg x\{$
 .join(", "). $\{cl.some((l) => (l > 0 ? CERT[l - 1]$
 : "Все литералы ложны — дизъюнкт НЕ выполнен.")`
 ok: $cl.some((l) => (l > 0 ? CERT[l - 1] === 1 : CERT[$
 })));

 /* ----- Пост -----

 interface Sample { n: number; ops: number }

 const GROWTH_LINES = [
 { id: "n", label: "n", fn: (n: number) => n, color: "red" },
 { id: "n log n", label: "n · log2 n", fn: (n: number) => n * Math.log2(n), color: "blue" },
 { id: "n²", label: "n²", fn: (n: number) => n * n, color: "green" },
 { id: "n³", label: "n³", fn: (n: number) => n * n * n, color: "purple" },
 { id: "2n", label: "2ⁿ", fn: (n: number) => Math.pow(2, n), color: "brown" },
];

 export const ComplexityViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const [tab, setTab] = useState<Tab>("hierarchy");
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [samples, setSamples] = useState<Sample[]>([]);

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);
 }

```

```

 if (liveN === null || liveOps === null) return;
 if (!Number.isFinite(liveN) || !Number.isFinite(liveOps)) return;
 setSamples((prev) => {
 const last = prev[prev.length - 1];
 if (last && last.n === liveN && last.ops === liveOps) return;
 const withoutN = prev.filter((s) => s.n !== liveN);
 return [...withoutN, { n: liveN, ops: liveOps }];
 });
 }, [liveN, liveOps]);

/** Оценка показателя роста по последним точкам (лог-лог)
const verdict = useMemo(() => {
 const pts = samples.filter((s) => s.ops > 0 && s.n > 0);
 if (pts.length < 3) return null;
 const xs = pts.map((p) => Math.log(p.n));
 const ys = pts.map((p) => Math.log(p.ops));
 const mx = xs.reduce((a, b) => a + b, 0) / xs.length;
 const my = ys.reduce((a, b) => a + b, 0) / ys.length;
 let num = 0;
 let den = 0;
 for (let i = 0; i < xs.length; i++) {
 num += (xs[i] - mx) * (ys[i] - my);
 den += (xs[i] - mx) ** 2;
 }
 const slope = den === 0 ? 0 : num / den;
 // экспонента на лог-лог растёт неограниченно: сравнение
 const half = Math.floor(pts.length / 2);
 const slopeTail = (() => {
 const q = pts.slice(half);
 if (q.length < 2) return slope;
 const lx = q.map((p) => Math.log(p.n));
 const ly = q.map((p) => Math.log(p.ops));
 const ax = lx.reduce((a, b) => a + b, 0) / lx.length;
 const ay = ly.reduce((a, b) => a + b, 0) / ly.length;
 let nn = 0;
 let dd = 0;
 for (let i = 0; i < lx.length; i++) {
 nn += (lx[i] - ax) * (ly[i] - ay);
 dd += (lx[i] - ax) ** 2;
 }
 return dd === 0 ? 0 : nn / dd;
 })();
 return slope > 0 && slopeTail > 0 ? 'рост' : 'стабильно';
});

```

```

<div className="flex flex-wrap gap-2 mb-4">
 {TABS.map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={"flex items-center gap-1.5 px-3 p-1.5 rounded-lg bg-slate-900 text-white" : "bg-slate-900/60 text-slate-500"}
 >
 {t.icon} {t.label}
 </button>
 })}
</div>

```

```

{ (tab === "reduce" || tab === "verify") && (
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setStep(0); setPlaying(true); }}
 className="p-1.5 rounded-lg bg-slate-900 text-white hover:text-white"><RotateCcw className="w-4 h-4" />
 <button onClick={() => { setPlaying(false); setStep(1); }}
 className="p-1.5 rounded-lg bg-slate-900 border border-slate-700 text-slate-500 hover:bg-slate-800">
 /></button>
 <button onClick={() => setPlaying((p) => !p)}
 className="p-1.5 rounded-lg bg-slate-900 border border-slate-700 text-slate-500 font-bold hover:bg-purple-500">
 {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 </button>
 <button onClick={() => { setPlaying(false); setStep(2); }}
 className="p-1.5 rounded-lg bg-slate-900 border border-slate-700 text-slate-500 font-bold hover:bg-purple-500">
 /></button>

 {tab === "reduce" ? "Reduce" : "Verify"}

 </div>
)}

```

```

{/* ----- ИЕРАРХИЯ ----- */
{tab === "hierarchy" && (
 <div className="space-y-4">
 <div className="bg-slate-950 rounded-xl p-3 border border-slate-800">
 <svg viewBox="0 0 500 380" className="w-full h-380">
 {CLASSES.map((c) => {

```

```

 const meta = BOX_LABEL[b];
 const active = i === reduceFrame.active
 return (
 <React.Fragment key={b + i}>
 <div
 className="rounded-lg border px-3"
 style={{
 borderColor: active ? meta.c :
 background: active ? meta.c + "
 opacity: active ? 1 : 0.45,
 boxShadow: active ? `0 0 18px $
 minWidth: 96,
 }}
 >
 <span className="text-[11px] font
 </div>
 {i < reduceFrame.bboxes.length - 1 &
 </React.Fragment>
);
 }}}
 </div>
 </div>
 <div className="bg-slate-900/70 border border
 <p className="text-sm font-bold text-white
 text-purple-400" />{reduceFrame.title}</p>
 <p className="text-xs text-slate-400 leadin
 </div>
</div>
)}

{/* _____ ВЕРИФИКАТОР _____
{tab === "verify" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <p className="text-[10px] uppercase trackin
 <div className="flex flex-wrap gap-2 mb-3">
 {CERT.map((v, i) => (
 <span key={i} className={"font-mono tex

```

```

 })
 </div>
 <div className="bg-slate-900/70 border border-
 <p className="text-sm font-bold text-white l
 <p className="text-xs text-slate-400 leadin
 определение NP: короткий сертификат + быстр
 </div>
</div>
})

{/* _____ ПОСТ _____
{tab === "growth" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <div className="flex flex-wrap items-center
 <span className="text-[10px] uppercase tr
 <span className="font-mono text-xs bg-sla
 ?? "-">
 <span className="font-mono text-xs bg-sla
 e0ps ?? "-">
 <button onClick={() => setSamples([])} cl
 0 rounded-lg px-2 py-1">очистить график</bu
 </div>
 <svg viewBox="0 0 500 240" className="w-ful
 {/* оси в лог-лог */}
 <line x1={44} y1={200} x2={486} y2={200}
 <line x1={44} y1={14} x2={44} y2={200} st
 <text x={265} y={222} textAnchor="middle"
 <text x={16} y={110} fill="#64748b" fontS
 {GROWTH_LINES.map((g) => {
 const pts: string[] = [];
 for (let i = 0; i <= 20; i++) {
 const n = 1 + i * 0.8;
 const v = g.fn(n);
 const x = 44 + (Math.log(n) / Math.lo
 const y = 200 - (Math.log(Math.max(1,
 pts.push(`${x.toFixed(1)}},${Math.max(
 }
)
)
)
)
}

```

```

 { /* живые переменные */
 <div className="mt-3 flex flex-wrap items-center">

 {vars ? (
 ["n", "ops", "cls", "k", "cert", "ok"].map((name) => {
 vars[name] === undefined ? null : (

 {name} = {JSON.stringify(vars[name]).slice(0, 100)}

)
 })
) : (

 открой панель Python: переменные <code class="font-mono text-slate-300">k</code>, <code className="font-mono text-slate-300">cert</code>
 управляют этой демонстрацией

)}
 </div>
 </div>
);
};

```

```
export default ComplexityViz;
```

---

ФАЙЛ 39/110: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";

interface CodeLine {

```

```

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {

```

```

 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).",
 activeCodeLine: [7,8,9]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) –",
 activeCodeLine: [13,14,15,16,17]
},

```



```

 .filter(({ step }) => step.currentNode === v && (
 return candidates.at(-1)?.index ?? null;
));
const [dfsAuto, setDfsAuto] = useState(false);
const dfsTimer = useRef<NodeJS.Timeout | null>(null);

useEffect(() => {
 if (dfsAuto) {
 dfsTimer.current = setInterval(() => {
 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
 return prev + 1;
 });
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">Шаг {step}
 <div className="flex items-center gap-1 bg-slate-800 p-1">
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
 disabled={dfsIdx === 0}
 aria-label="Шаг назад" title="Шаг назад"
 className="p-1.5 hover:bg-slate-700 rounded text-white">Назад
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px] ${dfsAuto ? "bg-rose-600 text-white" : "bg-slate-800 text-white"}>
 {dfsAuto ? <><Pause className="h-3 w-3" /> : <><Next className="h-3 w-3" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0);

```

```

 className="transition-all duration-500"
 }}}
 {step.currentNode !== null && step.currentNode !== null ? (
 const n = GRAPH_NODES.find(x => x.id === step.currentNode.id);
 if (!n) return null;
 return <circle cx={n.x} cy={n.y} r={18} fill={step.currentNode.fill}
 >;
)}()
 {GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(n.id);
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b" : "#f8d7da"}
 stroke={cur ? "#fff" : st ? "#34d399" : "#000"}
 strokeWidth={2.5}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y+3} textAnchor="middle">
 {n.label}</text>
 {vis && (
 <div
 <rect x={n.x-18} y={n.y-28} width={36} height={56}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor="center">
 {step.low[n.id] || "?"}</text>
 </div>
)}
 </g>;
 })}
</svg>
<div className="flex items-center justify-between">
 Шаг {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-10 rounded" style={style}>
 <div className="bg-indigo-600 h-full transition-all duration-300" style={style}>
 </div>
 </div>
</div>

```

```

 }`}>Вершина {GRAPH_NODES.find(n=>n.
 <span className="text-slate-500 m
 </div>
))
 }
 </div>
</div>

{/* Log */}
<div className="bg-indigo-950/10 border borde
 <p className="text-[11px] text-slate-300 le
 <div className="mt-2 pt-2 border-t border-s
 Мосты:</
 <span className="font-mono font-bold text
 {step.bridges.length > 0 ? step.bridges

 </div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 40/110: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,
import { LiveDistTable } from './LiveDistTable';

interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;

```

```

nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
});

const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' pq = PriorityQueue(); pq.push({v: start, d: 0})' },
 { line: 4, text: ' while not pq.empty():' },
 { line: 5, text: ' d, u = pq.pop() // Извлекаем элемент с минимальным расстоянием' },
 { line: 6, text: ' if d > dist[u]: continue' },
 { line: 7, text: ' for v, weight in graph.neighbours(u):' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' dist[v] = dist[u] + weight' },
 { line: 10, text: ' pq.push({v: v, d: dist[v]})' },
 { line: 11, text: ' return dist // Все кратчайшие пути найдены' },
];

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false, B: false };
 const prev: Record<string, string | null> = { A: null, B: null };

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пикетта',
 activeCodeLine: 2,
 });
});

for (let iter = 0; iter < nodes.length; iter++) {

```

```

 log: ` Улучшение (Релаксация)! Новое кратчайшее
 activeCodeLine: 9,
 });
 }
}
}

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Дейкстра всё доставил! Все возможные кратчайшие
 activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">Зарядка...
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(vars?.done);

```

```

 onClick={() => setIsPlaying(!isPlaying)}
 disabled={compilerLinked}
 title={compilerLinked ? HINT_TEXT : "Проигр
 className={`flex items-center gap-1 px-4 py-2
 disabled:cursor-not-allowed ${
 isPlaying ? 'bg-amber-600 hover:bg-amber-500' : ''
 }`}
 >
 {isPlaying ? 'Пауза' : 'Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); if (c
 disabled={compilerLinked || currentStepIndex
 title={compilerLinked ? HINT_TEXT : "Следую
 className="flex items-center gap-1 bg-indigo-600
 cursor-not-allowed text-white px-3 py-2 round
 >
 ⏏
 </button>
</div>
</div>

<LiveDistTable
 dist={liveDist}
 prev={livePrev}
 done={liveDone}
 demoIds={nodes.map((node) => node.id)}
 title="Ваши расстояния из кода (Дейкстра)"
/>

<div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/20
 justify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indigo-950/20
 font-bold shadow-sm">
 ⏏ {currentStepIndex + 1} из {steps.length}
 </div>
 </div>

```

```

 <circle
 cx={({uNode.x + vNode.x} / 2} cy={({uNode.y + vNode.y} / 2})
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : '#f87192'}
 />
 <text
 x={({uNode.x + vNode.x} / 2} y={({uNode.y + vNode.y} / 2})
 fill={isChecking ? '#f59e0b' : '#f87192'}
 fontSize="11" fontWeight="bold" text={edge.w}
 >
 {edge.w}
 </text>
 </g>
);
 }
}

{/* Вершины */}
{nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited[node.id];
 const dist = currentStep.distances[node.id];

 return (
 <g key={node.id} className="transition-all duration-300">
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVisited ? '#f87192' : '#1e293b'}
 stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f59e0b' : '#f87192'}
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-300"
 />
 <text x={node.x} y={node.y + 5} fill="white" font-weight="bold">
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill="white" font-weight="bold">
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 </g>
);
})}

```

```

 600 text-slate-300'
 }` }>
 {edge.v}<span cl
 <span className={isCheckingTh
an>

);
 }}}
</div>
</div>
);
}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>
 <h4 className="text-lg font-bold text-rose-950">
 <div className="overflow-x-auto">
 <table className="w-full text-left border">
 <thead>
 <tr className="border-b border-rose-950">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред.
 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n]
 const prev = currentStep.previous[n]
 const vis = currentStep.visited[n]
 const isCurr = currentStep.currentN

 return (

```



```

 });
 }}}
 </div>
 </div>
</div>
</div>
</div>
});
}

```

---

ФАЙЛ 41/110: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord } from '...';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from '...';

interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'end';
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];

```

```

 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр
 codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
}

generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibМемо(${n - 1})
 codeLine: 4,
 memoState: { ...memoMap }
});

const res1 = simulateFib(n - 1);
const res2 = simulateFib(n - 2);
memoMap[n] = res1 + res2;

generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: мемо[${n}] = ${res1} +
 codeLine: 5,
 memoState: { ...memoMap }
});

 return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
```

```

 ...baseStep,
 n: liveK ?? baseStep.n,
 memoState,
 desc: `Компилятор: k=${liveK ?? "-"}; таблица m
 }
 : baseStep;

return (
 <div className="bg-slate-900 rounded-2xl border border-
 /* Header & Controls */
 <div className="flex flex-col lg:flex-row lg:item
 <div>
 <div className="flex items-center gap-3 mb-2">
 <div className="p-2 bg-emerald-500/10 borde
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text
 </div>
 <p className="text-sm text-slate-400">
 Смотри, как таблица DP кэширует результаты
 </p>
 </div>

 <div className="flex flex-wrap items-center gap
 <div className="flex items-center gap-2 bg-sl
 N:
 <select
 value={targetN}
 onChange={(e) => setTargetN(Number(e.targ
 disabled={isPlaying}
 className="bg-transparent text-white font
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

```

```
</div>
```

```
/* DP Table View */
```

```
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded" >
```

```
 <h4 className="text-xs font-bold uppercase tracking-wider">
```

```
 <div className="flex flex-wrap items-center gap-2">
```

```
 {Array.from({ length: targetN }, (_, i) => i + 1)}
```

```
 .map(num => {
```

```
 const isCached = num in memoState || num <= 2
```

```
 const val = num <= 2 ? 1 : memoState[num]
```

```
 const isCurrent = currentStep?.n === num
```

```
 return (
```

```
 <div
```

```
 key={num}
```

```
 className={`flex flex-col items-center justify-center`}
```

```
 {isCurrent
```

```
 ? 'bg-emerald-950 border-emerald-500' :
```

```
 isCached
```

```
 ? 'bg-slate-900 border-emerald-500' :
```

```
 'bg-slate-900/50 border-slate-800'}
```

```
 `}
```

```
 >
```

```

```

```
 <span className={`text-xl md:text-2xl font-medium`}
```

```
 {isCached ? val : '0'}
```

```

```

```
 {num <= 2 &&
```

```
 </div>
```

```
)
```

```
 })
```

```
 </div>
```

```
{currentStep && (
```

```
 <div className="mt-6 pt-4 border-t border-slate-800">
```

```
 <div className="text-sm font-medium text-slate-400">
```

```
 <span className="inline-block w-2 h-2 rounded" style={{
```

```
 background-color: {currentStep.desc},
```

```
 </div>
```

```


 </h5>
 <p className="text-sm text-slate-400">
 В классической рекурсии для $N=\{targetN\}$ мы вычисляем значение в таблице, мы момента.
 </p>
 <div className="p-4 bg-slate-900 rounded-xl text-slate-300">
 Всего шагов симуляции: {steps.length}

 </div>
 </div>
)}

 {activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl border">
 <div className="bg-slate-900 px-6 py-3 border">

 </div>
 <pre className="p-6 font-mono text-sm space-x-1">
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine === lineNum;
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4 py-2 ${
 isActive
 ? 'bg-emerald-600/20 border-l-4 border-emerald-400'
 : 'text-slate-300 hover:bg-slate-800'
 }`}
 >
 {lineNum}
 {line}
 </div>
);
 })}
 </pre>
 </div>
);
)}

```

```

 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
 { u: 0, v: 1 }, { u: 0, v: 2 },
 { u: 1, v: 3 }, { u: 2, v: 4 },
 { u: 1, v: 2 },
 { u: 3, v: 4 }
];

const ekey = (a: number, b: number) => `${Math.min(a, b)}-${Math.max(a, b)}`;
const DEG = C1_NODES.map((n) => C1_EDGES.filter((e) => ekey(e.u, e.v) === n.id));
const ODD = C1_NODES.map((n) => n.id).filter((id) => DEG[id] % 2 !== 0);

/**
 * Билет 13 – «Графы. Эйлеров цикл».
 *
 * Две части:
 * 1. игра «пройди все рёбра» кликами (Эйлер не прощает
 * 2. критерий: степени вершин и число нечётных (0 → цикл)
 *
 * Синхронизация с Python-компилятором (PAGE_SYNC["euler"]);
 * path – маршрут (массив вершин) рисуется поверх графа;
 * пройденны, deg/odd – степени и нечётные вершины, last – последняя
 * step – номер шага.
 */
export function EulerSimulator() {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<boolean[]>([]);
 const [c1Msg, setC1Msg] = useState("Клики на вершину");
 const [c1Done, setC1Done] = useState(false);

```

```

 }
 setC1Done(true);
 } else {
 setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $
 }
};

```

/\* — что говорит компилятор — \*/

```

const live = useMemo(() => {
 if (!vars) return null;
 const rawPath = vizArray(vars.path) ?? vizArray(vars.path);
 const path = (rawPath ?? []).map((x) => (typeof x === "number" ? x : vizNumber(x))).filter((x): x is number => x !== null && Number.isFinite(x));
 const usedRaw = vizArray(vars.used) ?? vizArray(vars.used);
 const usedFlags = (usedRaw ?? []).map((x) => Boolean(x));
 const degRec = vizRecord(vars.deg) ?? vizRecord(vars.deg);
 const deg = degRec
 ? C1_NODES.map((n) => {
 const val = degRec[String(n.id)] ?? degRec[n.id];
 const num = typeof val === "number" ? val : vizNumber(val);
 return num ?? DEG[n.id];
 })
 : null;
 const oddRaw = vizArray(vars.odd);
 const odd = oddRaw ? oddRaw.map((x) => (typeof x === "number" ? x : vizNumber(x))) : [];
 const last = vizNumber(vars.last) ?? vizNumber(vars.last);
 const step = vizNumber(vars.step) ?? vizNumber(vars.step);
 if (path.length === 0 && usedFlags.length === 0 && last === 0 && step === 0) {
 // рёбра маршрута: пары соседних вершин
 const keys: string[] = [];
 for (let i = 0; i + 1 < path.length; i++) keys.push(`${path[i]}-${path[i+1]}`);
 usedFlags.forEach((f, i) => {
 if (f && C1_EDGES[i]) keys.push(ekey(C1_EDGES[i]));
 });
 }
 return {
 path,
 keys: Array.from(new Set(keys)),
 };
});

```

```

 <Undo className="h-3.5 w-3.5" /> Сброс
 </button>
 </div>

 <div className="bg-slate-950 rounded-xl p-4 border
 -hidden">
 <div className="absolute inset-0 bg-[radial-gra
 <svg className="w-full h-[260px] z-10 relative"
 {C1_EDGES.map((e, i) => {
 const u = C1_NODES.find(n => n.id === e.u)!
 const v = C1_NODES.find(n => n.id === e.v)!
 const key = ekey(e.u, e.v);
 const used = visitedKeys.includes(key);
 // номер этого ребра в маршруте – подписывае
 const order = path.length > 1 ? (() => {
 for (let s = 0; s + 1 < path.length; s++)
 return null;
 })() : null;
 const isLastEdge = order !== null && order === path.length - 1;
 return <g key={i}>
 <line x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke={isLastEdge ? "#a5b4fc" : used ? "#f472b4" : "#999"}
 strokeWidth={isLastEdge ? 5 : used ? 4 : 2}
 className="transition-all duration-300"
 {order !== null && (
 <text x={({u.x + v.x} / 2} y={({u.y + v.y} / 2})
 fill="white" font-size="10px">{order}</text>
)}
 </g>;
)))
 {C1_NODES.map(n => {
 const isLast = lastId === n.id;
 const visited = path.includes(n.id);
 const isOdd = odd.includes(n.id);
 return <g key={n.id} className="cursor-pointer">
 {isLast && <circle cx={n.x} cy={n.y} r={12}
 fill={isLast ? "#6366f1" : visited ? "#f472b4" : "#999"}
 stroke="white" strokeWidth={1}>
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#f472b4" : "#999"}
 stroke="white" strokeWidth={1}>
 </g>
 })}
 </div>
 </div>
 </div>
</div>

```



```

 <>

 {live.path.map((id) => C1_NODES[id].label)}.

 ойдено: {live.keys.length}/{C1_EDGES.length}

 {odd.length}
 {live.step !== null &&
 x-2 py-1">step = {live.step}}
 </>
) : (

 открой панель Python и запусти код: массив
 том графе, а <code className="font-mono text-slate-500">
 есчитают критерий

)}
 </div>
 </div>
);
}

```

ФАЙЛ 43/110: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect } from 'react';
import { liveMatrix, pickLiveVar, useVizRuntime, vizNum } from 'viz';

interface Step {
 k: number; i: number; j: number;
 matrix: (number | null)[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.

```

```

 6: [{ v: 1, w: 1 }],
 });

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Инициализация матрицы прямых путей, ∞ (99',
 activeCodeLine: 2,
 });

 const N = 7;
 for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через про
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];

```

```

const liveK = vizNumber(vars?.k);
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
/**
 * Матрица из кода пользователя – СВОЕЙ размерности.
 * протискивались в демо-сетку 7x7: при n < 7 появлял
 * при n > 7 компонент падал на nodeNames[i].split(..
 */
const liveSource = pickLiveVar(vars, ["d", "dist", "d
const liveMat = liveMatrix(liveSource?.value);
const compilerLinked = liveK !== null || liveI !== nu

const rows = liveMat?.rows ?? baseStep.matrix.length;
const cols = liveMat?.cols ?? (baseStep.matrix[0]?.len
const shownMatrix: (number | null)[][] = liveMat
 ? Array.from({ length: rows }, (_, i) => Array.from
 : baseStep.matrix;

const demoSize = baseStep.matrix.length;
const sizeMismatch = compilerLinked && liveMat && (li
 ? `Демо-схема графа нарисована для ${demoSize} вершн
 ерности; схема графа остаётся демонстрационной.`
 : null;
const labelOf = (idx: number) => nodeNames[idx]?.spli

const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 k: liveK ?? -1,
 i: liveI ?? -1,
 j: liveJ ?? -1,
 matrix: shownMatrix,
 updated: !!runtime?.changed?.some(({name}) => nam
 log: `Компилятор: k=${liveK ?? "-"}, i=${liveI
 ame}})` : ""}. Подсветка взята из реальных пер
 }
 : baseStep;

```

```

<path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth=
<path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
</defs>
{Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id
 const vNode = nodesSvg.find((n) => n.id
 const isTarget = currentStep.i === u &&
 const isVia = (currentStep.i === u && c
 let strokeColor = '#475569'; let marker
 if (isTarget) { strokeColor = '#10b981'
 else if (isVia) { strokeColor = '#818cf8
 return (
 <g key={` ${u} - ${edge.v} `}>
 <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarg
 <circle cx={({uNode.x + vNode.x} / 2
isVia ? '#818cf8' : '#64748b'} strokeWidth
 <text x={({uNode.x + vNode.x} / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
 </g>
);
 });
});
}}}
{nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a
 else if (isI) { fill = '#b45309'; stroke =
 else if (isJ) { fill = '#047857'; stroke =
 return (
 <g key={node.id} className="transition-

```

```

 {edge.v} ({edge.w})

)
 }
}
</div>
);
}}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Матрица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div className="flex justify-between items-center">
 <h4 className="text-lg font-bold text-rose-950">
 </div>
 {sizeMismatch ? (
 <p className="mb-3 rounded-lg border border-rose-950">
 {sizeMismatch}
 </p>
) : null}
 <div className="overflow-x-auto">
 <table className="w-full text-center border border-rose-950">
 <thead><tr className="bg-rose-950/40 text-rose-950">
 <th className="p-2 border-r border-rose-950">
 {Array.from({ length: cols }, (_, idx) =>
 <th key={idx} className={`p-2 border-rose-950 ${
 labelOf(idx)} {idx}</th>
)}}
 </tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b border-rose-950">
 <td className={`p-2 bg-rose-950/40 text-rose-950 ${
 tStep.k === i ? 'text-amber-400' : ''}`}>
 {labelOf(i)} {i}
 </td>

```

-----  
}

-----  
ФАЙЛ 44/110: src/components/GraphComponentsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

-----  
import React, { useCallback, useContext, useEffect, useLayoutEffect, useReducer } from 'react';  
import { ChevronLeft, ChevronRight, Play, Pause, Rotate } from 'react-feather';  
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from '../VizChapterContext';

/\*\*

\* Билет 8 – «Графы. Компоненты связности».

\*

\* Две вкладки:

\* 1. «Обход» – DFS/BFS-покраска: внешний цикл по непокрашенным вершинам, каждый новый запуск = +1 к счётчику компонент.

\* 2. «DSU» – рёбра приходят по одному, union склеивает компоненты, comps уменьшается.

\*

\* Синхронизация с Python-компилятором (PAGE\_SYNC["graphs"]):

\* - comp / color / used – массив разметки: вершины покрашены / не покрашены / использованы

\* - parent / p – массив DSU: рисуем множества

\* - v – текущая вершина (пульсирует)

\* - stack / dq / q – фронт обхода (подсвечивается)

\* - comps / c – счётчик компонент (большая часть)

\* - i / e / step – номер шага: двигает по кадру

\*/

type Mode = "dfs" | "dsu";

interface Node {

id: number;

label: string;

x: number;

y: number;

}

```

 edge: [number, number] | null;
}

/** Покадровая раскладка DFS-обхода: ровно тот порядок,
function buildDfsFrames(): DfsFrame[] {
 const frames: DfsFrame[] = [];
 const comp = new Array(NODES.length).fill(0);
 const push = (cur: number | null, frontier: number[],
 => {
 frames.push({ title, note, comp: [...comp], cur, fr
 });
 let c = 0;
 push(null, [], c, "Старт: все вершины серые", "comp =
 дой непокрашенной.");
 for (let start = 0; start < NODES.length; start++) {
 if (comp[start] !== 0) continue;
 c += 1;
 const stack = [start];
 comp[start] = c;
 push(start, stack, c, `Новый запуск: c = ${c}, стар
 цвет ${c} и кладём в стек.`);
 while (stack.length) {
 const v = stack[stack.length - 1];
 const next = ADJ[v].find((to) => comp[to] === 0);
 if (next === undefined) {
 stack.pop();
 push(v, stack, c, `Из ${v} идти некуда – возвра
 ;
 continue;
 }
 comp[next] = c;
 stack.push(next);
 push(next, stack, c, `${v} → ${next}: красим в цв
 }, кладём ${next} в стек.`, [v, next]));
 }
 push(null, [], c, `Компонента ${c} готова`, `Стек п
 } вершин. Ищем следующую серую вершину.`);
 }
}

```

```

 return frames;
}

const DFS_FRAMES = buildDfsFrames();
const DSU_FRAMES = buildDsuFrames();

/** Корень DSU-множества с полным сжатием пути. */
const dsuRoot = (parent: number[], v: number): number => {
 let cur = v;
 const seen = new Set<number>();
 while (parent[cur] !== cur && !seen.has(cur)) {
 seen.add(cur);
 cur = parent[cur];
 }
 return cur;
};

export const GraphComponentsViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const [mode, setMode] = useState<Mode>("dfs");
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);
 const timer = useRef<number | null>(null);

 const frames = mode === "dfs" ? DFS_FRAMES : DSU_FRAMES;
 const maxStep = frames.length - 1;
 const frame = frames[Math.min(step, maxStep)] as DfsFrame;

 /* — вкладка сообщает компилятору, какой код подстав. */
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode);
 }, [chapterId, mode]);

 /* — автопроигрывание — */
 useEffect(() => {
 if (!playing) return;
 timer.current = window.setInterval(() => {

```



---

```
const liveStep = vars ? (vizNumber(vars.step) ?? vizN
```

```
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 goTo(liveStep);
}, [liveStep, goTo]);
```

```
/* — какая разметка сейчас в силе: код пользователя
/** Массив parent, который сейчас рисуем: из кода пол
const parentNow = liveParent ?? (mode === "dsu" ? fra
/** Номер множества по parent[]: корни нумеруются в п
const setOf = useCallback(
 (parent: number[], id: number): number => {
 const roots = Array.from(new Set(parent.map((_, i) => i)));
 return roots.indexOf(dsuRoot(parent, id)) + 1;
 },
 []
);
```

```
const compOf = (id: number): number => {
 if (liveComp && liveComp[id] !== undefined && liveC
 if (parentNow) return setOf(parentNow, id);
 return frame.comp?.[id] ?? 0;
};
```

```
const frontier = liveFrontier ?? (mode === "dfs" ? (f
const cur = liveCur ?? frame.cur;
const comps = liveComps ?? frame.comps;
const edge = frame.edge;
const title = runtime && vars ? `${frame.title} · K
```

```
const switchMode = (m: Mode) => {
 setMode(m);
 setStep(0);
 setPlaying(false);
};
```

```
return {
```

```

 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1.5 transition-colors"
 >
 {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 </button>
 <button onClick={() => { setPlaying(false); goToNext(); }}
 className="flex items-center gap-1.5 px-3 py-1.5 border border-slate-700 text-slate-500 rounded-lg bg-slate-900 transition-colors"
 >
 <ChevronRight className="w-4 h-4" />
 </button>

 КОМПОНЕНТ: {comps}

 </div>

 {/* поле */}
 <div className="bg-slate-950 rounded-xl p-3 border border-slate-700" >
 <div className="absolute inset-0 bg-[radial-gradient(circle_1px_1px_1px_1px)]" >
 <svg viewBox="0 0 500 370" className="w-full h-full" >
 {EDGES.map(([u, v], i) => {
 const a = NODES[u];
 const b = NODES[v];
 const active = edge && ((edge[0] === u && edge[1] === v) || (edge[0] === v && edge[1] === u));
 const sameComp = compOf(u) > 0 && compOf(v) > 0 && compOf(u) === compOf(v);
 return (
 <line
 key={i}
 x1={a.x} y1={a.y} x2={b.x} y2={b.y}
 stroke={active ? "#fbbf24" : sameComp ? "#fbbf24" : "#fbbf24"}
 strokeWidth={active ? 4 : sameComp ? 2 : 1}
 strokeDasharray={sameComp ? undefined : [4, 4]}
 className="transition-all duration-300"
 opacity={sameComp || active ? 1 : 0.55}
 />
);
 })}
 </svg>
 </div>
 </div>
 </div>
</div>

```

```

 <text x={436} y={41.5 + i * 15} f
 </g>
);
 }}}
</g>
</svg>
</div>

{/* подпись кадра */}
<div className="mt-3 bg-slate-900/70 border borde
 <p className="text-sm font-bold text-white mb-1
 {runtime && vars && <Terminal className="w-3.
 {title}
 </p>
 <p className="text-xs text-slate-400 leading-re
</div>

{/* живые переменные из компилятора */}
<div className="mt-3 flex flex-wrap items-center
 <span className="text-[10px] uppercase tracking
 {vars ? (
 ["v", "comps", "c", "i", "e", "comp", "parent
 vars[name] === undefined ? null : (
 <span key={name} className="text-[11px] f
 py-1">
 {name} = {JSON.stringify(vars[name]).sl

)
)
) : (

 открой панель Python – переменные <code cla
 erald-400">parent</code>, <code className="
 comps</code> покрасят этот граф сами

)}
 </div>
 </div>

```

```

 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};

type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, just for fun

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Зависит от ${vis.size} узлов, visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Процесс узла ${v}, adj[v] = ${adj[v]}, stack: ${stack}, visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Обратный шаг от узла ${v}, queueOrStack: [...stack], visitedNodes: Array.from(vis) });
 }
}

```

```

const [mode, setMode] = useState<"dfs" | "bfs">("df
const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);

const steps = useMemo(() => mode === 'dfs' ? genera
useVizStepSync(stepIdx, setStepIdx, steps.length -
 const v = vizString(vars.v);
 const structure = vizArray(vars.stack) ?? vizArra
 const visited = vizArray(vars.visited);
 if (v === null && !structure && !visited) return
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) =>
 (v === null || step.node === v) &&
 (!structure || step.queueOrStack?.length ===
 (!visited || step.visitedNodes?.length === vi
);
 return candidates.at(-1)?.index ?? null;
});

useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [mode]);

useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
}

```



```

 { /* Controls */ }
 <div className="flex bg-slate-900 border
 <button onClick={() => {setAutoPlay
="Шаг назад" title="Шаг назад" className="p
0 disabled: hover: bg-transparent">
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={() => setAutoPlay(
justify-center min-w-[80px] ${mode === 'df
>
 {autoPlay ? <><Pause size={16}
 </button>
 <button onClick={() => {setAutoPlay
= steps.length - 1} aria-label="Шаг вперед"
00 rounded-lg disabled: opacity-30 disabled:
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={() => {setAutoPlay
ame="p-2 text-slate-500 hover: text-rose-400
 <RefreshCw strokeWidth={3} size
 </button>
 </div>

 { /* Description Log */ }
 <div className="flex-1 bg-slate-900/50
ow-hidden">
 <div className="text-[10px] text-sl
{steps.length}</div>
 <div className="text-sm text-slate-
 {step.desc}
 </div>
 </div>
 </div>
</div>
);
}

```

```

 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) l = l;
 if (r < n && a[r].priority > a[largest].priority) l = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '👤' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];

```



```

const [shakeEmpty, setShake] = useState(false)
const [busy, setBusy] = useState(false)
const [healingPatient, setHealing] = useState<Patient>()

const addLog = (msg: string) => setLog(prev => [msg, ...prev])

// --- INSERT: Новый пациент поступает в приемный покой
const insertPatient = useCallback((name: string, priority: number) => {
 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выписки')
 setShake(true)
 setTimeout(() => setShake(false), 400)
 return
 }
 setBusy(true)

 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)]
 const finalName = name || preset.name.split(' ')[0]
 const finalEmoji = preset.emoji
 const finalSymptom = preset.symptom

 const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
 }

 const nextHeap = heapInsert(heap, newPatient).map(x => x)
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === priority)
 const marked = nextHeap.map(x => x.id === (target?.id || ''))

 setHeap(marked)
 addLog(` Поступил пациент: ${finalName} с тяжестью ${priority}`)
}, [uid, heap])

```

```

// Step 2: Root exits the tree, tree reorganizes
setHeap(prev => prev.map((x, i) => i === 0 ? { ..

setTimeout(() => {
 // Run binary heap extraction
 setHeap(prev => {
 if (prev.length === 0) return [];
 const a = prev.map(x => ({ ...x }));
 a[0] = { ...a[a.length - 1] };
 a.pop();
 return siftDown(a).map(x => ({ ...x, animStat
 }));

 // Patient gets healed in the bed
 setTimeout(() => {
 setHealing(prev => prev ? { ...prev, animStat
 addLog(` Успех! Пациент ${topPatient.name} п
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
}, 650);
}, [busy, heap]);

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat
 setLog([' База данных скорой помощи перезагружена.
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);

```

```

 </p>
 </div>
 </div>

 { /* УПРАВЛЕНИЕ */ }
 <div className="flex flex-col lg:flex-row items-s

 { /* Добавить случайного */ }
 <button
 onClick={handleRandomInsert}
 disabled={busy || heap.length >= 15}
 className="flex-1 min-w-[150px] bg-gradient-to
 opacity-40 disabled:cursor-not-allowed text
 ve:scale-95 transition-all cursor-pointer f
 >
 Привезти по Скорой (INSERT)
 </button>

 { /* Добавить кастомного */ }
 <form onSubmit={handleCustomInsert} className="
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border borde
 er-emerald-500 transition-colors placeholde
 />
 <div className="flex flex-col items-center min
 <span className="text-[9px] font-mono text-
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e
 className="w-20 accent-emerald-500 cursor

```

```
<div className="absolute top-4 left-4 text-[10px] font-monospace">
 Сортировка по тяжести (Двоичная куча / Max-Heap)
</div>
```

```
<div className="relative w-full h-full min-h-[200px]">
 <svg className="absolute inset-0 w-full h-full" viewBox="0 0 1000 1000">
 {edges}
 </svg>
```

```
{visualHeap.map((patient, idx) => {
 const pos = nodePos(idx);
 const isRoot = idx === 0;
 const isCompilerNode = liveI === idx;
 return (
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center justify-center
 -translate-x-1/2 -translate-y-1/2 transition-all duration-300
 ${patient.animState === 'in' ? 'animate-in' : ''}
 ${patient.animState === 'winner' ? 'animate-winner' : ''}
 ${patient.animState === 'out' ? 'animate-out' : ''}
 ${isCompilerNode ? 'ring-4 ring-amber-500' : ''}
 ${isRoot ? 'bg-rose-950/80 border-rose-950/80' : ''}
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 {patient.priority}
 {patient.priority}
 >
)
})}
```

```
{/* Tooltip */}
```

```

 <Heart className="w-4 h-4 text-rose-500" />
 </div>
 </div>
 </div>
) : {
 <div className="text-center text-slate-600" >
 Ждём вас
 <p className="text-xs font-bold font-mono">Ваше имя</p>
 <p className="text-[10px] mt-1">Ждём вас</p>
 </div>
 <div className="w-full h-3 bg-gradient-to-r from-slate-500 to-slate-600" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4" >
 <div className="text-[10px] font-mono text-slate-400" >
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between`}
 >
 {idx === 0 ? Действие : {entry}
 {entry}
 </div>
 })}
 </div>
</div>
);
};

```

```

 idle: "#334155",
 idleStroke: "#475569",
 text: "#e2e8f0",
 dim: "#94a3b8",
 active: "#6366f1",
 done: "#10b981",
 warn: "#f59e0b",
 bad: "#f43f5e",
 edge: "#475569",
 };

export const Svg: React.FC<{ children: React.ReactNode;
 const paused = useContext(DemoPauseContext);
 return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза — курсор на карточке" : "наведи"}
 </div>
 </div>
);
};

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
```

```

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
 // i -> j напрямую 9; через k=1 получаем 3+4=7
 const pos: Record<string, [number, number]> = { i: [4, 9],
 const viaOpen = step >= 1;
 const relaxed = step >= 2;
 return (
 <Svg
 caption={
 step === 0
 ? "k ещё запрещена: путь i → j = 9"
 : step === 1
 ? "Разрешаем пересадку через k: 3 + 4 = 7"
 : "7 < 9 → D[i][j] = 7"
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.ok} />
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.ok} />
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.ok} />

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C.i} />
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={C.v} />
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={C.j} />
 </Svg>
);
};

```

```

 caption={
 step === 0
 ? "Вершины – объекты (города, слова, состояния)"
 : step === 1
 ? "Рёбра – связи между ними"
 : "Вес ребра – цена перехода"
 }
 >
 {edges.map(([u, v, w], i) => {
 const a = pos[u];
 const b = pos[v];
 return (
 <g key={i}>
 <Edge a={a} b={b} color={step >= 1 ? C.active : C.idle}
 {step >= 2 && (
 <text
 x={({a[0] + b[0]} / 2}
 y={({a[1] + b[1]} / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
)}
 </g>
);
)}
)}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={C.idle} />
))}
 </Svg>
);
};

```

```

/** Сжатие координат: разрежённые числа → плотные индексы
export const CoordCompressDemo: React.FC = () => {

```



```

 <rect x={startX + i * (boxW + 4)} y={98} wi
 <text x={startX + i * (boxW + 4) + boxW / 2
 {sorted.indexOf(v)}
 </text>
 </g>
 }}}
</Svg>
);
};

/* ----- Splay: три случая поворота ---

type Pt = [number, number];

const SplayFrames: React.FC<{
 frames: { pos: Record<string, Pt>; edges: [string, st
 captions: string[];
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : i

 return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 </Svg>
)
}

```

```

 captions=["Зигзаг: g влево, x вправо", "Поворот НИ
frames=[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
 { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
]}
 />
);

```

ФАЙЛ 49/110: src/components/hints/demosGraph.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
 const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
)}})
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h

```

```

 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]} />
 })}
 {nodes.map((n) => {
 const idx = placed.indexOf(n.id);
 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} height={24} />
 <text x={n.x} y={n.y} textAnchor="middle" dx={
 {idx >= 0 && <circle cx={n.x + 30} cy={n.y} r={10} />
 {idx >= 0 && (
 <text x={n.x + 30} y={n.y - 12} textAnchor="middle" />
)}
 } />
 </g>
);
 })}
 <text x={224} y={62} textAnchor="middle" fontSize={14} />
 <text x={224} y={74} textAnchor="middle" fontSize={14} />
 </Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Пусть" : ""}>
 <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
 <text x={130} y={54} textAnchor="middle" fontSize={14} />
 <Node x={60} y={70} r={20} fill={C.active} label="3" />
 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
 <text x={60} y={106} textAnchor="middle" fontSize={14} />
 <text x={200} y={106} textAnchor="middle" fontSize={14} />
 <text x={130} y={22} textAnchor="middle" fontSize={14} />
 </Svg>
);
};

```

```

 <circle key={k} cx={x} cy={y} r={12} fill="none" stroke={fill} />
) : (
 <Node key={k} x={x} y={y} label={k} fill={k} stroke={k} />
)
)}
 </Svg>
);
};

export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10], E: [190, 80]
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц`>
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ? "red" : "black"} />
 <text x={{(pos[e.u][0] + pos[e.v][0]) / 2} y={pos[e.v][1] - 10}
 textAnchor="middle" fontSize={9} fontWeight="bold"} />
 </g>
);
 })}
 </Svg>
 {Object.entries(pos).map(([k, [x, y]]) => (<Node key={k} x={x} y={y} fill={k} stroke={k} />))}
);
};

```

```

 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b=
 {heap.map((v, i) => {
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label=
 }}}
 </Svg>
);
};

```

```

export const StackDemo: React.FC = () => {
 const states = [["A"], ["A", "B"], ["A", "B", "C"], [
 const step = useLoop(states.length, 800);
 const items = states[step];
 const popping = step >= 3;
 return (
 <Svg caption={popping ? "pop: забираем ВЕРХНИЮ таре
 <rect x={90} y={22} width={80} height={96} rx={6}
 {items.map((v, i) => {
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={
 fill={i === items.length - 1 ? (popping ? C
 <text x={130} y={111 - i * 26} textAnchor="mi
 </g>
 }}}
 <text x={130} y={16} textAnchor="middle" fontSize=
 </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight=
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>

```

```

 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общие

 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]} }
 {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x} :
 </Svg>
);
};

export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? levels[step - 1] : [];
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, M
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]} }
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId
)} }
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
 </Svg>
);
};

```

```

 {nodes.map((n, i) => (
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={n.h}
 fill={n.lvl === step ? C.active : C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }} />
 <text x={n.x} y={n.y} textAnchor="middle" domClass={n.domClass} />
 </g>
))}
 </Svg>
);
};

```

```

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];
 const pref = a.reduce<number[]>((acc, v) => [...acc, v], []);
 const step = useLoop(4, 1200);
 const l = 1, r = 3;
 const show = step >= 1;
 return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -` : ''}
 o префиксные суммы P">
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }} />
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }} />
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
 </Svg>
);
};

```

```

 </Svg>
);
};

```

```

export const NpDemo: React.FC = () => {
 const step = useLoop(2, 1400);
 return (
 <Svg caption={step === 0 ? "Найти решение – долго (1000 итераций)" : "Найти решение – быстро (1400 итераций)"}>
 <ellipse cx={130} cy={64} rx={112} ry={48} fill="#333" />
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333" />
 <text x={92} y={64} textAnchor="middle" dominantBaseline="middle" font-size="16">
 Найти решение – долго (1000 итераций)
 </text>
 <text x={206} y={34} textAnchor="middle" font-size="16">
 Найти решение – быстро (1400 итераций)
 </text>
 <circle cx={196} cy={84} r={13} fill={step === 1 ? "red" : "black"} />
 <text x={196} y={84} textAnchor="middle" dominantBaseline="middle" font-size="16">
 {step === 1 ? "Найти решение – долго (1000 итераций)" : "Найти решение – быстро (1400 итераций)"}
 </text>
 <text x={130} y={124} textAnchor="middle" font-size="16">
 Найти решение – долго (1000 итераций)
 </text>
 </Svg>
);
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс "${s.substring(0, len)}" является префиксом "${s}"` : "Префикс-функция"}>
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20} fill={step > 0 && (idx < len || (idx > i - 1 ? "red" : "black")} style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle" font-size="16">
 {ch}
 </text>
 <text x={32 + idx * 33} y={70} textAnchor="middle" font-size="16">
 {idx <= i && step > 0 ? C.warn : C.idle}
 </text>
 </g>
))}
 </Svg>
);
};

```



---

ФАЙЛ 51/110: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components" |
 | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
```

---

ФАЙЛ 52/110: src/components/hints/SimulatorModal.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../../data/vizStepBuilder";
import { VizLiveStatus } from "../VizLiveStatus";

interface Props {
 vizId: string | null;
 onClose: () => void;
}

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

 const viz = getViz(vizId ?? undefined);
 if (!vizId || !viz) return null;
 const Component = viz.Component;
```

```

import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

export interface HintAnchor {
 termId: string;
 rect: DOMRect;
}

interface Props {
 anchor: HintAnchor | null;
 onClose: () => void;
 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
}

const CARD_W = 320;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor,
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number}>({
 top: 0, left: 0,

 useEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

```

```

 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo-500" >
 <div className="flex items-start gap-2 px-3 pt-3" >
 <h4 className="text-sm font-bold text-indigo-400" >
 <button
 onClick={onClose}
 className="text-slate-500 hover:text-white"
 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />
 </button>
 </div>

 <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
 <p className="text-[13px] leading-relaxed text-slate-300">
 {entry.demo && <MiniDemo kind={entry.demo} />}
 </p>
 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed text-slate-300">
 {entry.formal}
 </p>
)}
 {entry.complexity && (
 <div className="text-[11px] font-mono text-slate-300">
 Сложность: {entry.complexity}
 </div>
)}
 </div>

 <div className="p-3" >
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center text-white rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
 </div>
 </div>
 </div>

```

```

 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0 }
];

const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 return false;
};

const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'S') return 0;
 if (id === 'A') return 0;
 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return null;
};

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
 if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
 return "stroke-slate-700 stroke-dasharray-4";
 }
 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 6) return "stroke-emerald-500";
 }
};

```

```

 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1
 return e.newWeight;
 }
 return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;
 if (e.id === 'CA' && step === 6) return true;
 return false;
};

const isEdgeTextEmerald = (e: any) => {
 if (e.id === 'AB' && step > 4) return true;
 if (e.id === 'BC' && step > 5) return true;
 if (e.id === 'CA' && step > 6) return true;
 return false;
};

const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соед
 ми.
 Это наша база для расчета потенциал
 </div>
);
 }
};

```

```

 case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S
 Все новые веса ребер в графе стали
 При этом старые кратчайшие пути ост
 </div>
);
 default: return "";
 }
};

return (
 <div className="bg-slate-800 p-4 rounded-xl border">
 <h4 className="text-sm md:text-base font-bo">

 Визуализация: Перевзвешивание алгоритмом
 </h4>

 <div className="flex flex-col md:flex-row gap-4">
 <div className="bg-[#0f172a] flex-1 min-h-100 inter p-4">
 <svg width="400" height="300" class="dark">
 <defs>
 <marker id="arrow-slate" viewBox="0 0 10 5"
 start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" />
 </marker>
 <marker id="arrow-slate-dim" viewBox="0 0 10 5"
 uto-start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" />
 </marker>
 <marker id="arrow-amber" viewBox="0 0 10 5"
 start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" />
 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 5"
 tart-reverse">

```

```

 isS ? 'fi
 `}>
 {getEdgeContent
 </text>
 </g>
)
 }}}

 { /* Nodes */
 {nodes.map(n => {
 if (!isNodeVisible(n.id)) r
 const pot = getPotential(n.

 return (
 <g key={n.id} className=
 <circle
 cx={n.x} cy={n.y}
 className={`str
 ${n.id === 'S'
 ? 'fill-[#2
 : 'fill-sla
 />
 <text x={n.x} y={n.y}
- white">
 {n.label}
 </text>

 {pot !== null && (
 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-1
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
)}

```



```
 <div key={i} className={`w-125' : i < step ? 'bg-indigo-900' : 'bg-slate-500'}
 >>>
 </div>
 </div>
 </div>
</div>
);
}
```

---

ФАЙЛ 55/110: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...";
import {
 Play,
 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
```

```

 return candidates.at(-1)?.index ?? null;
 });
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 // Generate step-by-step Kosaraju steps
 const generatedSteps: AlgoStep[] = [];
 const visited = new Set<number>();
 const order: number[] = [];
 const sccMap: Record<number, number> = {};

 const recordStep = (
 phase: AlgoStep["phase"],
 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u })
 : [...initialEdges],
 });
 };

 recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый",
 null,
 false,
);
 });

```

```

 }
 }

 const topoOrder = [...order].reverse();
 recordStep(
 "transpose",
 `Первый проход завершен! Стек выхода в топологическом порядке`,
 null,
 true,
 -1,
);

 // Transpose adj
 const adjT: number[][] = Array.from({ length: 5 },
 initialEdges.forEach((e) => adjT[e.v].push(e.u));

 // Reset visited for phase 2
 visited.clear();
 let sccId = 0;

 const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе`,
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
 };
};

```

```

 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,
 order: [],
 transposed: false,
 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
 });

 useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
 };

 const getSccColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
 };

 return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-

```



```

 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />
);
 }}}

{/* Nodes */}
{initialNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited =
 step.visited.has(node.id) ||
 (step.phase === "dfs1" && step.currentNode === node.id);
 const sccId = step.sccMap[node.id];

 let classes = "fill-slate-800 stroke-slate-800";
 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 } else if (isVisited && step.phase === "dfs2") {
 classes = "fill-emerald-700 stroke-emerald-700";
 }

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={classes}
 strokeWidth="3"
 />
 <text
 x={node.x}

```

```

00 overflow-y-auto">
<h4 className="text-xs font-bold text-slate-400">
 Порядок обхода и стек
</h4>

<div className="bg-slate-900 border border-slate-800 p-4">
 <p className="text-xs text-white leading-relaxed">
 {step.desc}
 </p>
</div>

<div className="flex-1 space-y-4 overflow-y-auto">
 {/* Top-Sort output / Order stack representation */}
 <div>

 СТЕК ВЫХОДА (DFS 1 ORDER):

 <div className="flex flex-wrap gap-1 bg-slate-900 p-4">
 {step.order.length === 0 ? (

 Пустой стек

) : (
 [...step.order].reverse().map(({val, idx}) => (
 <div
 key={idx}
 className="bg-indigo-950 border border-indigo-800 p-2 text-white text-sm flex items-center gap-1"
 >
 {val}
 </div>
))
)}
 </div>
 </div>

 {/* List of actions log */}
 <div>

 ЛОГ ОПЕРАЦИЙ:

 </div>

```

```
 </div>
 </div>
 </div>
 </div>
);
};
```

---

ФАЙЛ 56/110: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useContext, useEffect, useId, useLayoutEffect, useReducer, useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizStepBus } from '../data/vizStepBus';
```

```
interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}
```

```
interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'rejected';
 color: string;
}
```

```
interface StepLog {
 title: string;
```



```

const [heapQueue, setHeapQueue] = useState<string[]>()
const [activeCheatTab, setActiveCheatTab] = useState<

const [logs, setLogs] = useState<StepLog[]>([
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Предста
 овы начинать!',
 type: 'info',
 },
]);

// --- KRUSKAL STEPS ---
const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные верши
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Include A-B
 () => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
 },
 // Step 3: Inspect D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',

```

```

 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю — мы перекрашиваем всё в один цвет!',
 type: 'warning'
 }, ...prev]);
},
// Step 8: Include A-D (Merge to purple)
() => {
 setVertices(prev => prev.map(v => v.color === 'blue' ? 'purple' : v.color));
 setEdges(prev => prev.map(e => e.status === 'included' ? 'excluded' : e.status));
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий кланы',
 type: 'success'
 }, ...prev]);
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',
 description: 'НО: это ребро соединяет вершины B и E, которые уже соединены!',
 type: 'warning'
 }, ...prev]);
},
// Step 10: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе получится цикл!',
 type: 'error'
 }, ...prev]);
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',

```

```

 description: 'Оно соединяет вершины С и F, кото
 type: 'warning'
 }, ...prev]);
},
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветн
 type: 'info'
 }, ...prev]));
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в осто́ве',
 description: 'Вершина H перекрашена в фиолетовы
 type: 'success'
 }, ...prev]));
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. С

```

```

setVertices(prev => prev.map(v => v.id === 4 ? {
setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E
setEdges(prev => prev.map(e => ['3-4', '1-4', '4-
setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вер
 description: 'Мы выбрали вершину E как стартовую
 ритетную очередь (Heap): D-E(3), B-E(6), E-F(
 type: 'info'
}, ...prev]));
},
// Step 2: Pop D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро
 description: 'Плесень тянется по ребру D-E к вер
 type: 'info'
 }, ...prev]));
},
// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D:
 type: 'success'
 }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вер
 type: 'info'

```

```

 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро В-С (вес 4)',
 description: 'Плесень тянется к вершине С.',
 type: 'info'
 }, ...prev]));
 },
 // Step 9: Include B-C, add C's edges
 () => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
 setLogs(prev => [{
 title: 'Успех: Вершина С захвачена',
 description: 'Ребро C-F(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
 },
 // Step 10: Pop B-E (6) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро В-Е (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет вершину В.',
 type: 'warning'
 }, ...prev]));
 },
 // Step 11: Reject B-E
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-F (10)']);
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро В-Е отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
 },
 // Step 12: Pop E-F (7)

```

```

 type: 'success'
 }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]);
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]);
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]);
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
 setLogs(prev => [{

```

```

 setLogs(prev => [{
 title: ' Успех: Плесень захватила весь граф
 description: 'Все 9 вершин поглощены. Итоговый
 иной – мы разрастались из одной точки с помо
 type: 'success'
 }, ...prev]));
 },
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)
 // Node 2(C) -> B-C(4)
 // Node 3(D) -> D-E(3)
 // Node 4(E) -> D-E(3)
 // Node 5(F) -> E-F(7)
 // Node 6(G) -> D-G(8)
 // Node 7(H) -> G-H(10)
 // Node 8(I) -> H-I(12)
 setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8
]));
 setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбира
 description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Concurrently merge into Kolkhozes (Compo
 () => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)

```

```

 e.status === 'included' || e.id === '0-3' ? { .
 });
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский
 description: 'Оба колхоза объединились по мосту
 type: 'success'
 }, ...prev]));
 },
 // Step 5: Check remaining edges & mark as rejected
 () => {
 setEdges(prev => prev.map(e => e.status === 'pend
 setLogs(prev => [{
 title: ' Успех: MST построен Борувкой за 2 шаг
 description: 'Все лишние ребра (В-Е, Е-Н, С-Е,
 вес: 51. Благодаря параллельности, мы потрати
 type: 'success'
 }, ...prev]));
 }
];

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const introLog = (algo: 'kruskal' | 'prim' | 'boruvka
 if (algo === 'kruskal') {
 return [{
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа
 type: 'info',

```



```

const goToStep = useCallback((n: number) => {
 const target = Math.max(0, Math.min(currentSteps.length, n));
 setVertices(initialVertices);
 setEdges(initialEdges);
 setHeapQueue([]);
 setLogs(introLog(activeAlgo));
 for (let i = 0; i < target; i++) currentSteps[i]();
 setStepIndex(target);
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [activeAlgo, currentSteps]);

```

```

/** Номер кадра из переменных кода: i / step / число
const selectByVariables = useCallback<VizStepSelector>((v) => {
 const i = vizNumber(v.i) ?? vizNumber(v.step);
 if (i !== null) return i + 1;
 const mst = vizArray(v.mst) ?? vizArray(v.answer) ?? vizArray(v.mst);
 if (mst) return mst.length;
 const taken = vizArray(v.taken) ?? vizArray(v.used);
 if (taken) return Math.max(0, taken.length - 1);
 return null;
}, []);

```

```

useVizStepSync(stepIndex, goToStep, currentSteps.length);

```

```

/** A..I ↔ id 0..8 */
const idOf = (name: string | number | null | undefined) => {
 if (name === null || name === undefined) return null;
 if (typeof name === 'number') return Number.isInteger(name) ? name : null;
 const c = name.trim().toUpperCase().replace(/^[A-I]/, '');
 return c ? c.charCodeAt(0) - 65 : null;
};

```

```

/** Разбор одного ребра из кода: (2, 'A', 'B') / [2, 'A', 'B']
const parseEdge = (item: unknown): { u: number; v: number; e: string } | null => {
 if (item === null || item === undefined) return null;
 if (typeof item === 'string') {
 const nums = item.match(/-?\d+(?:\.\d+)?/g);
 const letters = item.match(/[A-Ia-i]/g);

```

```

 /** Метка множества вершины, как её записал код (бу
const rawComp = new Map<number, string>();
 /** То же, но только если метка – вершина графа: по
const parentIdx = new Map<number, number>();
const feed = (id: number, val: unknown) => {
 const label = String(typeof val === 'string' ? va
 rawComp.set(id, label);
 const inner = typeof val === 'string' ? val : typ
 const pid = idOf(inner);
 if (pid !== null) parentIdx.set(id, pid);
};
if (parentRec) {
 Object.entries(parentRec).forEach(([k, val]) => {
 const id = idOf(k);
 if (id === null) return;
 feed(id, val);
 });
} else if (parentArr) {
 parentArr.forEach((val, id) => {
 if (id > 8) return;
 feed(id, val);
 });
}
/**
 * Ключ множества вершины: если код дал DSU-массив
 * идём по цепочке до корня – иначе вершины одной к
 * разные цвета. Для произвольных меток («comp1») б
 */
const compOfVertex = new Map<number, string>();
rawComp.forEach((label, id) => {
 if (parentIdx.size === 0) {
 compOfVertex.set(id, label);
 return;
 }
 let cur = id;
 const seen = new Set<number>([id]);
 for (let hop = 0; hop < 16; hop++) {
 const nxt = parentIdx.get(cur);

```

```

 if (heapKey.has(k)) return { ...e, status: 'in_heap' };
 if (cheapKey.has(k)) return { ...e, status: 'inspected' };
 return { ...e, status: 'pending' as const };
 });
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [edges, live]);

const effVertices: Vertex[] = useMemo(() => {
 if (!live) return vertices;
 return vertices.map((v) => {
 const root = live.compOfVertex.get(v.id);
 if (root !== undefined) return { ...v, color: colorOf(root) };
 if (live.takenVertices.includes(v.id)) return { ...v, color: 'none' };
 if (live.curU === v.id || live.curV === v.id) return { ...v, color: 'none' };
 return { ...v, color: 'none' };
 });
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [vertices, live]);

const getColorClass = (color: string, id: number) => {
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 text-white';
 case 'blue': return 'bg-blue-500 border-blue-300 text-white';
 case 'purple': return 'bg-purple-600 border-purple-300 text-white';
 case 'mold': return 'bg-emerald-500 border-emerald-300 text-white';
 case 'gold': return 'bg-amber-400 border-amber-200 text-white';
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIn === 1)
 return 'bg-slate-700 border-emerald-500 text-white';
 return 'bg-slate-700 border-slate-500 text-slate-200';
 }
};

const getEdgeStroke = (status: string, color: string) => {
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';

```

```

 : 'text-slate-400 hover:text-slate-500'
 }
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow-md rounded-xl border-2
 : 'text-slate-400 hover:text-slate-500'
)
 }
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md rounded-xl border-2
 : 'text-slate-400 hover:text-slate-500'
)
 }
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
</div>

<button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-center gap-2 py-200 font-semibold text-xs rounded-xl border-2"
 >
 Сброс
 </button>

```

```

 })
 {live.takenEdges.length > 0 && {

 в остове: {live.takenEdges.map((e) => {

 })}

 })
 {live.compOfVertex.size > 0 && {

 компонент: {new Set(Array.from(live.compOfVertex))}

 })
 {live.takenVertices.length > 0 && {

 дерево выросло: {live.takenVertices.map((v) => {

 })}

 })
 {live.heapEdges.length > 0 && {

 в куче: {live.heapEdges.length}

 })
 {live.cheapestEdges.length > 0 && {

 cheapest: {live.cheapestEdges.length}

 })
 {live.total !== null && {

 вес остова = {live.total}

 })
 </>
) : (

 открой панель Python и запусти код вкладки
 'ка'>»: переменные
 {' '}<code className="font-mono text-purple" style="font-family: monospace;">
e className="font-mono text-purple-300">u</code>

```



```

 </div>
)})
 </div>

 {stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom
 shadow-xl z-20">
 <p className={"font-bold text-base " +
 'ld-300' : 'text-rose-300'}>
 MST построено с помощью {activeAlgo}
 </p>
 <p className="text-slate-300 text-xs mt
 Общий вес минимального остова: 51. Вс
 </p>
 </div>
)}
 </div>

 {/* Logs / Heap */}
 <div className="bg-slate-950 rounded-2xl border
 <div className="p-4 bg-slate-900 border-b b
 <h4 className="font-bold text-slate-200 t
 Ход выполнения
 </h4>
 </div>

 {activeAlgo === 'prim' && (
 <div className="p-3 bg-slate-900/60 borde
 <p className="text-[11px] font-bold tex
 ⚡ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5"
 {heapQueue.length === 0 ? (
 <span className="text-xs text-slate
) : (
 heapQueue.map(({item, idx}) => (
 <span key={idx} className={"px-2 p
 0 shadow-md animate-pulse" : "bg-slate-800

```

```

 /* Cheat Sheet: Complexity & Code for all 3 algo
 <div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-400
 <h4 className="text-xl font-bold text-white
 </div>

 <div className="flex bg-slate-950 p-1 rounded
 <button
 onClick={() => setActiveCheatTab('kruskal
 className={"px-3 py-1.5 rounded text-xs f
 == 'kruskal' ? "bg-indigo-600 text-white" : "
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={"px-3 py-1.5 rounded text-xs f
 == 'prim' ? "bg-emerald-600 text-white" : "
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka
 className={"px-3 py-1.5 rounded text-xs f
 == 'boruvka' ? "bg-rose-600 text-white" : "
 >
 Борувка (Билет 20)
 </button>
 </div>
 </div>

 {activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo

```



```
2. Жадный выбор ребер
```

```
edges.sort() # $O(E \log E)$
```

```
mst = []
```

```
for w, u, v in edges:
```

```
 if union(u, v):
```

```
 mst.append((u, v, w))`}
```

```
 </pre>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
}}
```

```
{activeCheatTab === 'prim' && {
```

```
 <div className="grid grid-cols-1 lg:grid-cols-
```

```
 <div className="lg:col-span-1 space-y-4">
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 <Clock className="w-3.5 h-3.5 text-em
```

```
 </p>
```

```
 <p className="text-2xl font-black text-r
```

```
 <p className="text-slate-400 text-xs mt
```

```
 </div>
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 <Award className="w-3.5 h-3.5 text-em
```

```
 </p>
```

```
 <p className="text-2xl font-black text-r
```

```
 <p className="text-slate-400 text-xs mt
```

```
 </div>
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 ложении:</p>
```

```
 <p className="text-xs text-slate-300 le
```

```
 ие чистые вершины через самое дешевое досту
```

```
 </div>
```

```
</div>
```

```
<div className="lg:col-span-2">
```

```

 p>
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-rose-500">
 </p>
 <p className="text-2xl font-black text-white">
 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 ении:</p>
 <p className="text-xs text-slate-300 leading-5">
 , организуя массовое распараллеленное слияние
 </p>
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-rose-500">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">
{`def boruvka(n, edges):
 parent = list(range(n))
 mst = []
 while len(mst) < n - 1:
 # Для каждого колхоза ищем самый дешёвый мост на
 cheapest = [-1] * n
 for u, v, w in edges:
 ru, rv = find(u), find(v)
 if ru != rv:
 if cheapest[ru] == -1 or cheapest[ru][2] > w:
 cheapest[ru] = (u, v, w)
 if cheapest[rv] == -1 or cheapest[rv][2] > w:
 cheapest[rv] = (u, v, w)
 # Объединяем все колхозы по выбранным мостам
 for bridge in cheapest:
 if bridge != -1 and union(bridge[0], bridge[1]):
 mst.append(bridge)`}

```

```

const LIGHT_CSS = `
:root { color-scheme: light; }
body { background: #ffffff !important; color: #111827; }
.lite-wrap { max-width: 46rem; margin: 0 auto; padding: 1.6; }
.lite-wrap h1 { font-size: 1.75rem; font-weight: 700; }
.lite-wrap h2 { font-size: 1.35rem; font-weight: 700; color: #111827; }
.lite-wrap h3 { font-size: 1.1rem; font-weight: 700; }
.lite-wrap h4 { font-size: 1rem; font-weight: 700; margin: 0 0 1.2rem 0; }
.lite-wrap p, .lite-wrap li { color: #1f2937; font-size: 1rem; }
.lite-wrap a { color: #1d4ed8; }
.lite-meta { color: #4b5563; font-size: .85rem; margin: 0 0 1.2rem 0; }
.lite-topics { font-size: .8rem; color: #374151; margin: 0 0 1.2rem 0; }
.lite-viz { margin: 1.25rem 0; padding: .8rem 1rem; border: 1px solid #d1d7db; }
.lite-viz h4 { margin: 0 0 .35rem; font-size: .95rem; }
.lite-viz ul { margin: .35rem 0 0 1.1rem; padding: 0; }
.lite-viz code, .lite-wrap code { font-family: ui-monospace; font-size: 1rem; padding: 0 .25rem; }
.lite-wrap pre { background: #f3f4f6; border: 1px solid #d1d7db; padding: 1.45; }
.lite-wrap table { border-collapse: collapse; width: 100%; }
.lite-wrap th, .lite-wrap td { border: 1px solid #d1d7db; padding: .5rem; }
.lite-wrap th { background: #f3f4f6; }
.lite-wrap details { margin: .6rem 0; border: 1px solid #d1d7db; }
.lite-wrap summary { cursor: default; font-weight: 700; }
.lite-toc { font-size: .92rem; }
.lite-toc ol { margin: .4rem 0 0 1.2rem; }
@media print {
.lite-wrap { max-width: none; padding: 0; }
h2 { page-break-before: always; }
h2:first-of-type { page-break-before: avoid; }
pre, table, details { page-break-inside: avoid; }
}
`;

```

/\*\* Раскрываем все спойлеры и вычищаем служебную разметку

```

 <nav className="lite-toc">
 <h3>Содержание</h3>

 {list.map((c) => (
 <li key={c.id}>
 <a href={`?lite=1&topic=${encodeURIComponent(
 chapterTopics[c.id]?.length ? ` – биле
 ""}

))}

 </nav>
)}

```

```

{list.map((chapter) => {
 const viz = getViz(chapter.id);
 const topics = chapterTopics[chapter.id] ?? [];
 const syncKeys = Object.keys(PAGE_SYNC).filter(
 return (
 <article key={chapter.id} id={chapter.id}>
 <h2>{chapter.title}</h2>
 <p className="lite-topics">
 {topics.length > 0 && <>Билет{topics.leng
 {chapter.category ? <>Раздел: {chapter.ca
 id страницы: <code>{chapter.id}</code>
 </p>

 <div
 dangerouslySetInnerHTML={{
 __html: linkifyTicketsHtml(toPlainTextH
 }}
 />

 {viz && (
 <div className="lite-viz">
 <h4>Визуализация: {viz.title}</h4>
 {viz.hint && <p>{viz.hint}</p>}
 {syncKeys.map((k) => {

```

```


 {q.options.map((option, j) => (
 <li key={j}>
 {j === q.correctIndex ?

)}}

 <p className="lite-meta">{q.explan

)}}

</div>
)}
</article>
);
)}}
</div>
);
};

export default LiteGuide;

```

ФАЙЛ 58/110: src/components/LiveDistTable.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import type { DebugValue } from "../data/debugRunner";

interface LiveDistTableProps {
 /** dist из кода пользователя: имя вершины → расстояние */
 dist: Record<string, DebugValue> | null;
 /** prev/p – предок на кратчайшем пути (необязательно */
 prev?: Record<string, DebugValue> | null;
 /** done/visited – зафиксированные вершины (необязате. */
 done?: DebugValue[] | null;
 /** Идентификаторы вершин, нарисованных на схеме демо

```

```
// Все вершины совпали со схемой – таблица не нужна,
if (unknown.length === 0) return null;

const doneSet = new Set((done ?? []).map((value) => S
const shown = keys.slice(0, 64);

return (
 <div className="mb-6 rounded-xl border border-amber
 <p className="text-[11px] leading-relaxed text-am
 {title} В вашем коде вершины{" "}

 {unknown.slice(0, 8).join(", ")}
 {unknown.length > 8 ? " ..." : ""}

 , а на схеме демо-графа нарисованы{" "}
 {demoIds.join(", ")}
 показаны отдельной таблицей. Чтобы они управлял
 (кнопка-«глаз» в панели Python подставляет его
 </p>
 <div className="mt-2.5 flex flex-wrap gap-1.5">
 {shown.map((key) => {
 const parent = prev ? fmt(prev[key]) : null;
 const fixed = doneSet.has(key);
 return (
 <span
 key={key}
 className={`inline-flex items-baseline gap-1
 fixed
 ? "border-emerald-500/40 bg-emerald-500/10
 : "border-slate-700 bg-slate-900/70 t
 `}
 title={fixed ? "вершина зафиксирована (done
 >
 <b className={fixed ? "text-emerald-300"
 =
 {fmt(dist[key])}
 {parent && parent !== "-" ? <span classNa
 {fixed ? <span className="text-emerald-400
```



```

useEffect(() => {
 document.body.style.overflow = open ? "hidden" : ""
 return () => {
 document.body.style.overflow = ""
 }
}, [open]);

useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escape"
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey)
}, []);

return (
 <>
 <div className="lg:hidden sticky top-[57px] z-40" >
 <button
 onClick={() => setOpen(true)}
 className="w-full flex items-center gap-2.5 p-2"
 aria-label="Открыть содержание"
 >

 <List className="w-4 h-4" />

 Содержание • {index + 1} из {total}

 {title}

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]" >
 <div className="absolute inset-0 bg-slate-950" >
 <div className="absolute inset-y-0 left-0 w-[100%] h-[100%]
 te-[tocIn .18s ease-out]" >

```



```

 onToggleCompiler: () => void;
}

/**
 * Шапка приложения.
 *
 * Вкладки «Тренажёры» больше нет: все демонстрации живы
 * пособия (каждая тема = текст + визуализация + Python
 *
 * Две кнопки PDF:
 * * «PDF пособия» — компактный автоматический сборник
 * весь текст тем и снимки визуализаций, БЕЗ кода па
 * * «PDF (код)» — полный дамп исходников проекта (370
 */
export const Navbar: React.FC<NavbarProps> = ({ compile
 const [busy, setBusy] = useState(false);

 const saveBook = async () => {
 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 };

 return (
 <header className="bg-slate-900 border-b border-sla
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
 ap-3 lg:gap-0">
 <div className="flex items-center gap-3 w-full
 <div className="bg-gradient-to-tr from-indigo
 k-0">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-v
 Универсальное пособие

```

```

 title="PDF пособия, PDF с кодом проекта,
 >
 <FileDown className="w-4 h-4" /> Скачать
</summary>
<div className="absolute right-0 top-full m
shadow-xl shadow-black/50 lg:static lg:mt-0
lg:p-0 lg:shadow-none">
<a
 id="download-guide-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/gu
download="guide_lite.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
ver:bg-indigo-600/30 border border-indigo-5
 title="Компактный PDF всего пособия: текст
 run export:guide)"
>
 <FileDown className="w-4 h-4" /> PDF пособи

<a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/fu
download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
over:bg-emerald-600/20 border border-emeral
 title="Полный PDF-дамп исходного кода проек
>
 <FileDown className="w-4 h-4" /> PDF (код)

<a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}export/fu
download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"

```

```

* Четыре вкладки:
* 1. «Планарность» — K5 и K3,3 с подсвеченными пересечениями;
* 2. «Жадная покраска» — $\chi \leq \Delta + 1$: вершины красятся по порядку,
* достаётся наименьший цвет, не занятый соседями;
* 3. «Двудольность» — проверка 2-раскрашиваемости BFS:
* нечётный цикл = двумя красками не обойтись;
* 4. «Покраска рёбер» — $\chi' \leq \Delta + 1$ (Визинг), а для двудольных
* $\chi' = \Delta$ (теорема Кёнига).
*
* Два графа на выбор: «основной» (8 вершин, $\Delta = 3$, есть цикл К5)
* и «двудольный» (6 вершин, $\Delta = 3$, $\chi = 2$, $\chi' = 3$).
*
* Синхронизация с Python-компилятором (PAGE_SYNC["planar"]);
* color/col — массив цветов вершин (красит кружки на панели);
* ecolord/ec — массив цветов рёбер, v — текущая вершина;
* u — сосед, chi/k — число цветов, delta/D — максимальная степень;
* i/step — номер кадра.
*/

```

```

type Tab = "planar" | "greedy" | "bipartite" | "edges";
type GraphKey = "main" | "bip";

```

```

interface Graph {
 key: GraphKey;
 label: string;
 n: number;
 edges: [number, number][];
 adj: number[][];
 pos: { x: number; y: number }[];
 names: string[];
}

```

```

const PALETTE = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];
const colorOf = (c: number) => (c >= 1 && c <= PALETTE.length) ? PALETTE[c - 1] : "#000000";

```

```

function makeGraph(key: GraphKey): Graph {
 if (key === "bip") {
 const edges: [number, number][] = [[0, 3], [0, 4], [1, 2], [1, 5], [2, 3], [2, 4], [3, 5], [4, 5]];

```

```

bad?: [number, number] | null;
}

/** Жадная покраска вершин: $\chi \leq \Delta + 1$. */
function greedyFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const color = new Array(g.n).fill(0);
 const delta = Math.max(...g.adj.map((a) => a.length))
 frames.push({
 title: "Старт: ни одна вершина не покрашена",
 note: `color = [0]·n. Порядок обхода – просто по номерам.
 Конечно уложится в $\Delta + 1 = \${delta + 1}$ цветов: у вершины v
 color: [...color], v: null, u: null, e: null,
 });
 for (let v = 0; v < g.n; v++) {
 const used = Array.from(new Set(g.adj[v].filter((x) => x[0] === v)));
 let c = 1;
 while (used.includes(c)) c += 1;
 color[v] = c;
 const deg = g.adj[v].length;
 frames.push({
 title: `Вершина  $\${g.names[v]}$ : берём цвет  $\${c}$ `,
 note: used.length
 ? `Соседи уже заняли цвета $\{\${used.join(", ")}\}$.
 Всегда хватит ($\chi \leq \Delta + 1$).`
 : `У вершины $\${g.names[v]}$ ещё нет покрашенных соседей.
 color: [...color], v, u: null, e: null, used,
 });
 }
 const chi = Math.max(...color);
 frames.push({
 title: `Готово: жадная покраска использовала  $\${chi}$  цветов`,
 note: ` $\chi_{\text{жадн}} = \${chi}$. Истинное хроматическое число χ
 , а теорема Брукса ($\chi \leq \Delta$, если граф не полный и не двудельный)
 color: [...color], v: null, u: null, e: null,
 });
 return frames;
}

```

```

 title: `Конфликт: ${g.names[v]} и ${g.names[u]}
 note: `Ребро (${g.names[v]}, ${g.names[u]})
 покрасить: $\chi \geq 3$ `,
 color: [...color], v, u, e: null, bad: conflict
 }));
 }
 }
}
frames.push({
 title: conflict ? "Граф НЕ двудольный" : "Граф двудольный",
 note: conflict
 ? `BFS упёрся в нечётный цикл (${g.names[conflict]})
 краски — их и даёт жадный алгоритм на вкладке «Жадная покраска»
 : "Все вершины разбиты на две доли, рёбер внутри доли нет,
 справедлива теорема Кёнига: $\chi' = \Delta$ (рёберная покраска)",
 color: [...color], v: null, u: null, e: null, bad: false
});
return frames;
}

```

```

/** Жадная покраска рёбер: $\Delta \leq \chi' \leq \Delta + 1$ (Визинг), для
function edgeFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const ecol = new Array(g.edges.length).fill(0);
 const inc: Map<number, number>[] = Array.from({ length: g.n }, () => new Map());
 const delta = Math.max(...g.adj.map(a => a.length));
 frames.push({
 title: "Старт: рёбра не покрашены",
 note: `Правильная рёберная покраска: у каждой вершины
 рёбер одной вершины различны, а по теореме Визинга
 $\chi' \leq \Delta + 1$ `,
 color: new Array(g.n).fill(0), ecol: [...ecol],
 });
 g.edges.forEach(([a, b], i) => {
 const usedA = Array.from(inc[a].values());
 const usedB = Array.from(inc[b].values());
 const used = Array.from(new Set([...usedA, ...usedB]));
 let c = 1;

```

```

const [graphKey, setGraphKey] = useState<GraphKey>("m
const [step, setStep] = useState(0);
const [playing, setPlaying] = useState(false);

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
}, [chapterId, tab]);

const graph = useMemo(() => makeGraph(graphKey), [grap
const frames = useMemo(() => {
 if (tab === "greedy") return greedyFrames(graph);
 if (tab === "bipartite") return bipartiteFrames(grap
 if (tab === "edges") return edgeFrames(graph);
 return [] as Frame[];
}, [tab, graph]);
const maxStep = Math.max(0, frames.length - 1);

useEffect(() => {
 setStep(0);
 setPlaying(false);
}, [tab, graphKey]);

useEffect(() => {
 if (!playing) return;
 const t = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 1300);
 return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? vizNumber(vars.step) ?? vizNum

```

```

 <Tabs tab={tab} setTab={setTab} />
 <PlanarityDemo />
 <p className="text-[11px] text-slate-500 mt-2">
 Формула Эйлера $V - E + F = 2$ и оценка $E \leq 3V$
 </p>
 </div>
);
}

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl">
 <Header />
 <Tabs tab={tab} setTab={setTab} />

 <div className="flex flex-wrap items-center gap-2">

 {[["main", "bip"] as GraphKey[]].map((k) => (
 <button
 key={k}
 onClick={() => setGraphKey(k)}
 className={`px-3 py-1.5 rounded-lg text-[11px]
 ${bgSlate90060} text-slate-400 hover:text-white`}
 >
 {makeGraph(k).label}
 </button>
))]}
 </div>

 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setStep(0); setPlaying(false) }}
 rounded-lg bg-slate-900 border border-slate-700
 >
 <button onClick={() => go(-1)} disabled={step === 0}
 border border-slate-700 text-slate-300 hover:text-white
 >
 <button onClick={() => setPlaying((p) => !p)}
 text-xs font-bold hover:bg-indigo-500">
 {playing ? <Pause className="w-4 h-4" /> : <Play />}
 </button>
 <button onClick={() => go(1)} disabled={step === 0}

```





```
 <Palette className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold text-white">K5/K3,3</h3>
 <p className="text-xs text-fuchsia-300">K5/K3,3</p>
 </div>
 </div>
</div>
);

const Tabs: React.FC<{ tab: Tab; setTab: (t: Tab) => void}> = ({ tab, setTab }) => {
 <div className="flex flex-wrap gap-2 mb-4">
 {TABS.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={["flex items-center gap-1.5 px-3 py-2",
 "bg-slate-900/60 text-slate-400 text-white" : "bg-slate-900/60 text-slate-400"]
 }>
 {t.icon} {t.label}
 </button>
))}
 </div>
);

export default PlanarColoringViz;
```

---

ФАЙЛ 63/110: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5</h4>
 <p className="text-xs text-slate-400">Просто смотри</p>
 </div>
);
}
```





```

 /** Кнопка «открыть страницу в пособии» – не нужна, е
 onOpenGuide?: () => void;
 /** Скрыть панель. */
 onClose: () => void;
}

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) =>
 setStderr: (options: { batched: (message: string) =>
 setStdin: (options: { stdin: () => string | number |
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promi
};

let runtimePromise: Promise<PyodideRuntime> | null = null

/**
 * Подмена интерпретатора для проверок: DOM-харнес (jsd
 * из node_modules) и CI-скриншоты кладут сюда готовый
 * на CDN. В браузере у пользователя поле не определено
 */
type PythonRuntimeProvider = PyodideRuntime | (() => Pr

declare global {
 interface Window {
 __algv0PythonRuntime?: PythonRuntimeProvider;
 }
}

function getPythonRuntime(): Promise<PyodideRuntime> {
 const injected = typeof window !== "undefined" ? wind
 if (injected) return typeof injected === "function" ?
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_I
 (module as PyodideModule).loadPyodide({ indexURL:

```

```
interface Snippet {
 label: string;
 tip: string;
 code: string;
}

const SNIPPETS: { group: string; items: Snippet[] }[] = [
 {
 group: "Сортировка",
 items: [
 {
 label: "arr.sort() / sorted()",
 tip: "In-place сортировка по возрастанию и новая",
 code: 'arr = [5, 2, 8, 1]\narr.sort()
) # копия по убыванию\n',
 },
 {
 label: "sorted(key=...)",
 tip: "Сортировка по ключу: рёбра по весу, пары",
 code: 'edges = [(3, "A", "B"), (1, "B", "C"), (
 es)\n',
 },
],
 },
 {
 group: "Циклы",
 items: [
 {
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j – как обход матрицы",
 code: "n, m = 3, 4\nfor i in range(n):",
 },
 {
 label: "while ...",
 tip: "Классический while-счётчик – как n-1 итерация",
 code: "i = 1\nwhile i < 5:\n print(f\"итерация {i}\")\ni += 1",
 },
],
 },
]
```

```

 {
 group: "Вывод",
 items: [
 {
 label: "f-строка",
 tip: "Подставить значения переменных шага в строку",
 code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}]", dist[i][j])'
 },
],
 },
],
};

/**
 * Переживает скрытие панели: код пользователя не теряется
 *
 * Хранилище — ПО СТРАНИЦАМ. Раньше слот был один, и код
 * восстанавливался на новой странице: редактор показывал
 * и визуализация жила значениями чужого кода.
 */
const savedEditors = new Map<string, { code: string; baseVarName: string }>();

export function PythonCompiler({ chapterId, chapterTitle, chapterTitleShort, compilerProps }) {
 /** Активная вкладка демонстрации страницы (у sparse-кода) */
 const [demoId, setDemoId] = useState<string | null>(null);
 const sync = useMemo(() => getPageSync(chapterId, demoId), [chapterId, demoId]);
 const syncVarBases = useMemo(
 () => {
 [...new Set(
 sync.variables.flatMap((v) =>
 v.name
 .split(/\\s*(?:,|\\/)\s*/)
 .map(baseVarName)
 .filter((name) => /^[A-Za-z_][A-Za-z0-9_]*$/))
)],
 [sync]
),
 [sync]
);
};

```

```

 return () => {
 aliveRef.current = false;
 };
 }, []));

// Код хранится по страницам, поэтому при открытии па
// бывает: рассинхрон возможен только после смены вкл
const [desyncedFrom, setDesyncedFrom] = useState<stri
const prevTplRef = useRef({ base: initTemplate, full:

/** Активная трасса. input нужен, чтобы изменение std
const [debug, setDebug] = useState<{ result: DebugRes
/** Esc оставляет неизменённый текст в редакторе, пок
const [editHold, setEditHold] = useState<string | nul
/** Автопроход включается пользователем; первый шаг п
const [playing, setPlaying] = useState(false);
const [playbackSpeed, setPlaybackSpeed] = useState<num
 if (typeof window === "undefined") return 1;
 const saved = Number(window.localStorage.getItem(PL
 return PLAYBACK_SPEEDS.includes(saved as (typeof PL
));
const listRef = useRef<HTMLDivElement>(null);

useEffect(() => {
 window.localStorage.setItem(PLAYBACK_SPEED_KEY, Str
}, [playbackSpeed]);

useEffect(() => {
 savedEditors.set(chapterId, { code, base: initTempl
}, [code, initTemplate, chapterId]);

// демонстрация сообщает, какая вкладка открыта – код
useEffect(() => setDemoId(null), [chapterId]);
useEffect(() => onVizDemo(chapterId, (d) => setDemoId
useEffect(() => () => clearVizState(chapterId), [chap

// Смена страницы (или вкладки демо): нетронутый реда
// инициализацию; свой код не затираем – предлагаем с
```

/\* — Пошаговый отладчик

/\*\* Связь больше не зависит от эталонных строк: совпа  
const hasViz = sync.variables.length > 0;

/\*\*

\* Снимок ПОСЛЕ выделенной строки. Контейнеры берём и  
\* (там уже виден append/st[i][j]=...), но индексы цикл  
\* события: после st[i][j]= Python уже мог увеличить  
\*/

const snapshotAt = useCallback((result: DebugResult,   
const step = result.steps[idx];  
const next = result.steps[idx + 1];  
const afterLocals = next?.locals ?? result.finalLoc  
const afterValues = next?.values ?? result.finalVal  
const beforeValues = step?.values ?? {};  
const line = step?.line ? source.split("\n")[step.l  
// Нас интересуют только простые скалярные цели (`i  
// `i +=`). Контейнеры и так всегда берутся из after  
// внутри print(f"i={i}") не будет ошибочно принят  
const assignment = line.match(  
 /^\\s\*([A-Za-z\_][A-Za-z0-9\_]\*)?(?:\\s\*,\\s\*[A-Za-z\_][A  
 );  
const assigned = new Set(assignment?.[1].split(",")

const variables = { ...afterValues };  
const locals = { ...afterLocals };  
for (const [name, value] of Object.entries(beforeVa  
 const scalar = value === null || typeof value !==  
 if (scalar && !assigned.has(name) && name in after  
 variables[name] = value;  
 if (step?.locals[name] !== undefined) locals[na  
 }  
}  
const changed = changedVars(step?.locals, locals);  
return {  
 line: step?.line,



```

const stepBy = useCallback(
 (delta: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

const stepTo = useCallback((idx: number) => goDebug(i

const stopDebug = useCallback(() => {
 setPlaying(false);
 setEditHold(code);
 setDebug(null);
 clearVizState(chapterId);
}, [chapterId, code]);

// Базовый интервал 650 мс; множитель скорости хранитс
useEffect(() => {
 if (!playing || !debug) return;
 if (debug.idx >= debug.result.steps.length - 1) {
 setPlaying(false);
 return;
 }
 const t = window.setTimeout(() => stepBy(1), 650 /
 return () => window.clearTimeout(t);
}, [playing, debug, stepBy, playbackSpeed]);

const debugRun = useCallback(async (autoPlay = false)
 if (running) return;

 const source = code;
 const input = stdin;
 const request = traceRequest;

```

```

 if (result.steps.length === 0) {
 // Шагов нет (только комментарии, синтаксическая
 // заменяется пустым листингом, а визуализация
 // показывает собственный встроенный сценарий (
 setDebug({ result, idx: 0, src: source, input,
 clearVizState(chapterId);
 setOutputOpen(true);
 setOutput(
 result.error
 ? `Ошибка:\n${result.error}`
 : printed || "В коде нет исполняемых строк
);
 setRuntimeMessage(
 result.error
 ? "Код не выполнен — редактор остался отк
 : "Нет шагов трассы: связь с визуализацией
);
 } else {
 setDebug({ result, idx: 0, src: source, input,
 setPlaying(autoPlay && result.steps.length > 1)
 // Программа что-то напечатала — значит, вывод
 // результат своего же print() панель не имеет
 if (printed.trim() !== "") setOutputOpen(true);
 setOutput(
 (printed || "Программа ничего не вывела — тра
 (result.error ? `\\n\\nОшибка (выполнение пре
);
 if (result.error) setOutputOpen(true);
 setRuntimeMessage(
 result.truncated
 ? `Выполнение остановлено после ${DEBUG_STE
 : "Трасса готова автоматически. ← → — шаги,
);
 }
} catch (error) {
 if (!aliveRef.current) return;
 const message = errorText(error);
 setDebug({ result: { steps: [], error: message, t

```

```

 e.stopPropagation();
 e.preventDefault();
 setPlaying((v) => !v);
 }
 if (e.key === "Escape") stopDebug();
};
// capture-фаза: чтобы глобальный обработчик навига
window.addEventListener("keydown", onKey, true);
return () => window.removeEventListener("keydown",
}, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
/**
 * Панель показывает листинг трассы, только если шаги
 * Пустой код или синтаксическая ошибка больше не пря
 * раньше на их месте возникал пустой листинг без воз
 * «исчезала» и кнопка, и само поле ввода.
 */
const showTrace = !!debug && debug.result.steps.length
/** Трасса построена по старому тексту: пользователь
const stale = !!debug && debug.src !== code;
/** Листинг трассы можно свернуть, но редактор от это
const [traceOpen, setTraceOpen] = useState(true);
const curDebug = debug ? { step: debug.result.steps[d
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
 if (curDebugLine === undefined || !listRef.current)
 listRef.current
 .querySelector(`[data-line="${curDebugLine}"]`)
 ?.scrollIntoView({ block: "nearest" });
}, [curDebugLine]);

// производные данные шага: что изменилось, в каком п
// Строка, которую назначает текущий шаг, подсвечивае
// локали берём со следующего шага трассы (для послед
// трейсером на возврате из фрейма). Значит на строке
const shownLocals = useMemo(() => {
 if (!debug || !curDebug?.step) return {};

```

```

return (
 <aside
 aria-label="Python-компилятор"
 className="fixed z-40 inset-x-0 bottom-0 h-[58dvh]
 -[600px] flex flex-col bg-slate-900 border-t lg
 style={typeof window !== "undefined" ? {
 ...(width && window.innerWidth >= 1024 ? { width:
 ...(height ? { height: Math.min(height, window.h
 } : undefined)}
 >
 {/* Верхний край меняет высоту на телефоне и деск
 {onHeightChange && (
 <div
 role="separator"
 aria-orientation="horizontal"
 aria-label="Потяните, чтобы изменить высоту т
 title="Потяните, чтобы изменить высоту термин
 className="absolute -top-1.5 left-0 right-0 h
 onPointerDown={(event) => {
 const el = event.currentTarget;
 el.setPointerCapture(event.pointerId);
 const startY = event.clientY;
 const startH = el.parentElement?.getBoundin
 const onMove = (moveEvent: PointerEvent) =>
 const maxHeight = window.innerHeight - (w
 const minHeight = Math.min(260, maxHeight
 const next = Math.round(startH + startY -
 onHeightChange(Math.max(minHeight, Math.m
 });
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp
 el.removeEventListener("pointercancel", on
 };
 el.addEventListener("pointermove", onMove a
 el.addEventListener("pointerup", onUp as Ev
 el.addEventListener("pointercancel", onUp a
 })
)
)
}

```

```

<Terminal className="w-4 h-4 text-emerald-400 s
<span className="text-[13px] font-bold text-whi
<span className="hidden sm:inline text-[10px] t

<div className="ml-auto flex flex-wrap items-ce
 <Tooltip
 side="bottom"
 content={running ? "Python выполняется..." : "
 д и запустить автопроход"}
 >
 <button
 type="button"
 onClick={runOrTogglePlayback}
 disabled={running}
 className={`inline-flex items-center gap-
0 ${
 playing
 ? "bg-amber-500/15 text-amber-300 hov
 : "bg-emerald-500/15 text-emerald-300
 }`}
 aria-label={running ? "Python выполняется
 >
 {running ? <Loader2 className="h-4 w-4 an
 " />}
 {playbackSpeed}x
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Скопировать к
 <button
 type="button"
 onClick={() => void copyCode()}
 className="p-1.5 rounded-lg text-slate-40
 aria-label="Скопировать код"
 >
 {copied ? <Check className="w-4 h-4 text-
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Вернуть комме

```

```

 setCode(next ? template : "");
 setDesyncedFrom(null);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 clearVizState(chapterId);
 setTraceRequest((n) => n + 1);
 }}
 className={`p-1.5 rounded-lg transition-colors
 refOn
 ? "bg-indigo-500/30 text-white ring-1
 : "text-indigo-300 hover:text-white h
 }`
 aria-label={refOn ? "Убрать эталонный код
ктор"}
 aria-pressed={refOn}
>
 {refOn ? <Eye className="w-4 h-4" /> : <E
</button>
</Tooltip>
<label
 className="ml-1 inline-flex items-center gap-1
 der border-slate-700 hover:border-emerald-500
 title="Скорость автоматического проигрывани
>
 {running ? <Loader2 className="w-3.5 h-3.5
ld-400" />}
 {running
 <select
 aria-label="Скорость автопрохода"
 value={playbackSpeed}
 onChange={(event) => setPlaybackSpeed(Num
 className="cursor-pointer rounded bg-slate
 >
 {PLAYBACK_SPEEDS.map((speed) => (
 <option key={speed} value={speed}>{speed
)))}
 </select>

```

```

 hasViz ? "text-emerald-400" : "text-slate-400"
 }` }
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет визита"}

</Tooltip>
</div>
</div>

{stripTab === "vars" ? (
 <div className="px-3 pb-2.5 space-y-2 max-h-300">
 <div className="flex items-center gap-2 min-h-300">
 {onOpenGuide ? (
 <Tooltip side="bottom" content="Открыть руководство" >
 <button
 type="button"
 onClick={onOpenGuide}
 className="inline-flex items-center justify-center gap-2
 text-slate-300 hover:text-white hover:bg-slate-700" >
 {chapterTitle}
 </button>
 </Tooltip>
) : (
 <span className="inline-flex items-center justify-center gap-2
 text-slate-300 max-w-full">
 {chapterTitle}

)}
 </div>
 <div>
 {desyncedFrom !== null && (
 <Tooltip
 side="bottom"
 content="В редакторе ваш код, а страница — сгенерированная.
 Если что-то не так, нажмите на иконку. Если всё в порядке,
 то заменён)."
 >
 <button

```

```

 tabIndex={0}
 className="cursor-help inline-flex items-center justify-center gap-1"
 text-[10px] text-slate-400 hover:text-white
 >
 <ArrowRightLeft className="w-3 h-3 text-slate-400" />
 {s.title}
 </Tooltip>
)}
 </div>
) : (
 <div className="px-3 pb-2.5 max-h-36 overflow-y-auto">
 {SNIPPETS.map(({gItem}) => {
 <div key={gItem.group}>
 <div className="text-[9px] font-bold uppercase">{gItem.group}</div>
 <div className="flex flex-wrap gap-1.5">
 {gItem.items.map(({s}) => {
 <Tooltip key={s.label} side="bottom">
 <button
 type="button"
 onClick={() => insertSnippet(s)}
 className="inline-flex items-center justify-center gap-1"
 text-[10.5px] text-indigo-300 hover:text-white
 title=""
 >
 <Plus className="w-3 h-3 text-slate-400" />
 {s.label}
 </button>
 </Tooltip>
)}}
 </div>
 </div>
)}
 </div>
) : (
 <p className="text-[10px] text-slate-600">Hello, world!</p>
)

```



```

}}
spellCheck={false}
aria-label="Код Python"
className="flex-[2] min-h-[110px] w-full resi
line-none focus:ring-2 focus:ring-inset foc
placeholder="Напишите Python-код... (пустой ред
/>

```

```
{showTrace && debug && {
 <div className={`flex flex-col border-t borde
 /* панель управления шагами */
 <div className="shrink-0 flex flex-wrap ite
 <Tooltip content={playing ? `Пауза автопро
 роиграть трассу заново со скоростью ${playb
 л)`}>
 <button
 type="button"
 onClick={runOrTogglePlayback}
 className={`mx-0.5 inline-flex items-
 r-500/15 hover:bg-amber-500/25` : "text-еме
 aria-label={playing ? `Пауза автопрох
 >
 {playing ? <Pause className="w-4 h-4"
 <span className="text-[9px] font-bold
 </button>
 </Tooltip>
 <Tooltip content="В начало трассы">
 <button type="button" aria-label="В нач
 }} disabled={!curDebug?.step} className="p-
 transition-colors">
 <SkipBack className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг назад (←) – автопро
 <button type="button" aria-label="Шаг н
 bled={!curDebug?.prev} className="p-1 round
 tion-colors">
 <ChevronLeft className="w-4 h-4" />
```

```

 tabIndex={0}
 className={`cursor-help inline-flex items-center justify-center gap-2
"border-emerald-500/40 bg-emerald-500/10 text-emerald-500 transition-colors duration-200"
 >
 {hasViz ? <Link2 className="w-3 h-3" href="#">
 {hasViz ? "имена → демо" : "свободный"}

 </Tooltip>
 <Tooltip content={traceOpen ? "Свернуть лог трассы и переменные текущего шага."}>
 <button
 type="button"
 aria-label={traceOpen ? "Свернуть лог трассы" : "Показать лог трассы"}
 aria-expanded={traceOpen}
 onClick={() => setTraceOpen((v) => !v)}
 className="ml-auto inline-flex shrink-0 items-center justify-center gap-2
ext-[10px] font-bold text-slate-300 transition-colors duration-200"
 >
 {traceOpen ? <ChevronDown className="w-3 h-3" /> : <ChevronUp className="w-3 h-3" />}
 ЛОГ ТРАССЫ
 </button>
 </Tooltip>
 <Tooltip content="Остановить прогон (Esc)"}
 на показывает встроенное демо. Автозапуск не работает.
 <button
 type="button"
 aria-label="Остановить прогон трассы"
 onClick={stopDebug}
 className="inline-flex shrink-0 items-center justify-center gap-2
ext-[10px] font-bold text-rose-300 transition-colors duration-200"
 >
 <CircleStop className="w-3.5 h-3.5" />
 стоп
 <kbd className="rounded border border-rose-500/40 px-1 text-rose-300" />
 </button>
 </Tooltip>
 </div>

```

```

 {debugOrdered.length === 0 && !debug.result
 <div className="mt-1 text-[11px] text-slate-500">
 /div>
 })
 <div className="mt-1 flex flex-wrap gap-1">
 {debugOrdered.map(([name, value]) => {
 const changed = debugChanged[name];
 const synced = syncVarSet.has(name);
 return (
 <span
 key={name}
 className={`inline-flex items-baseline gap-1.5`}
 changed={changed}
 synced={synced}
 ? "border-amber-500/50 bg-amber-500/10"
 : "border-emerald-500/40 bg-emerald-500/10"
 : "border-slate-700 bg-slate-700/10"
 >

 {name}

 {value}

 {synced ? "synced" : "out of sync"}

);
 })}
 </div>
 </div>
</>
})
</div>
)}
{debug && !showTrace && debug.result.error ? (
 <pre className="shrink-0 max-h-28 overflow-auto py-2 font-mono text-[11px] leading-snug text-slate-500">
 {debug.result.error}
 </pre>
) : null}

```

```
 className="flex-1 min-h-[70px] overflow-auto
 ading-5 text-slate-300"
 >
 {output}
 </pre>
)}
 <div className="px-3 py-1.5 border-t border-sla
 </div>
</aside>
);
}
```

---

ФАЙЛ 65/110: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray, vizString } from '../v
```

```
interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
 { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue-500 to-purple-500', bubbleText: 'Матана!' },
 { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet-500 to-indigo-500', bubbleText: 'Порцию борща!' },
 { name: 'Кот Борис', emoji: '🐱', color: 'from-amber-500 to-orange-500', bubbleText: 'Плиз.' },
]
```

```

const addLog = (msg: string) => setLog(prev => [msg,

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяже.
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoj

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.
 }, 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog('⚠ DEQUEUE: В очереди никого нет! Повар ск
 setTimeout(() => setShake(false), 400);
 return;

```

```

 setQueue(prev => prev.map(c => ({ ...c, animState
 }, 500));
};

```

```

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-in
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 tex
 <p className="text-xs text-slate-300 mt-1 lea
 В очереди за супом всё честно. Кто первый в
 Новые гости встают строго в конец оч
 Здесь нет обхода и блата – только строгий п
 </p>
 </div>
 </div>
 </div>

```

```

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap
 <button
 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-t
 d:opacity-40 disabled:cursor-not-allowed te
 ive:scale-95 transition-all cursor-pointer
 >
 Встать в очередь (ENQUEUE)
 </button>

```

```

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-t
 opacity-40 disabled:cursor-not-allowed text
 :scale-95 transition-all cursor-pointer fle
 >

```

```
ase font-mono tracking-wider">
```

```
 ПОВАР
```

```

```

```
</div>
```

```
</div>
```

```
{/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
```

```
<div className="flex flex-col items-cente
```

```
 <span className="text-2xl animate-pulse
```

```
 <span className="text-[8px] font-mono t
```

```
</div>
```

```
{/* СПИСОК ОЧЕРЕДИ */}
```

```
<div className="flex-1 flex items-end gap
```

```
 {displayQueue.length === 0 ? {
```

```
 <div className="flex-1 flex flex-col
```

```
 </
```

```
 <p className="text-xs font-bold fon
```

```
 <p className="text-[10px]">Все сыты
```

```
 </div>
```

```
 } : {
```

```
 displayQueue.map(({customer, idx}) => {
```

```
 const isHead = idx === 0;
```

```
 const isTail = idx === displayQueue
```

```
 return (
```

```
 <div
```

```
 key={customer.id}
```

```
 className={`
```

```
 flex flex-col items-center ga
```

```
 ${customer.animState === 'in'
```

```
 ${customer.animState === 'eat
```

```
 ${customer.animState === 'out
```

```
 `}
```

```
 >
```

```
 {/* Облачко мыслей */}
```

```
 <div className={`
```

```
 absolute -top-16 left-1/2 -tr
```

```
8 text-center text-slate-200 leading-tight
```

```
</div>
```

```
{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
```

```
<div className="md:col-span-3 bg-slate-900 rounded-3xl" style={{width: 100%; height: 100%;}}>
```

```
 <div className="absolute top-4 left-4 text-[11px] font-mono">
```

```
 Сытые гости (Архив FIFO)
```

```
 </div>
```

```
 <div className="absolute top-4 right-4 flex items-center justify-center" style={{width: 100%; height: 100%;}}>
 <div style={{width: 100%; height: 100%;}}>
```

```
 <Sparkles className="w-3.5 h-3.5 animate-pulse" style={{width: 100%; height: 100%;}}>
```

```
 СЫТЫ
```

```
 </div>
```

```
<div className="flex-1 flex flex-col justify-between" style={{width: 100%; height: 100%;}}>
```

```
 {servedHistory.length === 0 ? (
```

```
 <div className="flex-1 flex flex-col items-center justify-center" style={{width: 100%; height: 100%;}}>
```

```
 Здесь 6
```

```
 <p className="text-xs font-bold font-mono">Здесь 6</p>
```

```
 <p className="text-[10px] mt-1">Здесь 6</p>
```

```
 </div>
```

```
) : (
```

```
 <div className="flex flex-col gap-1.5 w-full" style={{width: 100%; height: 100%;}}>
```

```
 {servedHistory.map((item, idx) => (
```

```
 <div
```

```
 key={idx}
```

```
 className="w-full py-1.5 px-3 rounded-3xl" style={{width: 100%; height: 100%;}}>
```

```
 <div style={{width: 100%; height: 100%;}}>
```

```
 <
```

```
 {item}
```

```
 </div>
```

```
 </div>
```

```
 </div>
```

```
 </div>
```

```
</div>
```

```
{/* Пол */}
```

```
<div className="w-full h-3 bg-gradient-to-r from-slate-900 to-slate-800" style={{width: 100%; height: 100%;}}>
```



```

 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR',
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: T
 const [simulationStep, setSimulationStep] = useState<
 const [bfsQueue, setBfsQueue] = useState<number[]>([]
 const [currentNode, setCurrentNode] = useState<number
 const [speechText, setSpeechText] = useState<string>('
 'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
 буль!')
);
 const [isTalking, setIsTalking] = useState<boolean>(fa
 const [isBlinking, setIsBlinking] = useState<boolean>

 /* — связь с Python-компилятором (PAGE_SYNC["aho-com
 words/patterns — набор образцов: бор ПЕРЕСТРАИВАЕТ
 state — текущая вершина автомата, step — шаг BFS,
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
 }, [chapterId]);

 const liveWordsRaw = vars ? vizArray(vars.words) ?? v
 const liveWordsSig = useMemo(() => (liveWordsRaw ? JS
 const liveState = vars ? vizNumber(vars.state) ?? viz
 const liveStep = vars ? vizNumber(vars.step) ?? vizNu
 const liveQueue = vars ? vizArray(vars.queue) : null;
 const liveNodes = vars ? vizNumber(vars.nodes) : null;

```

```

useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
}, []);

// Эффект разговора при смене текста
useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false),
 return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {}
 };
 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],

```

```

 setSimulationStep(0);
 setBfsQueue([]);
 setCurrentNode(null);
 setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД)
 S, чтобы расставить fail-ссылки!`
);
 };

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове S`);
 return;
 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем построить БПД');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Запуск BFS

```

```

 for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char]) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
 }

 logs += `Для #${u} ('${char}') fail = #${failTarget}`;
 }

 if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переходим к следующему';
 }

 setNodes(updatedNodes);
 setBfsQueue(newQueue);
 setCurrentNode(r);
 setSpeechText(logs);
 };

 return (
 <div className="bg-slate-800/90 rounded-2xl p-6 border" >
 {/* Шапка виджета */}
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Интерактивный конструктор: Говорящий Эхо-

```

```

 fill="#be123c"
 className="origin-[35px_100px] animate-
 />

 {/* Плавники */}
 <path d="M 90 55 L 70 30 L 110 55 Z" fill=
 <path d="M 90 145 L 70 170 L 110 145 Z" f
 <path
 d="M 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate
 />

 {/* Глаз (с морганием) */}
 <circle cx="145" cy="85" r="12" fill="#ff
 {isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85"
) : (
 <
 <circle cx="148" cy="85" r="6" fill="
 <circle cx="151" cy="82" r="2" fill="
 </>
)}

 {/* Рот (анимация разговора) */}
 {isTalking ? (
 <path d="M 165 105 Q 155 120 165 125 Q
) : (
 <path d="M 165 110 Q 155 115 168 118" s
)}

 {/* Улыбка / жабры */}
 <path d="M 130 80 Q 125 100 130 120" stro
 <path d="M 120 85 Q 115 100 120 115" stro
</svg>

{/* Декоративные пузыри */}
<div className="absolute -top-1 right-4 tex

```

```

 </button>
 })
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-700 shadow-blue-900/40 transition-transform"
 >
 Шаг BFS

 </button>
 })
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-700 shadow-indigo-900/40 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезагрузка
 </button>
 })
 </div>
 </div>
</div>
</div>

```

```

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800" >
 <h5 className="text-white font-bold mb-3 flex items-center" >
 Дерево и Суффиксные (fail) ссылки
 </h5>

```

```

{(() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 })
})}

```

```

 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
 </g>
);
}
}

{/* Draw Fail Links */}
{simulationStep > 0 && Object.values(nodes).map(node => {
 if (node.id === 0 || node.fail === 0) return;
 // Wait, all nodes get fail link. Let's draw it
 // To avoid too many lines, let's draw it only when step >= 1 and computed it
 const target = nodes[node.fail];
 if (!target || !node.x || !node.y || !target.x || !target.y) return;

 // Simple bezier curve for fail link
 const isCurrent = currentNode === node.id;

 return (
 <path
 key={`fail-${node.id}`}
 d={`M ${node.x} ${node.y - 10} Q ${node.x + 16} ${node.y - 16} ${target.x} ${target.y - 10}`}
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(128, 128, 128)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
});

{/* Draw Nodes */}
{Object.values(nodes).map(node => {

```

```

 </text>
 {!isRoot && {
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
 }}
 </g>
);
 }}}
</svg>
);
}}({})
</div>

```

```

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
 <div className="bg-slate-900/50 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 Словарь (Паттерны)
 </h5>
 <div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => {
 <span
 key={w}
 className="bg-slate-800 border border-slate-700 padding-1 group"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:text-white"
 title="Удалить"
 >

```



```

<div className="max-h-[220px] overflow-y-auto" style={{ width: '100%' }}>
 <table className="w-full text-left border-collapse" style={{ width: '100%' }}>
 <thead>
 <tr className="border-b border-slate-700" style={{ width: '100%' }}>
 <th className="py-2 px-3" style={{ width: '10%' }}>ID Вершины</th>
 <th className="py-2 px-3" style={{ width: '10%' }}>Символ</th>
 <th className="py-2 px-3" style={{ width: '10%' }}>Родитель</th>
 <th className="py-2 px-3" style={{ width: '10%' }}>fail-ссылка</th>
 <th className="py-2 px-3" style={{ width: '10%' }}>term_link</th>
 <th className="py-2 px-3" style={{ width: '10%' }}>Дети</th>
 <th className="py-2 px-3" style={{ width: '10%' }}>Слова (dict)</th>
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800" style={{ width: '100%' }}>
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node.id
 const isInQueue = bfsQueue.includes(node.id)

 return (
 <tr>
 <td>
 {node.id}
 </td>
 <td>
 {node.symbol}
 </td>
 <td>
 {node.parent}
 </td>
 <td>
 {node.fail}
 </td>
 <td>
 {node.term_link}
 </td>
 <td>
 {node.children}
 </td>
 <td>
 {node.words}
 </td>
 </tr>
)
 })}
 </tbody>
 </table>

```

```

 .map({[c, id]} => `${c}→#`
 .join(', '))}
 </td>
 <td className="py-2.5 px-3">
 {node.words.length === 0 ? (

) : (

 {node.words.join(', ')}

)}
 </td>
 </tr>
);
}}
</tbody>
</table>
</div>
</div>
</div>

```

```

{/* что сейчас сообщил компилятор */}
<div className="mt-4 flex flex-wrap items-center">

 <Terminal className="w-3.5 h-3.5" /> из компилятора

 {compilerLinked ? (
 <>
 {liveWordsSig && words = {words.join(", ")}}
 {liveNodes !== null && вершин в бопе = {liveNodes}}
 {liveState !== null && state = {liveState}}
 {liveStep !== null && шаг BFS = {liveStep}}
 {liveQueue && queue = {JSON.stringify(liveQueue).slice(0, 100)}}
 </>
) : (
 компилятор не работает
)}
</div>

```

```
function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
}
go(1, 0, n - 1);
return tree;
}
```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```
function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
 void CODE;

 function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 }
 }
}
```

```

 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v] });
 const left = go(2 * v, l, m, qL, qR);
 const right = go(2 * v + 1, m + 1, r, qL, qR);
 const res = left + right;
 steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
 rgeChildren: [2 * v, 2 * v + 1], arrayHighlight: [v] });
 return res;
 }
 const ans = go(1, 0, n - 1, qL, qR);
 steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]`,
 ghlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
 return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

 const steps: Step[] = [];
 let id = 0;
 let l = qL + n;
 let r = qR + n + 1; // half-open [l, r)
 let res = 0;

 steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`,
 codeLine: 3, treeState: tree, highlightNodes: [l, r] });

 while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 }
 if (r & 1) {
 res += tree[r - 1] ?? 0;
 }
 l = (l + 1) / 2;
 r = (r - 1) / 2;
 }

 return steps;
}

```

```

" // -----",
" // -----",
" mid = (l + r) >> 1;",
" build(2*v, l, mid);",
" build(2*v+1, mid+1, r);",
" tree[v] = tree[2*v] + tree[2*v+1];",
"}"
];
const QUERY_TD_CODE = [
"query(v, l, r, ql, qr) {",
" if (ql > r || qr < l) return 0;",
" // -----",
" if (ql <= l && r <= qr) return tree[v];",
" // -----",
" mid = (l + r) >> 1;",
" // спускаемся в обоих детей",
" return query(left) + query(right);",
"}"
];

```

```

const QUERY_BU_CODE = [
"queryBU(ql, qr) {",
" res = 0;",
" l = ql + n; r = qr + n + 1;",
" // -----",
" while (l < r) {",
" if (l & 1) res += tree[l++];",
" // -----",
" if (r & 1) res += tree[--r];",
" l >>= 1; r >>= 1;",
" }",
" return res; }"
];

```

// ===== PLAYER COMPONENT =====

```

function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

```

```

 if (isHigh) cls = `${clr.ring} text-white ring-2 sc
 else if (isChild) cls = `${clr.childRing} text-ambe
 else if (has) cls = `${clr.filled} text-slate-200`;

 return (
 <div key={nd.v} className="flex flex-col items-ce
 <div className={`flex flex-col items-center jus
 ${cls}`}>
 <span className="text-[7px] opacity-60 font-m
 [{nd.l
 <span className="text-xs font-extrabold font-i
 </div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full l
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
)}
 </div>
);
 }, [step, clr]);

 return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 /* Header */
 <div className={`px-4 py-2.5 flex items-center ju
 -emerald-950/50' : 'bg-amber-950/50'} border-b
 <h4 className="font-bold text-sm text-white fle
 <span className="text-[10px] font-mono text-sla
 </div>

 /* Array strip */
 <div className="px-3 py-2 bg-slate-950 border-b b
 {arr.map((v, i) => {

```

```

 {step.result !== undefined && {
 <span className={`ml-auto shrink-0 font-mono
 ' : color === 'emerald' ? 'text-emerald-300
 = {step.result}

 }}
 </div>

```

```

{ /* Controls */}
<div className="px-3 py-2.5 bg-slate-950 border-t
 <div className="flex items-center bg-slate-900
 <button onClick={() => { setPlaying(false); s
 unded" title="Сброс"><RotateCcw className="
 <button onClick={() => { setPlaying(false); s
 Шаг назад" className="p-1.5 text-slate-400
 Name="w-3.5 h-3.5" /></button>
 <button onClick={() => setPlaying(!playing)}
 white transition-all ${playing ? 'bg-rose-5
 {playing ? <Pause className="w-3 h-3" /> :
 </button>
 <button onClick={() => { setPlaying(false); s
 р вперёд" title="Шаг вперёд" className="p-1
 <ChevronRight className="w-3.5 h-3.5" /></b
 </div>
 <select value={speed} onChange={e => setSpeed(N
 text-[9px] font-bold rounded px-1.5 py-1 outl
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
</div>

```

```

{ /* Progress bar */}
<div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
</div>
);

```

```

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border border-
 {/* ---- TOP BAR: array editor + query range ---- */}
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4">
 {/* Array input */}
 <form onSubmit={handleApply} className="flex-
 <div className="flex items-center justify-b
 <label className="text-[10px] font-bold up
 label>
 <button type="button" onClick={randomize}
 "><RefreshCw className="w-3.5 h-3.5" /></bu
 </div>
 <div className="flex gap-2">
 <input
 value={customInput}
 onChange={e => setCustomInput(e.target.v
 className="flex-1 bg-slate-900 border b
 one focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo
 ition-colors">Применить</button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1
 {arr.map((v, i) => (
 <span key={i} className={`border px-2 p
 ext-amber-200 ring-2 ring-amber-400/30` : "l
 <span className={liveI === i ? "text-

)}}
 <span className="text-xs text-slate-500 f
 </div>
 </form>

```



```

<button onClick={() => setView('build')} className=
 -colors ${view === 'build' ? 'border-indigo-500
 -slate-200'}`}>
 <Hammer className="w-3.5 h-3.5" /> Построить
</button>
<button onClick={() => setView('queries')} clas
 on-colors ${view === 'queries' ? 'border-emera
 er:text-slate-200'}`}>
 <Search className="w-3.5 h-3.5" /> Запрос: св
</button>
<button onClick={() => setView('theory')} class
 n-colors ${view === 'theory' ? 'border-blue-500
 te-200'}`}>
 <Info className="w-3.5 h-3.5" /> Почему / Сра
</button>
</div>

```

```

{/* ---- TAB CONTENT ---- */}

```

```

{view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
)}

```

```

{view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху
 icon="! "
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 >

```



```

/**
 * Группы – это разделы пособия (SECTIONS), а не случайные
 * связанные темы стоят рядом, порядок разделов не зависит от
 * страница встретилась первой.
 */
const groups = useMemo(() => {
 const q = query.trim().toLowerCase();
 const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
 return SECTIONS.map((section) => ({
 section,
 items: filtered.filter((c) => sectionOf(c.id).id === section.id).filter((g) => g.items.length > 0);
 })), [query]);

return (
 <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:p-3" >
 <div className="p-3 pb-2 shrink-0">
 <h3 className="text-xs font-bold text-slate-400">
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute left-0 top-0">
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам..."
 className="w-full bg-slate-800/70 border border-slate-500 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-y-1/2 text-slate-500 hover:text-indigo-500"
 aria-label="Очистить поиск"
 />
 <X className="w-3.5 h-3.5" />
)}
 </div>
 </div>
 </div>
)

```

```

nt-bold text-indigo-300">
 {num ? num[1] : "•"}

<span className="font-semibold text
 {label}
 {hasViz && (
 <span className="ml-1.5 inline-l
ая визуализация" />
)}

<ChevronRight
 className={`w-4 h-4 shrink-0 mt-0
 isSelected ? "text-indigo-400"
 }`
 />
</button>
);
 }}}
</div>
</div>
)}}
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-l
] text-slate-300 space-y-1.5 hidden lg:block">
 <div className="font-bold text-indigo-400 flex
 <Sparkles className="w-3.5 h-3.5" /> Как поль
 </div>
 <p className="leading-relaxed">
 <b className="text-slate-200">Порядок чтения
 номерам билетов 1 → 24, каждая опирается на пр
 Следующая тема» продолжают этот же порядок.
 </p>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по
 симулятора.
 </p>
 <p className="text-slate-400 italic">Стрелки ←

```

```

 { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
 { id: '3', name: 'Блюдец от торта', icon: ' ' },
]);
const [popMessage, setPopMessage] = useState<string>('');

// Queue state
const [queue, setQueue] = useState<Person[]>([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
const [dequeueMessage, setDequeueMessage] = useState<string>('');

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', icon: ' ' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный пирог"', icon: ' ' },
 { id: '3', text: 'Кандидат: "Привет, я испанский интеллект"', icon: ' ' },
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.5);
const [heapMessage, setHeapMessage] = useState<string>('');

// Stack handlers
const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: ' ' },
 { name: 'Кастрюля от супа', icon: ' ' },
 { name: 'Чашка от кофе', icon: '☹' },
 { name: 'Миска с остатками салата', icon: ' ' },
 { name: 'Тарелка от тако', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue([newPlate, ...queue]);
 setAddPlateMessage('');
};
const dequeuePlate = () => {
 if (queue.length > 0) {
 setQueue(queue.slice(1));
 setDequeueMessage('');
 }
};
const popMessageHandler = () => {
 if (popMessage.length > 0) {
 setPopMessage('');
 }
};
const dequeueMessageHandler = () => {
 if (dequeueMessage.length > 0) {
 setDequeueMessage('');
 }
};
const heapMessageHandler = () => {
 if (heapMessage.length > 0) {
 setHeapMessage('');
 }
};
const newCustomTextHandler = () => {
 if (newCustomText.length > 0) {
 setNewCustomText('');
 }
};
const newCustomProbHandler = () => {
 if (newCustomProb > 0) {
 setNewCustomProb(0);
 }
};

```

```

 setDequeueMessage(` Встал в конец очереди: ${newPe
 });

```

```

const servePerson = () => {
 if (queue.length === 0) {
 setDequeueMessage("⚠ Очередь пуста! Суп ждет нов
 return;
 }
 const firstPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setDequeueMessage(` Выдан суп первому в очереди (F
};

```

```

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
 setDequeueMessage(" Очередь восстановлена.");
};

```

// Heap handlers

```

const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью
};

```

```

const popHeap = () => {
 if (heap.length === 0) {

```

```

 { /* Tabs */ }
 <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
 <button
 onClick={() => setActiveTab('stack')}
 className={`flex items-center justify-between border
 activeTab === 'stack'
 ? 'bg-blue-600/20 border-blue-500 text-blue-600'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-200'}
 >
 <div className="flex items-center gap-3">
 <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'text-blue-600' : 'text-slate-200'}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">Stack
 <div className="text-xs opacity-80">LIFO
 </div>
 </div>

 DFS

 </div>
 </button>

 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between border
 activeTab === 'queue'
 ? 'bg-indigo-600/20 border-indigo-500 text-indigo-600'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-200'}
 >
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-indigo-600' : 'text-slate-200'}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">Queue
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>
 </div>
 </button>
 </div>
 </div>
</div>

```

```

 </p>
 </div>
 <div className="flex flex-wrap items-center" >
 <button
 onClick={addPlate}
 className="flex-1 md:flex-none flex item
 rounded-xl text-sm font-bold shadow-lg tra
 >
 <Plus className="w-4 h-4" /> Поставить
 </button>
 <button
 onClick={popPlate}
 className="flex-1 md:flex-none flex item
 border-blue-500/40 px-4 py-2.5 rounded-xl
 >
 Помыть верхнюю
 </button>
 <button
 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
 tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>

 {popMessage && {
 <div className="bg-blue-950/60 border borde
 imate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {popMessage}
 </div>
 }}
)}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2

```



```


 </div>
);
 }
}
</div>
})
<div className="mt-6 text-xs text-slate-500" >
 ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
})

{/* QUEUE TAB */}
{activeTab === 'queue' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-xl" >
 <div>
 <h3 className="text-lg font-bold text-white">
 Симуляция очереди за супом

 </h3>
 <p className="text-slate-400 text-xs mt-1">
 Кто первым пришел, тот первым получил суп
 </p>
 </div>
 <div className="flex flex-wrap items-center">
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex items-center justify-center gap-2.5 rounded-xl text-sm font-bold shadow-lg" >
 <Plus className="w-4 h-4" /> Встать в очередь
 </button>
 <button
 onClick={servePerson}

```

```


 </div>

 <div className="text-indigo-500 flex items-center">
 <ArrowRight className="w-6 h-6 animate-pulse" />
 </div>

 {queue.map((person, index) => {
 const isFirst = index === 0;
 return (
 <div
 key={person.id}
 className={`flex flex-col items-center justify-center
 ${isFirst
 ? 'bg-gradient-to-b from-indigo-500/30 to-white
 relative'
 : 'bg-slate-900/90 border-slate-800'}
 `}
 >
 {isFirst && (
 <span className="absolute -top-10 left-1/2 transform
 translate-x-50% tracking-wider shadow">
 Первый (Head)

)}
 {person.name}
 <span className={`font-bold text-2xl ${isFirst ? 'text-indigo-500' : 'text-white'}
 `}>{person.name}
 <span className={`text-[10px] font-mono ${isFirst ? 'bg-indigo-500/30 text-white' : 'bg-white text-slate-900'}
 `}>{isFirst ? 'Получает сун' : 'Входит'}
 </div>
);
 })}
 </div>
);
}

```

```

 onClick={resetHeap}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>

 {/* Add Form */}
 <form onSubmit={addHypothesis} className="bg-
s-12 gap-4 items-end">
 <div className="md:col-span-6">
 <label className="block text-xs font-bold">
 <input
 type="text"
 value={newCustomText}
 onChange={e => setNewCustomText(e.target
placeholder='например: "Привет, я лучший"'
 className="w-full bg-slate-900 border b
rder-emerald-500 transition-colors"
 />
 </div>
 <div className="md:col-span-3">
 <label className="block text-xs font-bold">
 Вероятность: <span className="text-emera
 </label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e
 className="w-full accent-emerald-500 cu
 />
 </div>

```

```

const isTop = index === 0;
const percent = (item.probability * 100).toFixed(1);
return (
 <div
 key={item.id}
 className={`flex items-center justify-between
 ${isTop ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]' : 'bg-slate-900/80 border-slate-800'}
 `}
 >
 <div className="flex items-center justify-between" >
 <span className={`flex items-center justify-between
 ${isTop ? 'bg-emerald-500 text-white' : 'bg-slate-900/80 text-slate-400'}
 `}>
 #{index + 1}

 <div>
 <div className={`font-bold text-white`} >
 {item.text}
 </div>
 {isTop && (

 Вершина Кучи (Root) • Будущее

)}
 </div>
 </div>

 <div className="flex items-center justify-between" >
 { /* Custom progress bar */ }
 <div className="hidden sm:block" >
 <div
 className={`h-full rounded-full`}
 style={{ width: `${percent}%` }}
 ></div>
 </div>
 </div>
 </div>
);

```

```

* 2. «Большое дерево» — те же случаи внутри настоящего
* поворот — кадры «прицел → отстёгиваем среднее по
* 3. «Собери сам» — песочница: подними узел кликами,
*/
```

```
type Tab = "cases" | "big" | "play";
```

```
export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({ defaultTab }) => {
 const [tab, setTab] = useState<Tab>(defaultTab);
 const chapterId = useContext(VizChapterContext);
```

```
 // вкладка = подрежим страницы: панель Python подстав
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);
```

```
 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white">

 </div>
 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Один
 </p>
 </div>
 </div>
 </div>
```

```
 <div className="flex flex-wrap gap-2 mb-5">
 {[
 { id: "cases" as Tab, label: "1 · Механика: Z"},
 { id: "big" as Tab, label: "2 · Большое дерево"},
 { id: "play" as Tab, label: "3 · Собери сам"
]}.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 >
```

-----  
\* Порядок поворотов выбираешь сам; правильный ли он по  
\* только после того, как узел окажется наверху (или по  
\*/

```
export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}
```

```
const leaf = (key: number): TNode => ({ key, left: null
```

```
const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
 {
```

```
 name: "Линия влево",
```

```
 hintCase: "Zig-Zig",
```

```
 target: 1,
```

```
 build: () => {
```

```
 const n1 = leaf(1);
```

```
 const n2: TNode = { key: 2, left: n1, right: leaf
```

```
 const n4: TNode = { key: 4, left: n2, right: leaf
```

```
 return { key: 6, left: n4, right: leaf(7) };
```

```
 },
```

```
 },
```

```
 {
```

```
 name: "Змейка",
```

```
 hintCase: "Zig-Zag",
```

```
 target: 3,
```

```
 build: () => {
```

```
 const n3 = leaf(3);
```

```
 const n2: TNode = { key: 2, left: leaf(1), right:
```

```
 const n4: TNode = { key: 4, left: n2, right: leaf
```

```
 return { key: 6, left: n4, right: leaf(7) };
```

```
 },
```

```

const x = path[path.length - 1];
const p = path[path.length - 2];
const gg = path.length >= 3 ? path[path.length - 3] : null;

if (p.left === x) {
 p.left = x.right;
 x.right = p;
} else {
 p.right = x.left;
 x.left = p;
}

if (!gg) return x;
if (gg.left === p) gg.left = x;
else gg.right = x;
return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number) {
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя)" };
}

/* ----- раскладка -----

interface Placed {
 key: number;

```

```

const [tree, setTree] = useState<TNode>(() => preset.t
const [history, setHistory] = useState<TNode[]>([]);
const [moves, setMoves] = useState<{ node: number; co
const [showAnswer, setShowAnswer] = useState(false);

const target = preset.target;
const solved = tree.key === target;

const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, tar
const expected = useMemo(() => (solved ? null : canon

const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с ке

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.r
 setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
};

```



```
</div>
```

```
<div className="bg-slate-900 rounded-xl border bo
```

```
 <svg
```

```
 viewBox={`0 0 ${width} ${height}`}
```

```
 className="h-auto block mx-auto"
```

```
 style={{ minWidth: Math.min(width * 1.5, 420)
```

```
 role="img"
```

```
 >
```

```
 {edges.map(([a, b], i) => {
```

```
 const na = nodes.find((n) => n.key === a);
```

```
 const nb = nodes.find((n) => n.key === b);
```

```
 if (!na || !nb) return null;
```

```
 const onPath = path.has(a) && path.has(b);
```

```
 return (
```

```
 <line
```

```
 key={i}
```

```
 x1={na.x}
```

```
 y1={na.y}
```

```
 x2={nb.x}
```

```
 y2={nb.y}
```

```
 stroke={onPath ? "#6366f1" : "#475569"}
```

```
 strokeWidth={onPath ? 3 : 1.8}
```

```
 style={{ transition: "all 700ms cubic-b
```

```
 />
```

```
);
```

```
])}
 </div>
```

```
{nodes.map((n) => {
```

```
 const isTarget = n.key === target;
```

```
 const clickable = !solved && n.depth > 0;
```

```
 return (
```

```
 <g
```

```
 key={n.key}
```

```
 onClick={() => clickable && clickNode(n
```

```
 style={{
```

```
 transform: `translate(${n.x}px, ${n.y
```

```
 transition: "transform 700ms cubic-be
```

```

onClick={() => setShowAnswer((s) => !s)}
className={`flex items-center gap-1.5 px-3 py-2
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-slate-500"
 : "bg-slate-800 border-slate-700 text-slate-500"
 }`}
>
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор" : "Сдаться / попросить подсказку"}
</button>

 поворотов: {moves.length} • минимум: {minimalMoves}

</div>

{showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40 p-2">
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
)}

{solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60 text-emerald-500"
 : "bg-rose-950/30 border-rose-800/60 text-rose-500"
 `}
 >
 <div className="flex items-center gap-2 font-mono">
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : <XCircle2 className="w-4 h-4" />}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} ходов`
 : `Узел ${target} в корне за ${moves.length} ходов`
 }
 </div>
 {wrong > 0 && (

```

```

* это и есть доказательство, что поворот ничего не
* * кадр «прицел» ничего не двигает, только подсвечива
* * на кадре «результат» остаются призраки – пунктирные
* местах и стрелки «откуда → куда»;
* * по умолчанию анимация НЕ крутится сама: пользователь
* идёт в своём темпе. Автопрокрутка включается кнопкой
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
 pos: Pos;
 edges: [NodeId, NodeId][];
 kind: "start" | "aim" | "result";
 title: string;
 text: string;
 /** Пара, которую крутим на этом шаге: кадр-«прицел». */
 focus?: [NodeId, NodeId];
 /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
 moved?: NodeId[];
 ms: number;
}
```

```
interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}
```

```

 kind: "result",
 title: "Стало: 20 – корень",
 text: "40 стало правым сыном двадцатки. Поддеревов

 ева от 40 – это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
 edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [["p", "x"], ["p", "g"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба

 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый сл

 зглаживает.",
 ms: RESULT_MS,
 },
],
};

```

```

/* ----- Zig-Zag -----
const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A: [148, 112],
 edges: [{"g", "p"}, {"g", "D"}, {"p", "A"}, {"p", "x"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p: [148, 112],
 edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p – дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым – нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
],
}

```

```

x: "x — ищем его",
p: "p — родитель",
g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(
 const ghosts = frame.kind === "result" ? (moved ?? [])

 return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" re
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

 {/* призраки: где узел стоял до поворота */}
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fil
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}

```

```

 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke="black"}
 <circle r={r} fill={c.fill} stroke={c.stroke}
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font-size={12}>
 {id}
 </text>
)}
 </g>
);
}
}
</svg>
);
};

```

```

/** Имя случая из кода пользователя → вкладка демонстрации */
const CASE_BY_NAME: Record<string, number> = {
 zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-z zigzig": 1,
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);

```

```

const duration = useMemo(() => Math.round(current.ms
const last = idx === total - 1;

useEffect(() => {
 setFrame(0);
}, [caseIdx]);

useEffect(() => {
 if (!playing) return;
 const t = setTimeout(() => setFrame((f) => (f + 1)
 return () => clearTimeout(t);
}, [playing, frame, duration, total]);

return (
 <div className="w-full bg-slate-950 rounded-2xl border
 <div className="flex gap-2 mb-4 overflow-x-auto no
 {CASES.map((c, i) => {
 <button
 key={c.key}
 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text
 i === caseIdx ? "bg-indigo-600 text-white
 }`}
 >
 {c.label}
 </button>
 })}
 </div>

 <div className="mb-3 text-xs sm:text-[13px] text-
 Кор
 ({active.rotat
 </div>

 {/* шаг + пояснение стоят НАД картинкой: сначала
 <div className="mb-3 rounded-xl border border-sla
 <div className="flex items-center gap-2 mb-1 fl
 <span

```



```
</div>
```

```
{/* индикатор автопрокрутки */}
```

```
<div className="h-1 w-full bg-slate-800 rounded-f
```

```
 <div
```

```
 key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `
```

```
 className="h-full bg-indigo-500"
```

```
 style={
```

```
 playing
```

```
 ? { animation: `demoProgress ${duration}m
```

```
 : { width: "100%", background: "#334155"
```

```
 }
```

```
 />
```

```
</div>
```

```
<div className="flex flex-wrap items-center gap-2
```

```
 <button
```

```
 onClick={() => {
```

```
 setPlaying(false);
```

```
 setFrame((f) => (f - 1 + total) % total);
```

```
 }}
```

```
 disabled={idx === 0}
```

```
 className="px-3 py-2 rounded-lg bg-slate-800
```

```
 d:hover:text-slate-300 text-xs font-bold tr
```

```
 >
```

```
 ← Назад
```

```
</button>
```

```
<button
```

```
 onClick={() => {
```

```
 setPlaying(false);
```

```
 setFrame((f) => (f + 1) % total);
```

```
 }}
```

```
 className="px-4 py-2 rounded-lg bg-indigo-600
```

```
 w-indigo-900/40"
```

```
 >
```

```
 {last ? "⏮ Сначала" : "Дальше →"}
```

```
</button>
```

```
<button
```

```
</div>
```

```
{/* что сейчас сообщил компилятор */}
```

```
<div className="mt-3 pt-3 border-t border-slate-800">
```

```

```

```
 <Terminal className="w-3.5 h-3.5" /> из компилятора
```

```

```

```
 {liveCase || liveX !== null || liveStep !== null ? (
```

```
 <>
```

```
 {liveCase && {
```

```

```

```
 case = {liveCase}
```

```

```

```
 })
```

```
 {liveX !== null && x = {liveX}}
```

```
 {liveP !== null && p = {liveP}}
```

```
 {liveG !== null && g = {liveG || "нет (Zig)"}}
```

```
 {liveStep !== null && step = {liveStep}}
```

```
 </>
```

```
) : {
```

```

```

```
 запусти код из панели Python: переменная <code>case</code> случай,
```

```
 а <code>liveG</code> — результат работы программы
```

```

```

```
 })
```

```
</div>
```

```
<p className="mt-2 text-[11px] text-slate-500">
```

```
 Листай кнопкой «Дальше» в своём темпе — кадры не пролетят мимо
```

```
</p>
```

```
</div>
```

```
);
```

```
};
```

```

 if (r) {
 n.right = r;
 r.parent = n;
 }
 return n;
}

export function bstInsert(root: SN | null, key: number) {
 if (!root) return { key, left: null, right: null, parent: null };
 const r = cloneWithParents(root)!;
 let cur: SN = r;
 for (;;) {
 if (key < cur.key) {
 if (!cur.left) {
 cur.left = { key, left: null, right: null, parent: cur };
 return r;
 }
 cur = cur.left;
 } else {
 if (!cur.right) {
 cur.right = { key, left: null, right: null, parent: cur };
 return r;
 }
 cur = cur.right;
 }
 }
}

```

/\*\* Поднять child над его родителем (один атомарный поворот)

```

export function rotateUp(child: SN): void {
 const p = child.parent;
 if (!p) return;
 const g = p.parent;
 if (p.left === child) {
 p.left = child.right;
 if (child.right) child.right.parent = p;
 child.right = p;
 p.parent = child;
 }
}

```

```

export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
 kind: SplayStepKind;
 root: SN | null;
 aimEdge?: [number, number];
 /** Кадр cut: ребро, по которому отстёгивается среднее */
 cutEdge?: [number, number];
 cutLabel?: string;
 /** Куда β переедет: сторона у верхнего узла. */
 cutDest?: string;
 pathKeys?: number[];
 caseLabel?: string;
 note: string;
 rotCount: number;
}
```

/\*\* Полный пошаговый прогон splay(key): спуск → (прицел

```
export function splaySteps(
 rootIn: SN | null,
 key: number,
 isInsert = false,
): { steps: SplayStep[]; found: boolean } {
 let root: SN | null = cloneWithParents(rootIn);
 const steps: SplayStep[] = [];
 if (!root)
 return { steps: [{ kind: "done", root: null, note:

// --- спуск ---
let cur: SN | null = root;
const pathKeys: number[] = [];
let found = false;
let last: SN = root;
while (cur) {
 last = cur;
 pathKeys.push(cur.key);
 if (key === cur.key) {
 found = true;
 break;
 }
}
```

```

 cutLabel: mid ? `β = поддереву ${mid.key}` : "β п
 cutDest: mid ? (p.left === child ? "справа у p" :
 caseLabel: label,
 note: mid
 ? `Отстёгиваем среднее поддереву β = ${mid.key}
 й у них общий – после поворота переедет ${p.l
 : `Среднего поддерева нет (β пусто): ${child.ke
 rotCount,
 });
 const before = collectEdges(root);
 rotateUp(child);
 fixRoot();
 rotCount += 1;
 const diff = diffEdges(before, collectEdges(root));
 steps.push({
 kind: "rot",
 root: cloneWithParents(root)!,
 caseLabel: label,
 note: `Поворот: ${child.key} встал над ${p.key}.
 rotCount,
 });
 });
};

while (target.parent) {
 const p = target.parent;
 const g = p.parent;
 if (!g) {
 emitRotation(
 target,
 "Zig",
 [p.key, target.key],
 `Zig: у ${target.key} нет деда – хватит одного
);
 continue;
 }
 const pIsLeft = g.left === p;
 const xIsLeft = p.left === target;
 if (pIsLeft === xIsLeft) {

```

```

 emitRotation(
 target,
 "Zig-Zag",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key}
);
 }
}

steps.push({
 kind: "done",
 root: cloneWithParents(root)!,
 note: ` ${target.key} в корне за ${rotCount} ${rotCount}
 rotCount,
});
return { steps, found };
}

/* ----- служебное -----

const depthOf = (t: SN | null, key: number): number =>
 let d = 0;
 let cur = t;
 while (cur) {
 if (key === cur.key) return d;
 cur = key < cur.key ? cur.left : cur.right;
 d += 1;
 }
 return -1;
};

const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [...
 ...

export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1];

const buildInitial = (): SN => {
 let t: SN | null = null;
 for (const k of INITIAL_KEYS) t = bstInsert(t, k);
}

```

```

/>
<circle cx="55" cy="46" r="15" fill="#1e293b"
<text x="55" y="50" textAnchor="middle" fill=
<circle cx="30" cy="86" r="15" fill="#78350f"
<text x="30" y="90" textAnchor="middle" fill=
<circle cx="12" cy="126" r="13" fill="#0f172a"
<text x="12" y="130" textAnchor="middle" fill=
<circle
 cx="48" cy="126" r="13"
 fill={isCut ? "#78350f" : "#0f172a"}
 stroke={isCut ? "#f59e0b" : "#64748b"}
 strokeWidth={isCut ? 2.5 : 1.5}
 strokeDasharray={isCut ? "4,3" : "none"}
/>
<text x="48" y="130" textAnchor="middle" fill=
<circle cx="80" cy="126" r="13" fill="#0f172a"
<text x="80" y="130" textAnchor="middle" fill=
<text x="98" y="92" fill={isCut ? "#f59e0b" :
 {isCut ? " β отстёрнута" : "β при x"}}
</text>
<text x="16" y="160" fill="#64748b" fontSize=
</g>
<text x="163" y="96" fill="#10b981" fontSize="20"
<g transform="translate(186,6)">
 <text y="10" fill="#94a3b8" fontSize="11" font=
 <line x1="70" y1="46" x2="45" y2="86" stroke=
 <line x1="70" y1="46" x2="95" y2="86" stroke=
 <line x1="45" y1="86" x2="27" y2="126" stroke=
 <line
 x1="45" y1="86" x2="63" y2="126"
 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 3 : 2}
 />
 <circle cx="70" cy="46" r="15" fill="#059669"
 <text x="70" y="50" textAnchor="middle" fill=
 <circle cx="45" cy="86" r="15" fill="#1e293b"
 <text x="45" y="90" textAnchor="middle" fill=
 <circle cx="27" cy="126" r="13" fill="#0f172a"

```

```

 walk(n.right, d + 1);
 };
 walk(root, 0);
 return pos;
};

const SplayWalkViz: React.FC = () => {
 const initial = useMemo(buildInitial, []);
 const [steps, setSteps] = useState<SplayStep[] | null>();
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [input, setInput] = useState("");
 const [treeVers, setTreeVers] = useState(0);
 const baseRef = useRef<SN>(initial);
 const timer = useRef<ReturnType<typeof setTimeout> | null>();

 const runKey = (key: number, isInsert = false) => {
 const { steps: st } = splaySteps(baseRef.current, key, isInsert);
 setSteps(st);
 setIdx(0);
 setPlaying(true);
 };

 useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 baseRef.current = steps[steps.length - 1].root ?? null;
 setTreeVers((v) => v + 1);
 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v + 1), 100);
 return () => {
 if (timer.current) clearTimeout(timer.current);
 };
 }, [playing, idx, steps]);

 const step = steps ? steps[Math.min(idx, steps.length - 1)] : null;

```



```

const doInsert = () => {
 const v = parseInt(input, 10);
 if (Number.isNaN(v)) return;
 const withNew = bstInsert(baseRef.current, v);
 baseRef.current = withNew;
 setTreeVers((t) => t + 1);
 runKey(v, true);
 setInput("");
};

const stepColor = (k: number): string => {
 if (!step) return "#0f172a";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#0f172a";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e[0] === k) return "#1e293b";
 if (e[1] === k) return "#78350f";
)
 if (step.kind === "rot" && step.pathKeys?.includes(k)) return "#0f172a";
};

const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#64748b";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e.includes(k)) return "#f59e0b";
)
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 {/* Wanka */}
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

```

```

 <button
 onClick={() => {
 const shuffled = [...INITIAL_KEYS].sort(() =>
 Math.random() - 0.5);
 let r: SN | null = null;
 for (const k of shuffled) r = bstInsert(r, k);
 baseRef.current = r!;
 setSteps(null);
 setIdx(0);
 setPlaying(false);
 setTreeVers((v) => v + 1);
 }}
 className="px-3 py-2 rounded-lg text-xs font-medium"
 >
 Перемешать
 </button>
 </div>
 </div>

 {/* Плеер */}
 {steps && (
 <div className="flex flex-wrap items-center gap-2" >
 <button
 onClick={() => { setIdx(0); setPlaying(false); }}
 className="px-2 py-1.5 rounded-lg bg-slate-700 text-white disabled:opacity-50"
 >
 В начало
 </button>
 <button
 onClick={() => { setPlaying(false); setSteps(null); }}
 className="px-2 py-1.5 rounded-lg bg-slate-700 text-white"
 >
 Пауза
 </button>
 <button
 onClick={() => setPlaying(!playing)}
 className="px-2 py-1.5 rounded-lg bg-emerald-600 text-white"
 >
 ▶ Пуск
 </button>
 <button
 onClick={() => { setPlaying(false); setSteps(null); }}
 className="px-3 py-1.5 rounded-lg text-xs font-medium"
 >
 Очистить
 </button>
 </div>
)}
 </div>
</div>

```

```

 {isCutEdge && (
 <text x={({a.px + b.px) / 2 + 14} y=({
nospace" className="transition-all duration
 {step?.cutLabel} → {step?.cutDest
 </text>
 })}
 </g>
);
 })}
 {nodes.map((n) => {
 const aimE = step?.aimEdge ?? (step?.kind =
 const isAimNode = aimE?.includes(n.key);
 return {
 <g
 key={`n-${n.key}-${treeVers}`}
 transform={`translate(${n.px}, ${n.py})`
 onClick={() => {
 if (playing) return;
 const has = inorderKeys(baseRef.current)
 if (has) runKey(n.key);
 }}
 className="cursor-pointer"
 >
 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="#ffffff" fontSize="13"
 >
 {n.key}
 </text>
 </g>
 });
})}
</svg>

```

```

<table className="w-full text-sm">
 <tbody className="font-mono">
 <tr className="border-b border-slate-800">
 <td className="py-2 text-slate-400">г
 <td className="py-2 text-right text-slate-400">г
 </tr>
 <tr className="border-b border-slate-800">
 <td className="py-2 text-slate-400">в
 <td className="py-2 text-right text-slate-400">в
 </tr>
 <tr>
 <td className="py-2 text-slate-400">п
 <td className="py-2 text-right text-slate-400">п
 </tr>
 </tbody>
</table>
) : (
 <p className="text-xs text-slate-400">Запис
 </p>
 <p className="text-xs text-slate-500 mt-3">Од
 щений дешевлеет.</p>
</div>
<div className="w-full lg:w-1/2 bg-slate-900/60">
 <h4 className="text-sm font-bold text-slate-200">
 <div className="bg-slate-950/80 rounded-lg p-4">
 {CODE_LINES.map((l) => {
 const zig = step?.caseLabel === "Zig" &&
 const zz1 = step?.caseLabel === "Zig-Zig"
 const zz2 = step?.caseLabel === "Zig-Zig"
 const za1 = step?.caseLabel === "Zig-Zag"
 const za2 = step?.caseLabel === "Zig-Zag"
 const active = zig || zz1 || zz2 || za1 |
 return (
 <div key={l.line} className={`py-1 px-2
 border-400" : "text-slate-400"}`>
 <span className="text-slate-600 selec
 {l.text}
 </div>

```

```

let counter = 0;

export const StackViz: React.FC = () => {
 const [dirtyStack, setDirtyStack] = useState<Plate[]>(
 [
 { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'idle' },
 { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'idle' },
 { id: 'p3', name: 'Блюдо от торта', emoji: '🍰', state: 'idle' },
]
);

 const runtime = useVizRuntime();
 const liveStack = vizArray(runtime?.variables?.st) ?? [];
 const displayStack: Plate[] = liveStack
 ? liveStack.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: " ",
 color: "from-indigo-500/20 to-indigo-600/20 border",
 isDirty: true,
 animState: "idle",
 }))
 : dirtyStack;

 const [cleanRack, setCleanRack] = useState<Plate[]>([]);
 const [isWashing, setIsWashing] = useState(false);
 const [showSoap, setShowSoap] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string[]>([]);

 const addLog = (msg: string) => setLog(prev => [...prev, msg]);

 // PUSH: Положить грязную тарелку сверху стопки
 const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {

```

```

// Step 1: Start washing animation (sponge and soap)
setDirtyStack(prev => prev.map(p => p.id === topPlate.id)
setShowSoap(true);

setTimeout(() => {
 // Step 2: Wash complete. Plate becomes clean and
 const cleanPlate: Plate = {
 ...topPlate,
 isDirty: false,
 animState: 'clean',
 };

 setCleanRack(prev => [cleanPlate, ...prev].slice(0, 1));
 setDirtyStack(prev => prev.slice(0, prev.length - 1));
 setShowSoap(false);
 setIsWashing(false);
 addLog(` Готово! Чистая тарелка ${topPlate.emoji}`, 850);
}, [dirtyStack, isWashing]);

const reset = () => {
 counter = 0;
 setDirtyStack([
 { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'in' },
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'in' },
 { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', state: 'in' },
]);
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
};

const topIndex = displayStack.length - 1;

```

```

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 {/* ИНТЕРАКТИВНАЯ КУХНЯ */}
 <div className="grid grid-cols-1 md:grid-cols-12" >

 {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
 <div className="md:col-span-7 bg-slate-900 rounded-3xl
 [380px]">
 <div className="absolute top-4 left-4 text-[10px] font-mono">
 Грязная стопка (Стек вызовов / LIFO)
 </div>

 {/* Визуализация раковины */}
 <div className="absolute top-4 right-4 flex items-center gap-2">
 <div className="r-slate-800/80 text-[10px] font-mono">

 РАКОВИНА
 </div>

 {/* Мыльные пузыри при мытье */}
 {showSoap && (
 <div className="absolute inset-0 pointer-events-none">
 <div className="absolute w-6 h-6 rounded-full bg-white"></div>
 <div className="absolute w-4 h-4 rounded-full bg-white"></div>
 <div className="absolute w-5 h-5 rounded-full bg-white"></div>
 <div className="absolute text-4xl anim-crown"></div>
 </div>
)}
 </div>

```

```

>
 {plate.em
 <span className="text-slate-800 tex
 {isTop && (
 <span className="text-[9px] bg-bl
 LIFO

)}
 </div>
);
 }}}
</div>
))

 {/* Столешница раковины */}
 <div className="w-full h-4 bg-gradient-to-r f
</div>

 {/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК
 <div className="md:col-span-5 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[1
 Сушилка для чистой посуды
 </div>

 <div className="absolute top-4 right-4 flex i
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pu
 ЧИСТО
 </div>

 <div className="flex-1 flex flex-col justify-
 {cleanRack.length === 0 ? (
 <div className="flex-1 flex flex-col item

 <p className="text-xs font-bold font-mo
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (

```



ФАЙЛ 75/110: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useRef, useMemo } from "react";
import { useVizRuntime, vizArray, vizNumber, vizString } from "viz-runtime";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";
```

```
function generateKmpSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const pi = new Array(n).fill(0);
 const steps: any[] = [];
```

```
 steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });
```

```
 for (let i = 1; i < n; i++) {
 let j = pi[i - 1];
```

```
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 }
```

```
 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 // Сдвиг j назад: откат j = pi[j - 1] = pi[j - 1];
 j = pi[j - 1];
 }
```

```
 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 }
```

```
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
}
```

```

 if (i + z[i] < n) {
 steps.push({ i, l, r, zTarget: z[i], zSource:
 [i] } ` разные. Стр. ` });
 } else {
 steps.push({ i, l, r, z: [...z], desc: `Унѐ
 }

 steps.push({ i, l, r, z: [...z], desc: `Фиксирывае
 }

 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 steps.push({ i, l, r, z: [...z], desc: ` 0
 }

 if (z[i] === pattern.length) {
 steps.push({ i, l, r, z: [...z], found: true
 }
 }
 return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp"
 const [mode, setMode] = useState<"kmp" | "z">(default
 const [pattern, setPattern] = useState("aba");
 const [text, setText] = useState("abacaba");

 // Ensure inputs are valid dynamically
 const safePattern = pattern || " ";
 const safeText = text || " ";

 const { s: demoString, steps, patternLength: demoPa
 if (mode === "kmp") return generateKmpSteps(safe
 else return generateZSteps(safePattern, safeText
 }, [mode, safePattern, safeText]);
 const runtime = useVizRuntime();
 const runtimeString = vizString(runtime?.variables?

```

```

const liveR = vizNumber(vars?.r);
const compilerLinked = liveI !== null || hasLiveVal
const step = compilerLinked
 ? {
 ...baseStep,
 i: liveI ?? baseStep.i,
 j: liveJ ?? baseStep.j,
 l: liveL ?? baseStep.l,
 r: liveR ?? baseStep.r,
 [mode === "kmp" ? "pi" : "z"]: hasLiveValues
 desc: `Компилятор: i=${liveI ?? "-"}${mode ===
 }
 : baseStep;

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
 КМП (π-ФУНКЦИЯ)
 </button>
 <button onClick={() => setMode("z")
 className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </button>
 </div>

 <div className="flex items-center gap-2
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={pattern} onChange
xt-white text-xs font-bold font-mono px-2 p
 </div>
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te

```

```

 return (
 <div key={index} className=
 {/* L/R markers for Z */
 {mode === "z" && step.l
 <div className="abs
 }}
 {mode === "z" && step.r
 <div className="abs
 }}

 {/* Index */}
 <span className={`text-

 {/* Char Box */}
 <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
 {char}
 </div>

 {/* Value array box */}
 <div className="text-[1
slate-700/50 font-mono font-bold">
 {(mode === "kmp" ?
 </div>

 {/* KMP fingers */}
 {mode === "kmp" && step
 <div className="abs

 <span classNam
 </div>
 }}
 {mode === "kmp" && step
 <div className="abs
 <span classNam
 </div>
 }}
)
 }
 z-20">

```

```
</div>
```

```
<div className="flex flex-col sm:flex-row gap-4" data-bbox="267 143 1000 188">
 {/* Controls */}
```

```
 <div className="flex bg-slate-900 border border-slate-800 rounded-lg p-4" data-bbox="267 190 1000 285">
 <button onClick={prevStep} disabled={steps.length === 0} className="text-slate-400 hover:text-white hover:bg-slate-800 rounded-md px-4 py-2">
 <Undo strokeWidth={3} size={16} />
 </button>
```

```
 <button onClick={playPause} className="text-slate-400 hover:text-white hover:bg-slate-800 rounded-md px-4 py-2" data-bbox="267 286 1000 380">
 {autoplay ? <Play size={16} /> : <Pause size={16} />}
 </button>
```

```
 <button onClick={nextStep} disabled={steps.length === 0} className="text-slate-400 hover:text-white hover:bg-slate-800 rounded-md px-4 py-2" data-bbox="267 382 1000 476">
 <SkipForward strokeWidth={3} size={16} />
 </button>
```

```
 <button onClick={reset} aria-label="Reset" className="text-slate-400 hover:text-white hover:bg-slate-800 rounded-lg ml-2 border-l border-slate-800 px-4 py-2" data-bbox="267 478 1000 572">
 <RefreshCw strokeWidth={3} size={16} />
 </button>
```

```
</div>
```

```
 {/* Description Log */}
```

```
 <div className="flex-1 bg-slate-900/50 border border-slate-800 rounded-lg p-4 overflow-hidden" data-bbox="267 665 1000 710">
```

```
 <div className="absolute top-0 right-0 text-slate-400 font-mono text-sm" data-bbox="401 712 1000 757">
```

```
 <div data-bbox="267 759 1000 804">
```

```
 <div className="text-[10px] text-slate-400" data-bbox="267 806 1000 851">
```

```
 {steps.length}
 </div>
 </div>
```

```
</div>
```

```
</div>
```

```
 {/* Why this matters card */}
```

```
}

```

ФАЙЛ 76/110: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from "react";

interface TooltipProps {
 /** Содержимое подсказки (можно JSX). */
 content: React.ReactNode;
 /** Элемент, на который наводят курсор / фокус. */
 children: React.ReactNode;
 /** С какой стороны показывать. По умолчанию – сверху
 side?: "top" | "bottom";
 className?: string;
}

/**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры
 */
export const Tooltip: React.FC<TooltipProps> = ({ content,
 const [open, setOpen] = useState(false);

 const hiddenShift = side === "top" ? "translate-y-1"

 return (
 <span
 className={`relative inline-flex ${className ?? ""}
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}

)
}
```

```

 label: string;
 name: string;
}

interface TEdge {
 u: number;
 v: number;
}

const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (0)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный" },
 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы" },
 { id: 4, x: 640, y: 150, label: "4", name: "Машинное" },
];

const dagEdges: TEdge[] = [
 { u: 0, v: 1 },
 { u: 0, v: 2 },
 { u: 1, v: 3 },
 { u: 2, v: 3 },
 { u: 3, v: 4 },
];

interface TStep {
 desc: string;
 visited: Set<number>;
 fullyProcessed: Set<number>;
 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
};

```

```

const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} (${dagNodes[u].name})`,
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in topological order)
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} от ${u}`,
 u,
);
 }
 }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in reverse
recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'`,
 u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой вершины ${i}`,
 null,
);
 dfs(i);
 }
}

```



```

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 500 text-white"
 >
 {isPlaying ? {
 <Pause className="w-4 h-4 ml-1" />
 } : {
 <Play className="w-4 h-4 ml-1" />
 }}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить сортировку"
 title="Сбросить сортировку"
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>

```

```

 (step.currentNode === edge.u || step.currentNode === edge.v ||
 step.visited.has(edge.v));

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#334155"}
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-arrow" : "dag-arrow"})`}
 className="transition-all duration-300"
 />
);
 }
}

{/* Nodes */}
{dagNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited = step.visited.has(node.id);
 const isProcessed = step.fullyProcessed.has(node.id);

 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";

 void textColor;

 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 } else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#f1f3f4";
 stroke = "#c6c8ca";
 textColor = "text-slate-400";
 }
})}

```

```

 </text>
 </g>
);
 }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

 <div className="bg-slate-900 border border-slate-800">
 <p className="text-xs text-indigo-300 min-h-12">
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">
 {step.topoList.length === 0 ? (

 ожидаем обработку узлов...

) : (
 getTopoListString()
)}
 </div>
 </div>
 </div>

```

```
 >
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>
);
};
```

---

ФАЙЛ 78/110: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useContext, useEffect, useMemo, useState } from "react";
import { Terminal } from "lucide-react";
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from
```

```
/**
```

```
 * Интерактивный Treap – шесть режимов:
```

```
 * build – сборка: ВИДИМАЯ сортировка точек (и чем сортировать, тем лучше)
```

```
 * координатная плоскость НАВЕРХУ (ассоциативная)
```

```
 * чек-лист инвариантов (автопроверка, как у AVL)
```

```
 * split – разрез по x с двумя корзинами (грамотная)
```

```
 * merge – склейка двух деревьев по y.
```

```
 * erase – удаление ПРОИЗВОЛЬНОЙ точки: найти по x, удалить по y
```

```
 * layers – миф «дети в массиве по $2k/2k+1$ »: работает
```

```
 * search – «отсортирую бинарным поиском»: поиск не сортировкой
```

```
 * сортировка платит $\geq n \cdot \log n$ сравнений, сортировка
```

```
 * сортировка платит $\geq n \cdot \log n$ сравнений, сортировка
```

```
 * Канонический набор: все x и все y различны $\Rightarrow$ дерево
```

```
 * Узел идентифицируется парой $id = "x:y"$.
```

```
*/
```

```
export interface Pair {
```

Универсальное пособие • полный исходный код

```

export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p) => p.key > 0)

const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((p1, p2) => p1.key - p2.key)

/* ----- чистые алгоритмы (экспортируются для тестов) ----- */

export function insert(root: TNode | null, p: Pair): TNode | null {
 if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade, left: null, right: null }
 if (p.key <= root.key) root.left = insert(root.left, p)
 else root.right = insert(root.right, p)
 return root
}

export function buildTree(ps: Pair[]): TNode | null {
 let root: TNode | null = null
 for (const p of sortedByY(ps)) root = insert(root, p)
 return root
}

export interface SplitResult {
 l: TNode | null
 r: TNode | null
 trace: { atId: NodeId; goLeft: boolean; note: string }[]
}

export function splitTree(root: TNode | null, x: number): SplitResult {
 const trace: SplitResult["trace"] = []
 const go = (t: TNode | null): { l: TNode | null; r: TNode | null } | null {
 if (!t) return { l: null, r: null }
 if (t.key <= x) {
 const sub = go(t.right)
 trace.push({ atId: t.id, goLeft: true, note: `(${t.key} > ${x}) о правое поддерево.` })
 t.right = sub.l
 return { l: t, r: sub.r }
 }
 const sub = go(t.left)
 trace.push({ atId: t.id, goLeft: false, note: `(${t.key} < ${x}) о левое поддерево.` })
 t.left = sub.r
 return { l: sub.l, r: t }
 }
 const result = go(root)
 return { l: result?.l, r: result?.r, trace }
}
```

```

 trace.push({ atId: t.id, note: `${key} < ${t.key}` });
 t.left = go(t.left);
 return t;
 }
 if (key > t.key) {
 trace.push({ atId: t.id, note: `${key} > ${t.key}` });
 t.right = go(t.right);
 return t;
 }
 trace.push({ atId: t.id, note: `Нашли (${t.key}), ${t.key}` });
 const m = mergeTree(t.left, t.right);
 trace.push(...m.trace.map(s => ({ atId: s.chosenId, note: s.note })));
 return m.root;
};
return { root: go(root), trace };
}

```

```

const cloneTree = (t: TNode | null): TNode | null =>
 t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null;

```

/\* ----- валидация treap (для чек-листа и стресс-теста) ----- \*/

```

export function checkTreap(root: TNode | null): { bst: boolean, heap: boolean } {
 let bst = true;
 let heap = true;
 let prev: number | null = null;
 const walk = (n: TNode | null, min: number, max: number, parentGrade: number | null): void => {
 if (!n) return;
 if (n.key <= min || n.key > max) bst = false;
 if (parentGrade !== null && n.grade > parentGrade) heap = false;
 walk(n.left, min, n.key, n.grade);
 if (prev !== null && n.key < prev) bst = false;
 prev = n.key;
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}

```

```

 subtitle?: string;
 }

const TreeView: React.FC<TreeViewProps> = ({ root, width, height }) => {
 const pos = useMemo(() => layoutTree(root), [root]);
 const nodes = useMemo(() => [...pos.entries()].map(([id, node]) => node));
 const edges = useMemo(() => treeEdges(root), [root]);
 const autoW = Math.max(420, ...nodes.map((n) => n.px + n.rx));
 const autoH = Math.max(260, ...nodes.map((n) => n.py + n.ry));
 const w = width ?? autoW;
 const h = height ?? autoH;
 const byId = (id: NodeId) => nodes.find((n) => n.id === id);
 return (
 <div>
 <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full">
 {edges.map((e) => {
 const a = byId(e.from);
 const b = byId(e.to);
 if (!a || !b) return null;
 return (
 <line
 key={`te-${e.from}-${e.to}`}
 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={edgeColor ? edgeColor(e.from, e.to) : 'black'}
 strokeWidth={2.5}
 strokeDasharray={edgeDash ?? edgeDash(e.from, e.to)}
 className="transition-all duration-500"
 />
);
 })}
 {nodes.map((n) => {
 const p = pairParts(n.id);
 const pulse = pulseId === n.id;
 return (
 <g key={`tn-${n.id}`} transform={`translate(${n.px}, ${n.py})`}
 <circle r={pulse ? 24 : 20} fill={nodeColor(n)}
 />
 <rect x={-10} y={-10} x2={10} y2={10} fill="white"
 />
 <text y={-2} fill="#fff" fontSize="13" fontFam
);
 })}
 </svg>
 </div>
);
};

```

```

<svg
 viewBox={`0 0 ${PL_W} ${PL_H}`}
 className={`w-full ${interactive ? "touch-none select-none" : ""}`}
 onPointerMove={onMove}
 style={interactive ? { cursor: "crosshair" } : undefined}
>
 <defs>
 <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" strokeDasharray="4 3"/>
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" strokeDasharray="4 3"/>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" strokeDasharray="4 3"/>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PY(0)} stroke="black" strokeDasharray="4 3"/>
 {[0, 10, 20, 30, 40].map((k) => {
 <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16} fill="black" fontSi
 })}
 {[{-4, 0, 4, 8].map((g) => {
 <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) + 16} fill="black" fontSi
 })}
 <text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8" fontSi
 <text x={PX(1) - 26} y="14" fill="#94a3b8" fontSi

 {drawnEdges.map((e, i) => {
 const a = byPair(e.a);
 const b = byPair(e.b);
 const isBad = badEdge && ((badEdge.a.key === e.a.key && badEdge.a.grade === e.b.grade) || (badEdge.a.key === e.b.key && badEdge.a.grade === e.a.grade));
 return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y} stroke="black" strokeDasharray={isBad ? "4,3" : "none"} strokeDasharray="4 3"/>
 })}

 {/* временная линия игры */}
 {interactive && selected && cursor && (
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDasharray="4 3"/>
)}

```



```

export const CANON_GAME: GEdge[] = (() => {
 const es: GEdge[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ a: { key: n.key, grade: n.g
 if (n.right) { es.push({ a: { key: n.key, grade: n.g
 };
 walk(buildTree(POINTS));
 return es;
})();

const sameP = (p: Pair, q: Pair) => p.key === q.key &&
const gpid = (p: Pair) => `${p.key}:${p.grade}`;

/** Цепочка родителей от точки вверх. */
const parentOf = (edges: GEdge[], id: string): GEdge |
 edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pa
 let cur: Pair | undefined = from;
 const seen = new Set<string>();
 while (cur) {
 if (sameP(cur, target)) return true;
 if (seen.has(gpid(cur))) return false;
 seen.add(gpid(cur));
 cur = parentOf(edges, gpid(cur))?.a;
 }
 return false;
};

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 спр
export function forestIsBst(edges: GEdge[], all: Pair[])
 const kids = new Map<string, Pair[]>();
 for (const e of edges) kids.set(gpid(e.a), [...(kids.
 const hasParent = new Set(edges.map((e) => gpid(e.b))
 const roots = all.filter((p) => !hasParent.has(gpid(p
 const ok = (n: Pair | undefined, lo: number, hi: numb
 if (!n) return true;

```



```

}

export function checkGameState(
 points: Pair[],
 edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected: boolean } {
 const n = points.length;
 if (n < 2) return { heapOk: true, singleParent: true, connected: true };
 const heapOk = edges.every((e) => e.a.grade > e.b.grade);
 const parentCount = new Map<string, number>();
 for (const e of edges) parentCount.set(pid(e.b), (parentCount.get(pid(e.b)) || 0) + 1);
 const singleParent = [...parentCount.values()].every(v => v <= 1);
 // связность: компоненты через union-find
 const comp = new Map<string, string>(points.map((p) => [p, p]));
 const find = (x: string): string => {
 while (comp.get(x) !== x) {
 comp.set(x, comp.get(x)!);
 x = comp.get(x)!;
 }
 return x;
 };
 for (const e of edges) {
 const ra = find(pid(e.a));
 const rb = find(pid(e.b));
 if (ra !== rb) comp.set(ra, rb);
 }
 const comps = new Set(points.map((p) => find(pid(p))));
 const connected = comps.size === 1;
 const countOk = edges.length === n - 1;
 // BST: дети по x, inorder не убывает; корень один; э.б.
 const children = new Map<string, Pair[]>();
 for (const e of edges) children.set(pid(e.a), [...(children.get(pid(e.a)) || []), e.b]);
 const roots = points.filter((p) => !parentCount.has(pid(p)));
 let bstOk = roots.length === 1;
 if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {

```

```

export function edgesToTree(edges: GameEdge[]): TNode |
 const parentOf = new Map<string, GameEdge>();
 for (const e of edges) parentOf.set(pid(e.b), e);
 const roots = edges.filter((e) => !parentOf.has(pid(e.a)));
 if (!roots.length && edges.length) return null;
 const nodes = new Map<string, TNode>();
 const make = (p: Pair): TNode => ({
 id: pid(p),
 key: p.key,
 grade: p.grade,
 left: null,
 right: null,
 });
 const rootPair = roots.length ? roots[0].a : edges.length > 0 ? edges[0] : null;
 if (!rootPair) return null;
 const build = (p: Pair): TNode => {
 const node = make(p);
 nodes.set(pid(p), node);
 const kids = edges.filter((e) => pid(e.a) === pid(p));
 if (kids.length > 2) return node; // невалидно — по
 kids.sort((u, v) => u.key - v.key);
 const left = kids.filter((k) => k.key <= p.key); //
 const right = kids.filter((k) => k.key > p.key);
 if (left.length > 1 || right.length > 1) return node;
 if (left[0]) node.left = build(left[0]);
 if (right[0]) node.right = build(right[0]);
 return node;
 };
 return build(rootPair);
}

```

```

const BuildMode: React.FC = () => {
 const [raw, setRaw] = useState(DEMO_RAW);
 const live = useTreapLive();

```

```

 // набор точек из Python-кода пересобирает плоскость
 useEffect(() => {
 if (!live.pairsText || live.pairsText === raw) return;

```

```

 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return; }
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.length;
 if (connIdx < total) setConnIdx(connIdx + 1);
 else setPlaying(false);
 }
 }, 900);
 return () => clearTimeout(t);
 }, [playing, phase, sortIdx, connIdx, algo, order, canonEdges]);

const startConnect = (ord: ConnectOrder) => {
 setOrder(ord);
 setPhase("connect");
 setConnIdx(0);
 setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? Math.log2(n) : 0) : 0;
const connEdgesAll = order === "y-desc" ? canonEdges : canonEdges.reverse();
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && connIdx >= connEdges.length;

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>([]);
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>({ x: 0, y: 0 });
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
 setGameEdges([]);
 setSel(null);
 setCursor(null);
};

```

```

 };
 const pointUp = (p: Pair) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
 };
 const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
 if (!sel) return;
 const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
 setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
 };

 const undoEdge = () => {
 setGameEdges((prev) => prev.slice(0, -1));
 setGameMsg(null);
 };
 const showSolution = () => {
 setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
 setHintUsed(true);
 setGameMsg("Решение показано – собери сам в следующем наборе");
 };

 return (
 <div className="space-y-5">
 {/* Произвольный список */}
 <div className="bg-slate-900/60 rounded-xl p-4 border border-slate-700">
 <div className="flex flex-wrap items-center justify-between">
 <h4 className="text-sm font-bold text-slate-300">Задача
 <button onClick={() => { setRaw(DEMO_RAW); setGameEdges(canonEdges); }}>
 demo-набор
 </button>
 </div>
 <textarea
 value={raw}

```

```

 {a === "quicksort" ? "quicksort (ср
 </button>
)))
 </>
) : (
 <>
 <span className="text-xs text-slate-400
 <button
 onClick={() => startConnect("y-desc")}
 className={`px-3 py-1.5 rounded-lg te
bg-emerald-600 text-white" : "bg-slate-700
 >
 по y ↓ (алгоритм)
 </button>
 <button
 onClick={() => startConnect("left-right
 className={`px-3 py-1.5 rounded-lg te
"bg-emerald-600 text-white" : "bg-slate-700
 >
 слева направо
 </button>
 <button
 onClick={startGame}
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg te
go-600 text-white" : "bg-indigo-500/80 text
 >
 соединить сам
 </button>
 </>
)}
</div>
</div>
<Plane
 points={userPairs}
 drawnEdges={phase === "game" ? gameEdges : ph
 pending={null}
 interactive={phase === "game"}

```

```

 })
 </div>
</div>

{/* Чек-лист инвариантов */}
<div className={`rounded-xl p-4 border transition
 er-slate-700 bg-slate-900/40`} `>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {phase === "game" ? (
 <>
 {verdict.heapOk ? " " : " "} каждое р
 {verdict.singleParent ? " " : " "} у
 {verdict.connected ? " " : " "} все т
 {verdict.countOk ? " " : " "} линий р
 {verdict.bstOk ? " " : " "} обход сле
 </>
) : (
 <>
 {connDone ? " " : " "} каждое ребро в
 {connDone ? " " : " "} обход слева-на
 {connDone ? " " : " "} линий ровно <b
 </>
)}

 {phase === "game" && (
 <div className="flex flex-wrap gap-2 mt-3">
 <button onClick={undoEdge} disabled={!gameE
 white disabled:opacity-30">↶ Убрать линию</
 <button onClick={resetGame} disabled={!gameE
 ate-700 text-white disabled:opacity-30"> С
 <button onClick={showSolution} disabled={ga
 te disabled:opacity-30"> Показать решение<
 </div>
)}
</div>

{/* Победное дерево в игре */}

```



```

 }, [live.mid]));
 useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.step, steps.length]);
 const done = idx >= steps.length - 1;

 const inL = new Set<NodeId>();
 const inR = new Set<NodeId>();
 steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? inL.add(s.id) : inR.add(s.id)));

 const lTree = split.l;
 const rTree = split.r;

 return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Разрез
 <select value={x0} onChange={(e) => { setX0(Number(e.target.value)); }}>
 {steps.map((s) => <option key={s.id} value={s.id}>{s.id}</option>)}
 </select>
 <button onClick={() => setStepIdx(Math.max(0, live.step - 1))}>
 -
 </button>
 <button onClick={() => setStepIdx(Math.min(live.step + 1, steps.length - 1))}>
 +
 </button>
 шаг {live.step}
 </div>

 <div className="bg-slate-900/40 rounded-xl p-4 border border-slate-700">
 <h4 className="text-sm font-bold text-slate-300">Шаг 1: разрез
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={(id) => (inL.has(id) ? "#1e3a8a" : "#f43f00")}
 nodeStroke={(id) => (inL.has(id) ? "#60a5fa" : "#f43f00")}
 subtitle={done ? "Каждый узел покрашен ровно" : ""}
 </TreeView>
 </div>
 </div>
);
 }
}

```

```

useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [live.step, steps.length]);
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;
const cur = steps[idx];

return (
 <div className="space-y-4">
 <div className="flex items-center gap-3">
 <button onClick={() => setStepIdx(Math.max(0, idx - 1))}
 bg-slate-700 text-white disabled:opacity-30">←
 <button onClick={() => setStepIdx(Math.min(stepIdx + 1, steps.length - 1))}
 font-bold bg-indigo-600 text-white disabled:opacity-30">→
 mar {idx + 1} / {steps.length}
 </div>

 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 <div className="bg-sky-950/20 rounded-xl p-3 border border-slate-700">
 <h5 className="text-sm font-bold text-sky-300">Левое дерево
 <TreeView root={l} width={480} height={220} nodeSize={16} />
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 border border-slate-700">
 <h5 className="text-sm font-bold text-rose-300">Правое дерево
 <TreeView root={r} width={480} height={220} nodeSize={16} />
 </div>
 </div>

 <div className="text-xs text-slate-300 bg-slate-900 p-4">
 {cur?.chosenId ? (
 <div>📌 {cur.note}</div>
) : (
 <div>Готово: {done ? "деревья склеены обратно в исходное состояние" : ""}</div>
)}
 </div>
 </div>
)

```

```

const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;

const pathIds = useMemo(() => {
 const ids: NodeId[] = [];
 for (const s of steps.slice(0, idx + 1)) if (s.atId
 return ids;
}, [steps, idx]);
const pathSet = new Set(pathIds);

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Удалить
 <select value={key} onChange={(e) => { setKey(N
 te-700 rounded-lg px-3 py-1.5 text-sm font-mo
 {POINTS.map((p) => <option key={p.key} value=
 </select>
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:o
 шаг {i
 </div>

 {!done ? (
 <div className="bg-slate-900/40 rounded-xl p-4 l
 <TreeView
 root={fullTree}
 nodeColor={(id) => (pathSet.has(id) ? "#783
 nodeStroke={(id) => (pathSet.has(id) ? "#f5
 subtitle="Куча сама умеет удалять только ве
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4 l
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={res.root} subtitle="Валидность

```

```

const nodeCount = slots.filter(Boolean).length;

return (
 <div className="space-y-4">
 <div className="flex gap-2">
 <button onClick={() => setPick("heap")} className="text-white" : "bg-slate-700 text-slate-300"`>
 <button onClick={() => setPick("treap")} className="text-white" : "bg-slate-700 text-slate-300">
 </div>

 {pick === "heap" ? (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-slate-300">
 <div className="flex gap-2 overflow-x-auto mb-2">
 {heapArr.slice(1).map((v, i) => (
 <div key={i} className="px-3 py-2 rounded">
 <div className="text-slate-500 text-[10px]>
 <div className="text-white font-bold">{
 <div className="text-emerald-400 text-[10px]>
 </div>
)}}
 </div>
 <p className="text-xs text-slate-400">Полное
 → $2i/2i+1$ попадает в узел.</p>
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-rose-300">
 <TreeView root={full} subtitle="Попробуй «пос
 " />
 <div className="flex gap-2 overflow-x-auto mt-2">
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 ro

```

```

const [stage, setStage] = useState<0 | 1 | 2>(0); //
const [bidx, setBidx] = useState(0);

// шаги бинарного поиска на отсортированном массиве д
const bsSteps = useMemo(() => {
 const steps: { lo: number; hi: number; mid: number;
 let lo = 0, hi = sortedByKey.length - 1;
 while (lo <= hi) {
 const mid = Math.floor((lo + hi) / 2);
 if (sortedByKey[mid].key === target) { steps.push(
 нашли за ${steps.length + 1} сравнение(й).` });
 if (sortedByKey[mid].key < target) { steps.push({
 } – вся левая половина выброшена.` }); lo = mid
 else { steps.push({ lo, hi, mid, found: false, no
 ` }); hi = mid - 1; }
 }
 return steps;
}, [sortedByKey]);

const wrongMid = Math.floor((shuffled.length - 1) / 2
const comparisons = stage === 2 ? bidx + 1 : 0;

return (
 <div className="space-y-4">
 <div className="flex gap-2">
 {stage < 2 && (
 <button onClick={() => { setStage(stage === 0
 digo-600 text-white">
 {stage === 0 ? "Найти y для x=7 бинарным по
 </button>
)}
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsSte
 -1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
)}
 <button onClick={() => { setStage(0); setBidx(0

```

```

 </div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
 const [edges, setEdges] = useState<GEdge[]>([]);
 const [sel, setSel] = useState<Pair | null>(null);
 const [hover, setHover] = useState<Pair | null>(null);
 const [msg, setMsg] = useState<string>("Кликни точку-1");

 const v = gameVerdict(edges, POINTS);

 const click = (p: Pair) => {
 if (!sel) {
 setSel(p);
 setMsg(`Выбрана (${p.key}; ${p.grade}). Теперь кликай!`);
 return;
 }
 if (sameP(sel, p)) {
 setSel(null);
 return;
 }
 const res = tryConnect(edges, sel, p);
 if (res.ok) {
 const parent = sel.grade > p.grade ? sel : p;
 const child = sel.grade > p.grade ? p : sel;
 setEdges([...edges, { a: parent, b: child }]);
 setMsg(`Линия (${parent.key}; ${parent.grade})`);
 setSel(null);
 } else {
 setMsg("x " + res.reason);
 setSel(null);
 }
 };
};

```

```

const isHover = hover && sameP(hover, p) &&
return {
 <g key={`gp-${p.key}:${p.grade}`} onClick=
l)} className="cursor-pointer">
 <circle cx={PX(p.key)} cy={PY(p.grade)}>
 <circle cx={PX(p.key)} cy={PY(p.grade)}
 "#a5b4fc" : isHover ? "#94a3b8" : "#38bdf8"
transition-all duration-200" />
 <text x={PX(p.key)} y={PY(p.grade) + 3.5}
space">{p.key}</text>
 <text x={PX(p.key)} y={PY(p.grade) + 24}
}</text>
 </g>
);
}}}
</svg>
</div>

```

```

{/* Судья – условия, как у Ахо–Корасика */}
<div className={`rounded-xl p-4 border transition
ate-900/40`}>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {v.hear ? " " : " "} каждое ребро – вниз
 {v.oneParent ? " " : " "} у каждой точки
 {v.acyclic ? " " : " "} циклов нет
 {v.bst ? " " : " "} обход слева-направо –
 {v.complete ? " " : " "} линий n-1 = {POI

 {win && {
 <p className="mt-3 text-sm font-bold text-еме
 ения просто не существует.</p>
 }}
</div>

```

```

<div className="text-xs text-slate-300 bg-slate-9
</div>

```

```

);

```

```

const n = vars ? vizNumber(vars.n) : null;
return { vars, pairsText, mid, key, step, height, n }
};

```

```

/* ===== КОРПУС ===== */

```

```

type Tab = "build" | "split" | "merge" | "erase" | "lay

```

```

const TABS: { id: Tab; label: string }[] = [
 { id: "build", label: "Собрать" },
 { id: "split", label: "Split ≈ " },
 { id: "merge", label: "Merge " },
 { id: "erase", label: "Erase " },
 { id: "layers", label: "Миф: 2k/2k+1" },
 { id: "search", label: "Поиск ≠ сортировка" },
 { id: "game", label: "Игра " },
];

```

```

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");
 const chapterId = useContext(VizChapterContext);
 const live = useTreapLive();

```

```

 // вкладка = подрежим страницы: панель Python подстав.
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);

```

```

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Пар

```



```


 открой панель Python: переменные <code clas
 merald-300">mid</code>, <code className="for
 step</code> управляют этой демонстрацией

 })
</div>

{tab === "build" && <BuildMode />}
{tab === "split" && <SplitMode />}
{tab === "merge" && <MergeMode />}
{tab === "erase" && <EraseMode />}
{tab === "layers" && <LayersMode />}
{tab === "search" && <SearchMode />}
{tab === "game" && <GameMode />}
</div>
});
};

```

ФАЙЛ 79/110: src/components/VizLiveStatus.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { liveMatrix, pickLiveVar, useVizRuntime } from
import { VizSourceBadge } from "../VizSourceBadge";

interface VizLiveStatusProps {
 /** Название демонстрации – подписывается в режиме вс
 demoTitle?: string;
 className?: string;
}

/** Имена, по которым визуализации обычно читают матрицу
const MATRIX_NAMES = ["d", "dist", "dp", "D", "S", "P",

/**

```

```
import React from "react";

import { ArticulationPointsViz } from "../ArticulationPointsViz";
import BellmanFordViz from "../BellmanFordViz";
import BfsViz from "../BfsViz";
import { DPVisualizer } from "../DPVisualizer";
import { DfsBridgesSimulator } from "../DfsBridgesSimulator";
import DijkstraViz from "../DijkstraViz";
import { EulerSimulator } from "../EulerSimulator";
import FloydViz from "../FloydViz";
import { GraphComponentsViz } from "../GraphComponentsViz";
import { GraphTraversalViz } from "../GraphTraversalViz";
import { ComplexityViz } from "../ComplexityViz";
import { HeapViz } from "../HeapViz";
import { JohnsonViz } from "../JohnsonViz";
import { KosarajuViz } from "../KosarajuViz";
import { KruskalSimulator } from "../KruskalSimulator";
import MnemonicCards from "../MnemonicCards";
import { PlanarColoringViz } from "../PlanarColoringViz";
import { QueueViz } from "../QueueViz";
import { SalmonAutomatonWidget } from "../SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "../SegmentTreeVisualizer";
import SplayPlayground from "../SplayPlayground";
import { StackViz } from "../StackViz";
import { StringAlgorithmsViz } from "../StringAlgorithmsViz";
import { TopologicalSortViz } from "../TopologicalSortViz";
import { WaterfallAnimationWidget } from "../WaterfallAnimationWidget";
import ChapterImage from "../ChapterImage";
import { Simulator } from "../Simulator";
import { PrefixSumViz } from "../visualizers/PrefixSumVisualizer";
import { SparseTableViz } from "../visualizers/SparseTableVisualizer";
import { TreapBuildViz } from "../TreapBuildViz";

export interface VizEntry {
```

```

"sparse-table": {
 title: "Разрезанная таблица (1D и 2D)",
 hint: "Наведите курсор на любую ячейку – подсветится",
 Component: SparseTableViz,
},
treap: {
 title: "Treap: плоскость → дерево → Split/Merge/Erge",
 hint: "7 вкладок: Собрать (сортировка + соединение) +

 обери дерево мышкой, судья-инварианты объясняет к",
 Component: TreapBuildViz,
},
"prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число – увидите, из чего оно слож",
 Component: PrefixSumViz,
},
"dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
},
"splay-tree": {
 title: "Splay: механика → большое дерево → собери с",
 hint: "Три вкладки: покадровая механика поворотов в

 песочница «подними узел сам» с проверкой порядка.",
 Component: SplayPlayground,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component: },
"top-sort": { title: "Топологическая сортировка", Component: },
"scc-kosaraju": { title: "Компоненты сильной связности", Component: },
"graph-articulation": { title: "Точки сочленения", Component: },
"bridges-code": { title: "Мосты: DFS + tin/low", Component: },
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: },
"planarity-euler-formula": {
 title: "Планарность и покраска: $K_5/K_{3,3}$, $\chi \leq \Delta+1$, д",
 hint: "Четыре вкладки: непланарные K_5 и $K_{3,3}$, жадна

 ашего кода красит вершины напрямую.",
 Component: PlanarColoringViz,
}

```

```
 hint: "Две вкладки: покраска обходом (каждый новый
 nt из вашего кода красят вершины.",
 Component: GraphComponentsViz,
 },
 "complexity-classes": {
 title: "Классы сложности и сведения",
 hint: "Иерархия $P \subseteq NP \subseteq PSPACE \subseteq EXP$, сведение $A \leq$
 Component: ComplexityViz,
 },
 "string-kmp": { title: "Префикс-функция (КМП)", Component: () },
 "string-z-func": { title: "Z-функция", Component: () },
 "aho-corasick": {
 title: "Ахо-Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням
 Component: () => (
 <div className="space-y-10">
 <WaterfallAnimationWidget />
 <SalmonAutomatonWidget />
 </div>
),
 },
 intro: { title: "Мнемокарточки", Component: MnemonicCards },
 "everyday-basics": {
 title: "Бытовой тренажёр: стек, очередь, куча",
 hint: "Тарелки, очередь в столовой и мешок гипотез
 Component: Simulator,
 },
};
```

```
export const getViz = (id?: string): VizEntry | undefined
```

---

ФАЙЛ 81/110: src/components/VizSourceBadge.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { Cpu, Film } from "lucide-react";
```

ажмите «шаг вперёд» (→) или «в конец трассы» — визуализация их не увидит: список имён этой темы есть : "Сейчас показана встроенная демонстрация компонента. Визуализация переключится на ваши переменные."

```
return (
 <div className={`flex flex-wrap items-center gap-1.5`} >
 <Tooltip side="bottom" content={tip}>
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center gap-1.5`}
 linked
 ? "border-emerald-500/50 bg-emerald-500/10"
 : awaiting
 ? "border-sky-500/50 bg-sky-500/10 text-slate-500"
 : "border-slate-600 bg-slate-800/80 text-slate-500"
 >
 >
 {linked ? <Cpu className="h-3 w-3" /> : awaiting}
 {linked ? "данные из компилятора" : awaiting}
 {linked && detail ?

 </Tooltip>

 {!linked && !awaiting && demoTitle ? (

 сценарий: {demoTitle}

) : null}

 {mismatch ? (
 <Tooltip side="bottom" content={mismatch}>
 <span
 tabIndex={0}
 className="cursor-help rounded-full border border-slate-600 p-2"
 >
 >
```

```

3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, d: 100, r: 100,
4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, d: 100, r: 100,
// Ветка H -> I -> S
5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, d: 100, r: 100,
6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, d: 100, r: 100,
// Ветка S -> H -> E
7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, d: 100, r: 100,
8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, d: 100, r: 100,
9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, d: 100, r: 100,
 = 2 (HE)
};

```

// Нормальные переходы бора

```

const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
 0: { H: 1, S: 7 },
 1: { E: 2, I: 5 },
 2: { R: 3 },
 3: { S: 4 },
 4: {},
 5: { S: 6 },
 6: {},
 7: { H: 8 },
 8: { E: 9 },
 9: {},
};

```

```

export const WaterfallAnimationWidget: React.FC = () => {
 // Переключатель режимов: 'training' (Полный BFS обход)
 const [activeMode, setActiveMode] = useState<'training' | 'test'>('training');

```

```

 /* — связь с Python-компилятором (PAGE_SYNC["aho-corasick"])
 text — сканируемый текст, state — вершина автомата
 matches — найденные вхождения (индекс конца, слово)
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

```

```

 useEffect(() => {

```

```

 if (typeof it === "string") {
 const m = it.match(/(\d+)\D+([A-Za-zA-Яа-я])/);
 if (m) return { index: Number(m[1]), word: m[2] };
 }
 return null;
 })
 .filter((x): x is { index: number; word: string } => x)
 if (parsed.length) setFoundMatches(parsed);
}
// eslint-disable-next-line react-hooks/exhaustive-deps
}, [vars]));

const compilerLinked = Boolean(liveText || (liveState

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>('');
const [currentIndex, setCurrentIndex] = useState<number>(0);
const [currentNode, setCurrentNode] = useState<number>(0);
const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>('');
 'Режим поиска: Введите текст и жмите «Следующий шаг».';
);
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({ from: 0, to: 0 });
const [foundMatches, setFoundMatches] = useState<{ index: number; word: string }[]>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/ /g, '');
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {
 setCurrentIndex(0);

```

```

const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Логика

 setJumpPath(null);
} else {
 logMsg += `Поток по '${char}' у вершины #${originalCurr} переходит

 по fail-ссылке в #${curr} (${WATERFALL_FAIL_LINK}).

 `;
 setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
 curr = nextNode;
}

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
 if (WATERFALL_NODES[temp].is_terminal) {
 WATERFALL_NODES[temp].words.forEach((word) => {
 currentMatches.push({ index: currentIndex, word });
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWordsForLog.join(' ')}

 setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMatches]);
}

```



```
targetNodeId: 2,
title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
desc: 'Переходим на Depth 2 к вершине #2 (HE). Родитель :
а-разведчик плавает в ROOT и ищет ветку по "E".',
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'E',
checkingBranchesFrom: 0,
},
{
targetNodeId: 5,
title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'I',
checkingBranchesFrom: 0,
},
{
targetNodeId: 8,
title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ
desc: ' ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
)'. Рыба плавает в ROOT и ищет ветку "H". У корня
суффикс "H"!',
codeLine: 4,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'H',
checkingBranchesFrom: 0,
},
{
targetNodeId: 3,
title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
е подходят. fail[HER] = ROOT.',
```

```

 },
];

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainStep];

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border-1px border-slate-700" >
 { /* Шапка и переключатель режимов */ }
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Анимация водопада: Полное обучение автомата
 </h4>
 <p className="text-sm text-slate-400" >
 Визуализация пошагового построения всех функций
 </p>
 </div>
 </div>
 </div>

 { /* Переключатель режимов */ }
 <div className="bg-slate-900 p-1.5 rounded-xl border-1px border-slate-800" >
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p-2 transition-colors duration-200
 ${activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'}
 `}
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корня)
 </button>
 <button
 onClick={() => setActiveMode('search')}

```

```

 {step}
 {idx === trainStep &&
 </button>
 </div>
 </div>

 <div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => Math.min(prev + 1, TRAINING_STEPS_INFO.length))}
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg-slate-600 text-white font-weight-bold py-2 px-3 transition-all"
 >
 Назад
 </button>
 <button
 onClick={() => setTrainStep((prev) => Math.min(prev + 1, TRAINING_STEPS_INFO.length))}
 disabled={trainStep === TRAINING_STEPS_INFO.length}
 className="flex-1 bg-indigo-600 hover:bg-indigo-500 text-white font-weight-bold py-2 px-3 transition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.length ? "Готово" : "Далее"}
 </button>
 </div>
 </div>

 {/* Блок с кодом построения fail-ссылки и описанием */}
 <div className="lg:col-span-2 bg-slate-900/90 text-white p-4">
 <div>
 <h5 className="text-white font-weight-bold mb-3">Код построения fail-ссылки
 <div>

 {failLink} Код BFS построения fail-ссылки

 Вершина: #{targetTrainingNode.id} ({"{targetTrainingNode.parentId}"})

 </div>
 </div>
 </div>

```

```

 <h5 className="text-slate-200 font-bold mb-2">
 <Lightbulb className="w-4 h-4 text-amber-500"/>
 </h5>
 <div className="bg-slate-800 p-4 rounded-lg w-full min-w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
 </div>
 </div>
 </div>
) : (
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3">
 <div className="bg-slate-900/60 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-2">
 » Текст для сканирования
 </h5>
 <p className="text-xs text-slate-400 mb-4">
 Введи текст, чтобы лосось просканировал
 </p>
 <input
 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-slate-700 focus:outline-none focus:border-blue-500"
 />
 </div>
 <div>
 <div className="mb-4 flex flex-wrap gap-1">

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}

```

```

 </h5>
 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex item
border-blue-500 text-blue-300 font-bold' :
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && <span <
инициализация}
 </div>
 <div className={`p-1.5 rounded flex item
border-rose-500 text-rose-300 font-bold an
 2. while (curr !== root && !trie
 {activeSearchCodeLine === 2 && <span <
рыжок по fail!}
 </div>
 <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold' :
 3. curr = trie[curr].get(char)
 {activeSearchCodeLine === 3 && <span <
ем по течению}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
 4. if (is_terminal[curr]) repor
 {activeSearchCodeLine === 4 && <span <
овпадение!}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-300 font-bold m
 <AlertCircle className="w-3.5 h-3.5 tex
 </h5>
 <div className="bg-slate-800 p-3.5 rounde
tems-center">
 {searchLog}
 </div>
</div>

```

```

 { depth: 2, y: 260, label: 'Depth 2 (HE, I
 { depth: 3, y: 360, label: 'Depth 3 (HER,
 { depth: 4, y: 460, label: 'Depth 4 (HERS
].map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14"
 />
 <text x="32" y={level.y + 4} fill="#94a
 {level.label}
 </text>
 </g>
)})

```

```

 /* Рендер прямых переходов (потоков воды)
 Object.entries(TRIE_TRANSITIONS).map(([from
 const fromNode = WATERFALL_NODES[Number(f
 return Object.entries(children).map(([cha
 const toNode = WATERFALL_NODES[toId];

```

```

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'sea
 type === 'normal';

```

```

 // Подсветка в режиме обучения (когда и
 let isTrainingExamined = activeMode ===
 let isMatchBranch = isTrainingExamined &

```

```

 return (

```

```

 });
 });
}

{/* Рендер fail-ссылки (пунктирные водопады)
Object.values(WATERFALL_NODES).map((node) => {
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.fail];

 // В режиме обучения показываем fail-ссылку
 // Найдем индекс шага, на котором обработать
 const stepIdxForNode = TRAINING_STEPS_INFO[node.fail];
 if (activeMode === 'training' && trainStep === stepIdxForNode) {

 let isHighlighted = activeMode === 'search' || activeMode === 'fail';
 let isJustEstablished = activeMode === 'training' && trainStep === stepIdxForNode;

 // Вычисляем изогнутую кривую для красоты
 const dx = targetNode.x - node.x;
 const dy = targetNode.y - node.y;
 const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -10 : 10);
 const cy = node.y + dy / 2 - 30;

 return (
 <g key={`fail-${node.id}`}>
 <path
 d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`}
 fill="none"
 stroke={isHighlighted ? '#f43f5e' : '#10b98f'}
 strokeWidth={isHighlighted || isJustEstablished ? 6 : 2}
 strokeDasharray="6,6"
 className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
 opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b98f">fail
)}
 </g>
);
 }
});
}

```

```

 fill={isCurrent || isTargetChild ?
 fontSize={isCurrent || isTargetChild
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 {/* Метка искомой вершины в режиме об
 {isTargetChild && {
 <g transform={`translate(${node.x},
 <rect x="-35" y="-12" width="70"
 <text x="0" y="2" fontSize="10" f
 ИЩЕМ FAIL
 </text>
 </g>
 }}

 {/* Индикатор словарных слов */}
 {hasWords && {
 <g transform={`translate(${node.x +
 <circle cx="0" cy="0" r="9" fill=
 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
 }}
 </g>
);
 }}}

 {/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ *
 <g>
 {/* Круг-подложка под рыбу */}
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"

```



```

 className="bg-emerald-950 border border-
-sm font-bold shadow animate-[scaleUp_0.3s_
>
 <CheckCircle2 className="w-4 h-4 text
 {match.word}
 <span className="text-[10px] bg-emera
 Индекс {match.index}

 </div>
)})
</div>
})

```

```

{/* что сейчас сообщил компилятор */}
<div className="mt-6 flex flex-wrap items-center
 <span className="text-[10px] uppercase tracking
 <Terminal className="w-3.5 h-3.5" /> из компи.

 {compilerLinked ? (
 <>
 {liveText && <span className="text-[11px] f
 >text = {liveText}}
 {liveI !== null && <span className="text-[1
 py-1">i = {liveI}{liveText ? ` (${liveText.
 {liveState !== null && (
 <span className="text-[11px] font-mono bg
 state = {liveState}{WATERFALL_NODES[liv

)})
 {liveMatchesSig && <span className="text-[1
 d px-2 py-1">matches: {foundMatches.map((m)
 {liveFail !== undefined && <span className=
 ed-md px-2 py-1">fail = {JSON.stringify(liv
 </>
) : (


```

```

* а при выбранном запросе показывается формула вклю
*/
```

```
const A1 = [3, 1, 4, 1, 5, 9, 2, 6];
```

```
const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];
```

```
const cellBase =
 "flex items-center justify-center rounded-md font-monosp
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] te
```

```
export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 const chapterId = useContext(VizChapterContext);
```

```
// Вкладка 2D = своё демо (свой скелет): компилятор п
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode === "2d"
}, [chapterId, mode]);
```

```
return (
 <div className="w-full bg-slate-950 rounded-2xl border
 <div className="flex gap-2 mb-4 overflow-x-auto no
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm
```

```

 ? { from: 0, to: hover.i - 1 }
 : hover?.kind === "a"
 ? { from: hover.i, to: hover.i }
 : null;

```

```
const sum = pref[range.r + 1] - pref[range.l];
```

```

return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 {/* индексы */}
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={`_${cellBase} !h-5`}>
 {i}
 </div>
))}
 </div>

 {/* массив a */}
 <div className="flex gap-1.5 items-center mb-1">
 <div className="w-[46px] shrink-0 text-right">
 {A1.map((v, i) => {
 const inCover = covered && i >= covered.f
 const inRange = i >= range.l && i <= range.r
 const isDrivenAdd = shownDriven !== null && i === shownDriven
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: "a" })}
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r ? r : i))}
 className={`_${cellBase} cursor-pointer`}>
 {v}
 {isDrivenAdd
 ? "bg-amber-500 text-black ring-2"
 : inCover
 ? "bg-indigo-500 text-white ring-2"
 : "bg-white text-black ring-2"}
 </div>
)
)}
 </div>
 </div>
 </div>
 </div>
 </div>
)
)

```

```

 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"
 : "bg-slate-900 text-slate-300"
 }` }
 title={blank || masked ? `P[${i}] – n` : ""}
 >
 {blank || masked ? "." : String(shown)}
</div>
);
 }}}
</div>
</div>
</div>

```

```

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border border-slate-700" >
 {shownDriven !== null && !hover ? (
 <p className="text-slate-300">
 <b className="text-emerald-400">Компилятор
 i =

 {hasLiveP && <> =
 Пустой массив

 </p>
) : hover?.kind === "pref" ? (
 hover.i === 0 ? (
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0 – длина
 отрезка, начинающегося с нуля.
 </p>
) : (
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала
 </p>
)
) : (
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала
 </p>
)
)
}

```

```

 });
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveI = vizNumber(vars?.i);
 const liveJ = vizNumber(vars?.j);

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array<number>().length);
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i][j - 1];
 }
 }
 return p;
 }, [n, m]);

 const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
 const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

 /**
 * Матрицы из кода пользователя рисуются В СВОЕЙ размерности.
 *
 * Раньше значения S[i][j] протискивались в демо-сетку,
 * размере матрицы часть ячеек становилась «пустыми»,
 * шаги компилятора не совпадали с картинкой.
 */
 const livePrefSource = pickLiveVar(vars, ["S", "P", "g"]);
 const liveS = liveMatrix(livePrefSource?.value);
 const hasLiveS = liveS !== null;

 const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
 const liveA = liveMatrix(liveArrSource?.value);

```

```

// Границы запроса держим внутри реальной сетки (у по
const r1 = Math.max(0, Math.min(rect.r1, prefRows - 1)
const c1 = Math.max(0, Math.min(rect.c1, prefCols - 1)
const r2 = Math.max(r1, Math.min(rect.r2, prefRows - 1)
const c2 = Math.max(c1, Math.min(rect.c2, prefCols - 1)

const D = cellAt(r1, c1);
const B = cellAt(r1, c2 + 1);
const Cc = cellAt(r2 + 1, c1);
const Aa = cellAt(r2 + 1, c2 + 1);
const total = Aa !== null && B !== null && Cc !== null;
const fmt = (v: number | null) => (v === null ? "-" : v);

const cornerOf = (i: number, j: number) => {
 if (i === r2 + 1 && j === c2 + 1) return "A";
 if (i === r1 && j === c2 + 1) return "B";
 if (i === r2 + 1 && j === c1) return "C";
 if (i === r1 && j === c1) return "D";
 return null;
};

return (
 <div className="space-y-5">
 {sizeMismatch ? (
 <p className="rounded-lg border border-amber-500">
 {sizeMismatch}
 </p>
) : null}
 <div className="grid gap-5 lg:grid-cols-2">
 /* Исходная матрица */
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 Исходная матрица {liveA ? `${liveArrSource?}` : ""}
 чтобы задать прямоугольник
 </div>
 <div className="inline-block">
 {srcGrid.map((row, i) => (
 <div key={i} className="flex gap-1.5 mb-1"

```

```

const isCompilerCell = liveI === i &&
const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border-
return {
 <div
 key={j}
 onMouseEnter={() => setHover({ i,
 onMouseLeave={() => setHover(null
 className={`${cellBase} cursor-he
 isCompilerCell
 ? "bg-emerald-600 text-white
 : isHover
 ? "bg-amber-500 text-white
 : blank
 ? "bg-slate-950 text-slate-
 : cornerColor
 }`}
 title={hasLiveS ? `${livePrefSour
 >
 {blank ? "." : String(shownValue)
 {corner && !isHover && {
 <span className="absolute -top-
ht">
 {corner}

 }}
 </div>
);
 }}}
</div>
)))
</div>

```







```

 : null;

/** Размерность таблицы пользователя: сетка демо 8x4,
const liveTable = liveMatrix(runtimeVars?.st);
const dimMismatch = liveTable
 ? liveTable.rows !== N || liveTable.cols !== LOG
 ? `Демо-сетка построена для массива из ${N} элемен
 блица ниже рисует ячейки демо-сетки: значения з
 панели переменных компилятора.`
 : null
 : hasLiveSource && liveSource && liveSource.length
 ? `Демо-массив из ${N} элементов, а в вашем коде
 ся демо-размера ${N}x${LOG}.`
 : null;

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-500">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$

```

```

 : typeof v === "object"
 ? JSON.stringify(v)
 : String(v);
 return (
 <div key={idx} className="flex flex-col i
 <div
 className={`flex items-center justify
w,44px)}] h-[clamp(26px,8vw,44px)] text-[cla
 inCover ? "bg-indigo-500 text-white
 }`}
 >
 {display}
 </div>
 <span className="text-[9px] text-slate-
 </div>
);
 }}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
 {hover && covered ? (
 <p className="text-slate-300">
 <b className="text-sky-400">
 ST[{hover.i}][{hover.j}] = {compilerLin
 j]}
 {" "}
 – это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\text{hover.j}}$ = {1 << hover.j}:{" "}

 min({arr1D.slice(covered.from, covered.

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое чи
 </p>
)
}

```

```

// Уровень j=0 – это исходный a[i], от
// очищаемого рабочего массива и не д
const sourceValue = j === 0 ? liveSou
const runtimeValue = liveValue ?? sou
const runtimeHasValue = runtimeValue
const manualVisible =
 isValid &&
 (j === 0 || computeSteps.findIndex(
const isVisible = compilerLinked ? run

let isActive = !!liveTarget && liveTa
let isSrc1 = false;
let isSrc2 = false;

if (compilerLinked && liveTarget && l
 isSrc1 = i === liveTarget.i && j ===
 isSrc2 = i === liveTarget.i + (1 <<
} else if (!compilerLinked && step >
 const currentStep = computeSteps[ste
 isActive = currentStep.i === i && c
 if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 =
 if (currentStep.i + (1 << (current
 }
}

// Готовые вычисленные уровни демо не
// уровень j=0 всегда берём из уже су
const shownValue = compilerLinked ? r
const displayValue =
 shownValue === undefined || shownVa
 ? "."
 : typeof shownValue === "object"
 ? JSON.stringify(shownValue)
 : String(shownValue);

return (
 <div key={j} className="relative fl

```

```

 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-events-none"
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 <= isSrc2 ? 40 : 0) : 40,
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc2 ? 40 : 0} 0 0`}
 fill="none"
 stroke={isSrc1 ? "#10b981" : "#000000"}
 strokeWidth="3"
 />
 </motion.svg>
 </div>
 </div>
);
}
</React.Fragment>
)
</div>
</div>
</div>

```

```

 /* Formula helper text */
 <div className="bg-slate-900 border border-slate-800 p-4">
 {compilerLinked && liveTarget ? (
 <p className="text-base text-slate-300">

 i =
 — активна ячейка
 {(hasLiveTable || hasLiveSource) && <> = ?.[liveTarget.j] ?? (liveTarget.j === 0 ? liveTarget.i : liveTarget.j)

 </p>
) : null}
 </div>
)
}

```

```
};
```

```
const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: number }>({ r: 0, c: 0 });
 const [locked, setLocked] = useState<{ r: number; c: number }>({ r: 0, c: 0 });
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveK = vizNumber(vars?.k);
 /**
 * Шаг, закреплённый кликом по вкладке. Без него live
 * перетирал выбор пользователя и вкладки «Шаг 1/2/3»
 * клик есть, а матрица не меняется. Закрепление живё
 */
 const [pinnedK, setPinnedK] = useState<number | null>(null);
 const prevLiveK = useRef<number | null>(liveK);
 useEffect(() => {
 if (liveK !== prevLiveK.current) {
 prevLiveK.current = liveK;
 setPinnedK(null);
 }
 }, [liveK]);
 const liveR = vizNumber(vars?.r);
 const liveC = vizNumber(vars?.c);
 const liveTable = vizRecord(vars?.st2);
 const hasLiveSource = !!vars && Object.prototype.hasOwnProperty.call(vars, 'source');
 const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'table');
 const liveTableEmpty = hasLiveTable && Object.keys(liveTable).length === 0;
 const codeCell = liveR !== null && liveC !== null ? {
 r: liveR, c: liveC, k: liveK
 } : null;
 const compilerLinked = codeCell !== null || liveK !== null;
 const shownK = pinnedK !== null
 ? pinnedK
 : liveK !== null
 ? Math.max(0, Math.min(3, Math.trunc(liveK)))
 : hasLiveSource
 ? 0
 : k;
}
```

```

 onClick={() => { setK(step); setPinnedK(s
 className={`px-4 py-2 rounded-md text-sm
 'text-slate-400 hover:text-white hover:bg-
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${s
 </button>
 }}}
 </div>
 {pinnedK !== null && compilerLinked && (
 <p className="-mt-6 mb-6 text-[11px] text-sla
 Шаг закреплён вручную: когда <span className
 демонстрация вернётся к шагу кода.
 </p>
)}

 <div className="flex flex-col items-center gap-
 scrollbar">
 [...Array(shownK + 1)].map((_, i) => shownK
 const stepL = 1 << step;
 const stepSize = 8 - stepL + 1;
 const isCurr = step === shownK;

 return (
 <React.Fragment key={step}>
 {idx > 0 && <div className="text-slate
 <div className={`flex flex-col items-c
 <h4 className={`font-bold mb-3 flex
 }>
 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
 rder-indigo-800/40 text-indigo-300' : 'bg-s
 </h4>
 <div className="max-w-full overflow-
 <div className={`grid gap-[2px] p-
 go-900/40 shadow-[0_0_25px_rgba(99,102,241,
 eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step

```

```

md:scale-[1.08]';
 } else if (isTop && !isLeft)
 highlightClass = 'bg-blue-500';
] md:scale-[1.08]';
 } else if (!isTop && isLeft)
 highlightClass = 'bg-emerald-500';
 9,0.3)] md:scale-[1.08]';
 } else {
 highlightClass = 'bg-amber-500';
 3)] md:scale-[1.08]';
 }
 zIndex = 'z-10';
 }
}

return {
 <div
 key={idx}
 onMouseEnter={() => { if(isCurr) setCurr(idx); }}
 onMouseLeave={() => { if(isCurr) setCurr(-1); }}
 onClick={() => {
 if(isCurr) {
 if(locked && lockedBy === idx) return;
 else setLocked({r: idx, c: idx});
 }
 }}
 className={`flex items-center justify-center gap-2`}
 style={{width: clamp(24px, 6.5vw, 38px), height: clamp(24px, 6.5vw, 38px),
 background: highlightClass, border: 1px solid black, border-radius: 5.2vw, 30px}}
 h-[clamp(20px, 5.2vw, 30px)] min-[width: 20px, height: 20px]}
 title={hasLiveTable ? `ст2[${idx}]` : ''}
 >
 {isBlank ? "•" : String(showLiveTable ? table[idx][0] : 0)}
 </div>
);
}}}
</div>
</div>
</div>

```



```

) : (
 <div className="flex flex-col gap-4 animate-
 <div className="bg-[#0a0fle] rounded-xl l
ate-300 shadow-inner overflow-x-auto">
 // Те
 int<
="text-amber-300">1);

 // Кв
 ST</
 <span className="text-rose
e="text-amber-300">1], <sp
 <span className="text-blue
ext-amber-300">1][k-<span className=
pan>

 <span className="text-emer
="text-amber-300">1][k-<span classNa
span>

 <span className="text-ambe
white">c + pL][k-<span className="te
ame="text-slate-500">// Правый Низ<b
 {''}}));
 </div>

 {activeCell ? (
 <div className="bg-slate-950 p-5 round
.2)] mt-2 transition-all">
 <p className="text-slate-300 mb-3 t
 Пример для ячейки:
 {locked && <span className="text
ded border border-rose-500/20"><Lock size={
 </p>
 <p className="text-indigo-300 borde
 <span className="font-bold text-w
 }][{shownK}][{shownK}]

 </p>

 <div className="space-y-2 pt-3 pl-2

```

```

 список матриц (слева) вниз, чтобы увидеть,
 </div>
) : (
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L}
 </div>
)}
 </div>
)}
</div>
</div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);
 const runtime = useVizRuntime();

 // Координаты запроса – те же захардкоженные имена,
 useEffect(() => {
 const vars = runtime?.variables;
 const nr1 = vizNumber(vars?.r1);
 const nc1 = vizNumber(vars?.c1);
 const nr2 = vizNumber(vars?.r2);
 const nc2 = vizNumber(vars?.c2);
 if (nr1 !== null) setR1(Math.max(0, Math.min(7, M
 if (nc1 !== null) setC1(Math.max(0, Math.min(7, M
 if (nr2 !== null) setR2(Math.max(0, Math.min(7, M
 if (nc2 !== null) setC2(Math.max(0, Math.min(7, M
 }, [runtime]));

 const queryVars = runtime?.variables;
 const liveCoords = ["r1", "c1", "r2", "c2"].map((na

```

```
0 border-rose-500/50 border scale-105 z-10'
 {val2D[r][c]}
 </div>
```

```
)
}}}
```

```
<div
 className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
 style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
/>
```

```
/* Показываем 4 угла в самой матрице
<div className="absolute pointer-e
shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
 <div className="absolute pointer-e
shadow-[0_0_10px_#3b82f6]" style={{ top: `
width: `calc(`${(Ly / 8) * 100}% - 16px)`,
 <div className="absolute pointer-e
ld-400 shadow-[0_0_10px_#10b981]" style={{
8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
 <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
 </div>
```

```
<div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
 <label className="flex items-cen
 <span className="text-slate-
```

```

ext-amber-300">1); <span className="
 int<
ext-amber-300">1); <span className="
 int<
"text-slate-500">// Выс: {Lx}

 int<
"text-slate-500">// Шир: {Ly}
<b

 // 2.
 int<

В запроса)

r1][<span className="text-white bg-r

во, чтобы правый край уперся в c2)<b

r1][<span className="text-white bg-b

рх, чтобы нижний край уперся в r2)<b

nded">r2 - Lx + 1][<span className="

и влево от c2)

">r2 - Lx + 1][<span className="text
 {'}'}));
</div>

<div className="w-full flex justify-cent
 <div className="flex flex-col items-ce
 <h4 className="text-slate-400 font-bo
 Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{{kx}}][{k
 </h4>

```

```

 <div className="text-center font-bold
wrap bg-[#0a0f1e] py-3 px-2 rounded-lg bord
 <span className="text-slate-400 fon

 <span className="text-blue-400 m
n>
 <span className="text-emerald-40
span>
 <span className="text-amber-400 r
 <span className="text-slate-4
 <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
 </div>
 </div>
</div>
)
}
```

---

## РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

---

---

ФАЙЛ 85/110: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

---

```

 </div>
 </main>
</div>
```

```

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal')....
```

```

 btn.classList.remove('outline', 'outline-1');
 });

 if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded-md transition-colors');
 if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded-md transition-colors');
 if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors' : 'px-3 py-1 rounded-md transition-colors');
 document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded-md transition-colors');
 document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded-md transition-colors');
 document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors' : 'px-3 py-1 rounded-md transition-colors');
}

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu-btn');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer-btn');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('hidden');
 if (isClosed) {
 mobileDrawer.classList.remove('hidden');
 if (drawerOverlay) {
 drawerOverlay.classList.remove('hidden');
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
 }
 } else {
 mobileDrawer.classList.add('hidden');
 if (drawerOverlay) {
 drawerOverlay.classList.add('hidden');
 }
 }
 }

 mobileMenuBtn.addEventListener('click', toggleDrawer);
 closeDrawerBtn.addEventListener('click', toggleDrawer);
 drawerOverlay.addEventListener('click', toggleDrawer);
});

```

```
 if (entry.isIntersecting && !document.querySelector('#active-toc-link')) {
 // Highlight active TOC link locally
 const id = entry.target.id;
 document.querySelectorAll('aside .toc-link').forEach(link => {
 if (link.getAttribute('href') === `#${id}`) {
 link.classList.add('bg-active');
 } else {
 link.classList.remove('bg-active');
 }
 });
 }
 });
}, observerOptions);

document.querySelectorAll('section[id^="questions"]').forEach(section => {
 observer.observe(section);
});
});
</script>
</body>
</html>
```

---

ФАЙЛ 86/110: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 10.5 КБ

---

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Пособие по алгебре: От пиццы к предикатам</title>
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js?features=IntersectionObserver"></script>
 <script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.2/es5/tex-mml-chtml.js"></script>
```

```

 }

 mjx-container {
 max-width: 100% !important;
 }
 mjx-container[display="true"] {
 overflow-x: auto;
 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
 }
 mjx-container[display="true"]::-webkit-scrollbar
 display: none;
 }

 @media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font
 .uno-card::before, .uno-card::after { font-
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px;

 .uno-card.mini { width: 28px !important; he
 us: 4px; }
 .uno-card.mini::before, .uno-card.mini::aft

 .uno-equation .text-2xl { font-size: 1.2rem
 .uno-equation.gap-4 { gap: 0.5rem !important

 /* Allow horizontal scroll for equations in
 .formula-scroll { flex-wrap: nowrap !importan
 px; }
 }
 .uno-inner {
 position: absolute; width: 110%; height: 75%
 }

```



```

 background-color: #fcf8eb !important; /* light brown ink */
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient(45deg, transparent, transparent 2px, #3b2818 2px, #3b2818 4px);
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Serif";
 }
 body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
 background-color: rgba(255, 255, 255, 0.7);
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
 }
 body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #cbd5e1; }
 body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-800 { border-color: #94a3b8 !important; }
 body.theme-notebook .font-mono { font-family: "Monospace"; }
 body.theme-notebook #top-controls { background-color: #f1f3f4; }

 /* CREEPY BLINKING */
 @keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1); }
 96.5% { transform: scaleY(0.1); }
 }
 .creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
 }
 .creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
 }
 .creepy-emoji {

```

```

<marker id="arrow-orange" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#f9731
</marker>
<marker id="arrow-pink" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b
</marker>
<marker id="arrow-cyan" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4b
</marker>
<marker id="arrow-indigo" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf
</marker>
<marker id="arrow-lime" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63
</marker>
<marker id="arrow-green" markerWidth="6" ma
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade8
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-red">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>

```

```

<!-- Левое меню (Оглавление) -->
<aside id="mobile-drawer" class="fixed inset-y-0
 nslate-x-full transition-transform duration-300
 w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
 <div class="h-full flex flex-col p-6 overflow-y-auto
 lg:rounded-2xl lg:border lg:border-slate-700
 <div class="flex justify-between items-center
 <h3 class="font-black text-transparent bg-clip-text
 ider text-sm">
 Содержание
 </h3>
 <button id="close-drawer-btn" class="text-slate-500
 <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6
 <path stroke-linecap="round stroke-width="2" d="M6 18
 </svg>
 </button>
 </div>
 <nav class="space-y-3 text-sm font-medium">
 <div>
 <details class="group" open>
 <summary class="list-none cursor-pointer text-slate-500

 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-sm">
 1.1
 1.2
 1.3
 1.4
 </div>
 </details>
 </div>
 <div>
 <details class="group">
 <summary class="list-none cursor-pointer text-slate-500

 2. Теория групп

 </summary>
 <div class="pl-6 space-y-2 text-sm">
 2.1
 2.2
 2.3
 </div>
 </details>
 </div>
 </nav>
 </div>
 </div>
 </div>
 </div>

```

```
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-500">
 5. Группа корней из единицы

 </summary>
 <div class="pl-6 space-y-2 text-fuchsia-500">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-lime-500 pl-3 font-bold text-lime-300">
 6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text-lime-500">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-lime-500 pl-3 font-bold text-lime-300">
```

```

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q9-1" class="bloc
 <a href="#q9-2" class="bloc
 <a href="#q9-3" class="bloc
 <a href="#q9-4" class="bloc
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-10" clas
der-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Г

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q10-1" class="blo
 <a href="#q10-2" class="blo
 <a href="#q10-3" class="blo
a>
 <a href="#q10-4" class="blo
 <a href="#q10-5" class="blo
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-11" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
 11. НОД и НОК многочлен

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q11-1" class="blo
 <a href="#q11-2" class="blo
нственности НОД
 <a href="#q11-3" class="blo

```

```
der-fuchsia-500 pl-3 font-bold text-fuchsia
 14. Кратность корня. Ко

 </summary>
 <div class="pl-6 mt-2 space-y-2"
 <a href="#q14-1" class="blo
 <a href="#q14-2" class="blo
 <a href="#q14-3" class="blo
 deg)
 <a href="#q14-4" class="blo
 ition-colors border border-rose-500/30"> K
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-15" clas
 der-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критери

 </summary>
 <div class="pl-6 mt-2 space-y-2"
 <a href="#q15-1" class="blo
 <a href="#q15-2" class="blo
 <a href="#q15-3" class="blo
 <a href="#q15-4" class="blo
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-16" clas
 der-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многоч

 </summary>
 <div class="pl-6 mt-2 space-y-2"
 <a href="#q16-1" class="blo
```

```

 document.querySelectorAll('details.' + detailId)
 detail.addEventListener('toggle', () => {
 if (detail.open) {
 document.querySelectorAll('details.' + otherId)
 if (other !== detail) {
 other.removeAttribute('open')
 }
 }
 });
 });
});

// Scroll spy logic
document.addEventListener('DOMContentLoaded', () => {
 const sections = document.querySelectorAll(
 "#q5, #q6, #q7, #q8, #q9, #q10, #q11, #q12, #q13, #q14, #q15, #q16, #q17, #q18, #q19, #q20, #q21, #q22, #q23, #q24, #q25, #q26, #q27, #q28, #q29, #q30, #q31, #q32, #q33, #q34, #q35, #q36, #q37, #q38, #q39, #q40, #q41, #q42, #q43, #q44, #q45, #q46, #q47, #q48, #q49, #q50, #q51, #q52, #q53, #q54, #q55, #q56, #q57, #q58, #q59, #q60, #q61, #q62, #q63, #q64, #q65, #q66, #q67, #q68, #q69, #q70, #q71, #q72, #q73, #q74, #q75, #q76, #q77, #q78, #q79, #q80, #q81, #q82, #q83, #q84, #q85, #q86, #q87, #q88, #q89, #q90, #q91, #q92, #q93, #q94, #q95, #q96, #q97, #q98, #q99, #q100"
);
 let isScrolling = false;

 document.querySelectorAll('.group-item-link').forEach(link => {
 link.addEventListener('click', () => {
 isScrolling = true;
 setTimeout(() => isScrolling = false, 200);
 });
 });

 const observer = new IntersectionObserver((entries) => {
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;
 }
 });

 // Highlight active link
 document.querySelector('a[href="#' + visibleId + '"]').style.fontWeight = 'bold';
 if (a.href.endsWith('#')) {
 a.href = a.href + visibleId;
 }
 });
});

```

```
modal.style.zIndex = '99999';
modal.style.display = 'flex';
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в ' + fileName;
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за особенностей веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
```



```

 btnContainer.appendChild(closeB
 box.appendChild(btnContainer);
 modal.appendChild(box);
 document.body.appendChild(modal
 }

```

```

function downloadMD() {
 let clone = document.querySelec

 // Smart markdown parsing based

 // Smart markdown parsing based
 clone.querySelectorAll('h2').fo
 clone.querySelectorAll('h3').fo
 clone.querySelectorAll('.analog
 clone.querySelectorAll('li').fo
 clone.querySelectorAll('svg').fo
 let caption = '';
 if (el.nextElementSibling &
 caption = el.nextElement
 }
 el.textContent = '\n\n[Илл
 });
 clone.querySelectorAll('.img-ca

 let text = clone.innerText;
 text = text.replace(/\n{3,}/g,

 if (window !== window.top) {
 fallbackModal(text, 'Markdow
 return;
 }
 const blob = new Blob([text], {
 const url = URL.createObjectURL
 const link = document.createEle
 link.href = url;
 link.download = 'vyisshaya_alge
 document.body.appendChild(link)

```

```

body.bg-white [class*="f
nt; }
body.bg-white [class*="f
nt; }
body.bg-white [class*="f
body.bg-white .analogy-l
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-l
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuc
body.bg-white .text-lim

```

```

ont-bold flex items-center justify-center gap-4
 Word
 </button>

-sm font-bold flex items-center justify-center gap-4
 PDF

 <button onclick="downloadMD()">
-bold flex items-center justify-center gap-4
 Markdown
 </button>
 <button onclick="downloadHTML()">
t-sm font-bold flex items-center justify-center gap-4
 HTML
 </button>
</div>

<!-- Смена темы -->
<div>
 <div class="text-xs text-slate-500">
 <div class="flex items-center gap-4">
 <button id="theme-dark" onclick="toggleTheme('dark')">
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
 <button id="theme-light" onclick="toggleTheme('light')">
r justify-center text-amber-50 hover:text-white
">* </button>
 <button id="theme-terminal" onclick="toggleTheme('terminal')">
-center justify-center text-green-400 hover:text-white
терминала"> </button>
 </div>
</div>
</div>

</div>
</aside>

<!-- Основной контент -->

```

```

<!-- ===== WTF CONTAINER =====
<div id="wtf-container" class="hidden">
 <h2 class="text-3xl font-black text-cen

 <div class="overflow-x-auto w-full pb-4">
 <div class="grid grid-cols-3 gap-4">
 <!-- Column headers -->
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
 <div class="bg-cyan-900/40 bord
сные числа</div>

 <!-- Row 1: База -->
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slat

 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
 <div class="text-xs text-orange
 <div class="font-bold text-slat

 <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slat

 <!-- Row 2: Базовые операции / Евклид
 <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slat


```

```

 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

 <!-- Row 5: Корни / Уравнения -->
 <div class="hidden md:block"></div>

- l-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

- l-4 border-cyan-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>

- l-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>

- l-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

 <div class="hidden md:block"></div>

 <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
 <div class="col-span-1 md:col-span-3">
 b-2 border-b border-emerald-500/30 pb-2">Кл

```

```
:col-span-3 bg-red-900/20 p-4 border-l-4 border-r-4
 <div class="text-xs text-red-500">
 <div class="font-bold text-slate-500">

 </div>
</div>

<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="mb-6">
 <div class="flex items-center mb-6">
 <div class="bg-red-500/20 p-3 rounded">
 <svg class="w-8 h-8 text-red-500"
rg/2000/svg">
 <path stroke-linecap="round"
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border-r-4">
 </div>

 <div class="prose prose-invert max-w-3xl">
 <p class="text-lg">
 Хватит тупить. Доказательств
 есь для того, чтобы жать на кнопки. Вот жес
 </p>

 <h2 class="text-2xl font-bold border-b-2">
(Меметика)</h2>

 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-blue-400">
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5">
 <span class="text-w
k$).
 <span class="text-w
 <span class="text-w
```

```
 <span class="text-w
Просто перепиши формулу с буквой k .
 <span class="text-w
усок k , замени его на жареного карася из

 </div>
```

```
 <!-- Двойная делимость -->
 <div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-ora
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 sp
 <span class="text-w
вверх по уравнениям).
 <span class="text-w
рху вниз, показывая, что $\Delta \vdots r_i$
 <span class="text-w
х. Твой d больше.

 </div>
```

```
 <h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```
 <div class="grid grid-cols-1 md
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
со следующим. Если в конце 0 – пациент мертв
 </div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
 c . Пока получается 0 – корень жив. Как то
```

```

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 x двух, потом последних.

 <code class="text-x">
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 есть сложение — есть ноль. Если есть умножение
 <code class="text-x">
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 авь их драться. Умножение всегда врывается
 <code class="text-x">
 </div>

</div>
</section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====>
```

---



```
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText));
 } catch {
 const href = (sheet as CSSStyleSheet).href;
 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен – пропускаем */
 }
 }
 }
 return parts.join("\n");
}
```

```
const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid`
```

```

 a.setAttribute("href", `#ch-${a.dataset.goto}`);
 a.removeAttribute("data-goto");
 });
} catch {
 /* ссылки – улучшение, а не обязательная часть файла
}
}

const ids: string[] = [];
host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
 const id = el.dataset.termId;
 if (!id) return;
 if (!ids.includes(id)) ids.push(id);
 const link = document.createElement("a");
 link.className = "term-hint";
 link.href = `#term-${id}`;
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
 link.textContent = el.textContent ?? "";
 el.replaceWith(link);
});

// <details> в файле лучше раскрыть: иначе код и разбросан
host.querySelectorAll("details").forEach((d) => d.setOpen());

return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escapeHtml(
 e.title
)}>
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
</div>`;
 })
 .join("");
}

```

```
${opts.body}
 </article>
```

```
${opts.gloss}
```

```
 <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}
```

```
/**
```

```
 * Картинки встраиваются в файл data-URL: автономная ко
 * иллюстрации офлайн (раньше src вел на ассет приложен
 * картинки исчезали – оставались только самые первые,
 */
```

```
async function inlineImages(html: string): Promise<string> {
 const doc = new DOMParser().parseFromString(`<body>${html}</body>`, 'text/html');
 const imgs = Array.from(doc.querySelectorAll("img[src]"));
 await Promise.all(
 imgs.map(async (img) => {
 const src = img.getAttribute("src") ?? "";
 if (!src || src.startsWith("data:")) return;
 try {
 const res = await fetch(src);
 if (!res.ok) return;
 const blob = await res.blob();
 if (blob.size > 4_000_000) return;
 const data = await new Promise<string>((resolve, reject) => {
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
 });
 img.setAttribute("src", data);
 } catch {
 /* картинка недоступна – глава останется со ссы
 }
 })
);
}
```

```
}

/** Выгрузить всё пособие одним файлом со сквозным оглавлением
export async function downloadBookHtml(chapters: Chapters) {
 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

 const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a></li>`)
 .join("\n");

 for (const c of chapters) {
 const { body: rawBody, termIds } = prepareBody(c.content);
 // картинки глав встраиваются сразу: книга одна, асы разные
 const body = await inlineImages(rawBody);
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="background-color:#f9f9f9; padding:1.5rem 0 0 0">
<p style="text-align:right;font-size:.72rem;margin:0 0 1.5rem 0">Глава ${c.title}</p>
${body}
</div>`);
 }

 const body = `<nav id="toc" style="background:#0f172a;color:#fff;padding:1.5rem 0 0 0">
<h2 style="color:#fff;font-size:1.05rem;font-weight:800;margin:0">Оглавление</h2>
<ol style="margin:0;padding-left:1.1rem;font-size:.85rem">

</nav>
${bodies.join("\n")}`;

 const html = wrap({
 title: "Универсальное пособие по алгоритмам",
 subtitle: `Все темы, ${chapters.length} шт.`,
 css,
 body,
 gloss: glossarySection(allTerms),
 origin: window.location.origin,
 });
}
```

```

/** «билет», «билеты», «билетов», «билете», «билетам»...
const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{

/**
 * Константы TreeWalker числами: глобальный `NodeFilter`
 * во всех окружениях (в jsdom его нет) – с числами разн
 */
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

/** Внутри чего ссылки не ставим: код, уже существующие
const SKIP_SELECTOR = "code, pre, script, style, a, but

/** Одна страница на диапазон? «билеты 18–20» → mst, а
function chapterForRange(from: number, to: number): string {
 let found: string | undefined;
 for (let n = from; n <= to; n += 1) {
 const id = ticketToChapter.get(n);
 if (!id) return undefined;
 if (found && found !== id) return undefined;
 found = id;
 }
 return found;
}

function hrefFor(id: string, lite: boolean): string {
 return lite ? `?lite=1&topic=${encodeURIComponent(id)}
}

/**
 * Строковая версия – для текстовой (`?lite=1`) версии,
 * через `dangerouslySetInnerHTML`, а эффекты разметки
 * HTML режется на «тег» и «текст», меняются только тек
 * вне `pre/code/a`.
 */
export function linkifyTicketsHtml(html: string, opts:

```

```

const walker = document.createTreeWalker(root, SHOW_TEXT,
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return FILTER_REJECT;
 const text = node.nodeValue ?? "";
 if (/билет/i.test(text)) return FILTER_REJECT;
 return FILTER_ACCEPT;
 },
 true);

```

```

const targets: Text[] = [];
let current = walker.nextNode();
while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
}

```

```

let created = 0;
for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 TICKET_RE.lastIndex = 0;
 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;
 while ((m = TICKET_RE.exec(text)) !== null) {
 const from = Number(m[1]);
 const to = m[2] ? Number(m[2]) : from;
 const id = chapterForRange(from, to);
 if (!id || id === opts.selfId) continue;
 if (m.index > last) frag.appendChild(document.createTextNode(
 const a = document.createElement("a");
 a.className = "ticket-link";
 a.dataset.goto = id;
 a.setAttribute("href", hrefFor(id, opts.lite ?? false));
 a.textContent = m[0];
 frag.appendChild(a);
 last = m.index + m[0].length;
));
 }
}

```

```

*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"), "utf-8");
const registryBody = registry.slice(registry.indexOf("VIZUALIZATORY"), registry.length);
const vizIds = new Set([...registryBody.matchAll(/^\s{2}\d+/g)].map(m => m[0].trim()));

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 };
});

```

}

ФАЙЛ 91/110: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

/\*\*

\* Проверка покрытия глав всплывающими подсказками:  
 \* в каждой ли теме находятся термины из глоссария.  
 \*

\* node scripts/audit-terms.mjs

\*/

import { build } from "esbuild";

import path from "node:path";

const ROOT = process.cwd();

await build({

entryPoints: ["src/data/content.ts", "src/data/glossa

bundle: true, format: "esm", platform: "node",

outdir: "tmp/audit-terms", outExtension: { ".js": ".m

external: ["react", "react-dom", "motion", "lucide-re

});

const { chapters } = await import(path.join(ROOT, "tmp/

const { GLOSSARY\_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.\\*\+\?\^\\$\{\}\(\)|[\]\[\]]/g, "\\

const re = new RegExp(

`(?!<![\\p{L}\\p{N}\_-])(\${GLOSSARY\_LABELS.map((l) => e  
 "giu"

);

const map = new Map(GLOSSARY\_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts

const MAX\_PER\_SURFACE = 2;

const MAX\_PER\_TERM = 8;



---

ФАЙЛ 92/110: scripts/export/build-guide-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРӨК: 677 | РАЗМЕР: 10.5 КБ

---

```
/**
 * Компактный PDF пособия: public/export/guide_lite.pdf
 *
 * текст всех тем (без служебной разметки)
 * + снимки визуализаций, если они отрендерены (tmp/viz-
 * capture-viz-shots.mjs в CI через headless-браузер)
 * + иллюстрации страниц (jpg из src/assets)
 * + текстовое описание каждой демонстрации и переменных
 * Python-компилятор (работает и без браузера – локально)
 * – БЕЗ кода панели компилятора и без навигации приложения
 *
 * Запуск: npm run export:guide
 * Цель: 40–60 страниц (жёсткий предел – 100).
 *
 * Вёрстка целиком на собственном курсоре (cy): pdfkit
 * сам, длинные таблицы/код/боксы режутся на фрагменты
 */
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, ensureDir, humanSize } from "../utils";
import { htmlToBlocks } from "../html-to-blocks.mjs";

const GUIDE_PDF_PATH = path.join(OUT_DIR, "guide_lite.pdf");
const SHOTS_DIR = path.join(ROOT, "tmp", "viz-shots");
const ENTRY = path.join(ROOT, "tmp", "guide-entry.ts");

/** Картинки-иллюстрации страниц (те же, что показывает
const CHAPTER_IMAGES = {
 dijkstra: "src/assets/dijkstra-kind.jpg",
 "bellman-ford": "src/assets/bellman-ford-truck.jpg",
```

```

/**
 * Убираем символы, которых нет в DeJaVu (эмодзи и пиктограммы)
 * на их месте появляется «пустой» глиф. Стрелки, математические
 * геометрические фигуры DeJaVu знает — их оставляем.
 */
const UNSUPPORTED = /[\u{1F000}-\u{1FAFF}\u{2700}-\u{2700}
 22}]/gu;
/** Русское склонение счётных существительных: 1 страница
const plural = (n, one, few, many) => {
 const mod10 = Math.abs(n) % 10;
 const mod100 = Math.abs(n) % 100;
 if (mod10 === 1 && mod100 !== 11) return one;
 if (mod10 >= 2 && mod10 <= 4 && (mod100 < 10 || mod100 > 10))
 return many;
};

const plain = (text) =>
 String(text ?? "")
 .replace(UNSUPPORTED, "")
 .replace(/\s{2,}/g, " ")
 .replace(/\u0000/g, "")
 .trim();

/* ----- данные приложения (через esbuild) ----- */

async function loadAppData() {
 ensureDir(path.dirname(ENTRY));
 fs.writeFileSync(
 ENTRY,
 [
 'export { chapters, chapterTopics } from "../src/data/chapters";',
 'export { PAGE_SYNC } from "../src/data/vizSync";',
 'export { VIZ_REGISTRY } from "../src/components/vizRegistry";',
 'export { quizzes } from "../src/data/quizzes";',
].join("\n")
);
};
const res = await build({

```

\*

\* Важно: на время печати нижнее поле страницы уменьш

\* решает, что текст не влез, и молча добавляет пусты

\*/

```
stampFooter() {
 if (this.pages === 0 || !this.doc.page) return;
 const prevBottom = this.doc.page.margins.bottom;
 this.doc.page.margins.bottom = 16;
 const y = PAGE.height - PAGE.margin + 4;
 this.doc.font(this.fonts.regular).fontSize(7.6).fil
 this.doc.text("Универсальное пособие • алгоритмы и
 width: CONTENT_W * 0.62,
 align: "left",
 lineBreak: false,
 ellipsis: true,
 });
 this.doc.text(`${this.footerTitle} • ${this.pages}`
 width: CONTENT_W * 0.38,
 align: "right",
 lineBreak: false,
 ellipsis: true,
 });
 this.doc.page.margins.bottom = prevBottom;
 this.doc.y = this.cy;
}
```

```
newPage(footerTitle) {
 this.stampFooter();
 if (footerTitle !== undefined) this.footerTitle = f
 this.doc.addPage({ size: [PAGE.width, PAGE.height],
 this.pages += 1;
 this.cy = TOP;
}
```

```
firstPage(footerTitle) {
 if (footerTitle !== undefined) this.footerTitle = f
 this.doc.addPage({ size: [PAGE.width, PAGE.height],
 this.pages += 1;
```

```

 current = sentence;
 continue;
 }
 for (const word of sentence.split(/\s+/)) {
 const withWord = current ? `${current} ${word}` : word;
 if (measure(withWord) <= max) current = withWord;
 else {
 flush();
 current = word;
 }
 }
 flush();
 return chunks.length ? chunks : [source];
}

rule(color = "#c7d2fe", width = 1) {
 const y = this.cy;
 this.doc.moveTo(PAGE.margin, y).lineTo(PAGE.margin + width, y);
 this.cy = y + 8;
}

h1(rawText, { size = 20, color = "#111827", rule = true } = {}) {
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + (rule ? 14 : 6) + gapBefore, 22);
 this.cy += gapBefore;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(color);
 this.cy = Math.max(this.cy, this.doc.y) + 2;
 if (rule) this.rule();
}

h2(rawText, { size = 13.5, color = "#1e293b" } = {}) {
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + 6, 20);
}

```

```

const marker = ordinal ? `${ordinal}.` : "*";
const width = CONTENT_W - indent - 4;
const opts = { font: this.fonts.regular, size, width };
const chunks = this.splitToFit(text, opts, BOTTOM);
chunks.forEach((chunk, index) => {
 const h = this.height(chunk, opts);
 this.ensure(h + 2.5, 6);
 const y0 = this.cy;
 if (index === 0) {
 this.doc.font(this.fonts.regular).fontSize(size)
 .text(marker, { align: "right", lineGap: 1 });
 }
 this.doc.font(this.fonts.regular).fontSize(size).text(chunk);
 this.cy = Math.max(y0 + h, this.doc.y) + 2.5;
 if (this.cy > BOTTOM) this.newPage();
});
}

```

```

/** Код: режим на фрагменты по высоте страницы, каждый фрагмент
code(rawText, { size = 8.0 } = {}) {
 const font = this.fonts.mono;
 const inner = CONTENT_W - 22;
 const lines = String(rawText ?? "").replace(UNSUPPORTED_CHARS, "");
 // собираем фрагменты, которые влезают в одну рамку
 let i = 0;
 while (i < lines.length) {
 // Если у низа страницы осталось меньше минимальной высоты
 // СРАЗУ: иначе рамка уедет за BOTTOM, и pdfkit с ошибкой вылетит
 // в обход рендерера (без колонтитула и нумерации)
 if (BOTTOM - this.cy - 16 < 60) this.newPage();
 const avail = BOTTOM - this.cy - 16;
 const maxH = avail;
 let chunk = [];
 let h = 0;
 while (i < lines.length) {
 const candidate = chunk.concat(lines[i]).join(" ");
 const ch = this.height(candidate, { font, size, width });
 if (chunk.length && ch > maxH) break;
 }
 }
}

```

```

 this.doc
 .font(isHeader ? this.fonts.bold : this.fonts.regular)
 .fontSize(size)
 .fillColor("#1f2937")
 .text(String(cells[i] ?? ""), x, y + pad, { w: width })
 }
 this.doc.rect(PAGE.margin, y, CONTENT_W, h).lineWidth(1)
 return h;
 };

 this.ensure(rowHeight(header) + 8, 14);
 let y = this.cy;
 rows.forEach((cells, idx) => {
 const row = [];
 for (let i = 0; i < cols; i++) row.push(cells[i]);
 const h = rowHeight(row);
 if (y + h > BOTTOM) {
 this.cy = y;
 this.newPage();
 y = this.cy;
 if (idx > 0) y += drawRow(header, y, true, false);
 }
 y += drawRow(row, y, idx === 0, idx % 2 === 1);
 });
 this.cy = y + 8;
}

quote(rawText) {
 const text = plain(rawText);
 if (!text) return;
 const font = this.fonts.oblique ?? this.fonts.regular;
 const width = CONTENT_W - 22;
 const opts = { font, size: 9.5, width, lineGap: 1 };
 // Длинная цитата режется по страницам заранее: рисовать
 // текст уйдёт за BOTTOM и pdfkit добавит страницу
 const chunks = this.splitToFit(text, opts, BOTTOM - 10);
 chunks.forEach((chunk) => {
 const h = this.height(chunk, opts);
 });
}

```

```

const lines = rawLines.map((l) => plain(l)).filter(
const inner = CONTENT_W - 20;
let rest = lines.slice();
let first = true;
while (rest.length || first) {
 first = false;
 const headH = this.height(title, { font: this.font
 this.ensure(headH + 24, 16);
 const y0 = this.cy;
 // сколько строк влезет на эту страницу
 const avail = BOTTOM - y0 - headH - 16;
 const taken = [];
 let h = 0;
 while (rest.length) {
 const ch = this.height(rest[0], { font: this.font
 if (taken.length && h + ch > avail) break;
 taken.push(rest.shift());
 h += ch;
 }
 const boxH = headH + h + 12;
 this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, 1
 this.doc.font(this.fonts.bold).fontSize(9.2).fill
 let y = y0 + 7 + headH;
 for (const l of taken) {
 this.doc.font(this.fonts.regular).fontSize(size
 ;
 y += this.height(l, { font: this.fonts.regular,
 }
 this.cy = y0 + boxH + 7;
 if (rest.length) this.newPage();
}
}
}

```

/\* \_\_\_\_\_ сборка \_\_\_\_\_

```

async function main() {
 const fonts = {

```

```

 Count, "интерактивная демонстрация", "интерактивн
 { size: 10.5, color: "#4b5563" }
);
 R.para(
 "Автоматический компактный сборник: весь текст тем
 уализации. Навигация приложения и код панели Pytho
 b Pages.",
 { size: 9.6, color: "#374151" }
);
 R.para(`Дата сборки: ${today}`, { size: 9, color: "#6b728b" });

 const missingViz = chapters.filter((c) => !VIZ_REGISTERED[c.id]);
 R.box("Как устроено пособие", [
 "• Страница = один или несколько билетов: текст, анимация,
 "• У каждой страницы есть интерактивная демонстрация, которую можно
 совывают её напрямую.",
 "• В этом PDF демонстрации показаны снимками (когда вы их
 управляют.",
 missingViz.length
 ? `• Страницы без демонстрации: ${missingViz.map(c => c.title).join(", ")}`
 : "• Демонстрация и синхронизация с компилятором Pytho
]));

 /* — содержание — */
 R.newPage("Содержание");
 R.h1("Содержание", { size: 18 });
 chapters.forEach((c, i) => {
 const topics = chapterTopics[c.id] ?? [];
 R.para(
 `${String(i + 1).padStart(2, "0")}.  ${c.title}${c.description ? " : " + c.description : ""}`,
 { size: 9.3, gapAfter: 2 }
);
 });

 /* — темы — */
 let shotsTotal = 0;
 for (const chapter of chapters) {

```



```

 if (img && fs.existsSync(path.join(ROOT, img))) {
 R.h2("Иллюстрация");
 R.image(path.join(ROOT, img), { maxHeight: 300, c
 }

 /* демонстрация: снимки + описание синхронизации с
 const viz = VIZ_REGISTRY[chapter.id];
 const syncKeys = Object.keys(PAGE_SYNC).filter((k)
 if (viz || syncKeys.length) {
 R.h2("Демонстрация и связь с компилятором");
 if (viz) {
 R.para(viz.title, { size: 10.2, color: "#1e293b
 if (viz.hint) R.para(viz.hint, { size: 9.1, col
 }

 const shots = shotsFor(chapter.id);
 shotsTotal += shots.length;
 for (const shot of shots) {
 R.image(shot.file, {
 maxHeight: 330,
 caption: shot.name ? `${viz?.title ?? chapter
 });
 }

 for (const key of syncKeys) {
 const sync = PAGE_SYNC[key];
 const demo = key.includes("#") ? key.split("#")
 const title = demo ? `Вкладка «${demo}»${sync.v
 const lines = [];
 if (sync.stepNote) lines.push(`Шаг демонстрации
 sync.variables.forEach((v) => lines.push(`• ${v
 lines.push("• Референсный код демонстрации в эт
 if (lines.length) R.box(title, lines);
 }

 if (!shots.length) {
 R.para(
 "Снимки демонстрации добавляются автоматически

```

```

 if (R.pages > 100) console.warn(`                ВНИМАНИЕ: ${R.pages} страниц`);
 if (physical !== R.pages) {
 console.error(`                ОШИБКА: pdftk добавил ${physical} страниц`);
 process.exitCode = 1;
 }
}

main().catch((err) => {
 console.error("Ошибка сборки PDF пособия:", err.message);
 process.exit(1);
});

```

ФАЙЛ 93/110: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

```

/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,
 ensureFile } from "../utils";

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }

 const pages = fs

```

```
main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

---

ФАЙЛ 94/110: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМ: 1000

---

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недоступен)
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir, } from "scripts/export/paths";

/** Расширения, которые считаем «кодом» и включаем в документацию */
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yaml", ".yml",
]);

/** Явные исключения: генерируемое, бинарное и просто служебные */
const EXCLUDE_FILES = new Set(["package-lock.json", "build", "dist"]);
const EXCLUDE_DIRS = ["node_modules", "dist", "build", "public"];

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
```

```

 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileType
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.s
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
 }

 return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.start
 .filter((rel) => !EXCLUDE_FILES.has(path.basename(r
 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 }));
 }

```

/\*\* Достаём заголовки глав из данных пособия (без запус

```

function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("s
 const src = fs.readFileSync(path.join(ROOT, rel), "
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 }
 }
}

```

```
push();
```

```
push(HR);
```

```
push("
```

ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДН

```
push(HR);
```

```
let current = null;
```

```
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
```

```
 if (cat !== current) {
```

```
 current = cat;
```

```
 push();
```

```
 push(`  # ${cat}`);
```

```
 }
```

```
 push(`  ${String(i + 1).padStart(3, " ")}: ${rel}`)
```

```
});
```

```
push();
```

```
push();
```

```
current = null;
```

```
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
```

```
 if (cat !== current) {
```

```
 current = cat;
```

```
 push(HR);
```

```
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
```

```
 push(HR);
```

```
 push();
```

```
 }
```

```
 const content = fs.readFileSync(path.join(ROOT, rel)
```

```
 const lines = content.split("\n");
```

```
 push(SUBHR);
```

```
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
```

```
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |
```

```
 push(SUBHR);
```

```
 push();
```

```
 push(content.replace(/\n+$/, ""));
```

```
 push();
```

```
 push();
```

Универсальное пособие • полный исходный код

```

/** Куда кладём финальные артефакты (эта папка отдаётся
export const OUT_DIR = path.join(ROOT, "public", "export
/** Промежуточные PNG-страницы (в .gitignore, в CI уезж
export const PAGES_DIR = path.join(ROOT, "tmp", "export

export const TXT_PATH = path.join(OUT_DIR, "full_code_a
export const PDF_PATH = path.join(OUT_DIR, "full_code_a

/** Параметры вёрстки страницы (подобраны под A4 @150dp
export const PAGE = {
 /** DPI растра. Можно переопределить: EXPORT_DENSITY=
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
 font: "DejaVu-Sans-Mono",
 fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
 pointsize: 8,
 /** Максимум символов в строке (далее – жёсткий пере
 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапк
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
```

```
});
```

```
/** Пустые элементы (без закрывающего тега). */
```

```
const VOID = new Set(["br", "hr", "img", "input", "meta
```

```
/** Блочные теги: их содержимое собирается в отдельный
```

```
const BLOCK = new Set([
 "h1", "h2", "h3", "h4", "h5", "h6", "p", "li", "pre",
 "summary", "figcaption", "table", "tr", "td", "th", "t
```

```
const HEADING = new Set(["h1", "h2", "h3", "h4", "h5",
```

```
/**
```

```
* Разбор HTML в дерево: { tag, attrs, children }, текст
```

```
* Намеренно простой: страницы пособия генерируются нами
```

```
*/
```

```
export function parseHtml(html) {
```

```
 const root = { tag: "#root", attrs: {}, children: []
```

```
 const stack = [root];
```

```
 const re = /<!--[\s\S]*?-->|<\s*\/([a-zA-Z0-9]+)\s*>|<([a-zA-Z0-9]+)
```

```
 let m;
```

```
 while ((m = re.exec(html)) !== null) {
```

```
 const [full, closeTag, openTag, attrsStr, selfClose
```

```
 if (full.startsWith("<!--")) continue;
```

```
 if (closeTag) {
```

```
 const tag = closeTag.toLowerCase();
```

```
 // закрываем до совпадения (прощает незакрытые <pre>
```

```
 for (let i = stack.length - 1; i >= 1; i--) {
```

```
 if (stack[i].tag === tag) {
```

```
 stack.length = i;
```

```
 break;
```

```
 }
```

```
 }
```

```
 continue;
```

```
 }
```

```

 .replace(/←/g, "←")
 .replace(/&/g, "&")
 .replace(/&#(\d+);/g, (_, d) => String.fromCharCode(d))

const clean = (s) => decodeEntities(s).replace(/\s+/g, " ")

const isEl = (n) => n.tag !== "#text";
const isBlockEl = (n) => isEl(n) && BLOCK.has(n.tag) && !isTextEl(n);

/** Есть ли у узла блочные потомки (тогда его текст «разбит» на строки) */
const hasBlockChild = (node) => node.children.some(isBlockEl);

/** Текст инлайнового поддерева в порядке документа. */
function inlineText(node) {
 let out = "";
 for (const c of node.children) {
 if (c.tag === "#text") {
 out += c.text;
 } else if (SKIP.has(c.tag)) {
 continue;
 } else if (BLOCK.has(c.tag)) {
 continue;
 } else if (c.tag === "br") {
 out += "\n";
 } else {
 out += inlineText(c);
 }
 }
 return clean(out);
}

/** Весь текст поддерева (для ячеек таблиц и подписей). */
function collectText(node) {
 if (node.tag === "#text") return clean(node.text);
 if (SKIP.has(node.tag)) return "";
 if (node.tag === "table") return tableRows(node).map(
 row => row.map(c => collectText(c)).join(" ")
).join("\n");
}

```



```

const emitListItem = (li, ordinal, depth) => {
 const text = collectText(li);
 if (text) push({ type: "li", text, ordinal: ordinal
 // вложенные списки внутри li
 li.children.forEach((c) => {
 if (c.tag === "ul" || c.tag === "ol") emitList(c,
 });
};

const emitList = (list, depth) => {
 let i = 0;
 for (const li of list.children) {
 if (li.tag !== "li") continue;
 i += 1;
 emitListItem(li, list.tag === "ol" ? i : null, depth);
 }
};

const walk = (node, listDepth = 0) => {
 if (node.tag !== "#root" && SKIP.has(node.tag)) return;

 if (node.tag === "table") {
 const rows = tableRows(node);
 if (rows.length) push({ type: "table", rows });
 return;
 }
 if (node.tag === "pre") {
 const text = decodeEntities(collectRawPre(node));
 if (text.trim()) push({ type: "code", text: text });
 return;
 }

 // есть блочные дети: идём по порядку, инлайн-пробе
 if (hasBlockChild(node)) {
 let buf = "";
 const flush = () => {
 const t = clean(buf);
 if (t) push({ type: "p", text: t });
 };
 }
};

```

```
/** То же самое, но простым текстом (для .txt-дампа и в
export function htmlToPlainText(html) {
 const lines = [];
 for (const b of htmlToBlocks(html)) {
 if (b.type === "heading") lines.push("", "#".repeat
 else if (b.type === "summary") lines.push("", "▶ "
 else if (b.type === "li") lines.push(" ".repeat(b.c
 else if (b.type === "code") lines.push("", ...b.tex
 else if (b.type === "table") {
 lines.push("");
 b.rows.forEach((r) => lines.push("| " + r.join("
 lines.push("");
 } else if (b.type === "quote") lines.push("> " + b.
 else if (b.type === "caption") lines.push(" " + b
 else lines.push(b.text, "");
 }
 return lines.join("\n").replace(/\n{3,}/g, "\n\n").tr
}
```

---

ФАЙЛ 97/110: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ

---

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child process";
```

```

const im = imageMagickCmd();
const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

fs.rmSync(PAGES_DIR, { recursive: true, force: true });
ensureDir(PAGES_DIR);

const total = pages.length;
const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`)
 `стр. ${idx + 1} / ${total}`;
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
});

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "DejaVuSansMono",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

-----  
\* визуализациям матрицы чужих размерностей и ловит пад  
\*/

```
import { act } from "react";
import { createRoot } from "react-dom/client";
import type { Root } from "react-dom/client";
import App from "../../src/App";
import { chapters, chapterTopics, SECTIONS, sectionOf }
import { quizzes } from "../../src/data/quizzes";
import { VIZ_REGISTRY } from "../../src/components/vizR
import { VizLiveStatus } from "../../src/components/Viz
import { PAGE_SYNC, buildInitTemplate } from "../../src
import { VizChapterContext, emitVizState, clearVizState
import { buildDebugRunner, type DebugResult } from "../
```

```
type Py = { runPythonAsync: (src: string) => Promise<un
```

```
export interface TestResult {
 group: string;
 name: string;
 ok: boolean;
 ms: number;
 detail: string;
 consoleErrors: string[];
}
```

```
const results: TestResult[] = [];
let consoleErrors: string[] = [];
const realConsoleError = console.error;
```

```
/* -----
/* утилиты
/* -----
```

```
const sleep = (ms: number) => new Promise((r) => setTim
```

```
async function settle(ms = 40) {
 await act(async () => {
 await sleep(ms);
```

```

 el.dispatchEvent(new window.MouseEvent("click", { bubble: true }));
 await sleep(20);
 });
}

/** Панель компилятора – все поиски внутри неё, чтобы не искать в документе
function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[aria-label="Compiler"]');
}

function panelText(): string {
 return (panel()?.textContent ?? "").replace(/\s+/g, " ");
}

function textarea(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>("textarea");
}

function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window.HTMLTextAreaElement.prototype, "value");
 if (!setter) throw new Error("нет нативного setter'a для HTMLTextAreaElement");
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubble: true }));
 });
}

async function key(k: string, init: KeyboardEventInit = { }) {
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("keydown", init));
 await sleep(20);
 });
}

/** Ждём появления признака в DOM (текст/предикат), перепроверяем каждые 100мс
async function waitFor(pred: () => boolean, timeoutMs = 1000) {
 const deadline = Date.now() + timeoutMs;
 while (Date.now() < deadline) {
 if (pred()) return true;
 await sleep(100);
 }
 return false;
}

```

```

 await act(async () => {
 root!.render(<App />);
 });
 await settle(80);
 return host;
 }

 /** Смонтировать одну визуализацию так, как это делает
 async function mountViz(vizId: string) {
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) host.remove();
 host = freshHost();
 const entry = VIZ_REGISTRY[vizId];
 if (!entry) throw new Error(`в реестре нет визуализации`);
 const Comp = entry.Component;
 root = createRoot(host);
 await act(async () => {
 root!.render(
 <VizChapterContext.Provider value={vizId}>
 <VizLiveStatus demoTitle={entry.title} />
 <Comp />
 </VizChapterContext.Provider>
);
 });
 await settle(60);
 return host;
 }

 const BAD_DOM = [
 { re: /\bNaN\b/, why: "NaN в разметке" },
 { re: /\bundefined\b/, why: "undefined в разметке" },
 { re: /\[object Object\]/, why: "[object Object] в разметке" },
];

```

```

 if (errs.length > 0) {
 ok = false;
 detail = `${detail}\nconsole.error (${errs.length})
 }
 results.push({ group, name, ok, ms, detail, consoleError});
 const mark = ok ? "PASS" : "FAIL";
 realConsoleError === console.error;
 process.stdout.write(`${mark} [${group}] ${name}${ok}
 if (!ok) process.stdout.write(detail.split("\n").map(
 return ok;
 }

/* -----
/* сценарии
/* -----

/** Фильтры прогона: HAR_CHAPTERS=segment-trees,floyd и
const onlyChapters = (process.env.HAR_CHAPTERS || "").split(",");
const skipGroups = new Set((process.env.HAR_SKIP || "").split(","));
/** HAR_SYNC=floyd,dijkstra – прогнать только эти записи
const onlySync = new Set((process.env.HAR_SYNC || "").split(","));

export async function runSuite({ py }: { py: Py }): Promise<Result> {
 const chapterList = onlyChapters.length ? chapters.filter(chapter =>
 const want = (group: string) => !skipGroups.has(group);
 // перехват console.error приложения (React пишет сюда)
 console.error = (...args: unknown[]) => {
 consoleErrors.push(args.map(a => (a instanceof Error ? a.message : a)));
 };

 /* — 1. Панель Python: реальный запуск на каждой главе
 for (const chapter of chapterList) {
 const group = "A. страницы";
 await test(group, `${chapter.id}: монтируется, бейджик
 await mountApp(chapter.id);
 const txt = bodyText();
 if (!txt.includes(chapter.title.slice(0, 18))) throw new Error("не
 if (!VIZ_REGISTRY[chapter.id]) return "визуализация не найдена";
 }
}

```

```

/* — 2. Флойд: матрицы ЛЮБОЙ размерности из кода полн
const floydCases: { name: string; vars: Record<string
 { name: "5x5 (меньше демо-7x7)", vars: { n: 5, k: 1
 }, [9, 6, 9, 9, 0]] }, expect: "5x5" },
 { name: "10x10 (больше демо – старый краш nodeNames
 from({ length: 10 }, (_, c) => (r === c ? 0 : (r
 { name: "3x3 с бесконечностями", vars: { n: 3, k: 0
 "3x3" },
 { name: "не квадратная 4x6", vars: { n: 4, k: 1, i:
 , 17, 18, 19, 0, 20]] }, expect: "Демо-схема граф
 { name: "словарь словарей d[0][1]", vars: { n: 4, k
 2: { 0: 9, 1: 9, 2: 0, 3: 1 }, 3: { 0: 3, 1: 9,
 { name: "ключи-пары \"(i, j)\"", vars: { n: 3, k: 0
 "(1, 2)": 2, "(2, 0)": 1, "(2, 1)": 9, "(2, 2)":
 { name: "плоский список – это НЕ матрица", vars: {
 { name: "80x80 (обрезка до 64)", vars: { n: 80, k:
 (_, c) => (r === c ? 0 : 1 + ((r + c) % 50))) }
 { name: "пустая матрица", vars: { n: 0, k: 0, i: 0,
 { name: "матрица под именем D и k/i/j вне диапазона
 [10, 11, 12, 0]] }, expect: "4x4" },
];

for (const c of floydCases) {
 await test("В. Флойд: размерности", c.name, async () => {
 await mountViz("floyd");
 await act(async () => {
 emitVizState("floyd", { line: 3, func: "<module
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\\s
 if (!/данные из компилятора/.test(txt)) throw new
 if (!txt.includes(c.expect)) throw new Error(`в р
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в
 return `бейдж: ${txt.match(/данные из компилятор
 });
}

```



```

 await sleep(40);
 });
 const grids = () => Array.from((host ?? document).querySelectorAll("div.grid"));
 let g0 = grids()[0];
 if (!g0) throw new Error("нет ни одной матрицы блоков");
 const firstCell = () => (g0.children[0] as HTMLElement)?.textContent;
 // «Оригинал (1x1)» = сама матрица A: значения видны в консоли
 if (firstCell() !== "1") throw new Error(`оригинал не 1x1`);
 if ((g0.children[7] as HTMLElement)?.textContent ?? "").trim() !== ""
 // шаги кликаются даже под live-k из компилятора
 await click(findButton("Шаг 2 (4x4)"), "вкладка шаг 2");
 const txt = (host?.textContent ?? "").replace(/\s+/g, " ");
 if (!/Матрица блоков 4x4/.test(txt)) throw new Error("не 4x4");
 g0 = grids()[0];
 const cls = (g0.children[0] as HTMLElement)?.className;
 // пустая активная клетка (код смотрит на (0,0), стили не заданы)
 if (!/border-dashed/.test(cls) || !/indigo/.test(cls))
 throw new Error(`пустая активная клетка не нарисована`);
 if (domDefects(host).length) throw new Error(`артефакты`);
 return "оригинал из A с первого шага, вкладка «Шаг 2»";
});

/* — 4. Дейкстра / Форд-Беллман: чужие имена вершин */
for (const [viz, vars] of [
 ["dijkstra", { u: 0, v: 3, w: 4, dist: [0, 4, 2, 6, 5] }],
 ["bellman-ford", { n: 5, m: 4, i: 1, u: 2, v: 4, w: [0, 4, 2, 6, 5] }],
]) as [string, Record<string, unknown>][] {
 await test("D. графы путей", `${viz}: вершины 0...4 в`);
 await mountViz(viz);
 const before = allButtons().length;
 const playEnabled = (findButton(/^Пуск|Пуск$/)) as HTMLElement;
 await act(async () => {
 emitVizState(viz, { line: 5, func: "<module>", vars });
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\s+/g, " ");
 if (!/данные из компилятора/.test(txt)) throw new Error("не 4x4");
}

```

```

 });

 if (!VIZ_REGISTRY[chapterId]) return `шагов ${result.steps.length}`;
 await mountViz(chapterId);
 const finalVars = result.finalValues ?? result.steps[result.steps.length - 1].finalValues;
 await act(async () => {
 emitVizState(chapterId, { line: result.steps[result.steps.length - 1].line,
 vars: Object.keys(finalVars) });
 await sleep(40);
 });
 const txt = (host?.textContent ?? "").replace(/\\s/g, " ");
 if (!/данные из компилятора/.test(txt)) throw new Error("некорректные данные");
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в визуализации: ${defects.join(", ")}`);
 return `шагов ${result.steps.length}${result.steps[result.steps.length - 1].finalValues ? " с переменными " : ""}`;
 });
}

```

/\* — 6. Поведение редактора: пустой код, ошибки, глаголы

```

const ch = "dijkstra";

```

```

await test("F. редактор", "только комментарии: редактор", async () => {
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "поиск");
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "# просто комментарий\n# и ещё один\n");
 const done = await waitFor(() => bodyText().include("комментарий"));
 if (!done) throw new Error("не дождался сообщения");
 if (!textarea()) throw new Error("редактор исчез после клика");
 if (!/встроенное демо/.test(bodyText())) throw new Error("некорректные данные");
 return "редактор на месте, бейдж «встроенное демо»";
});

```

```

await test("F. редактор", "синтаксическая ошибка: редактор", async () => {
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "поиск");

```

```

 if (!/встроенное демо/.test(bodyText())) throw new Error("не демо");
 if (allButtons().length < 5) throw new Error("кнопки не найдены");
 return `пустой код: кнопок ${allButtons().length},
 });

```

```

await test("F. редактор", "глаз: эталон попадает в редактор");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "поиск");
const ta = textarea();
if (!ta) throw new Error("нет редактора");
const init = ta.value;
const editorButtons = allButtons();
const ref = PAGE_SYNC[ch]?.code ?? "";
if (!ref) throw new Error("у страницы нет эталонного кода");
const done0 = await waitFor(() => RUN_DONE.some((m) => m === "done"));
if (!done0) throw new Error("инициализация не выполнена");

```

```

// — глаз ВКЛ: эталон лежит В РЕДАКТОРЕ, и редактор не должен быть пустым
await click(findButton("Вставить эталонный код"), "вставить");
const trace = await waitFor(() => !!findButton("/Остановить"));
if (!trace) throw new Error("эталон не запустился. попробуйте снова");
if (!textarea()) throw new Error("редактор исчез во время теста");
if (/Ошибка:/.test(panelText())) throw new Error("ошибка в панели");
const ta2 = textarea();
const firstRefLine = ref.split("\n").find((l) => l.trim().length > 0);
if (!ta2?.value.includes(firstRefLine)) {
 throw new Error(`выполняется не тот текст, что лежит в эталоне
 0, 100)} `);
}

```

```

const mustBe = [
 { what: "пуск/автопроход", re: /Запустить код|Продолжить/ },
 { what: "копирование кода", re: /Скопировать код|Копировать/ },
 { what: "заккрыть панель", re: /Заккрыть компилятор/ },
 { what: "шаг вперёд", re: /Шаг вперёд|вперёд/i },
 { what: "шаг назад", re: /Шаг назад|назад/i },
 { what: "свернуть листинг", re: /Свернуть листинг/ },
];
const btns = allButtons().join(" | ");

```

```

 await mountApp("floyd");
 await click(findButton("Боковая панель Python"), "п
 const ta2 = textarea();
 if (!ta2) throw new Error("на новой странице нет ре
 if (ta2.value.includes("MY-MARKER-14")) throw new E
 if (!ta2.value.includes("# «")) throw new Error("на
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta3 = textarea();
 if (!ta3?.value.includes("MY-MARKER-14")) throw new
 return "у каждой страницы свой код, возврат сохраня
 });

```

```

await test("F. редактор", "редактор живёт и во время
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const done = await waitFor(() => bodyText().include
 if (!done) throw new Error(`трасса не собралась. Па
 if (!textarea()) throw new Error("во время прогона
 if (!findButton(/Остановить прогон трассы/)) throw
 // Листинг сворачивается (освобождая место редактор
 // и редактор остаются на месте.
 const codeLines = () => panel()?.querySelectorAll("
 if (codeLines() === 0) throw new Error("в раскрытой
 await click(findButton("Свернуть листинг трассы"),
 await settle(80);
 if (!textarea()) throw new Error("после сворачивани
 if (codeLines() !== 0) throw new Error(`листинг не
 if (!findButton("В конец трассы")) throw new Error(
 await click(findButton("Показать листинг трассы"),
 await settle(80);
 if (codeLines() === 0) throw new Error("листинг не
 // Esc: прогон остановлен, трасса убрана, редактор
 await key("Escape");
 await settle(150);
 if (!textarea()) throw new Error("Esc убрал редактор
 if (findButton(/Остановить прогон трассы/)) throw n
 if (!/встроенное демо/.test(bodyText())) throw new

```

```
return `редактор: ${editorBtns} кнопок, листинг: ${
});
```

```
await test("Н. доступность", "бейдж честно различает
 await mountApp(ch);
 // 1) демо: компилятор ещё ничего не присылал
 if (!/встроенное демо/.test(bodyText())) throw new Error("демо не работает");
 if (/переменных пока нет/.test(bodyText())) throw new Error("переменные не пришли");
 await click(findButton("Боковая панель Python"), "показать панель Python");
 const listing = await waitFor(() => !!findButton(/Описание кода/));
 if (!listing) throw new Error("панель не показала трассу");
 // 2) код выполнен, отладчик на первой строке: переключиться на код
 const awaiting = await waitFor(() => /переменных пока нет/);
 if (!awaiting) throw new Error("после запуска бейдж не переключился");
 const code = $(bodyText().match(/встроенное демо|данные из компилятора/));
 // 3) дошли до конца трассы — картинка обязана жить
 await click(findButton("В конец трассы"), "в конец трассы");
 const linked = await waitFor(() => /данные из компилятора/);
 if (!linked) throw new Error("в конце трассы визуализация не появилась");
 // 4) сняли связь (Esc + пустой код) — снова демо
 await key("Escape");
 await settle(80);
 const ta = textarea();
 if (!ta) throw new Error("редактор не вернулся по Escape");
 typeInto(ta, "");
 const cleared = await waitFor(() => /встроенное демо/);
 if (!cleared) throw new Error("после очистки кода бейдж не переключился");
 if (/переменных пока нет/.test(bodyText())) throw new Error("переменные не пришли");
 return "демо → выполнено/переменных нет → данные из компилятора";
});
```

/\* — 7. Текстовый режим для PDF/парсеров —————

/\* — 9. Ссылки между билетами, разделы содержания, бейджи

```
await test("J. шапка", "кнопки загрузки не прячутся и
 await mountApp("segment-trees");
 const menu = document.querySelector("#download-menu");
 if (!menu) throw new Error("нет меню загрузки в шапке");
```

```
});
```

```
await test("I. ссылки", "содержание: разделы по поряд
 await mountApp("bridges-code");
 const sidebar = document.querySelector("#sidebar-ro
 if (!sidebar) throw new Error("нет сайдбара содержа
 const headers = Array.from(sidebar.querySelectorAll
 .map((el) => (el.textContent ?? "").trim()))
 .filter((t) => SECTIONS.some((sec) => sec.title === t))
 if (headers.length !== SECTIONS.length) {
 throw new Error(`в содержании ${headers.length} раз
 }
 const expected = SECTIONS.map((sec) => sec.title).join("
 if (headers.join(" | ") !== expected) throw new Error
 // 11 «Мосты» и 12 «Точки сочленения» читаются парой
 const items = Array.from(sidebar.querySelectorAll("li
 const iBridges = items.findIndex((t) => t.includes("Мосты"))
 const iArtic = items.findIndex((t) => t.includes("Точки сочленения"))
 if (iBridges < 0 || iArtic < 0) throw new Error(`в содержании
 if (Math.abs(iArtic - iBridges) !== 1) throw new Error("неверно
 // 2 «разреженная» и 5 «префиксные» – в одном разделе
 if (sectionOf("sparse-table").id !== sectionOf("prefixes").id)
 throw new Error("билеты 2 и 5 попали в разные разделы")
 }
 if (sectionOf("bridges-code").id !== sectionOf("graphs").id)
 throw new Error("билеты 11 и 12 попали в разные разделы")
 }
 // порядок чтения: номера билетов сверху вниз не убывающий
 const nums = Array.from(sidebar.querySelectorAll("b
 .map((b) => (b.querySelector("span")?.textContent ?? ""))
 .filter((t) => /^\\d/.test(t))
 .map((t) => Number.parseInt(t, 10));
 const ticketed = chapters.filter((c) => (chapterTopic === c.title));
 if (nums.length !== ticketed) {
 throw new Error(`номера билетов видны у ${nums.length} билетов`)
 }
 for (let i = 1; i < nums.length; i += 1) {
 if (nums[i] < nums[i - 1]) throw new Error(`порядок чтения не

```

```

 if (!shown().includes(list[0].question.slice(0, 24))
 throw new Error(`первый вопрос не показан: ${list[0].question}`);
 }
 const wrong = list[0].options.findIndex((_, i) => i !== list[0].correctIndex);
 await click(ask(list[0].options[wrong].slice(0, 20)));
 await settle(80);
 if (!/Неверно/.test(shown())) throw new Error("на неверный ответ");
 if (!shown().includes(list[0].explanation.slice(0, 24)))
 throw new Error("после ответа нет разбора – пользуйтесь подсказкой");
 }
 const nextBtn = ask("Следующий вопрос");
 if (!nextBtn) throw new Error("нет кнопки «Следующий вопрос»");
 await click(nextBtn, "дальше");
 await settle(80);
 if (!shown().includes(list[1].question.slice(0, 24)))
 // верный ответ
 await click(ask(list[1].options[list[1].correctIndex].slice(0, 20)));
 await settle(80);
 if (!/Верно/.test(shown())) throw new Error("на верный ответ");
 if (!/вопрос 2\\./.test(shown())) throw new Error(`с ${list[1].question} не совпало`);
 return `блиц из ${list.length} вопросов отвечает, правильно`;
 }));

await test("I. факты", "память: разреженная жирная, префиксная", async () => {
 const memoryRow = async (topic: string) => {
 await mountApp(topic);
 const rows = Array.from(document.querySelectorAll("table tbody tr"));
 const row = rows.find((r) => /^\\s*Память/.test(r.textContent));
 if (!row) throw new Error(`на странице «${topic}» не найдено содержимое`);
 return Array.from(row.querySelectorAll("td")).map((c) => c.textContent);
 };
 const sparse = await memoryRow("sparse-table");
 const prefix = await memoryRow("prefix-sums-2d");
 // билет 2: у префиксной N × M, у разреженной – с логарифмом
 if (!/log/.test(sparse.join(" "))) throw new Error("неверно задана разреженная таблица");
 const prefixCell = sparse.find((c) => !/log/.test(c));
 if (!prefixCell) throw new Error("в сравнении билетов не найдено префиксное значение");
});

```

```
});
```

/\* — 10. Сквозной путь: свой код чужой размерности, 1

```
await test("К. свой код", "Флойд 4x4 из редактора вед
 await mountApp("floyd");
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(
 ta,
 [
 "n = 4",
 "INF = 999",
 "d = [[0, 3, INF, 7], [8, 0, 2, INF], [5, INF, 0, 3], [6, 4, 6, 0]]",
 "k = 0",
 "i = 0",
 "j = 0",
 "for k in range(n):",
 " for i in range(n):",
 " for j in range(n):",
 " if d[i][k] + d[k][j] < d[i][j]:",
 " d[i][j] = d[i][k] + d[k][j]",
 "print(d[0], d[1][0])",
].join("\n")
);
 const ready = await waitFor(() => bodyText().includes("Готово"));
 if (!ready) throw new Error(`своя программа не выполнилась`);
 if (!bodyText().includes("[0, 3, 5, 6]")) {
 throw new Error(`Флойд на своей матрице 4x4 посчитал 0, 3, 5, 6`);
 }
 if (!textarea()) throw new Error("редактор исчез, попробуйте снова");
 await click(findButton("В конец трассы"), "в конец трассы");
 const linked = await waitFor(() => /данные из компилятора/);
 if (!linked) throw new Error("демонстрация не принята");
 const page = bodyText();
 if (!/4x4/.test(page)) throw new Error(`демонстрация не принята`);
 const defects = domDefects(document.querySelector("#defects"));
```



```

 await click(findButton("Боковая панель Python"), "п
 if (!textarea()) throw new Error("нет редактора");
 await click(findButton("Вставить эталонный код"), "
 const ready = await waitFor(() => bodyText().include
 if (!ready) throw new Error(`эталон Косарайю не вып
 if (/Ошибка:/.test(panelText())) throw new Error(`э
 const out = panelText();
 if (!/порядок выхода:/.test(out)) throw new Error(`
 // граф на экране: 0→1→2→0, 2→3, 3→4 – ровно две SC
 if (!/компонент: 2/.test(out)) throw new Error(`в гр
 if (!textarea()) throw new Error("редактор исчез, п
 await click(findButton("В конец трассы"), "в конец
 const linked = await waitFor(() => /данные из компи.
 if (!linked) throw new Error("демонстрация не приня
 const defects = domDefects(document.querySelector("
 if (defects.length) throw new Error(`артефакты в де
 return "эталон Косарайю: order считается DFS-ом, на
 });

```

```

await test("G. lite-режим", "?lite=1 отдаёт текст без
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) host.remove();
 host = freshHost();
 window.history.replaceState({}, "", "/?lite=1&topic
 root = createRoot(host);
 await act(async () => {
 root!.render(<App />);
 });
 await settle(120);
 const txt = bodyText();
 if (txt.length < 2000) throw new Error(`lite-режим
 if (/main\.py|Pyodide/.test(txt)) throw new Error("
 if (!/разреженн/i.test(txt)) throw new Error("в lite

```

```
/**
 * Аудит внутренних ссылок пособия: ничего не должно ве
 *
 * npm run verify:links
 *
 * Проверяет (по настоящим модулям приложения, собранным)
 * 1. билеты 1–24 закрыты страницами, id в chapterTopics
 * 2. разделы содержания: каждая страница в своём разд
 * 3. упоминания «билет N» в тексте ведут на существующи
 * 4. все data-goto / href="?topic=..." указывают на су
 * 5. якоря href="#..." есть в той же главе;
 * 6. glossary.simulator существует в реестре и (кроме
 * имеет свою страницу – иначе модалка не получит
 * 7. glossary.demo – работающая мини-демо;
 * 8. у каждого ключа PAGE_SYNC есть запись в реестре
 * 9. у каждой страницы есть визуализация;
 * 10. страницы связаны перекрёстными ссылками (темы не
 */
import * as esbuild from "esbuild";
import { mkdirSync, writeFileSync } from "fs";

const OUT = "tmp/verify/links-entry.mjs";
const FAILS = [];
const NOTES = [];
const ok = (cond, message) => (cond ? true : (FAILS.push(message), false));

mkdirSync("tmp/verify", { recursive: true });
writeFileSync(
 "tmp/verify/links-entry.ts",
 [
 'export { chapters, chapterTopics, SECTIONS, section',
 'export { GLOSSARY } from "../../src/data/glossary"',
 'export { VIZ_REGISTRY } from "../../src/components',
 'export { PAGE_SYNC } from "../../src/data/vizSync"',
 'export { quizzes } from "../../src/data/quizzes";',
 'export { MINI_DEMO_KINDS } from "../../src/componen',
].join("\n")
);
```

```

/* 2. разделы содержания */
for (const chapter of chapters) {
 const sec = sectionOf(chapter.id);
 ok(!sec, `страница «${chapter.id}» не попала ни в од
 const tickets = chapterTopics[chapter.id] ?? [];
 ok(
 tickets.every((n) => n >= sec.from && n <= sec.to),
 `билеты страницы «${chapter.id}» (${tickets.join(",
 });
}
{
 const bySection = SECTIONS.map((s) => ({
 s,
 nums: chapters.flatMap((c) => (sectionOf(c.id).id =
 }));
 for (let i = 1; i < bySection.length; i += 1) {
 const prevMax = Math.max(...bySection[i - 1].nums);
 const curMin = Math.min(...bySection[i].nums);
 ok(curMin > prevMax, `разделы пересекаются: «${bySe
 }
 const all = bySection.flatMap((x) => x.nums);
 for (let n = all[0]; n <= all[all.length - 1]; n += 1
 ok(all.includes(n), `в содержании дыра: билет ${n}
 }
}

/* 3–5. ссылки внутри текста страниц */
const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{
let ticketMentions = 0;
let explicitLinks = 0;
const withoutLinks = [];
for (const chapter of chapters) {
 const html = chapter.content ?? "";
 let mentionsHere = 0;

 // 3. упоминания «билет N» – только внутри текста, не
 const textOnly = html

```

```

 `гlossарий «${term.id}»: simulator "${term.simula
 ые компилятора`
 });
}
if (term.demo) ok(demoKinds.has(term.demo), `гlossарий
}

/* 8–9. реестр и синхронизация */
for (const key of Object.keys(PAGE_SYNC)) {
 const base = key.split("#")[0];
 ok(registryIds.includes(base), `PAGE_SYNC["${key}"]:
}
for (const id of registryIds) {
 if (!chapterIds.has(id) && !EXTRA_DEMOS.has(id)) {
 FAILS.push(`реестр: визуализация "${id}" не является
 }
}
for (const chapter of chapters) {
 ok(registryIds.includes(chapter.id), `страница «${cha
}

/* 96. файлы, на которые ведут кнопки шапки, действител
{
 const { existsSync, statSync } = await import("fs");
 for (const file of ["public/export/guide_lite.pdf", "
 ok(existsSync(file), `шапка ссылается на ${file}, н
 if (existsSync(file) && statSync(file).size < 1000)
 }
}

/* 10. связанные темы + блиц */
for (const id of withoutLinks) NOTES.push(`нет перекрёс
const noQuiz = chapters.filter((c) => !(quizzes[c.id] ?

console.log("————— ссылки и структура содержания —
console.log(` страниц: ${chapters.length} • разделов:
console.log(` упоминаний «билет N» в тексте: ${ticketM
console.log(` визуализаций в реестре: ${registryIds.le

```

```

mkdirSync(OUT_DIR, { recursive: true });

writeFileSync(
 `${OUT_DIR}/contract-entry.ts`,
 `import { buildDebugRunner } from "../../src/data/deb
import { PAGE_SYNC } from "../../src/data/vizSync";
import { chapters } from "../../src/data/content";
export const runnerOf = (src: string) => buildDebugRunn
export const sync = Object.fromEntries(Object.entries(P
export const declared = Object.fromEntries(Object.entri
export const chapterIds = chapters.map((c) => c.id);
`
);

const built = await esbuild.build({
 entryPoints: [`${OUT_DIR}/contract-entry.ts`],
 bundle: true,
 outfile: `${OUT_DIR}/contract-bundle.mjs`,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 sourcemap: false,
 loader: { ".css": "empty", ".png": "dataurl", ".jpg":
 logLevel: "warning",
});
void built;
const mod = await import(`${process.cwd()}/${OUT_DIR}/c

const py = await loadPyodide({ indexURL: "node_modules/
const printed = [];
py.setStdout({ batched: (m) => printed.push(m) });
py.setStderr({ batched: (m) => printed.push("[stderr] "

const only = process.argv.slice(2);
const ident = (t) => /^[A-Za-z_][A-Za-z0-9_]*$/.test(t)
/** «xk / x», «dist[v]», «(i-1)//2», «π» → набор python
const alts = (raw) => raw.split(/[\/[\],]|\\s+или\\s+/).m
```

```

const neverFilled = (mod.declared[key] ?? [])
 .map(alts)
 .filter((a) => a.some(ident))
 .map((a) => a.find((x) => ident(x) && seen.has(x)))
 .filter(Boolean)
 .filter((name) => {
 let sawValue = false;
 let sawNonEmpty = false;
 for (const step of [...result.steps, { values: result }]) {
 const v = step.values?.[name];
 if (v === undefined || v === null) continue;
 sawValue = true;
 if (!isEmpty(v)) sawNonEmpty = true;
 }
 return sawValue && !sawNonEmpty;
 });
if (neverFilled.length) issues.push(`объявлены, но ни

const stderr = printed.filter((p) => p.startsWith("[s
if (stderr.length) issues.push(`предупреждения: ${std

if (issues.length) problems++;
rows.push({ key, steps: result.steps.length, vars: seen
}

rows.sort((a, b) => (a.status === "ок") - (b.status ===
console.log("ЭТАЛОННЫЕ КОДЫ СТРАНИЦ (настоящий CPython
console.log("=".repeat(96));
for (const r of rows) {
 const mark = r.status === "ок" ? " ok " : " x ";
 console.log(`${mark}${r.key.padEnd(34)} шагов ${String
 === "ок" ? "" : r.status}`);
}
console.log("=".repeat(96));
console.log(`всего: ${rows.length} | с проблемами: ${
writeFileSync(`${OUT_DIR}/reference-code.json`, JSON.st
process.exit(problems > 0 ? 1 : 0);

```

```

const { window } = dom;

window.ResizeObserver = window.ResizeObserver ?? OBSERVER
window.IntersectionObserver = window.IntersectionObserver ?? OBSERVER
window.matchMedia =
 window.matchMedia ??
 ((media) => ({
 matches: false,
 media,
 onchange: null,
 addListener() {},
 removeListener() {},
 addEventListener() {},
 removeEventListener() {},
 dispatchEvent: () => false,
 }));
window.scrollTo = () => {};
window.HTMLElement.prototype.scrollIntoView = () => {};
window.HTMLElement.prototype.focus = window.HTMLElement.prototype.focus;
window.URL.createObjectURL = window.URL.createObjectURL;
window.URL.revokeObjectURL = window.URL.revokeObjectURL;
Object.defineProperty(window, "innerWidth", { value: window.innerWidth });
Object.defineProperty(window, "innerHeight", { value: window.innerHeight });
Object.defineProperty(window, "devicePixelRatio", { value: window.devicePixelRatio });
window.Element.prototype.getBoundingClientRect = function() {
 return { x: 0, y: 0, top: 0, left: 0, right: 400, bottom: 400 };
};

const copy = [
 "window",
 "document",
 "navigator",
 "HTMLElement",
 "HTMLInputElement",
 "HTMLTextAreaElement",
 "HTMLButtonElement",
 "HTMLAnchorElement",
 "HTMLDivElement",

```

ФАЙЛ 102/110: scripts/verify/live-matrix.test.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 103 | РАЗМЕР: 1.5 КБ

```
/**
 * Модульные проверки разбора «живых» данных компилятора.
 *
 * npm run verify:unit
 *
 * liveMatrix() – то, благодаря чему визуализации матриц
 * суммы 2D, разреженная таблица) принимают данные ЛЮБОГО
 * пользователя вместо захардкоженной демо-сетки.
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
writeFileSync(
 `${OUT_DIR}/unit-entry.ts`,
 `export { liveMatrix, pickLiveVar, clampIndex } from
`);
await esbuild.build({
 entryPoints: [`${OUT_DIR}/unit-entry.ts`],
 bundle: true,
 outfile: `${OUT_DIR}/unit-bundle.mjs`,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 loader: { ".css": "empty" },
 logLevel: "warning",
});
const { liveMatrix, pickLiveVar, clampIndex } = await import(
 "unit-bundle.mjs");

let passed = 0;
const failures = [];
function check(name, actual, expected) {
```



```

check("строка → null", m("abc"), null);
check("массив строк-мусора → null", m(["a", "b"], ["c"

/* — liveMatrix: особые значения -----
check('"inf" → +Infinity', cells(["inf", 1], [2, "-inf
check('числа-строки "42" → 42', cells(["1", "2"], ["3"
check("булевы значения ячейкой не считаются", m([[true,
check("обрывок до 64×64", shape(Array.from({ length: 80
 4", numbers: true }));
check("hasNumbers=false невозможна для живой матрицы",

/* — pickLiveVar -----
check("первое найденное имя по приоритету", pickLiveVar
check("берёт следующее имя, если первого нет", pickLive
check("ничего не найдено → null", pickLiveVar({ x: 1 },
check("нет снимка → null", pickLiveVar(undefined, ["d"]
check("ключ со значением undefined всё равно считается"

/* — clampIndex -----
check("null остаётся null", clampIndex(null, 5), null);
check("размер 0 → null", clampIndex(2, 0), null);
check("обрезка сверху", clampIndex(99, 5), 4);
check("обрезка снизу", clampIndex(-7, 5), 0);
check("дробный индекс усекается", clampIndex(2.9, 5), 2
check("NaN → null", clampIndex(Number.NaN, 5), null);
check("Infinity → null", clampIndex(Number.POSITIVE_INF

console.log("=".repeat(72));
console.log(`liveMatrix / pickLiveVar / clampIndex: пр
for (const f of failures) console.log(`  x ${f}`);
console.log("=".repeat(72));
process.exit(failures.length ? 1 : 0);

```

```

function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window, 'value').set;
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubble: true }));
 });
}

async function click(el: Element | null | undefined, label: string) {
 if (!el) throw new Error(`нет кнопки: ${label}`);
 await act(async () => {
 el.dispatchEvent(new window.MouseEvent("click", { bubbles: true }));
 await sleep(20);
 });
}

function findButton(match: RegExp): HTMLButtonElement | null {
 return Array.from(document.querySelectorAll("button")).find(
 b => match.test([b.getAttribute("aria-label"), b.textContent].join(" "))
) as HTMLButtonElement | undefined;
}

export async function probe() {
 const host = document.createElement("div");
 document.body.appendChild(host);
 window.history.replaceState({}, "", "/*topic=dijkstra");
 const root = createRoot(host);
 await act(async () => {
 root.render(<App />);
 });
 await settle(80);

 process.stdout.write("\n=== 1. открыли панель, ждём асинхронно...");
 await click(findButton(/Боковая панель Python/), "панель Python");
 state("t=0");
 for (const t of [300, 1000, 3000, 8000]) {
 await settle(t - (t === 300 ? 0 : 0));
 state(`t+${t}`);
 if (/Трасса готова|Нет шагов|Пустой код|Ошибка/.test(state)) {
 return;
 }
 }
}

```

```
 } else {
 process.stdout.write(" редактора нет (листинг) – ждём редактора\n");
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("keydown", {
 key: "Esc",
 }));
 await sleep(30);
 });
 state("после Esc");
 }
 }
}
```

---

ФАЙЛ 104/110: scripts/verify/README.md

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

---

# Проверки приложения (`npm run verify`)

Три уровня проверок, которые можно гонять локально и в CI. Для локального запуска нужен: DOM даёт `jsdom`, Python – `pyodide` из `node\_modules`. Браузер нужен: 3.12, что грузится в браузере пользователя, только без ч/б.

```
```bash
```

```
npm run verify          # всё сразу: unit → эталонные коды
npm run verify:unit     # 38 проверок разбора живых данных
npm run verify:code     # 46 эталонных кодов страниц по DOM
npm run verify:dom      # 139 сценариев: страницы, панели
npm run verify:probe    # диагностика панели Python во время запуска
```
```

Отчёты кладутся в `tmp/verify/` (каталог в .gitignore):  
помазочно по DOM-сценариям, `reference-code.json` – по кодам.

## Что именно проверяется

### `verify:unit` – `live-matrix.test.mjs`

Разбор данных, которые присылает компилятор, в матрицу. Матрица имеет размерности: список списков, рванные строки, словарь слов.

Универсальное пособие • полный исходный код

-----  
Фильтры: `HAR\_CHAPTERS=floyd,dijkstra`, `HAR\_SKIP=A,E`,

## Как это заведено в приложение

- \* `window.\_\_algv0PythonRuntime` — точка подмены интерпретатора `PythonCompiler.tsx`. В браузере пользователя поле не грузит Pyodide с CDN как раньше; харнес кладёт туда локальный
- \* Состояние визуализаций передаётся через `emitVizState` (`src/data/vizStepBus.ts`) — те же события, что шлёт и получает идёт по боевому пути, а не через внутренние API компонентов

-----  
ФАЙЛ 105/110: scripts/verify/run-browser-checks.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 96 | РАЗМЕР: 1.5 КБ

-----  
/\*\*  
 \* DOM-проверки приложения: jsdom + настоящий Pyodide и  
 \*  
 \* npm run verify:dom  
 \*  
 \* Фильтры:  
 \* HAR\_CHAPTERS=floyd,dijkstra только эти страницы  
 \* HAR\_SKIP=A,E пропустить группы (A  
 \* C матрицы, D графы, E  
 \* F редактор, G lite, H  
 \* HAR\_SYNC=floyd,mst#prim только эти записи эти  
 \*  
 \* Оркестратор: грузит CPython ДО создания jsdom (иначе  
 \* в браузере, и лезет за wasm на CDN), кладёт runtime и  
 \* window.\_\_algv0PythonRuntime и собирает реальные компоненты  
 \*/  
import \* as esbuild from "esbuild";  
import { writeFileSync, mkdirSync } from "fs";  
import { loadPyodide } from "pyodide";  
import { createDom } from "../dom-env.mjs";

```

 jsdomErrors.push(`unhandledRejection: ${reason instance
});
```

```
log("прогоняю сценарии...\n");
const results = await mod.runSuite({ py, window, document
const sum = mod.summary();
```

```
writeFileSync(`${OUT_DIR}/report.json`, JSON.stringify(

```

```
const failed = results.filter((r) => !r.ok);
console.log("\n" + "=".repeat(96));
console.log("СВОДКА");
console.log("=".repeat(96));
for (const [group, g] of Object.entries(sum.byGroup)) {
 console.log(` ${group.padEnd(26)} ok: ${String(g.ok)
}
```

```
console.log(`\n ВСЕГО ${sum.total}: прошло ${sum.passed}
if (failed.length) {
 console.log("\nПРОВАЛЫ:");
 for (const f of failed) console.log(` x [${f.group}]
}
```

```
if (jsdomErrors.length) {
 console.log("\nОШИБКИ ОКРУЖЕНИЯ (jsdom/неперехваченны
 for (const e of [...new Set(jsdomErrors)].slice(0, 10
}
```

```
const stray = consoleErrors.filter((e) => !/not wrapped
if (stray.length) {
 console.log(`\nCONSOLE.ERROR вне тестов (${stray.leng
 for (const e of [...new Set(stray)].slice(0, 10)) con
}
```

```
console.log(`\nотчёт: ${OUT_DIR}/report.json | время:
process.exit(sum.failed > 0 || jsdomErrors.length > 0 ?
```

```
function log(message) {
 process.stdout.write(`[verify:dom] ${message}\n`);
}
```

```

 "import.meta.env.BASE_URL": '"/',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
 logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/pages.js`);
log("зондирую панель...");
await mod.probe();
process.exit(0);
=====
```

## РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

```

ФАЙЛ 107/110: .github/workflows/deploy-pages.yml
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9 КБ

```

name: Deploy app to GitHub Pages

```
ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ
(у бота-интеграции нет admin-прав на них – API отвечает за них)
#
1. Settings → Pages → Build and deployment → Source
Если там «Deploy from a branch» (legacy) – шаг 2.
2. Settings → Environments → github-pages → Deployment protection rules
добавьте паттерн «arena/**» и ветку «main» (или «main/*» если не хотите деплоить из ветки)
иначе деплой из новой рабочей ветки отклоняется
```

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

```
uses: actions/upload-pages-artifact@v5
with:
 path: ./dist
```

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 # Ранняя и понятная ошибка вместо невнятного «Fail»
 # когда в репозитории Pages не переключён на сбор
 - name: Preflight – Pages собирается через GitHub
 env:
 GH_TOKEN: ${ secrets.GITHUB_TOKEN }
 run: |
 bt=$(gh api "repos/${ github.repository }/pages" |
 echo "Pages build_type: ${bt}")
 if ["${bt}" != "workflow"]; then
 echo "::error title=Pages не настроен::Settings
 те этот запуск. См. комментарий в начале de
 exit 1
 fi

 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
```

---

ФАЙЛ 108/110: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРOK: 128 | РАЗМЕР: 4.0

---

name: Rebuild code PDF

```
if: "!contains(github.event.head_commit.message, '[
```

```
steps:
```

- name: Checkout  
uses: actions/checkout@v6  
with:  
 fetch-depth: 0  
 persist-credentials: true
- name: Setup Node.js  
uses: actions/setup-node@v7  
with:  
 node-version: "24"  
 cache: npm
- name: Install ImageMagick + DejaVu fonts  
run: |  
 sudo apt-get update -qq  
 sudo apt-get install -y --no-install-recommen  
 # На Ubuntu политика ImageMagick иногда запре  
 # разрешаем то, что нужно пайплайну (text -> p  
 for policy in /etc/ImageMagick-6/policy.xml /  
 if [ -f "\$policy" ]; then  
 sudo sed -i 's/rights="none" pattern="\{P  
 fi  
done  
convert -version
- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
 EXPORT\_DENSITY: \${github.event.inputs.densi  
run: npm run export:png



```

- name: Commit rebuilt export back to the branch
 run: |
 git config user.name "github-actions[bot]"
 git config user.email "41898282+github-action@users.noreply.github.com"
 git add -- public/export/full_code_all_pages.
 if git diff --cached --quiet; then
 echo "Экспорт не изменился – коммит не нужен"
 exit 0
 fi
 git commit -m "chore(export): пересборка full_code_all_pages.
 git push origin "HEAD:${GITHUB_REF_NAME}"

```

ФАЙЛ 109/110: .github/workflows/verify.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 71 | РАЗМЕР: 2.3 КБ

name: Verify

# Проверки приложения на каждом пуше и PR:

```

verify:unit – разбор «живых» данных компилятора (lexer, parser, ast, ...)
verify:code – все эталонные Python-коды страниц выгрузки
под трейсером панели и создают обещания
verify:dom – приложение монтируется в jsdom: страница
матрицы чужих размерностей, кнопки, ...
#
```

# Headless-браузер не нужен: jsdom + pyodide из node\_modules

on:

push:

branches:

- "\*\*"

paths-ignore:

- "public/export/\*\*"  
- "\*\*/\*.md"

pull request:

```
run: npm run verify:code

- name: DOM-проверки приложения (jsdom)
 run: npm run verify:dom

- name: Отчёт проверок
 if: always()
 uses: actions/upload-artifact@v4
 with:
 name: verify-report
 path: tmp/verify/report.json
 if-no-files-found: ignore
```

---

## РАЗДЕЛ: ПРОЧЕЕ

---

---

ФАЙЛ 110/110: .ai\_skills/universal\_guide\_authoring/SKIL  
КАТЕГОРИЯ: Прочее | СТРОК: 252 | РАЗМЕР: 13.3 KB

---

# Universal Guide Authoring – шаблоны страниц пособия

Обязательный справочник для **\*\*создания и правки контента** репозитории. На него ссылаются `AGENTS.md` и `GEMINI.md` собираем **\*\*только\*\*** из блоков ниже – не придумываем новый «голый» markdown.

Контент живёт в `src/data/\*\*/\*.ts` как массив объектов `src/data/content.ts` (при одинаковом `id` побеждает бо

---  
## 1. Объект главы

```ts

```
-----
<!-- смысловой блок (один или несколько) -->
<div class="bg-slate-700/50 p-6 rounded-xl border-l
  <h3 class="text-xl font-bold text-emerald-400 mb-
    ...определение, абзацы, стат-сетка, аналогии, спол
  </div>
</div>
</section>
```
```

Цвет левой полосы и заголовка блока – семантика раздела

| Полоса / цвет                             | Смысл раздела   |
|-------------------------------------------|-----------------|
| ---                                       | ---             |
| `border-blue-500` + `text-blue-400`       | определение, «о |
| `border-emerald-500` + `text-emerald-400` | алгоритм,       |
| `border-indigo-500` + `text-indigo-400`   | теория, док     |
| `border-amber-500` + `text-amber-400`     | ловушки, част   |
| `border-rose-500` + `text-rose-400`       | ограничения, «к |

---

## ## 3. Библиотека блоков (точные классы)

### ### 3.1 Определение / строгое правило – всегда по центру

```
```html
<div class="bg-slate-900 p-4 rounded-lg mb-6 text-cente
  <p class="text-lg text-emerald-300 italic mb-2">Строг
  <p class="text-xl font-mono text-white">if d[v] > d[u]
</div>
```
```

### ### 3.2 Обычный абзац

```
```html
<p class="text-slate-300 text-sm mb-4">
  Текст с <b>акцентом</b> и инлайн-кодом
  <span class="font-mono text-emerald-300">dist[v]</span>

```

```
        Ограничения
    </div>
    <p class="text-slate-300 text-sm mb-4">...</p>
</div>
</div>
```
```

Второй карточкой может быть «Аналогия 2», «Ограничения» «Когда не работает» – маккуп тот же (пунктирная rose-pan

### ### 3.5 Инфо-блок с таблицей

```
```html
<div class="bg-slate-800/70 p-5 rounded-lg border border-
  <p class="text-sm font-bold text-white mb-2">  Заголо
  <p class="text-slate-300 text-sm mb-3">Пояснение пере
  <table class="w-full text-xs text-left border-collapse
    <thead>
      <tr class="text-slate-500 border-b border-slate-700
        <th class="py-1.5 pr-2 font-bold">Колонка</th>
        <th class="py-1.5 pr-2 font-bold">Колонка</th>
      </tr>
    </thead>
    <tbody>
      <tr class="border-b border-slate-800 align-top">
        <td class="py-2 pr-3 font-bold">Значение</td>
        <td class="py-2 pr-3 font-mono">O(log n)</td>
      </tr>
    </tbody>
  </table>
</div>
```
```

### ### 3.6 Спойлер `<details>` – **\*\*обязателен\*\*** для кода, и

```
```html
<details class="bg-slate-900/40 rounded-lg border border-
  <summary class="p-4 font-bold text-slate-400 outline-
```

8. Связь с другими билетами («тот же find/union – сердце»)

Демонстрацию вставлять не нужно: `ChapterView` монтирует по `id` главы после текста.

5. Контракт «демонстрация ↔ компилятор»

Каждая страница обязана иметь и визуализацию, и синхронизацию:

- `src/components/vizRegistry.tsx` – запись с ключом ****demoId**** (`title`, опциональный `hint`, `Component`).
- `src/data/vizSync.ts` – запись `PAGE\_SYNC[id]` или `PAGE\_SYNC[demoId]`, где `demoId` – второй аргумент `emitVizDemo(demoId, title)` (вкладки одной демонстрации).
- Поля записи: `vizTitle`, `stepNote` (что считается одной страницей), `variables: [{ name, role, range }]`, `code` (референт на код).
- Переменные, которые читает визуализация, должны быть одного из типов (`int`, `list`, `dict`, `str`): экземпляры классов снотификатора до графики не доезжают.
- Стартовый шаблон в редакторе компилятора – `buildInit` и инициализация переменных до строки `# --- тело алгоритма`.

Проверка перед коммитом: `npm run dev` → страница открывает редактор, подставляет код, шаг демонстрации подсвечивает нужные поля.

6. Экспорт в PDF (`npm run export:guide`)

Текст страниц парсится `scripts/export/html-to-blocks.mjs`, результат – `public/export/guide\_lite.pdf` (текст + визуализации, без кода компилятора и без навигации). Чтобы экспорт был полным:

- заголовки – только `h2/h3/h4`; списки – `ul/ol/li`; теги кода – `pre`; определения – обычный `p` (см. 3.1);
- ****смысл не должен держаться на эмодзи****: в PDF используются только латиница и цифры.